Causal Econometrics with Julia

Xiang Ao

2026-08-29

Table of contents

Preface

Most applied questions are causal. Did a program raise earnings? Did a drug reduce mortality? Did a policy change behavior? With experimental data, a difference in means can sometimes answer the question. With observational data, it almost never can. Causal econometrics is the business of saying what would have to be true for a number we can compute to mean the causal object we want.

This book is a working guide in Julia. It is for applied researchers who already know regression and probability, and who want to see the methods carried out end to end. Each chapter gives the identifying assumptions and then runs code on a real or simulated data set. Where it matters, the influence functions, score equations, and weighting schemes are written out so the link between the formula and the Julia call is visible.

Julia is not the usual choice for this material. R remains the default language of applied causal inference, and a companion R volume — Introduction to Causal Econometrics with Observational Data — covers the same ground. A messier working notebook, Topics on Econometrics and Causal Inference, holds the rougher posts that fed into both books. The Julia version exists because some of the heavier estimators — TMLE on large samples, distributional difference-in-differences, fully Bayesian g-computation — are uncomfortably slow in R, and because Julia's type system makes it possible to write small, focused estimation packages whose code is easy to read. Several such packages were written alongside this book and are used throughout: `CausalEstimate.jl` for unified TMLE and AIPW, `CausalGraphs.jl` for graph-based identification, `Lavaan.jl` for structural equation modeling, `Crumble.jl` for causal mediation, and a handful of smaller libraries for difference-in-differences, synthetic control, shift-share IV, and regression discontinuity. They are not requirements for following the text, but readers who want to see how an estimator is actually built will find the source short enough to read.

The book is organized in the order in which an applied project usually runs. Part I is identification. Part II is estimation. Part III is designs: difference-in-differences, synthetic control, instrumental variables, regression discontinuity, and shift-share IV. The remaining parts cover longitudinal settings, survival outcomes, mediation, and causal discovery. The appendix lists the Julia packages used in the examples.

The book is meant to be read at a desk with Julia running. Source files, datasets, and a `Project.toml` that pins package versions are available in the repository linked above; the "Edit this page" link at the foot of each chapter goes directly to the corresponding `.qmd` file. Corrections and suggestions are welcome through the issues tracker.

A note on the code you see. Hidden setup chunks (package loading, helper definitions) execute when the book is rendered but are not shown. Some visible code blocks are marked `eval: false`: these are displayed for readability but not run, either because they only show which packages a chapter uses or because they require a tool or package outside the book's default environment (these are

flagged where they appear). A block marked `eval: false` therefore shows intended usage rather than executed output.

How to cite

Ao, Xiang. *Causal Econometrics with Julia*. https://xiangao.github.io/causal_econometrics_julia/.

```
@book{ao_causal_econometrics_julia,
  author = {Ao, Xiang},
  title  = {Causal Econometrics with Julia},
  url    = {https://xiangao.github.io/causal_econometrics_julia/}
}
```

Part I.

Identification

1. Identification & Potential Outcomes

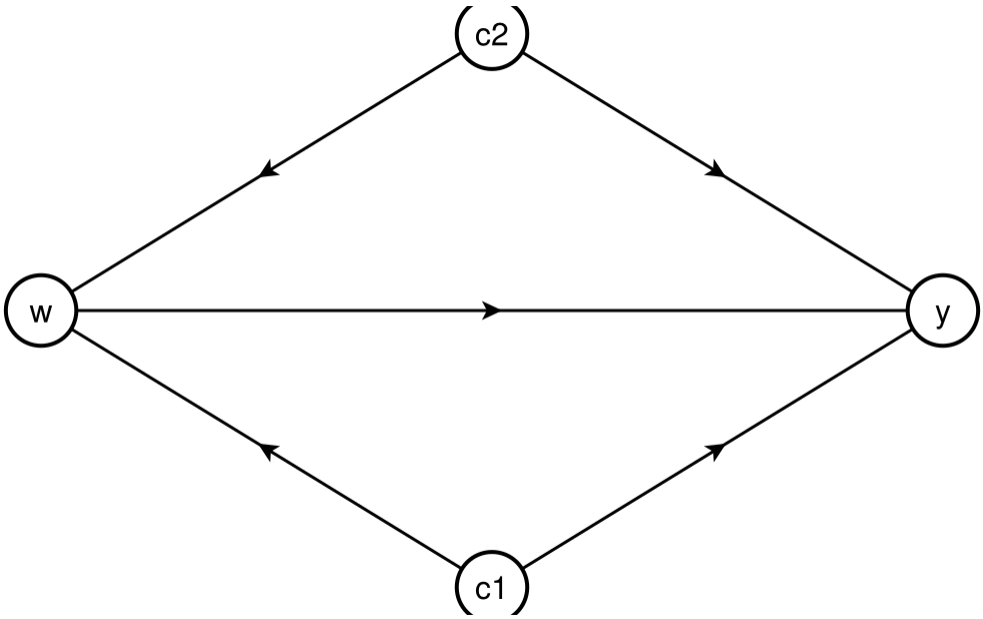
```
using CairoMakie
CairoMakie.activate!(type = "png")
using GraphMakie
using Graphs
using GeometryBasics
```

1.1. Why Do We Need Tools for Causal Inference?

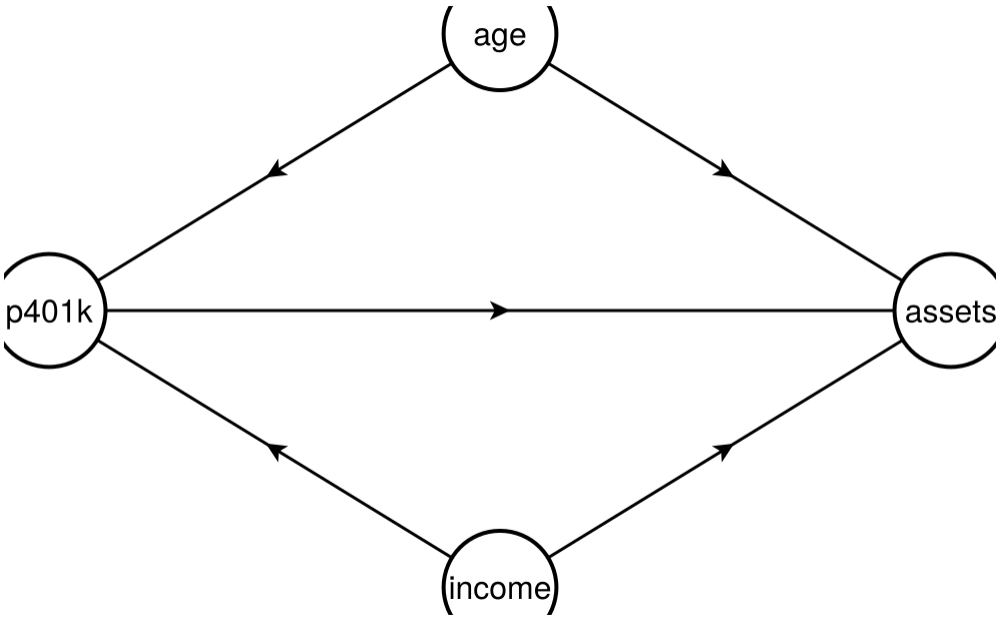
Association is not causation. We have known how to measure associations for a long time, but most econometric models are associational. The gap between the two arises from confounding.

1.2. Confounders

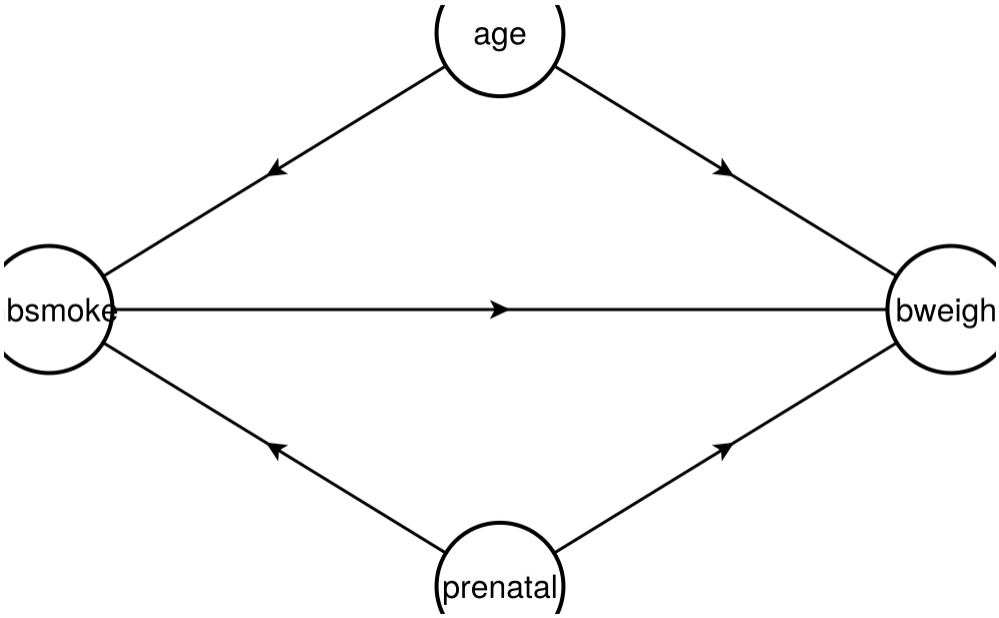
In this graph, w is the treatment, y the outcome, and c_1, c_2 are confounders. Both confounders affect both w and y, so the observed association between w and y may reflect confounding rather than a causal effect.



A concrete example: income and age may both drive 401(k) participation and assets. To identify the causal effect of participation on assets, we need to control for income and age.



Another example: prenatal care and maternal age may affect both smoking and birth weight. Even if smoking has no causal effect on birth weight, it will appear to because of these shared causes.



1.3. Controlled and Uncontrolled Confounders

If we can control for all confounders, correlation does imply causation. The two questions are: do we observe all confounders, and how do we control for them? In a randomized experiment neither question arises. In an observational study both do. If c_2 is unobserved, any estimate of the effect of w on y is biased.

1.4. Potential Outcome Setup

We study causal effects through the potential outcomes (counterfactual) framework. Each unit has two potential outcomes, $Y(1)$ and $Y(0)$, one for each treatment state. These are fixed before any assignment; the experiment tries to recover them.

The fundamental problem is that we never observe both: only the realized outcome $Y = WY(1) + (1 - W)Y(0)$ is seen. Individual treatment effects $\tau_i = Y_i(1) - Y_i(0)$ are not identifiable; only averages can be estimated.

We write W for treatment assignment and X for covariates.

1.5. ATE, ATT and ATU

The standard estimands are:

$$\tau_{ATE} = E[Y(1) - Y(0)] \tag{1.1}$$

$$\tau_{ATT} = E[Y(1) - Y(0)|W = 1] \tag{1.2}$$

$$\tau_{ATU} = E[Y(1) - Y(0)|W = 0] \tag{1.3}$$

$$\tau_{ATE} = \rho\tau_{ATT} + (1 - \rho)\tau_{ATU} \tag{1.4}$$

where $\rho = P(W = 1)$.

1.6. Identification: From Potential Outcome to Observed

We focus on τ_{ATT}. To identify it, we need:

$$E[Y(1)|W=1] \tag{1.5}$$

which is easy, because we observe it.

And we need

$$E[Y(0)|W=1] \tag{1.6}$$

To identify this, we'll need unconfoundedness:

$$E[Y(0)|W,X] = E[Y(0)|X] \tag{1.7}$$

and overlap:

$$\pi(x) = P(W=1|X=x) < 1 \tag{1.8}$$

(for the ATT only $\pi(x) < 1$ is needed; the ATE requires $0 < \pi(x) < 1$).

Unconfoundedness says that within each subgroup $X = x$, treated and control units have the same mean of $Y(0)$: $E[Y(0)|W=1, X=x] = E[Y(0)|W=0, X=x] = E[Y|W=0, X=x]$. Averaging over the covariate distribution of the treated,

$$E[Y(0)|W=1] = E\big[E[Y|W=0,X] \mid W=1\big], \tag{1.9}$$

which involves only observed data. Therefore ATT is identified:

$$\begin{aligned} \tau_{ATT} &= E[Y(1) - Y(0)|W=1] \\ &= E[Y|W=1] - E\big[E[Y|W=0,X] \mid W=1\big] \end{aligned} \tag{1.10}$$

Note the conditional-on-X assumption does NOT imply the marginal equality $E[Y(0)|W=1] = E[Y(0)|W=0]$: treated and control units have different covariate distributions, so the adjustment over X is essential. In a randomised experiment, unconfoundedness holds without conditioning on X (i.e. $E[Y(0)|W] = E[Y(0)]$); then $E[Y(0)|W=1] = E[Y(0)|W=0] = E[Y|W=0]$, the formula collapses to $E[Y|W=1] - E[Y|W=0]$, and a difference-in-means estimator is consistent.

$$\hat{\tau}_{ATT} = \bar{Y}_1 - \bar{Y}_0 \tag{1.11}$$

But in observational studies, most cases we cannot assume unconfoundedness. What can we do to justify unconfoundedness better?

1.7. Unconfoundedness and Overlap

Weaker Assumptions: Unconfoundedness (conditional independence, or ignorability), stated jointly for both potential outcomes since we are after the ATE here (as opposed to the ATT case above, which only required it for $Y(0)$):

$$E[Y(d)|W, X] = E[Y(d)|X] \quad \text{for } d \in \{0, 1\} \tag{1.12}$$

Overlap:

$$0 < \pi(x) = P(W = 1|X = x) < 1 \tag{1.13}$$

SUTVA (stable unit treatment value assumption): 1. No interference between units. No spillovers. 2. Consistency. No hidden versions of treatment or control. Treatment is clearly defined.

Within each subgroup of $X = x$, W is independent of $Y(0)$. With the overlap assumption, we ensure there is a comparable control unit for each treated unit.

However, these two criteria are often contradictory: we may need more covariates to satisfy conditional independence; meanwhile, if the dimension of covariates goes high, then it's likely in some partition we don't have control units.

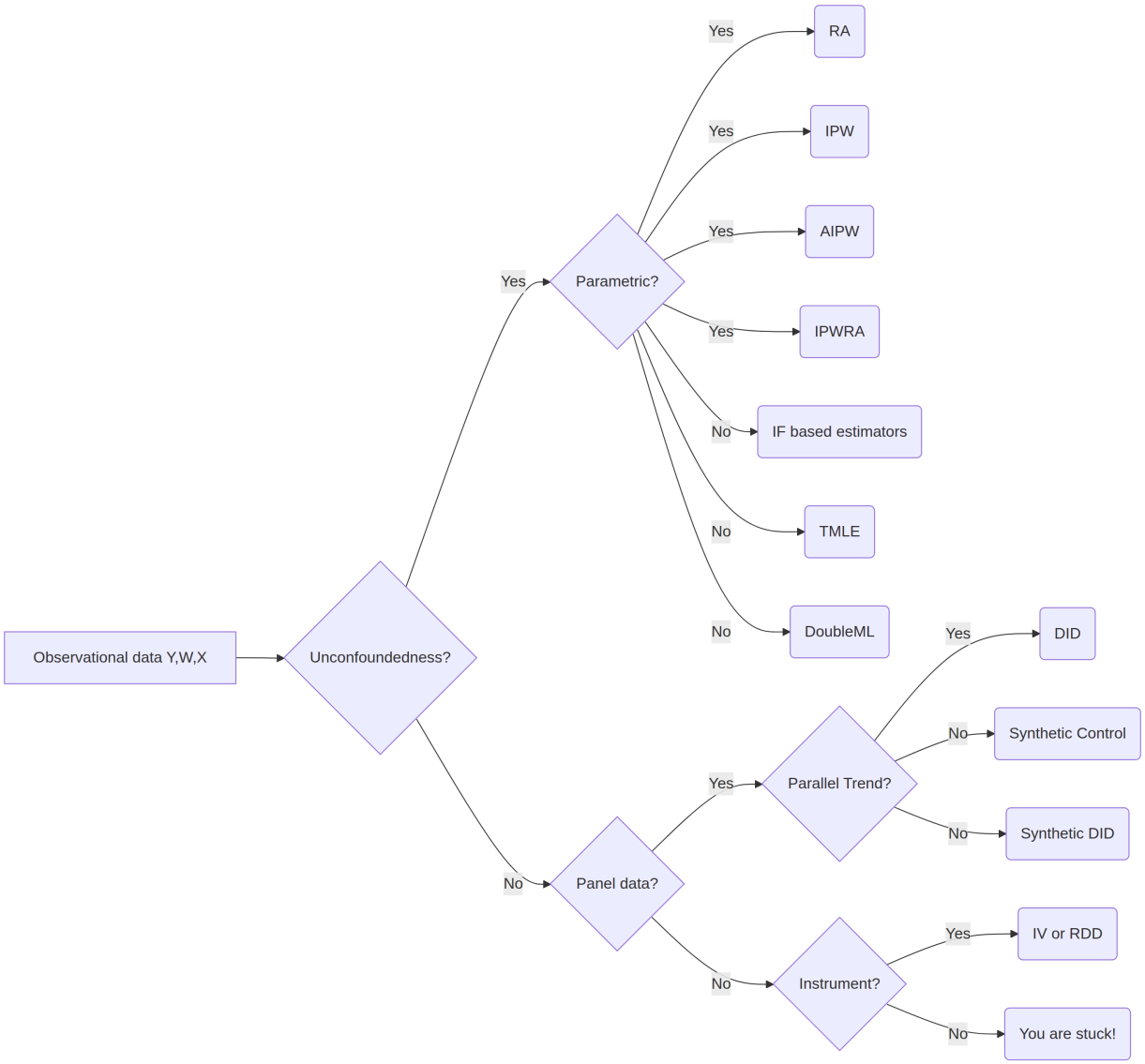
We need four assumptions for identification:

- Unconfoundedness (conditional independence, or ignorability)
- Overlap (positivity, common support)
- No interference
- Consistency

The last two together are SUTVA, the stable unit treatment value assumption (Rubin 1980).

1.8. Observational Data Flowchart

In observational studies, I summarise the decision what method to use for identification of causal effect:


Observational data Y,W,X
Unconfoundedness?
Yes
Parametric?
Yes
RA
Yes
IPW
Yes
AIPW
Yes
IPWRA
No
IF based estimators
No
TMLE
No
DoubleML
No
Panel data?
Yes
Parallel Trend?
Yes
DID
No
Synthetic Control
No
Synthetic DID
No
Instrument?
Yes
IV or RDD
No
You are stuck!

2. Five Estimands on One DGP

```
using DataFrames
using Distributions
using Random
using Statistics
using CairoMakie
CairoMakie.activate!(type = "png")
using GLM
```

Before choosing an estimator, we need to know the estimand. ATE, ATT, LATE, CATE and QTE are not five ways to estimate the same object. They are five different objects.

Here I use one simulated data set so we can calculate all of them. The simulation is useful because we observe both potential outcomes. In real data we do not, which is exactly why identification assumptions matter.

2.1. The data-generating process

Each person has an observed covariate X_i, an unobserved type U_i, and a binary treatment D_i. The instrument Z_i is a random offer. It changes the probability of treatment, but it has no direct effect on the outcome.

```
Random.seed!(42)
n = 20_000

X = rand(Uniform(0, 1), n)          # observed covariate, in [0,1]
U = rand(Normal(0, 1), n)           # latent type, unobserved
Z = rand(Bernoulli(0.5), n)         # random instrument

# Treatment assignment: depends on Z (the instrument) and U (selection on type)
# Higher U → more likely to take treatment regardless of Z (always-takers)
# Lower  U → less likely to take treatment regardless of Z (never-takers)
# Middle U → responsive to Z (compliers)
pD0 = @. clamp(0.10 + 0.20 * U, 0, 1)       # P(D=1 | Z=0, U)
pD1 = @. clamp(0.10 + 0.20 * U + 0.50, 0, 1)  # P(D=1 | Z=1, U) - adds 0.50

# ONE latent draw per unit, two thresholds: since pD1 >= pD0, this gives
# D1 >= D0 for every unit - monotonicity (no defiers) holds EXACTLY.
```

```
# (Drawing D0 and D1 with independent rand() calls would create defiers
#  by construction, even though pD1 > pD0.)
V  = rand(n)
D0 = V .< pD0   # potential treatment if not offered slot
D1 = V .< pD1   # potential treatment if offered slot
D  = ifelse.(Z .== 1, D1, D0)

# Heterogeneous treatment effect: depends on X
# Y(d) = baseline + d * tau(X) + 0.3 * U + noise
 (x) = 1.0 + 2.0 * x            # the true individual treatment effect function
Y0   = 0.5 .* X .+ 0.3 .* U .+ randn(n)
Y1   = Y0 .+  .(X)
Y    = ifelse.(D, Y1, Y0)

df = DataFrame(X=X, Z=Z, D=D, Y=Y)
first(df, 6)
```

	X	Z	D	Y
	Float64	Bool	Bool	Float64
1	0.173575	1	1	-1.64247
2	0.321662	1	1	1.29054
3	0.258585	1	1	2.52079
4	0.166439	1	1	-0.00756264
5	0.527015	1	1	4.47846
6	0.483022	0	0	1.91673

In the simulated data we have `Y0` and `Y1` for every observation. That lets us calculate the true estimands directly. With real data, each observation only reveals one potential outcome.

2.2. The five estimands

2.2.1. ATE — average treatment effect

$$\text{ATE} = \mathbb{E}[Y(1) - Y(0)] \tag{2.1}$$

ATE is the mean effect in the whole population. It compares the mean outcome if everyone were treated with the mean outcome if nobody were treated.

```
ate_true = mean(Y1 .- Y0)
@printf("ATE (population mean of Y1 - Y0) = %.3f\n", ate_true)

# What the integral over  (X) should be:  (1 + 2x) dx on [0,1] = 1 + 1 = 2
@printf("Theoretical ATE =  (1 + 2x)dx on [0,1] = 2.000\n")
```

```
ATE (population mean of Y1 - Y0) = 2.002
Theoretical ATE =  (1 + 2x)dx on [0,1] = 2.000
```

2.2.2. ATT — average treatment effect on the treated

$$\text{ATT} = \mathbb{E}[Y(1) - Y(0) \mid D = 1] \tag{2.2}$$

ATT is the mean effect for the units that actually took treatment. It differs from ATE when treatment selection is related to treatment-effect heterogeneity.

```
att_true = mean(Y1[D] .- Y0[D])
@printf("ATT (mean of Y1-Y0 conditional on D=1) = %.3f\n", att_true)
```

```
ATT (mean of Y1-Y0 conditional on D=1) = 2.006
```

Here ATT is close to ATE – and in fact the two are analytically identical in this DGP, exactly 2.0 in the population; the estimates above differ only by finite-sample simulation noise, not by any true gap between the estimands. The reason is mechanical: the individual treatment effect $\tau_i = 1 + 2X_i$ depends only on X_i, while selection into treatment depends on U_i (and the instrument Z_i below), and X_i is generated independently of U_i and Z_i. Since treatment-effect heterogeneity is unrelated to who selects into treatment or who complies with the instrument, averaging τ_i over the treated subpopulation or over compliers gives the same population average as over everyone – hence ATE = ATT = LATE = 2.0 exactly here. If selection (or compliance) also depended on X, these estimands would diverge, since ATE, ATT, and LATE are different averages of τ_i across different, non-identical subpopulations.

2.2.3. LATE — local average treatment effect (Imbens-Angrist)

$$\text{LATE} = \mathbb{E}[Y(1) - Y(0) \mid D(1) > D(0)] \tag{2.3}$$

LATE is the mean effect for compliers, the units whose treatment status is changed by the instrument. Under the usual IV assumptions, the Wald estimator identifies this effect, not the ATE. Identifying LATE by the Wald ratio additionally requires monotonicity (no defiers): $D(1) \geq D(0)$ for all i. This DGP imposes it by generating both potential treatments from one shared latent draw V_i with ordered thresholds ($D(z) = \mathbb{1}\{V_i < p_{Dz}\}$ and $p_{D1} \geq p_{D0}$), so no unit is pushed out of treatment by the offer. Raising the *probability* alone would not be enough: with independent draws for $D(0)$ and $D(1)$, about 3% of units would be defiers and the Wald ratio would no longer equal the complier mean.

```
compliers = (D1 .== 1) .& (D0 .== 0)
late_true = mean(Y1[compliers] .- Y0[compliers])
@printf("LATE (mean of Y1-Y0 conditional on complier status) = %.3f\n", late_true)
@printf("Share of compliers: %.3f\n", mean(compliers))

# Wald estimator from the data alone
wald = (mean(Y[Z .== 1]) - mean(Y[Z .== 0])) /
       (mean(D[Z .== 1]) - mean(D[Z .== 0]))
@printf("Wald IV estimate (should match LATE): %.3f\n", wald)
```

```
LATE (mean of Y1-Y0 conditional on complier status) = 2.004
Share of compliers: 0.460
Wald IV estimate (should match LATE): 2.004
```

With heterogeneous effects, IV averages over the people moved by the instrument. In this example the Wald estimate matches the complier mean because Z changes treatment only for that group.

2.2.4. CATE — conditional average treatment effect

$$\mathrm{CATE}(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x] \tag{2.4}$$

CATE keeps X in the estimand. In this DGP, $\mathrm{CATE}(x)$ is simply $\tau(x) = 1 + 2x$.

```
# Bin X and compute mean of (Y1 - Y0) in each bin
nbins = 20
edges = range(0, 1, length=nbins + 1)
centers = (edges[1:end-1] .+ edges[2:end]) ./ 2

cate_est = [mean((Y1 .- Y0)[(X .>= edges[k]) .& (X .< edges[k+1])])
            for k in 1:nbins]

# Label by the actual bin centers (bin 5 = [0.20,0.25) has center 0.225)
@printf("CATE at x=%.3f: estimated %.2f, true %.2f\n",
        centers[5],  cate_est[5],   (centers[5]))
@printf("CATE at x=%.3f: estimated %.2f, true %.2f\n",
        centers[17], cate_est[17],  (centers[17]))
```

```
CATE at x=0.225: estimated 1.45, true 1.45
CATE at x=0.825: estimated 2.65, true 2.65
```

2.2.5. QTE — quantile treatment effect

$$\mathrm{QTE}(q) = F^{-1}_{Y(1)}(q) - F^{-1}_{Y(0)}(q) \tag{2.5}$$

(We index quantiles by q to avoid a clash with the treatment-effect function $\tau(x)$.)

QTE compares the two marginal outcome distributions. It is the difference between a quantile under treatment and the same quantile under control. It does not follow the same person across the two potential outcomes.

```
qte_grid = 0.05:0.05:0.95
qte_est  = [quantile(Y1, q) - quantile(Y0, q) for q in qte_grid]

@printf("QTE(0.10) = %.3f\n", quantile(Y1, 0.10) - quantile(Y0, 0.10))
@printf("QTE(0.50) = %.3f\n", quantile(Y1, 0.50) - quantile(Y0, 0.50))
@printf("QTE(0.90) = %.3f\n", quantile(Y1, 0.90) - quantile(Y0, 0.90))
```

```
QTE(0.10) = 1.704
QTE(0.50) = 1.992
QTE(0.90) = 2.289
```

2.3. When the estimands differ

In the first DGP the scalar estimands are close because:

- X is uniform on [0, 1] (so ATE = average of over uniform $X = 2$)
- Treatment selection is on U (unobserved type), not X (which drives), so ATT ATE
- The instrument shifts compliers uniformly across X, so LATE ATE

If treatment selection depends on the effect modifier, the numbers separate. Here I make high-X units more likely to take treatment:

```
Random.seed!(7)
n = 20_000
X2 = rand(Uniform(0, 1), n)
U2 = rand(Normal(0, 1), n)
Z2 = rand(Bernoulli(0.5), n)
# High X → much more likely to take treatment (selection on X, the modifier)
pD0_2 = @. clamp(0.10 + 0.20 * U2 + 0.6 * X2, 0, 1)
pD1_2 = @. clamp(pD0_2 + 0.50, 0, 1)
V2    = rand(n)            # shared draw: monotonicity exact, as above
D0_2  = V2 .< pD0_2
D1_2  = V2 .< pD1_2
D2    = ifelse.(Z2 .== 1, D1_2, D0_2)
Y0_2  = 0.5 .* X2 .+ 0.3 .* U2 .+ randn(n)
Y1_2  = Y0_2 .+  .(X2)
Y2    = ifelse.(D2, Y1_2, Y0_2)

ate2  = mean(Y1_2 .- Y0_2)
att2  = mean(Y1_2[D2] .- Y0_2[D2])
late2 = mean((Y1_2 .- Y0_2)[(D1_2 .== 1) .& (D0_2 .== 0)])

@printf("Selection-on-X DGP:\n")
@printf("  ATE  = %.3f\n", ate2)
@printf("  ATT  = %.3f (now higher because treated have higher X → larger  )\n", att2)
@printf("  LATE = %.3f (average  (X) among compliers)\n", late2)
```

```
Selection-on-X DGP:
  ATE  = 1.999
  ATT  = 2.126 (now higher because treated have higher X → larger  )
  LATE = 1.917 (average  (X) among compliers)
```

Now ATT is larger than ATE. The treated group has higher X, and high X means a larger treatment effect. In this case reporting ATE or ATT changes the substantive answer.

2.4. Which estimand should you choose?

The estimand should come from the research question:

Policy question	Right estimand
"What if we treated everyone?"	ATE
"What did treating the currently-treated achieve?"	ATT
"What can the instrument tell us?" (e.g. policy expansion that's already in place)	LATE
"Who benefits most?"	CATE(x)
"Does the effect vary across the outcome distribution?"	QTE(q)
"What is the distribution of individual effects?"	Often unidentifiable; bounds required

It is fine for one paper to report more than one estimand, as long as each is named correctly. What is not fine is to report the coefficient that is easiest to estimate and call it "the causal effect."

2.5. Summary

- ATE, ATT and LATE are different averages of the individual treatment effect.
- CATE(x) keeps heterogeneity by covariates. QTE(q) describes changes in the outcome distribution.
- With heterogeneous effects, IV estimates LATE. It should not be described as ATE unless extra assumptions justify that interpretation. This is the clean case with no covariates. Once we add covariates linearly to 2SLS, even LATE needs a *rich covariates* condition that is rarely defended. See Blandhol et al. (2025) and the IV chapter section "When 2SLS with Covariates Is *Actually* LATE".
- Estimator choice comes after the estimand is fixed.

3. DAGs, ADMGs, Identification, and Estimation

```
using CausalGraphs
using CausalEstimate
import CausalGraphs: identify, make_graph, to_mermaid, markov_pillow, is_fix, is_p_fix, ID_alg
import CausalEstimate: estimate, confint, pvalue
using DataFrames
using Random
using Statistics
using Logging
```

The potential-outcomes chapter asked when an observed data set can tell us about $E[Y(1) - Y(0)]$. Graphs give one way to state the assumptions behind the answer. A graph records which variables are allowed to cause other variables, and whether some observed variables may share unobserved common causes. Once the graph is stated, identification is a graph problem. Estimation comes after that: estimate the functional that the graph identified.

I use `CausalGraphs.jl` for the graph and identification steps, then `CausalEstimate.jl` for back-door/a-fixable effects, and `CausalGraphs.estimate_causal` directly for p-fixable and nested-fixable effects. The order is the important part: graph first, identified functional second, estimator third.

3.1. DAGs and ADMGs

A directed acyclic graph, or DAG, has directed arrows such as `X -> A`. In causal work, an arrow means a direct causal relation is allowed by the model. Absence of an arrow is also a substantive assumption.

An acyclic directed mixed graph, or ADMG, adds bidirected arrows such as `A <-> Y`. A bidirected edge represents an unobserved common cause. This is useful because many econometric applications have variables we cannot measure: ability, preferences, latent health, firm quality, and so on.

The graph is not an estimator. It is a compact statement of assumptions. The workflow is:

1. write down a DAG or ADMG;
2. ask whether the target effect is identified;
3. estimate the identified functional from data.

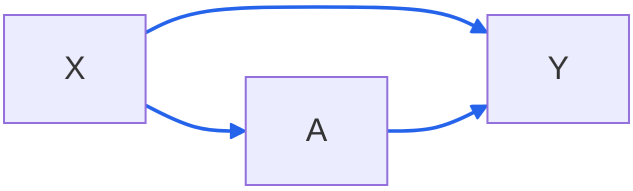
3.2. A DAG: Backdoor Adjustment

Start with a simple observational study. `A` is treatment, `Y` is the outcome, and `X` is an observed confounder:

```
g_dag = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
)
```

```
ADMG([:X, :A, :Y], [(:X, :A), (:X, :Y), (:A, :Y)], Tuple{Symbol, Symbol}[], Dict{Symbol, Vecto
```

```
println("```{mermaid}")
println(to_mermaid(g_dag; direction = "LR"))
println("```")
```



The graph says `X` affects both treatment and outcome. It also says there is no unobserved common cause between `A` and `Y` after we condition on `X`. In this case the treatment is a-fixable, which corresponds to backdoor-style identification.

```
id_dag = identify(g_dag, :A, :Y)
(strategy = id_dag.strategy,
 markov_pillow = markov_pillow(g_dag, :A; treatment = :A))
```

```
(strategy = :a_fixable, markov_pillow = [:X])
```

The Markov pillow is the adjustment set used by the estimator. Under this graph,

$$E[Y(a)] = E_X\{E(Y \mid A = a, X)\}. \tag{3.1}$$

So the graph has translated a causal query into an observed-data functional.

3.3. Estimating the DAG Effect with CausalEstimate

Here is a small simulated data set where `X` confounds the treatment-outcome association.

```
Random.seed!(2026)

n = 400
X = randn(n)
pA = logistic.(-0.2 .+ 0.8 .* X)
A = Float64.(rand(n) .< pA)
Y = 1 .+ 1.5 .* A .+ 0.7 .* X .+ 0.3 .* randn(n)

df_dag = DataFrame(X = X, A = A, Y = Y)
first(df_dag, 5)
```

	X	A	Y
	Float64	Float64	Float64
1	-0.91813	1.0	1.76741
2	0.313593	1.0	2.77638
3	-1.23885	0.0	0.130173
4	0.546413	0.0	1.43362
5	0.114729	0.0	0.906974

Because the graph is a-fixable, `GraphID` routes to AIPW backdoor adjustment. The call below estimates the average contrast $E[Y(1) - Y(0)]$.

```
dag_res = estimate(
    ATE(outcome = :Y, treatment = :A),
    GraphID(graph = g_dag),
    AIPW(crossfit = 5),
    df_dag,
)

effect_table("Backdoor ACE", dag_res)
```

	estimand	estimate	lower_95	upper_95	se
	String	Float64	Float64	Float64	Float64
1	Backdoor ACE	1.465	1.265	1.666	0.1023

The graph chose the adjustment set (Markov pillow of `A`) and the estimator used that set. Access it via `dag_res.components[:adjustment_set]`. The DGP sets the coefficient on `A` to 1.5, so the point estimate recovers the true ACE within sampling noise.

`crossfit = 5` is the package default, and the number of folds matters more than it looks. With `crossfit = 2` each nuisance is fit on only half the sample, and the extra noise in the out-of-fold predictions propagates into *both* the point estimate and the influence-function variance — fewer folds means a noisier estimate reported with a wider interval, not a cheap approximation to the same answer. Two folds is the smallest value the package allows; it is not a sensible default. The next chapter measures the size of this effect on real data.

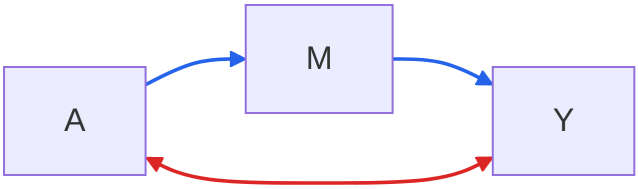
3.4. An ADMG: Front-Door Identification

Now suppose treatment and outcome share an unobserved common cause. A simple adjustment argument no longer works. The front-door ADMG has a mediator `M` that transmits the effect of `A` to `Y`, while `A` and `Y` are hidden-confounded:

```
g_fd = make_graph(
    vertices = [:A, :M, :Y],
    di_edges = [(:A, :M), (:M, :Y)],
    bi_edges = [(:A, :Y)],
)
```

```
ADMG([:A, :M, :Y], [(:A, :M), (:M, :Y)], [(:A, :Y)], Dict{Symbol, Vector{Symbol}}(), Dict{Symbo
```

```
println("```{mermaid}")
println(to_mermaid(g_fd; direction = "LR"))
println("```")
```



The bidirected edge `A <-> Y` means that ordinary backdoor adjustment is not available. But the graph is p-fixable, the ADMG generalization that includes front-door identification.

```
id_fd = identify(g_fd, :A, :Y)
(strategy = id_fd.strategy,
 a_fixable = is_fix(g_fd, :A),
 p_fixable = is_p_fix(g_fd, :A))
```

```
(strategy = :p_fixable, a_fixable = false, p_fixable = true)
```

The intuition is that `M` carries the causal effect of `A`, and `M` is not itself hidden-confounded with `A`. The graph lets us use mediator information to recover the causal effect even though `A` and `Y` are confounded.

3.5. Estimating the ADMG Effect with CausalGraphs

Simulate a front-door setting. The latent `U` is not included in the observed data; it is represented in the graph by `A <-> Y`.

```
Random.seed!(2027)

U = randn(n)
A_fd = Float64.(rand(n) .< logistic.(-0.1 .+ 0.8 .* U))
M = Float64.(rand(n) .< logistic.(-0.4 .+ 1.2 .* A_fd))
Y_fd = Float64.(rand(n) .< logistic.(-1 .+ 0.9 .* M .+ 0.7 .* U))

df_fd = DataFrame(A = A_fd, M = M, Y = Y_fd)
first(df_fd, 5)
```

	A	M	Y
	Float64	Float64	Float64
1	1.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	1.0
5	0.0	1.0	1.0

The DGP has a known truth. Under $do(A = a)$ the mediator is $M \sim \text{Bern}(\Lambda(-0.4 + 1.2a))$ while U keeps its marginal $N(0, 1)$ distribution, so

$$\tau = [\Lambda(0.8) - \Lambda(-0.4)]\ [g(-0.1) - g(-1.0)], \qquad g(c) = E_U[\Lambda(c + 0.7U)], \tag{3.2}$$

which evaluates to $0.2887 \times 0.1894 = 0.0547$. The naive contrast $E[Y \mid A = 1] - E[Y \mid A = 0]$ equals 0.158 in the population this DGP defines — almost three times the causal effect, because U raises both the chance of treatment and the outcome. That gap is what the front-door step removes. The table below reports the sample version of the naive contrast alongside the front-door estimate.

Since `identify()` returns `:p_fixable`, this is a front-door effect. `CausalEstimate.jl` currently handles backdoor (a-fixable) graphs; for p-fixable effects we call `CausalGraphs.estimate_causal` directly, which routes to NPS TMLE.

```
fd_res = with_logger(NullLogger()) do
    CausalGraphs.estimate_causal(
        a = [1.0, 0.0],
        data = df_fd,
        graph = g_fd,
        treatment = :A,
        outcome = :Y,
        n_iter = 80,
        cvg_criteria = 0.02,
        truncate_lower = 0.01,
        truncate_upper = 0.99,
    )
end

naive_fd = mean(df_fd.Y[df_fd.A .== 1]) - mean(df_fd.Y[df_fd.A .== 0])
```

```
vcat(
    DataFrame(estimand = ["Naive E[Y|A=1] - E[Y|A=0]"], estimate = [r4(naive_fd)],
              lower_95 = [missing], upper_95 = [missing], se = [missing]),
    effect_table("Front-door ACE", fd_res[:TMLE]),
    DataFrame(estimand = ["Truth"], estimate = [0.0547],
              lower_95 = [missing], upper_95 = [missing], se = [missing]);
    cols = :union,
)
```

	estimand	estimate	lower_95	upper_95	se
	String	Float64	Float64?	Float64?	Missing
1	Naive E[Y|A=1] - E[Y|A=0]	0.1014	*missing*	*missing*	*missing*
2	Front-door ACE	0.05239	0.02208	0.0827	*missing*
3	Truth	0.0547	*missing*	*missing*	*missing*

The naive contrast is far above the truth; the front-door estimate brackets it.

3.6. General ID Algorithm

Backdoor/a-fixable, p-fixable/front-door and nested-fixable are useful because they are easy to estimate. ADMGs can be more general. The Pearl-Shpitser ID algorithm asks the broader question:

```
Can P(Y | do(A)) be written using only the observed joint law P(V)?
```

The algorithm works recursively:

1. remove variables that are not ancestors of the outcome;
2. add interventions that are irrelevant after graph surgery;
3. split the remaining graph into bidirected districts;
4. identify each district as a factor of the observed law, if possible;
5. return a hedge witness when no such reduction is possible.

For the front-door graph, the general ID algorithm also succeeds:

```
id_alg_fd = ID_algorithm(g_fd, :A, :Y)
(identified = id_alg_fd.identified,
 expression = string(id_alg_fd.expression))
```

```
(identified = true, expression = "sum_{M} P(M | A) * sum_{A} P(A) * P(Y | A, M)")
```

The printed expression reuses `A` as the inner summation index; read it as $\sum_M P(M \mid A) \sum_{A'} P(Y \mid A', M) P(A')$, where the inner A' ranges over the marginal distribution of treatment and is distinct from the A being intervened on. The expression is symbolic. It is not yet an estimate. For finite-support variables, `CausalGraphs.estimate_causal` evaluates the symbolic functional by empirical probabilities and attaches a finite-support EIF standard error via the `:IDPlugin` key.

```
id_res = with_logger(NullLogger()) do
    estimate_id(
        a = [1.0, 0.0],
        data = df_fd,
        graph = g_fd,
        treatment = :A,
        outcome = :Y,
    )
end

effect_table("ID plug-in ACE", id_res[:IDPlugin];
             se = id_res[:IDPlugin].standard_error)
```

	estimand	estimate	lower_95	upper_95	se
	String	Float64	Float64	Float64	Float64
1	ID plug-in ACE	0.05346	0.02278	0.08413	0.01565

This plug-in estimator is useful for discrete examples and diagnostics. It is not a universal TMLE for every continuous ID functional.

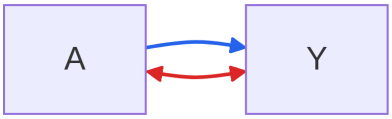
3.7. When the Graph Says No

The bow graph has both a direct causal edge and unobserved confounding between the same two variables:

```
g_bow = make_graph(
    vertices = [:A, :Y],
    di_edges = [(:A, :Y)],
    bi_edges = [(:A, :Y)],
)
```

```
ADMG([:A, :Y], [(:A, :Y)], [(:A, :Y)], Dict{Symbol, Vector{Symbol}}(), Dict{Symbol, Bool}(:A =>
```

```
println("```{mermaid}")
println(to_mermaid(g_bow; direction = "LR"))
println("```")
```



The effect of `A` on `Y` is not identified in this ADMG. The observed association mixes the direct causal effect with the latent common cause.

```
id_bow = identify(g_bow, :A, :Y)
id_alg_bow = ID_algorithm(g_bow, :A, :Y)

(route = id_bow.strategy,
 id_algorithm_identified = id_alg_bow.identified,
 hedge = id_alg_bow.hedge)
```

```
(route = :not_identified, id_algorithm_identified = false, hedge = (S = [:Y], F = [:A, :Y], tr
```

3.8. Practical Workflow

For applied work, a useful workflow is:

1. write the graph before looking at estimates;
2. use bidirected edges for latent common causes;
3. run `identify(graph, treatment, outcome)`;
4. if identified and a-fixable, call `estimate(ATE(...), GraphID(graph=...), AIPW(...), df)`; for p-fixable or nested-fixable effects, call `CausalGraphs.estimate_causal(...)`;
5. report the graph and the identification route along with the estimate.

4. Applied Graph Workflow: Smoking Cessation

```
using CausalGraphs
using CausalEstimate
import CausalGraphs: identify, make_graph, to_mermaid, markov_pillow
import CausalEstimate: estimate, confint, pvalue
using DataFrames
using DelimitedFiles
using Downloads
using Logging
using Random
```

The previous chapter used small graph examples. Here I apply the same graph-identification-estimation workflow to a real data set: the complete-case National Health and Nutrition Examination Survey I Epidemiologic Follow-up Study (NHEFS) data distributed through `causaldata` and mirrored by RDatasets.

The question is:

```
What is the effect of quitting smoking on weight change from 1971 to 1982?
```

The treatment is `qsmk`, an indicator for quitting smoking between baseline and follow-up. The outcome is `wt82_71`, weight change over the follow-up period. This is observational data. The effect is not identified by the data alone. It is identified only after we state the adjustment assumptions in the graph.

4.1. Data

For this example, keep the treatment, outcome, and baseline covariates that are commonly used in NHEFS smoking-cessation examples.

```
function read_rdatasets_csv(url, cols)
    local_file = Downloads.download(url; timeout = 120)
    x, header = readdlm(local_file, ',', Any, '\n'; header = true)
    raw = DataFrame(x, Symbol.(vec(header)))
    DataFrame((c => Float64.(raw[!, c]) for c in cols)...)
end

nhefs_url = "https://vincentarelbundock.github.io/Rdatasets/csv/causaldata/nhefs_complete.csv"
```

```
nhefs_cols = [
    :qsmk, :wt82_71,
    :sex, :race, :age, :school,
    :smokeintensity, :smokeyrs,
    :exercise, :active, :wt71,
]

nhefs = read_rdatasets_csv(nhefs_url, nhefs_cols);

(n = nrow(nhefs), variables = ncol(nhefs))
```

```
(n = 1566, variables = 11)
```

```
preview_table(nhefs)
```

	qsmk	wt82_71	sex	race	age	school	smokeintensity	smokeyrs	
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	
1	0.0	-10.09	0.0	1.0	42.0	7.0	30.0	29.0	...
2	0.0	2.605	0.0	0.0	36.0	9.0	20.0	24.0	...
3	0.0	9.414	1.0	1.0	56.0	11.0	20.0	26.0	...
4	0.0	4.99	0.0	1.0	68.0	5.0	3.0	53.0	...
5	0.0	4.989	0.0	0.0	40.0	11.0	20.0	19.0	...

4.2. Adjustment DAG

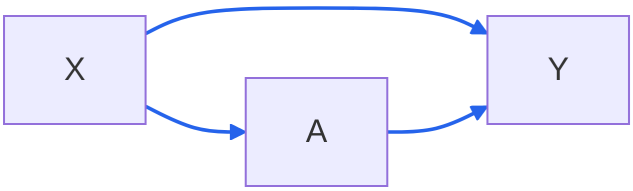
Start with the grouped graph

```
X -> A -> Y
X -----> Y
```

where `X` is the baseline covariate set, `A` is quitting smoking, and `Y` is later weight change. This graph says the baseline variables may affect both quitting and later weight change, and that there is no unmeasured common cause of quitting and later weight change after conditioning on those baseline variables.

```
conceptual_graph = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
);
```

```
println("```{mermaid}")
println(to_mermaid(conceptual_graph; direction = "LR"))
println("```")
```



For estimation, expand `X` into the observed NHEFS columns. The expanded graph is still an adjustment DAG: each baseline variable is allowed to point into quitting and into later weight change.

```
nhefs_vertices = [
    :sex, :race, :age, :school,
    :smokeintensity, :smokeyrs,
    :exercise, :active, :wt71,
    :qsmk, :wt82_71,
]

baseline = setdiff(nhefs_vertices, [:qsmk, :wt82_71])

nhefs_edges = vcat(
    [(w, :qsmk) for w in baseline],
    [(w, :wt82_71) for w in baseline],
    [(:qsmk, :wt82_71)],
)

nhefs_graph = make_graph(vertices = nhefs_vertices, di_edges = nhefs_edges);

(vertices = length(nhefs_vertices), directed_edges = length(nhefs_edges))
```

```
(vertices = 11, directed_edges = 19)
```

4.3. Identification

`CausalGraphs.identify` checks the graph before estimation. In this adjustment DAG, the quitting node is a-fixable. The Markov pillow is the sufficient adjustment set:

```
nhefs_id = identify(nhefs_graph, :qsmk, :wt82_71)

DataFrame(
    strategy = [string(nhefs_id.strategy)],
    markov_pillow = [join(string.(markov_pillow(nhefs_graph, :qsmk; treatment = :qsmk)), ", ")]
)
```

	strategy	markov_pillow
	String	String
1	a_fixable	sex, race, age, school, smokeintensity, smokeyrs, exercise, active, wt71

The graph therefore identifies the causal mean by the adjustment formula

$$E[Y(a)] = E_X\{E(Y \mid A = a, X)\}. \tag{4.1}$$

This formula is not a modeling claim by itself. It is the observed-data functional implied by the graph. Estimation still requires nuisance models for the outcome regression and treatment mechanism.

4.4. Estimation with CausalEstimate

The book keeps identification and estimation as separate steps. Here `CausalGraphs` supplies the graph result and `CausalEstimate` estimates the identified functional. Because the graph is a-fixable, `GraphID` routes to the AIPW average treatment effect machinery.

```
Random.seed!(2026)

nhefs_result = estimate(
    ATE(outcome = :wt82_71, treatment = :qsmk),
    GraphID(graph = nhefs_graph),
    AIPW(crossfit = 5),
    nhefs,
)

effect_table("Quit smoking vs continued smoking", nhefs_result)
```

	estimand	estimate	lower_95	upper_95	se
	String	Float64	Float64	Float64	Float64
1	Quit smoking vs continued smoking	3.452	1.986	4.918	0.748

Under the DAG, the estimate is the average effect of quitting smoking versus continuing smoking on weight change in the complete-case NHEFS population. The positive estimate is consistent with weight gain after quitting. The causal interpretation depends on the graph, especially the assumption that the measured baseline variables block the relevant backdoor paths.

Two settings deserve comment, because both move the number.

Folds. `crossfit = 5` is the package default and this chapter's seed makes the comparison reproducible. Dropping to `crossfit = 2` gives 3.866 with a standard error of 1.331 — a 95% interval of $[1.26, 6.48]$, more than five kilograms wide. Five folds gives 3.452 (0.748) and ten gives 3.312 (0.737). Two folds fits each nuisance on 783 observations, and the resulting noise in the out-of-fold predictions inflates the point estimate *and* the influence-function variance at the same time. Use the default, or more.

Nuisance estimation. The companion R chapter fits the same adjustment set by full-sample parametric nuisances (a linear outcome model interacted with treatment, a logistic propensity) and reports 3.293 with a standard error of 0.498. The propensity scores in this data set stay inside $[0.05, 0.75]$, so there is no overlap problem to explain the gap: the wider interval here is the variance price of cross-fitting flexible learners rather than trusting a correctly specified parametric model.

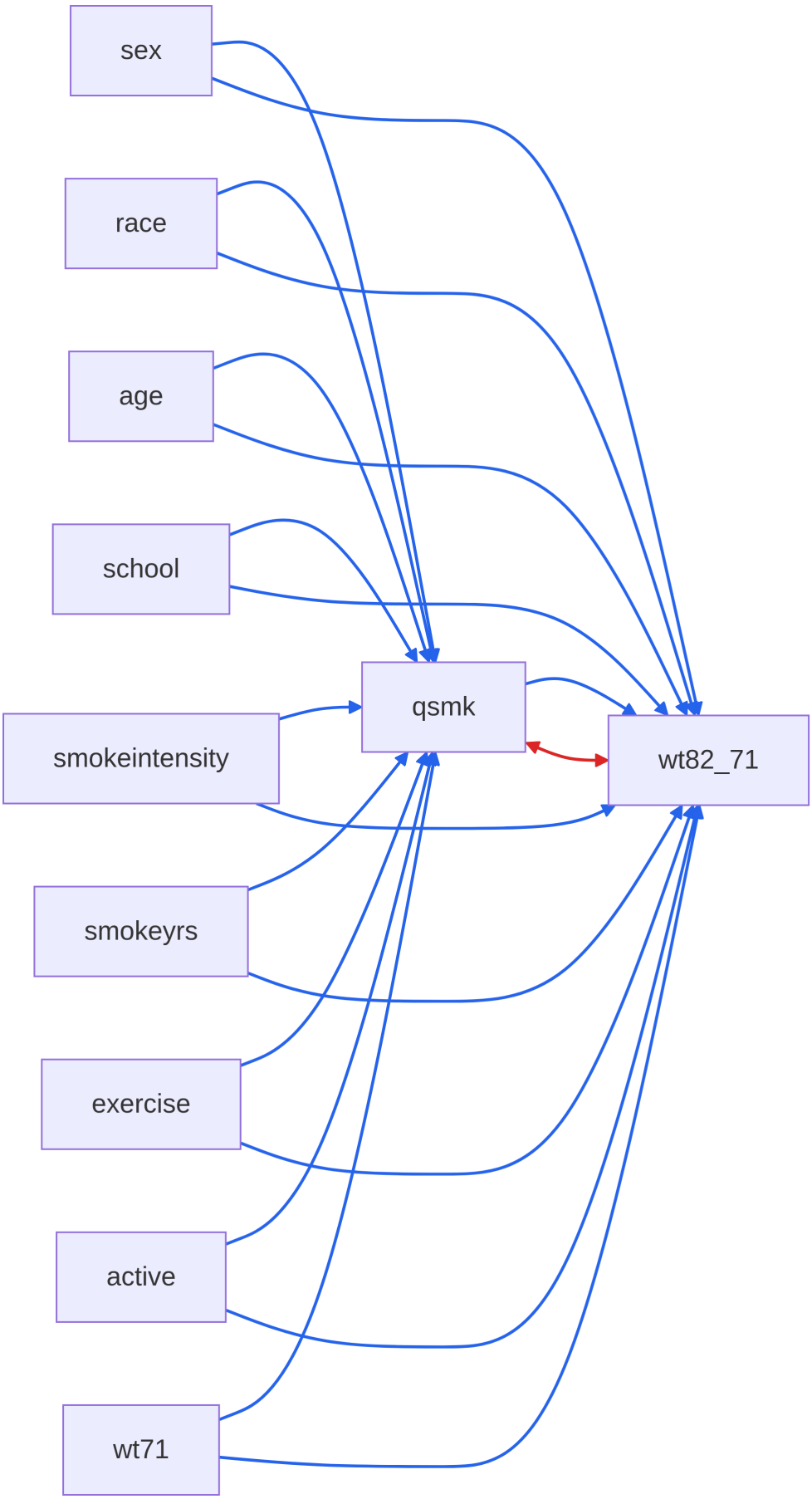
When the parametric model is right, it wins; the cross-fit estimator buys insurance against its being wrong.

4.5. Sensitivity Graph

Now encode a different assumption: quitting and later weight change share an unobserved common cause even after conditioning on the measured baseline variables. In an ADMG this is a bidirected edge, `qsmk <-> wt82_71`.

```
sensitivity_graph = make_graph(
    vertices = nhefs_vertices,
    di_edges = nhefs_edges,
    bi_edges = [(:qsmk, :wt82_71)],
)

sensitivity_id = identify(sensitivity_graph, :qsmk, :wt82_71);
```

```
println("```{mermaid}")
println(to_mermaid(sensitivity_graph; direction = "LR"))
println("```")
```


sex
race
age
school
qsmk
smokeintensity
wt82_71
smokeyrs
exercise
active
wt71

```
DataFrame(strategy = [string(sensitivity_id.strategy)])
```

	strategy
	String
1	not_identified

The identifying conclusion changes. Under this graph, the observed NHEFS variables are not enough to identify the effect. This is why the graph has to come before the estimator. The same data can imply different causal answers under different assumptions.

4.6. Structured Baseline Graph

The simple adjustment graph is often the clearest starting point. A more detailed DAG can record baseline ordering: demographics may precede education, smoking history, activity, and baseline weight; those variables may then affect quitting and later weight change.

```
structured_edges = [
    (:age, :school), (:age, :smokeyrs), (:age, :wt71),
    (:age, :active), (:age, :exercise), (:age, :qsmk), (:age, :wt82_71),

    (:sex, :smokeintensity), (:sex, :smokeyrs), (:sex, :wt71),
    (:sex, :qsmk), (:sex, :wt82_71),

    (:race, :school), (:race, :smokeintensity),
    (:race, :qsmk), (:race, :wt82_71),

    (:school, :smokeintensity), (:school, :qsmk), (:school, :wt82_71),

    (:smokeyrs, :smokeintensity), (:smokeyrs, :qsmk), (:smokeyrs, :wt82_71),
    (:smokeintensity, :qsmk), (:smokeintensity, :wt82_71),

    (:exercise, :wt71), (:exercise, :qsmk), (:exercise, :wt82_71),
    (:active, :wt71), (:active, :qsmk), (:active, :wt82_71),
    (:wt71, :qsmk), (:wt71, :wt82_71),

    (:qsmk, :wt82_71),
]

structured_graph = make_graph(vertices = nhefs_vertices, di_edges = structured_edges);
structured_id = identify(structured_graph, :qsmk, :wt82_71)

DataFrame(
    strategy = [string(structured_id.strategy)],
    same_adjustment_set = [
        Set(markov_pillow(structured_graph, :qsmk; treatment = :qsmk)) == Set(baseline)
```

```
    ],
)
```

	strategy	same_adjustment_set
	String	Bool
1	a_fixable	1

The check returns `true`, and the reason is worth spelling out, because it is not an accident of this data set. Every arrow we added runs between baseline variables, and every baseline variable is pre-treatment and in the adjustment set. A back-door path from `qsmk` to `wt82_71` has to leave `qsmk` through an incoming arrow, travel among the baseline variables, and then enter `wt82_71` from one of them. That last baseline variable has an arrow pointing into the outcome, so on this path it is a non-collider, and we are conditioning on it. The path is blocked whatever the arrows upstream of it look like. The measured parents of `qsmk` are therefore still the full baseline, and the identifying functional is unchanged.

That does not make the detail useless. It records what we believe about the baseline ordering, which a reader may want to see. But each arrow is a causal assumption that can be wrong, and here none of them buys anything for this estimand. The grouped adjustment graph asserts less and identifies the same thing.

5. Sensitivity Analysis

```
using DataFrames
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
```

The previous chapters often treat identification as yes or no. Applied work is usually less clean. We may believe the adjustment set is good, but still worry about some remaining confounding. Sensitivity analysis asks a practical question: how strong would unmeasured confounding have to be to change the conclusion?

> **Related reading**: An extended R-based treatment of the Cinelli-Hazlett robustness value with `sensemakr` and benchmark interpretations appears in the companion blog chapter on sensitivity analysis of *Topics on Econometrics and Causal Inference*.

Here I use three tools:

1. **Cinelli-Hazlett (2020) omitted-variable bias bounds** — robustness values and benchmark comparisons for OLS regression estimates.
2. **E-values (VanderWeele-Ding 2017)** — for risk-ratio estimates, the minimum strength an unmeasured confounder would need.
3. **Rosenbaum bounds** — the critical odds ratio of hidden bias that would render a matched-sample test insignificant.

There is no standard Julia package for these, but the formulas are short.

5.1. A simulated example

Use a simulation so the truth is known:

```
Random.seed!(2026)
n   = 2000
X1  = randn(n)
X2  = randn(n)
U   = randn(n)              # unmeasured confounder
```

```
D   = @. Float64(rand() < 1 / (1 + exp(-(0.5 * X1 + 0.5 * X2 + 0.6 * U))))
Y   = @. 1.0 * D + 1.5 * X1 + 1.5 * X2 + 1.0 * U + 0.5 * randn()

df  = DataFrame(Y = Y, D = D, X1 = X1, X2 = X2, U = U)

# Pull the coefficient out BY NAME. Indexing positionally (`[end]`, `[2]`) is a
# standing invitation to report the wrong number: `[end]` on `Y ~ D + X1 + X2`
# is the coefficient on X2, not on D, and in this DGP X2's true coefficient
# (1.5) is close enough to the confounded D estimate that the mistake does not
# look like one.
coef_of(m, name) = coef(m)[findfirst(==(name), coefnames(m))]

@printf("True treatment effect: 1.000\n")
@printf("Naive (D only):        %.3f\n",
        coef_of(lm(@formula(Y ~ D), df), "D"))
@printf("Adjusted (D, X1, X2):  %.3f  (biased because U is unobserved)\n",
        coef_of(lm(@formula(Y ~ D + X1 + X2), df), "D"))
@printf("Oracle (D, X1, X2, U): %.3f\n",
        coef_of(lm(@formula(Y ~ D + X1 + X2 + U), df), "D"))
```

```
True treatment effect: 1.000
Naive (D only):        2.787
Adjusted (D, X1, X2):  1.497  (biased because U is unobserved)
Oracle (D, X1, X2, U): 0.966
```

The adjusted regression is still biased because U is unobserved.

5.2. Cinelli-Hazlett robustness value

The robustness value is the minimum partial R^2 an unobserved confounder would need with both treatment and outcome to bring the estimate to zero.

The closed-form expression depends only on the t-statistic of the treatment coefficient and the regression's degrees of freedom:

$$RV_q = \tfrac{1}{2}\left[\sqrt{f_q^4 + 4f_q^2} - f_q^2\right], \qquad f_q = q\,\frac{|t|}{\sqrt{df}}, \tag{5.1}$$

where $|t|$ is the absolute t-stat of the treatment coefficient, df is the residual degrees of freedom, and q is the fraction of the estimate we want explained away ($q = 1$ for full explanation).

```
"""
    robustness_value(t_stat, df; q=1.0)
Minimum partial R² with both treatment and outcome that an unmeasured
confounder would need to fully explain away a fraction `q` of the estimate.
```

```
Cinelli & Hazlett (2020).
"""
function robustness_value(t_stat::Real, df::Real; q::Real=1.0)
    fq = q * abs(t_stat) / sqrt(df)
    return 0.5 * (sqrt(fq^4 + 4 * fq^2) - fq^2)
end

# Fit the adjusted regression and extract t-stat + dof
fit_adj   = lm(@formula(Y ~ D + X1 + X2), df)
t_adj     = coef(fit_adj)[2] / stderror(fit_adj)[2]
df_resid  = dof_residual(fit_adj)

rv  = robustness_value(t_adj, df_resid; q=1.0)
rv5 = robustness_value(t_adj, df_resid; q=1.0 - 1.96/abs(t_adj))   # q to reach significance

@printf("t-statistic on D:                %.2f\n", t_adj)
@printf("Residual df:                     %.0f\n", df_resid)
@printf("Robustness value (q=1):          %.4f  (= %.2f%% partial R²)\n",
        rv, 100rv)
@printf("RV to reach insignificance:      %.4f  (= %.2f%% partial R²)\n",
        rv5, 100rv5)
```

```
t-statistic on D:                29.24
Residual df:                     1996
Robustness value (q=1):          0.4745  (= 47.45% partial R²)
RV to reach insignificance:      0.4521  (= 45.21% partial R²)
```

The useful interpretation is comparative: compare the RV to the partial R^2 values of observed covariates.

5.2.1. Benchmarking against observed covariates

How strong are the observed covariates as a reference? Compute their partial R^2 contributions:

```
# Partial R² of a covariate: variance explained over a model that omits it
function partial_r2(full_fit, reduced_fit)
    ss_full     = sum(abs2.(residuals(full_fit)))
    ss_reduced  = sum(abs2.(residuals(reduced_fit)))
    1 - ss_full / ss_reduced
end

# Partial R² of X1 with the outcome (omit X1)
fit_noX1_Y = lm(@formula(Y ~ D + X2), df)
pR2_X1_Y   = partial_r2(fit_adj, fit_noX1_Y)

# Partial R² of X1 with the treatment (a separate regression)
```

```
fit_D_full = lm(@formula(D ~ X1 + X2), df)
fit_D_noX1 = lm(@formula(D ~ X2), df)
pR2_X1_D   = partial_r2(fit_D_full, fit_D_noX1)

@printf("Benchmark - X1:\n")
@printf("  Partial R² with Y: %.4f\n", pR2_X1_Y)
@printf("  Partial R² with D: %.4f\n", pR2_X1_D)
@printf("\nRobustness value:    %.4f\n", rv)
@printf("\nIf an unobserved confounder is as strong as X1 with BOTH Y and D,\n")
@printf("would it explain the estimate? %s\n",
        min(pR2_X1_Y, pR2_X1_D) > rv ? "YES - fragile result" : "NO - robust to this benchmark")
```

```
Benchmark - X1:
  Partial R² with Y: 0.6209
  Partial R² with D: 0.0489

Robustness value:    0.4745

If an unobserved confounder is as strong as X1 with BOTH Y and D,
would it explain the estimate? NO - robust to this benchmark
```

The benchmark comparison turns the RV into a sentence: an unmeasured confounder as strong as X_1 would or would not overturn the result.

Because this is a simulation we can check the machinery against the confounder that actually exists. U's partial R^2 is 0.787 with the outcome and 0.064 with the treatment. The second number is far below the robustness value, so the verdict "not explained away" is correct — and it is correct even though the adjusted estimate is badly biased. Those are different questions, and the distinction is the main thing to take from this section: a robustness value asks whether a confounder could drive the estimate to *zero*, not whether the estimate is *right*. Here it cannot, and the estimate is still 50% too large.

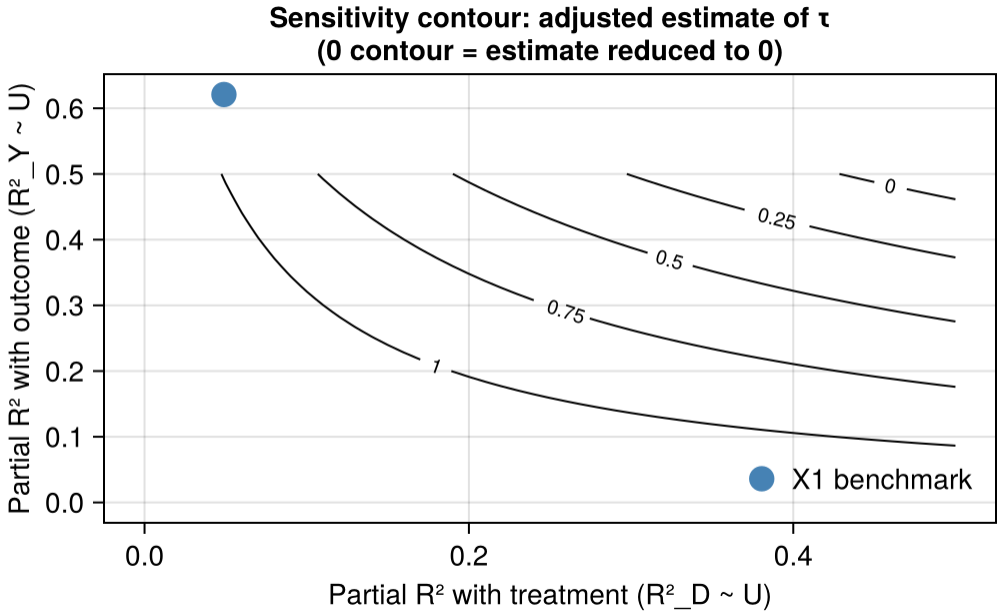
Feeding U's true strengths into the Cinelli-Hazlett bias formula gives a bound of 0.531, so the bias-corrected estimate is $1.497 - 0.531 = 0.966$ against a true effect of 1.000. The bound is doing its job: it is not a point correction one would apply in practice, since in real data the confounder's strengths are exactly what we do not know, but it confirms that the formula is calibrated rather than merely suggestive.

5.2.2. A sensitivity contour plot

The contour plot below shows, for each $(R^2_{D\sim U}, R^2_{Y\sim U})$ pair, the implied estimate of the treatment effect under that strength of unobserved confounding:

```
# Bias formula (Cinelli-Hazlett 2020, eqn 4):
# bias = se(^) * sqrt(df) * sqrt( R²_Y * R²_D / (1 - R²_D) )
# adjusted_estimate = ^ - bias (or + bias, the worst case is the larger absolute)
```

```
tau_hat = coef(fit_adj)[2]
se_hat  = stderror(fit_adj)[2]

grid = 0.0:0.01:0.5
bias_matrix = [se_hat * sqrt(df_resid) * sqrt(r2y * r2d / max(1 - r2d, 1e-8))
               for r2y in grid, r2d in grid]
adjusted    = tau_hat .- bias_matrix    # worst-case (downward) bias

fig = Figure(size = (700, 500))
ax  = Axis(fig[1, 1],
           xlabel = "Partial R² with treatment (R²_D ~ U)",
           ylabel = "Partial R² with outcome (R²_Y ~ U)",
           title  = "Sensitivity contour: adjusted estimate of  \n" *
                    "(0 contour = estimate reduced to 0)")
contour!(ax, grid, grid, adjusted, levels = [0.0, 0.25, 0.5, 0.75, 1.0],
         labels = true, color = :black)
# Mark the X1 benchmark
scatter!(ax, [pR2_X1_D], [pR2_X1_Y], color = :steelblue, markersize = 18,
         label = "X1 benchmark")
axislegend(ax, position = :rb, framevisible = false)
fig
```



A confounder above and to the right of the zero contour would reduce the estimate to zero. The benchmark dot shows the strength of an observed covariate for comparison.

5.3. E-values

For risk-ratio estimates, VanderWeele and Ding (2017) propose a simpler-to-interpret sensitivity statistic. For an observed risk ratio $RR > 1$, the **E-value** is the minimum strength of association (on the risk-ratio scale) that an unobserved confounder would need to have with both treatment and outcome to fully explain the observed association:

$$\text{E-value} = RR + \sqrt{RR(RR-1)}. \tag{5.2}$$

For $RR < 1$, use $1/RR$ first and apply the formula.

```
"""
    evalue(rr; lo=missing, hi=missing)
Compute the E-value for a risk ratio and (optionally) its 95% confidence
interval. Returns (E_point, E_ci) where E_ci is the E-value needed to
shift the lower CI bound past 1 (or upper past 1 if rr < 1).
"""
function evalue(rr::Real; lo=missing, hi=missing)
    e_point = rr >= 1 ? rr + sqrt(rr * (rr - 1)) : begin
        rr_inv = 1 / rr
        rr_inv + sqrt(rr_inv * (rr_inv - 1))
    end
    e_ci = missing
    if !ismissing(lo) && !ismissing(hi)
        if rr >= 1 && lo > 1
            e_ci = lo + sqrt(lo * (lo - 1))
        elseif rr < 1 && hi < 1
            hi_inv = 1 / hi
            e_ci = hi_inv + sqrt(hi_inv * (hi_inv - 1))
        else
            e_ci = 1.0   # CI crosses 1, no E-value needed
        end
    end
    return (e_point = e_point, e_ci = e_ci)
end

# Example: a study reports RR = 1.40 [1.20, 1.62]
ev = evalue(1.40; lo = 1.20, hi = 1.62)
@printf("RR = 1.40 [1.20, 1.62]\n")
@printf("E-value (point): %.2f\n", ev.e_point)
@printf("E-value (CI):    %.2f\n", ev.e_ci)
```

```
RR = 1.40 [1.20, 1.62]
E-value (point): 2.15
E-value (CI):    1.69
```

The interpretation is on the risk-ratio scale: a hidden confounder would need associations of this size with both treatment and outcome to explain away the result.

5.4. Rosenbaum bounds for matched studies

For matched estimators (propensity score matching, full matching, optimal matching), **Rosenbaum bounds** (Rosenbaum 2002) parametrise hidden bias as Γ, the maximum ratio of treatment odds between the two units of a matched pair due to unobservables. $\Gamma = 1$ corresponds to within-pair randomisation; $\Gamma = 2$ means one unit's odds of receiving treatment can be up to twice the other's.

For each value of Γ, the upper-bound p-value of the Wilcoxon signed-rank test on matched-pair outcome differences is:

```
# Simulate matched-pair differences from a known DGP
Random.seed!(7)
n_pairs = 500
true_effect = 0.5
diffs = true_effect .+ 0.8 .* randn(n_pairs)

using Distributions: Normal, cdf

"""
    rosenbaum_pvalue(diffs, gamma)
Upper-bound one-sided p-value of the Wilcoxon signed-rank test on matched
pair differences, allowing for hidden bias of magnitude `gamma`  1.
"""
function rosenbaum_pvalue(diffs, gamma)
    abs_d = abs.(diffs)
    sgn   = sign.(diffs)
    rks   = sortperm(sortperm(abs_d))   # ranks of |d|
    # Under Γ, prob of positive sign is bounded by Γ/(1+Γ)
    p_pos = gamma / (1 + gamma)
    T_obs = sum(rks[sgn .> 0])
    mu    = p_pos * sum(rks)
    v     = p_pos * (1 - p_pos) * sum(rks .^ 2)
    z     = (T_obs - mu) / sqrt(v)
    return 1 - cdf(Normal(), z)
end

gammas = 1.0:0.25:5.0
pvals  = [rosenbaum_pvalue(diffs, g) for g in gammas]
result = DataFrame(Gamma = gammas, p_upper = pvals)

# Critical Γ: smallest Γ where p exceeds 0.05
critical_idx = findfirst(p -> p > 0.05, pvals)
critical_gamma = critical_idx === nothing ? Inf : gammas[critical_idx]
```

```
@printf("Critical Γ (p > 0.05): %.2f\n", critical_gamma)
@printf("\n%-10s %s\n", "Γ", "p (upper bound)")
for (g, p) in zip(gammas, pvals)
    @printf("%-10.2f %.4f\n", g, p)
end
```

```
Critical Γ (p > 0.05): 3.00

Γ          p (upper bound)
1.00       0.0000
1.25       0.0000
1.50       0.0000
1.75       0.0000
2.00       0.0000
2.25       0.0001
2.50       0.0013
2.75       0.0121
3.00       0.0599
3.25       0.1799
3.50       0.3725
3.75       0.5887
4.00       0.7698
4.25       0.8891
4.50       0.9534
4.75       0.9827
5.00       0.9942
```

If the critical Γ is close to 1, small hidden bias could overturn the matched-pair result. (The reported value is the first point *on the 0.25 grid* at which significance is lost; the actual crossing lies between it and the preceding grid point.)

5.5. Reporting practice

Most observational causal estimates should have some sensitivity analysis. A minimum report is:

Estimator family	What to report
Linear regression	Cinelli-Hazlett RV + benchmark comparison
Risk-ratio analysis	E-value (point + CI)
Matched / pair-based test	Critical Γ from Rosenbaum bounds

When the result is fragile, the next step is either to improve measurement and adjustment or to report partial-identification bounds.

5.6. Summary

- Sensitivity analysis asks how strong hidden confounding must be to change the result.
- Robustness values are useful for regression estimates.
- E-values are useful for risk ratios.
- Rosenbaum bounds are useful for matched pairs.
- The basic formulas are easy to implement directly in Julia.

Part II.

Estimation

6. Estimation: RA, IPW, AIPW and IPWRA

```
using CausalEstimate
using Panelest
using MLJ
using GLMNet
using DecisionTree
using Vcov
using ReadStatTables
using DataFrames
using Statistics
using GLM
using Downloads
using Random
using Printf
using RegressionTables
```

Once the causal effect is identified, we need an estimator. This chapter covers four: regression adjustment (RA), inverse probability weighting (IPW), augmented IPW (AIPW), and inverse-probability-weighted regression adjustment (IPWRA).

6.1. Experimental Data

With experimental data the assumptions hold by design. A difference-in-means estimator suffices (under randomization ATE = ATT = ATU):

$$\hat{\tau} = \bar{Y}_1 - \bar{Y}_0 \tag{6.1}$$

A t-test or a regression of Y on W and X both recover the same estimate.

6.2. Adjustment Formula

The adjustment formula connects causal and statistical quantities:

$$\begin{aligned} E[Y(1) - Y(0)] &= E[Y(1)] - E[Y(0)] \\ &= E_X[E[Y(1)|X]] - E_X[E[Y(0)|X]] \\ &= E_X[E[Y(1)|W=1,X]] - E_X[E[Y(0)|W=0,X]] \\ &= E_X[E[Y|W=1,X]] - E_X[E[Y|W=0,X]] \end{aligned} \tag{6.2}$$

The steps are: linearity of expectation, iterated expectations, unconfoundedness plus overlap, and consistency.

6.3. Regression Adjustment

We estimate $E[Y \mid W = w, X = x]$ by regression.

We define a conditional mean function:

$$\begin{aligned} \mu_0(X) &\equiv E[Y|W = 0, X] \\ &= E[Y(0)|W = 0, X] \\ &= E[Y(0)|W = 1, X] \end{aligned} \tag{6.3}$$

The RA estimators are:

$$\hat{\tau}_{ATT} = \frac{1}{n_1} \sum_{i:W_i=1} \{Y_i - \hat{\mu}_0(X_i)\} \tag{6.4}$$

$$\hat{\tau}_{ATE} = \frac{1}{n} \sum_{i=1}^{n} \{\hat{\mu}_1(X_i) - \hat{\mu}_0(X_i)\} \tag{6.5}$$

where $\hat{\mu}_0(X_i)$ is the predicted value of Y_i from a regression of Y on X for the control group.

Target:

$$\begin{aligned} \tau_{ATT} &= E[Y|W = 1] - E[Y(0)|W = 1] \\ &= E_X\{E[Y|W = 1, X] - \mu_0(X) \mid W = 1\} \\ &= E_X\{E[Y|W = 1, X] \mid W = 1\} - (\alpha_0 + E(X \mid W = 1)\beta_0) \end{aligned} \tag{6.6}$$

The last term is a linear form of $\mu_0(X)$. We can specify other forms, but the idea is to model it with some functional form. For parametric forms, we need to make sure extrapolation does not go out of control.

6.4. Linear RA

Regression Adjustment is basically an imputation estimator. While we observe $E[Y|W = 1, X]$, we model $E[Y(0)|W = 1]$, based on unconfoundedness and a functional form (say linear form). We estimate β_0 on the control sample, then get the expected values for the treated sample, for each value of X.

Implementation of linear RA is easy. We regress Y on X and W and their interaction. It's shown that we need to de-mean X to get the effect correct.

```julia
using CSV, DataFrames, Statistics, GLM

# Load the pension dataset from the hdm R package (Chernozhukov et al.)
# Treatment: p401 (eligibility for 401k), Outcome: net_tfa (net total financial assets)
pension = CSV.read("data/pension.csv", DataFrame)

# Calculate means for de-meaning (treated group means)
mean_inc_treated = mean(pension[pension.p401 .== 1, :inc])
mean_age_treated = mean(pension[pension.p401 .== 1, :age])

# For ATT: we de-mean X by deducting the mean of X in the treated group
# For ATE: we de-mean X by deducting the mean of X in the whole sample
data = DataFrames.transform(pension,
    :inc => (x -> x .- mean_inc_treated) => :inc_dm,
    :age => (x -> x .- mean_age_treated) => :age_dm
)

lm_ra = lm(@formula(net_tfa ~ p401 * (inc_dm + age_dm)), data)
show_regression_html(lm_ra)
```

	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	2.410e+04	831.154	28.998	< 2e-16 ***
p401	1.416e+04	1397.304	10.134	< 2e-16 ***
inc_dm	0.772	0.030	25.670	< 2e-16 ***
age_dm	797.675	63.503	12.561	< 2e-16 ***
p401 & inc_dm	0.330	0.051	6.429	1.342e-10 ***
p401 & age_dm	815.335	133.260	6.118	9.810e-10 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

After demeaning the covariates, we include all interactions of demeaned covariates with treatment. Because the covariates are demeaned at *treated-group* means, the coefficient on W is the ATT (demeaning at whole-sample means would give the ATE instead).

6.4.1. Questions on RA

1. Why do we have to do interaction of W and X? What if we don't do it?
2. What if we don't de-mean X?

Answers:

1. It has been shown (Słoczyński 2022) that with heterogeneous treatment effect, and unequal divide between treated and control, we need to do the interaction term. Otherwise, we'll get biased estimates, for τ_{ATT} or τ_{ATE}. The intuition is that we need to model the difference between treated and control, and the difference is not constant across X.

2. If we don't de-mean X, then the coefficient on W is not the average treatment effect. We can still retrieve the ATT or ATE by calculating the marginal effect of W though.

6.5. IPW

Under unconfoundedness and overlap,

$$\begin{aligned} E[Y(w)] &= E[E[Y|W=w,X]] \\ &= \sum_x (\sum_y yP(y|w,x))P(x) \\ &= \sum_x \sum_y yP(y|w,x)P(x)\frac{P(w|x)}{P(w|x)} \\ &= \sum_x \sum_y yP(y,w,x)\frac{1}{P(w|x)} \\ &= \sum_x E[\mathbb{1}(W=w,X=x)Y]\frac{1}{P(w|x)} \\ &= E[\frac{\mathbb{1}(W=w)Y}{P(w|X)}] \end{aligned} \tag{6.7}$$

The IPW equation means that the expected value of the potential outcome is the weighted average of the observed outcome, where the weight is the inverse of the propensity score $P(w|x)$. $\mathbb{1}(W=w)$ is an indicator function, which is 1 if $W=w$ and 0 otherwise.

$$\begin{aligned} \tau_{ATE} &= E[Y(1)] - E[Y(0)] \\ &= E[\frac{WY}{\pi(X)}] - E[\frac{(1-W)Y}{1-\pi(X)}] \end{aligned} \tag{6.8}$$

The sample estimators are:

$$\hat{\tau}_{ATE,IPW} = \frac{1}{N}\sum_i \frac{W_i Y_i}{\hat{\pi}(X_i)} - \frac{1}{N}\sum_i \frac{(1-W_i)Y_i}{1-\hat{\pi}(X_i)} \tag{6.9}$$

$$\hat{\tau}_{ATT,IPW} = \bar{Y}_1 - \frac{1}{N_1}\sum_i \frac{\hat{\pi}(X_i)(1-W_i)Y_i}{1-\hat{\pi}(X_i)} \tag{6.10}$$

6.5.1. IPW Intuition

IPW removes confounding X by creating a pseudo-population in which X is independent of W. The intuition is that we can create a pseudo-population in which X is independent of W, by weighting the observations by the inverse of the propensity score. Once X is independent of W, X is not a confounder anymore. We can then estimate the effect of W on Y by comparing the weighted average of Y for $W=1$ and $W=0$.

Suppose there are two types of people, one type has high probability to be treated, the other one has low probability to be treated. Without adjusting for the propensity score, the treated group will be dominated by the first type, and the control group will be dominated by the second type. The difference between the treated and control group will be due to the difference in the composition of the two groups, not due to the treatment effect. By weighting the observations by the inverse of the propensity score, we can create a pseudo-population in which the two groups have the same composition. The difference between the treated and control group will be due to the treatment effect.

6.6. Doubly Robust Estimators

We can model both outcome and treatment assignment, and use both models to estimate the treatment effect. The estimator is called doubly robust because it is consistent if *either* the outcome regression *or* the propensity/treatment model is correctly specified — we only need one of the two to be right, not both.

6.6.1. AIPW

$$\mu_1^{AIPW} = E[\frac{W(Y-\mu_1(X))}{\pi(X)} + \mu_1(X)] \tag{6.11}$$

For the ATT, control units are weighted by the odds $\hat{\pi}(X_i)/(1-\hat{\pi}(X_i))$ rather than $1/(1-\hat{\pi}(X_i))$: this reweights the controls to the covariate distribution of the treated (one can check $\mathrm{E}[\frac{\pi(X)}{1-\pi(X)}(1-W)Y] = P(W=1)\,\mathrm{E}[Y(0)|W=1]$).

$$\mu_0^{AIPW} = E[\frac{(1-W)(Y-\mu_0(X))}{1-\pi(X)} + \mu_0(X)] \tag{6.12}$$

```julia
using CSV, DataFrames, GLM, Statistics

# Load Cattaneo (2010) birthweight dataset
# Treatment: mbsmoke (maternal smoking), Outcome: bweight (birthweight in grams)
data = CSV.read("data/cattaneo2.csv", DataFrame)

# Outcome Model (mu)
mu = lm(@formula(bweight ~ mbsmoke * (prenatal1 + mmarried + mage + fbaby)), data)

# Propensity Score Model (pi)
pi_model = glm(@formula(mbsmoke ~ mmarried + mage + fbaby + medu), data, Binomial(), LogitLink

# Predict Potential Outcomes for treated and control assignments
data_treated = copy(data)
data_treated.mbsmoke .= 1
mm1 = GLM.predict(mu, data_treated)

data_control = copy(data)
```

```
data_control.mbsmoke .= 0
mm0 = GLM.predict(mu, data_control)

# Predict Propensity Scores
pi1 = GLM.predict(pi_model, data)

# Assemble data for final AIPW computation
data.w = data.mbsmoke
data.y = data.bweight
data.m1 = mm1
data.m0 = mm0
data.pi1 = pi1

# Compute Doubly Robust potential outcomes
data.mu1 = @. (data.w * (data.y - data.m1) / data.pi1 + data.m1)
data.mu0 = @. ((1 - data.w) * (data.y - data.m0) / (1 - data.pi1) + data.m0)

# Calculate treatment effect
data.tau = data.mu1 .- data.mu0
tau_aipw = mean(data.tau)
@printf "ATE (AIPW) = %.4g\n" tau_aipw
```

```
ATE (AIPW) = -233.8
```

6.6.2. IPWRA

The idea of IPWRA is to combine RA and IPW. Basically RA with IPW weights.

Implementation:

- Estimate propensity score. Get predicted probability of treated for each observation.
- Estimate outcome model: two separate weighted equations, one for treated and one for control, using $1/\hat{\pi}(X)$ as weights for the treated group and $1/(1-\hat{\pi}(X))$ for the control group. (A single weighted regression with full treatment-covariate interactions would give numerically identical fits — the normal equations are block-diagonal — so the two-equation form is a convenience, not a requirement.)
- Get predicted values for setting everyone treated. Get predicted values for setting everyone control. These are the two potential outcomes.
- Take difference. The average is the ATE.

```
using CSV, DataFrames, GLM, Statistics

# Load Cattaneo (2010) birthweight dataset
data = CSV.read("data/cattaneo2.csv", DataFrame)

# 1. Estimate propensity score
pi_model = glm(@formula(mbsmoke ~ mmarried + mage + fbaby + medu), data, Binomial(), LogitLink
```

```
data.pi1 = GLM.predict(pi_model, data)

# Split data into treated and control
data1 = subset(data, :mbsmoke => x -> x .== 1)
data0 = subset(data, :mbsmoke => x -> x .== 0)

# 2. Estimate outcome models using inverse probability weights
wts1 = 1 ./ data1.pi1
wts0 = 1 ./ (1 .- data0.pi1)

# GLM.jl lm accepts weights via the `wts` keyword argument
mu1 = lm(@formula(bweight ~ prenatal1 + mmarried + mage + fbaby), data1; wts=wts1)
mu0 = lm(@formula(bweight ~ prenatal1 + mmarried + mage + fbaby), data0; wts=wts0)

# 3. Predict potential outcomes for the whole sample using the respective models
data_treated = copy(data)
data_treated.mbsmoke .= 1
mm1 = GLM.predict(mu1, data_treated)

data_control = copy(data)
data_control.mbsmoke .= 0
mm0 = GLM.predict(mu0, data_control)

# 4. Take the difference
data.w = data.mbsmoke
data.y = data.bweight
data.m1 = mm1
data.m0 = mm0

data.tau = data.m1 .- data.m0
tau_ipwra = mean(data.tau)
@printf "ATE (IPWRA) = %.4g\n" tau_ipwra
```

```
ATE (IPWRA) = -233.2
```

7. Matching Estimators

```
using DataFrames
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
```

Matching is a way to make treated and control observations more comparable. For each treated observation, we look for controls with similar covariates. Then we compare outcomes in this matched sample.

This does not solve endogeneity. Matching only helps with selection on observables. If treatment is selected on unobserved variables, matching will not fix the problem. Its value is that it makes overlap visible.

Julia does not yet have the same matching ecosystem as R's `MatchIt`, `cobalt`, and `WeightIt`. Here I implement the basic pieces directly: propensity-score matching, balance diagnostics, IPW, and entropy balancing. For full matching, CEM, CBPS, energy balancing, and better balance plots, see the R companion chapter.

> **Related reading**: The Matching and Weighting Part 1 and Treatment effects and matching chapters of *Topics on Econometrics and Causal Inference* walk through matching with `MatchIt` and Stata's `teffects` commands respectively.

7.1. Assumptions

Matching needs the same assumptions as other selection-on-observables methods:

1. **SUTVA** — no interference, no hidden treatment versions.
2. **Ignorability** — conditional on X, $D \perp (Y(0), Y(1))$.
3. **Overlap** — every x has positive probability of both treatments.

7.2. A simulated example

```
Random.seed!(42)
n  = 2000
X1 = randn(n)
X2 = randn(n)

# Treatment depends on covariates (confounding)
ps_true = @. 1 / (1 + exp(-(-0.5 + 0.6 * X1 + 0.4 * X2)))
D       = Float64.(rand(n) .< ps_true)

# Outcome: true treatment effect = 1, plus confounding through X1, X2
Y = @. 1.0 * D + 0.8 * X1 + 0.6 * X2 + 0.5 * randn()

df = DataFrame(Y = Y, D = D, X1 = X1, X2 = X2)
@printf("Naive difference in means: %.3f\n", mean(Y[D .== 1]) - mean(Y[D .== 0]))
@printf("True ATE: 1.000\n")
```

```
Naive difference in means: 1.713
True ATE: 1.000
```

7.3. Estimating the propensity score

```
ps_fit = glm(@formula(D ~ X1 + X2), df, Binomial(), LogitLink())
df.ps  = predict(ps_fit)

@printf("PS range: [%.3f, %.3f]\n", minimum(df.ps), maximum(df.ps))
```

```
PS range: [0.060, 0.927]
```

7.4. Nearest-neighbour 1:1 matching

```
"""
    nearest_neighbor_match(ps_treated, ps_control)
Return a vector of indices into the control group, matching each treated
unit to its nearest-neighbour control on PS. Sampling is without replacement.
"""
function nearest_neighbor_match(ps_treated, ps_control)
    n_t = length(ps_treated)
    avail = trues(length(ps_control))
    matches = zeros(Int, n_t)
    for (i, p) in enumerate(ps_treated)
        candidates = findall(avail)
```

```
        if isempty(candidates)
            break
        end
        diffs = abs.(ps_control[candidates] .- p)
        best  = candidates[argmin(diffs)]
        matches[i] = best
        avail[best] = false
    end
    matches
end

idx_T = findall(df.D .== 1)
idx_C = findall(df.D .== 0)

matched_C = nearest_neighbor_match(df.ps[idx_T], df.ps[idx_C])
matched_idx_C = idx_C[matched_C[matched_C .> 0]]
matched_idx_T = idx_T[1:length(matched_idx_C)]

@printf("Matched %d treated to %d controls\n",
        length(matched_idx_T), length(matched_idx_C))
```

```
Matched 818 treated to 818 controls
```

7.5. Balance diagnostics

The standardized mean difference before and after matching tells us whether the matched groups are balanced:

```
function smd(x_t, x_c)
    (mean(x_t) - mean(x_c)) / sqrt((var(x_t) + var(x_c)) / 2)
end

@printf("%-15s %12s %12s\n", "Covariate", "SMD before", "SMD after")
for v in [:X1, :X2]
    smd_before = smd(df[!, v][df.D .== 1], df[!, v][df.D .== 0])
    smd_after  = smd(df[!, v][matched_idx_T], df[!, v][matched_idx_C])
    @printf("%-15s %12.3f %12.3f\n", String(v), smd_before, smd_after)
end
```

```
Covariate         SMD before    SMD after
X1                     0.614        0.270
X2                     0.394        0.152
```

Standardized mean differences below 0.1 are usually treated as acceptable. If they remain large, the matching is not doing enough.

7.6. Estimating the ATT on matched data

After matching, the ATT is the simple difference in means:

```
att = mean(df.Y[matched_idx_T]) - mean(df.Y[matched_idx_C])
@printf("Matched ATT: %.3f  (true ATE = 1.000)\n", att)

# Standard error: paired-difference SE (treats matched pairs as paired)
diffs = df.Y[matched_idx_T] .- df.Y[matched_idx_C]
se    = std(diffs) / sqrt(length(diffs))
@printf("Paired-diff SE: %.3f\n", se)
@printf("95%% CI: [%.3f, %.3f]\n", att - 1.96 * se, att + 1.96 * se)
```

```
Matched ATT: 1.287  (true ATE = 1.000)
Paired-diff SE: 0.032
95% CI: [1.225, 1.350]
```

The matched estimate is still visibly biased (compare it to the truth of 1.0). The balance table above says why: with 818 treated units and only 1,182 controls, 1:1 matching without replacement exhausts the good controls, and the post-matching standardized mean differences (0.27 and 0.15) remain above the 0.1 threshold. Residual imbalance means residual confounding. The with-replacement, IPW, and entropy-balancing estimates below, which are not constrained to use each control at most once, all land much closer to the truth. For matched pairs, the paired-difference standard error is the natural first check.

7.7. Matching with replacement

With replacement, the same control can be used several times. This can improve balance when good controls are scarce, but the effective sample size is smaller.

```
function nn_match_replace(ps_treated, ps_control)
    [argmin(abs.(ps_control .- p)) for p in ps_treated]
end

matched_repl = nn_match_replace(df.ps[idx_T], df.ps[idx_C])
@printf("Average reuse: %.2f (1.0 = no replacement)\n",
        length(matched_repl) / length(unique(matched_repl)))

att_repl = mean(df.Y[idx_T]) - mean(df.Y[idx_C[matched_repl]])
@printf("Matched-with-replacement ATT: %.3f\n", att_repl)
```

```
Average reuse: 1.76 (1.0 = no replacement)
Matched-with-replacement ATT: 0.988
```

7.8. Matching vs weighting

IPW is very close to matching in spirit. Instead of dropping or pairing observations, we reweight observations by the inverse probability of receiving their observed treatment.

```
# These are the ATE (Horvitz-Thompson/Hajek) weights: treated by 1/ps and
# controls by 1/(1-ps). They target the ATE, not the ATT. With a homogeneous
# +1 effect here, ATE = ATT numerically, but the estimand is the ATE.
w = ifelse.(df.D .== 1, 1 ./ df.ps, 1 ./ (1 .- df.ps))
w_trim = min.(w, quantile(w, 0.99))

ate_ipw = sum(df.Y[df.D .== 1] .* w_trim[df.D .== 1]) / sum(w_trim[df.D .== 1]) -
          sum(df.Y[df.D .== 0] .* w_trim[df.D .== 0]) / sum(w_trim[df.D .== 0])
@printf("IPW ATE:                %.3f\n", ate_ipw)
@printf("Matched ATT (NN):       %.3f\n", att)
@printf("Matched ATT (replace): %.3f\n", att_repl)
@printf("True ATE:               1.000\n")
```

```
IPW ATE:                1.064
Matched ATT (NN):       1.287
Matched ATT (replace): 0.988
True ATE:               1.000
```

When IPW and matching disagree sharply, it is usually a warning about the propensity-score model or lack of overlap.

7.9. Entropy balancing

Entropy balancing chooses weights so that the weighted control group has the same covariate means as the treated group. It does not require a propensity-score model. R's `WeightIt` package implements this and related methods. In Julia, the basic optimization is short enough to write directly.

The entropy-balancing problem for the ATT: find weights w_i on the control units that solve

$$\min_{w_i \geq 0} \sum_i w_i \log w_i \quad \text{subject to} \quad \sum_i w_i x_{ik} = \bar{x}_k^{\text{treated}} \;\; \forall k, \quad \sum_i w_i = 1. \tag{7.1}$$

The dual problem has one parameter per balance constraint:

$$\min_{\lambda} \;\; \log\left(\sum_i \exp(-x_i'\lambda)\right) + \lambda' \bar{x}^{\text{treated}}. \tag{7.2}$$

This is a small convex problem, so Newton's method works well here.

```
"""
    entropy_balance(X_control, X_treated; tol=1e-8, maxiter=50)
Compute entropy-balancing weights on the control units so that the
weighted control mean of every covariate equals the treated mean.
Returns a length-`size(X_control, 1)` vector of normalised weights that
sum to 1.
"""
function entropy_balance(X_control::AbstractMatrix, X_treated::AbstractMatrix;
                         tol::Float64 = 1e-8, maxiter::Int = 50)
    target = vec(mean(X_treated, dims=1))   # length p target means
    Xc = X_control                          # n_c × p
    p  = size(Xc, 2)
       = zeros(p)

    for _ in 1:maxiter
        z = -Xc *
        m = maximum(z)
        w = exp.(z .- m)
        w ./= sum(w)                                    # normalised weights
        mean_w = vec(w' * Xc)                           # weighted control mean
        grad   = target .- mean_w                       # negative gradient
        if maximum(abs.(grad)) < tol
            break
        end
        # Hessian: Σ w_i (x_i - mean_w)(x_i - mean_w)'  (with tiny ridge for stability)
        Xc_centered = Xc .- reshape(mean_w, 1, :)
        H = Matrix((Xc_centered .* w)' * Xc_centered) + 1e-6 .* Matrix{Float64}(I, p, p)
        # Newton step on the convex dual L( )=log Σexp(-x )+ ·target,
        # whose gradient is `grad`=target-mean_w and Hessian is H:   ←   - H ¹ grad
        step = H \ grad
          .-= step
    end

    z = -Xc *
    m = maximum(z)
    w = exp.(z .- m)
    w ./ sum(w)
end

# Apply to the simulated example from earlier sections
Xc_mat = Matrix(df[df.D .== 0, [:X1, :X2]])
Xt_mat = Matrix(df[df.D .== 1, [:X1, :X2]])

w_ebal = entropy_balance(Xc_mat, Xt_mat)
@printf("Sum of weights: %.6f  (should be 1.0)\n", sum(w_ebal))
@printf("Max weight: %.4f, min weight: %.6f\n",
        maximum(w_ebal), minimum(w_ebal))
```

```
# Verify the mean-balance constraints
control_means_unweighted = mean(Xc_mat, dims=1)
control_means_balanced   = w_ebal' * Xc_mat
treated_means            = mean(Xt_mat, dims=1)

@printf("\n%-15s %12s %12s %12s\n",
        "Covariate", "Treated", "Control (raw)", "Control (ebal)")
for (j, name) in enumerate([:X1, :X2])
    @printf("%-15s %12.4f %12.4f %12.4f\n",
            String(name),
            treated_means[j], control_means_unweighted[j],
            control_means_balanced[j])
end
```

```
Sum of weights: 1.000000  (should be 1.0)
Max weight: 0.0095, min weight: 0.000079

Covariate            Treated Control (raw) Control (ebal)
X1                    0.3117      -0.2801       0.3117
X2                    0.2395      -0.1541       0.2395
```

The weighted control means equal the treated means because the optimization imposes that constraint. If overlap is poor, the weights can become very uneven.

The ATT estimate uses these weights on the control outcomes:

```
Yt = df.Y[df.D .== 1]
Yc = df.Y[df.D .== 0]

# ATT = mean(Y_treated) - weighted mean(Y_control)
att_ebal = mean(Yt) - dot(w_ebal, Yc)
@printf("Entropy-balanced ATT: %.3f  (true ATE = 1.000)\n", att_ebal)
```

```
Entropy-balanced ATT: 1.014  (true ATE = 1.000)
```

Entropy balancing is useful when mean balance is the main diagnostic. If we need balance over whole covariate distributions, energy balancing in R's `WeightIt` is a better tool. There is no Julia equivalent yet.

7.10. Matching, weighting, and direct estimation

The estimation chapter estimated a treatment effect by regression adjustment, IPW and AIPW. This chapter has estimated one by matching, by IPW and by entropy balancing. Those are not two

families of estimators. They are one estimand and several ways of building it, and it is worth saying how they line up.

The ATT is

$$\tau_{ATT} = E[Y \mid D = 1] - E_{X|D=1}\big[\, E[Y \mid D = 0, X] \,\big]. \tag{7.3}$$

The first term is observed. The second is not: it is the mean outcome the treated would have had untreated, averaged over the covariate distribution of the treated. Every method here forms that second term, and every one of them ends up as a weighted average of control outcomes, $\sum_{i:D_i=0} w_i Y_i$. What differs is where the weights come from.

Route	What is modelled	Weight on control i
Regression adjustment	the outcome, $\mu_0(x) = E[Y \mid D = 0, X = x]$	implicit in the fitted values used to predict at treated x
IPW	the propensity score $\pi(x)$	the odds $\pi(x_i)/(1 - \pi(x_i))$
Matching	neither; a distance rule	0 if unmatched, 1 if matched
Entropy balancing	the weights themselves	whatever satisfies the balance constraints
AIPW	both	the odds, applied to the residual $Y - \hat{\mu}_0(X)$

Weighting is IPW. That is the second row, and the odds are what the IPW section above computed. Entropy balancing does not change the estimator, only how the weights are found: IPW fits π by maximum likelihood and transforms it, while entropy balancing skips the likelihood and solves for weights satisfying the balance constraints directly. Its solution has the form $w_i \propto \exp(x_i'\lambda)$, which is the odds of a logistic propensity score — so entropy balancing is IPW for a logit model whose coefficients are set by exact mean balance rather than by likelihood.

Matching is weighting too, with the weights restricted to zero and one and chosen by a distance rule rather than by a model. The restriction is where the trade-off lives: matching sets some weights to zero, so it discards observations, while weighting keeps everyone and pays with extreme weights when overlap is poor. Matching buys stability with the estimand; weighting buys the estimand with variance.

AIPW uses both columns: start from regression adjustment, then add the odds-weighted average of the control residuals $Y - \hat{\mu}_0(X)$. It is consistent if either model is right.

Five routes, one data set. Each row is built by hand from the weight representation so the arithmetic is visible.

```
trt = df.D .== 1

# Outcome model fitted on controls only, then predicted everywhere
mu0_fit = lm(@formula(Y ~ X1 + X2), df[.!trt, :])
mu0 = predict(mu0_fit, df)

# Odds weights from the logistic propensity score fitted above
```

```
odds   = df.ps ./ (1 .- df.ps)
wt_ipw = ifelse.(trt, 1.0, odds)

# Weighted difference in means: the common shape of rows 2 to 4
hajek(w) = sum(df.Y[trt] .* w[trt]) / sum(w[trt]) -
           sum(df.Y[.!trt] .* w[.!trt]) / sum(w[.!trt])

# Matching as 0/1 weights on the units the matcher kept
w_nn = zeros(nrow(df))
w_nn[matched_idx_T] .= 1.0
w_nn[matched_idx_C] .= 1.0

# Entropy-balancing weights, treated at 1
w_eb = zeros(nrow(df))
w_eb[trt] .= 1.0
w_eb[.!trt] .= w_ebal

att_ra   = mean(df.Y[trt] .- mu0[trt])
att_ipw  = hajek(wt_ipw)
att_eb   = hajek(w_eb)
att_nn   = hajek(w_nn)
att_aipw = att_ra - sum((df.Y[.!trt] .- mu0[.!trt]) .* odds[.!trt]) / sum(odds[.!trt])

DataFrame(
    route = ["Regression adjustment (outcome model)",
             "IPW (propensity model)",
             "Entropy balancing (balance constraints)",
             "1:1 matching, no replacement (distance rule)",
             "AIPW (both models)"],
    ATT = round.([att_ra, att_ipw, att_eb, att_nn, att_aipw], digits = 3))
```

	route	ATT
	String	Float64
1	Regression adjustment (outcome model)	1.009
2	IPW (propensity model)	1.014
3	Entropy balancing (balance constraints)	1.014
4	1:1 matching, no replacement (distance rule)	1.287
5	AIPW (both models)	1.014

Two of these confirm the algebra directly. Expressing the matched comparison as a weighted mean with zero-one weights reproduces the 1.287 printed earlier, which is what "matching is a weighting estimator" means concretely. And entropy balancing returns 1.014, the same figure the entropy-balancing section printed, since that section computed the same weighted difference.

The spread is the part worth reading. Four of the five routes land between 1.009 and 1.014 against a true effect of 1.000. They agree because this DGP gives them every reason to: the treatment effect is constant at 1, so there is no heterogeneity for differing weights to average differently, and the propensity score runs from 0.060 to 0.927, so no unit carries an extreme weight. Under those two

conditions the routes are estimating the same number and differ only by noise. The R companion chapter runs the same five routes on the Lalonde data, where effects are heterogeneous and overlap is poor, and there they spread from $1,214 to $1,855.

The exception here is instructive. One-to-one matching without replacement gives 1.287, nearly 30% high, and it is the one route that throws data away: 364 of the 1,182 controls receive weight zero. Matching with replacement, which reuses good controls instead of discarding them, gave 0.988 earlier in the chapter. That gap is the cost of the zero weights, not of matching as an idea.

These are point estimates, and standard errors are not comparable across the rows: matching is not a smooth functional of the data, so the bootstrap is invalid for it, while IPW and AIPW have influence-function variances and AIPW attains the semiparametric efficiency bound that fixed-M matching does not.

7.11. When to use which method

Method	When to use	Limitation
NN matching (no replacement)	Simple, intuitive; ATT estimand	Drops unmatched units
NN matching (with replacement)	Few treated, many controls	Wasteful of controls
IPW	Good propensity model	Sensitive to extreme weights
Entropy balancing	Want exact mean balance; small covariate set	Balances means only, not distributions
Doubly robust (AIPW)	Combine matching/IPW with regression	More complex

For applied work in Julia, IPW or AIPW is usually more practical than 1:1 matching. Julia has good regression tools, but not yet a full matching toolkit. For a matched analysis with many diagnostics, I would still use R's `MatchIt` and `cobalt`.

7.12. Summary

- Matching is for selection on observables. It does not fix unobserved confounding.
- The main diagnostic is balance, not the treatment-effect coefficient.
- Propensity-score nearest-neighbor matching is easy to implement in Julia, but full matching and CEM are better handled in R.
- IPW and entropy balancing are often more practical Julia workflows.

8. Nonparametric Causal Methods

```
using DataFrames
using Distributions
using Random
using Statistics
using MLJ
using EvoTrees
using MLJLinearModels
using DecisionTree
using CausalEstimate
import CausalEstimate: estimate, confint, pvalue
using Printf
```

> **Related reading**: For an R-based TMLE simulation that compares super-learner libraries and stress-tests model misspecification, see the companion blog chapter on TMLE in *Topics on Econometrics and Causal Inference*.

8.1. Why Do We Need Nonparametric Methods?

- Parametric models are often mis-specified
- Nonparametric models are more flexible; less assumptions
- What kind of nonparametrics are good?

Considerations:

- Plug-in (g-computation) estimators are easy.
- The problem is not the decomposition into $E[Y(0)]$ and $E[Y(1)]$ itself: with correctly specified models and regularity, a plug-in estimator can be efficient. The problem appears once the nuisance models are flexible and regularised. Then the error in the nuisance estimate enters the plug-in target *linearly*, and the plug-in has no term that cancels it. An influence-function estimator adds exactly such a term, and the error that survives is a product of two nuisance errors rather than one. Section 8.2.1 does the algebra.
- So they can be consistent, but the convergence rate inherits the (often slow) nuisance rate, which is especially bad in high dimensions.
- Ideally we remove that first-order sensitivity using an orthogonal influence function, and get $\sqrt{n}$ efficiency.
- Influence-function-based (one-step/AIPW/TMLE) estimators are shown to be efficient and orthogonal to first-order nuisance error.

8.2. Influence Function Based Estimators

Estimators based on efficient influence function are often $\sqrt{n}$ consistent, meaning they converge to the truth as fast as $\sqrt{n}$ rate. This means the bias not only goes to 0 with n goes to infinity, but also as fast as $1/\sqrt{n}$ goes to 0. Why does this matter? Because we have limited sample size, when we are based on asymptotics, the faster the bias goes to 0 the smaller sample size we need.

If we have an estimator such that

$$\sqrt{n}(\hat{\psi} - \psi) = \sqrt{n}P_n(\phi) + o_P(1) \to N(0, var(\phi)) \tag{8.1}$$

Then this estimator is $\sqrt{n}$ consistent and asymptotically normal; and ϕ is the influence function. The efficient influence function is the one with variance at the lower bound (similar to parametric case for Cramer-Rao lower bound).

Unfortunately there is no universal formula for efficient influence function for every problem. We need to derive it for each problem. For example, ATE has an IF based estimator, but ATT needs a different formula; and LATE has a different formula too, etc. Deriving them is hard!

For the treated potential-outcome mean, $\psi_1 = E\{E(Y \mid X, A = 1)\}$, the efficient influence function is

$$\phi_1(Z; P) = \frac{A}{\pi(X)}\{Y - \mu_1(X)\} + \mu_1(X) - \psi_1 \tag{8.2}$$

where $\pi(X) = P(A = 1|X)$ is the propensity score model and $\mu_1(X) = E(Y|X, A = 1)$ is the treated outcome model. The ATE influence function is the difference between the treated and untreated components, $\phi_1 - \phi_0$.

For example, an IF-based bias-corrected estimator for the treated-arm mean

$$\hat{\psi}_1 = P_n\left[\frac{A}{\hat{\pi}(X)}\{Y - \hat{\mu}_1(X)\} + \hat{\mu}_1(X)\right] \tag{8.3}$$

where P_n means sample mean. This is the familiar AIPW estimator. It is known to be doubly robust.

Since we know the influence function, we can calculate its variance. This variance is the same as variance of $\hat{\psi}$ since they differ by a constant ψ. We can use any estimators to estimate the nuisance parameters, then get $\hat{\psi}$. Then the sample variance is the variance of the influence function; and the sample mean is the ATE.

In this estimator, both $\hat{\mu}$ and $\hat{\pi}$ can be done using any estimator, such as machine learning. One caveat: with adaptive learners (random forests, boosting), the nuisance fits should be *cross-fit* — predicted for each observation from a model trained without it — otherwise own-observation overfitting can bias the estimate. The manual AIPW below reuses full-sample fits to keep the code short; the `CausalEstimate.jl` calls use `crossfit=5`.

8.2.1. Why the Plug-In Is Off

The bullets above claimed that the plug-in inherits the nuisance error at first order and that the augmentation term removes it. That is algebra, not a simulation finding, so it is worth doing before we run anything.

Take $\psi_1 = E[\mu_1(X)]$ with $\mu_1(x) = E(Y \mid X = x, A = 1)$. The plug-in estimator is $\hat{\psi}_1^{\text{pi}} = P_n[\hat{\mu}_1(X)]$. Split its error into a sampling part and a nuisance part:

$$\hat{\psi}_1^{\text{pi}} - \psi_1 = \underbrace{(P_n - P)[\mu_1]}_{O_P(n^{-1/2})} + \underbrace{P[\hat{\mu}_1 - \mu_1]}_{\text{nuisance error, linear}} + \underbrace{(P_n - P)[\hat{\mu}_1 - \mu_1]}_{\text{smaller still}}. \tag{8.4}$$

The middle term is $E[\hat{\mu}_1(X) - \mu_1(X)]$, and it is linear in the nuisance error. If $\hat{\mu}_1$ converges at rate $n^{-\alpha}$, that term is of order $n^{-\alpha}$, and a regularised learner in even a moderate number of dimensions gives $\alpha < 1/2$. The nuisance error then dominates the sampling error, the estimator is not $\sqrt{n}$ consistent, and nothing in the plug-in cancels it.

Now do the same for the AIPW estimator of Equation 8.3. Evaluate it under the population distribution, holding the estimated nuisances fixed. Because $E[A\{Y - \hat{\mu}_1(X)\} \mid X] = \pi(X)\{\mu_1(X) - \hat{\mu}_1(X)\}$,

$$P\Big[\frac{A}{\hat{\pi}(X)}\{Y - \hat{\mu}_1(X)\} + \hat{\mu}_1(X)\Big] = \psi_1 + E\Big[\frac{\pi(X) - \hat{\pi}(X)}{\hat{\pi}(X)}\{\mu_1(X) - \hat{\mu}_1(X)\}\Big]. \tag{8.5}$$

The bias is a *product* of the two nuisance errors, and that is the entire difference between the two estimators. The term linear in $\hat{\mu}_1 - \mu_1$ has cancelled: the $\hat{\mu}_1$ that the outcome model carries in is subtracted back out by the $\hat{\mu}_1$ sitting inside the augmentation, and what survives is multiplied by the propensity-score error. By Cauchy-Schwarz the bias is bounded by $\|\pi - \hat{\pi}\| \, \|\mu_1 - \hat{\mu}_1\|$ up to the overlap constant, so rates of $o(n^{-1/4})$ on each nuisance suffice to make it $o(n^{-1/2})$ — slower than $\sqrt{n}$ on each piece, fast enough for $\sqrt{n}$ on the target.

Two familiar facts fall out of Equation 8.5 directly. Double robustness is immediate: if either factor is zero the whole bias is zero, which is why one correct model is enough. And the $n^{-1/4}$ requirement is the reason cross-fitting matters, since reusing the same observations to fit and to evaluate $\hat{\mu}_1$ makes the product term behave worse than the rate calculation assumes.

8.2.2. Simulation Setup

```
Random.seed!(1)
n = 1000
w1 = rand(Binomial(1, 0.5), n)
w2 = rand(Binomial(1, 0.5), n)
w3 = round.(rand(Uniform(0, 4), n), digits=3)
w4 = round.(rand(Uniform(0, 5), n), digits=3)

logistic(x) = 1.0 ./ (1.0 .+ exp.(-x))   # inverse logit (sigmoid), not the logit
```

```
prob_A = logistic.(-0.4 .+ 0.2.*w2 .+ 0.15.*w3 .+ 0.2.*w4 .+ 0.15.*w2.*w4)
A = rand.(Binomial.(1, prob_A))

prob_Y1 = logistic.(-1.0 .+ 1.0 .- 0.1.*w1 .+ 0.3.*w2 .+ 0.25.*w3 .+ 0.2.*w4 .+ 0.15.*w2.*w4)
prob_Y0 = logistic.(-1.0 .+ 0.0 .- 0.1.*w1 .+ 0.3.*w2 .+ 0.25.*w3 .+ 0.2.*w4 .+ 0.15.*w2.*w4)

Y_1 = rand.(Binomial.(1, prob_Y1))
Y_0 = rand.(Binomial.(1, prob_Y0))
Y = Y_1 .* A .+ Y_0 .* (1 .- A)

W = DataFrame(w1=w1, w2=w2, w3=w3, w4=w4)
df = DataFrame(w1=w1, w2=w2, w3=w3, w4=w4, A=A, Y=Y, Y_1=Y_1, Y_0=Y_0)

True_Psi = mean(df.Y_1 .- df.Y_0)
@printf "True ATE = %.4g\n" True_Psi
```

```
True ATE = 0.184
```

The line above reports `mean(Y.1 - Y.0)` for this particular draw. That is a legitimate quantity — the *sample* average treatment effect — but it is not a fixed benchmark, and it is easy to mistake for one. The potential outcomes are binary, so at $n = 1000$ this realized average has a sampling standard deviation of about 0.020: two runs of the same DGP under different seeds can differ by 0.04 without anything being wrong. (The R and Julia editions of this chapter print 0.21 and 0.184 for exactly this reason.) Estimator standard errors in this chapter are around 0.03, so a noisy target of that size would swamp the comparison we are trying to make.

The *population* ATE is a definite number, and since the DGP is fully specified we can integrate it out rather than simulate it:

```
# w1, w2 ~ Bernoulli(0.5); w3 ~ U(0,4); w4 ~ U(0,5). Average the true individual
# effect over that covariate distribution on a trapezoid grid.
function population_ate()
    g3 = range(0, 4; length = 1601)
    g4 = range(0, 5; length = 1601)
    trap(g) = (w = fill(step(g), length(g)); w[1] /= 2; w[end] /= 2; w)
    w3w, w4w = trap(g3), trap(g4)
    total = 0.0
    for w1 in (0, 1), w2 in (0, 1)
        acc = 0.0
        for (j, u4) in enumerate(g4), (i, u3) in enumerate(g3)
            lin0 = -1 - 0.1w1 + 0.3w2 + 0.25u3 + 0.2u4 + 0.15w2 * u4
            acc += w3w[i] * w4w[j] * (logistic(lin0 + 1) - logistic(lin0))
        end
        total += acc / (4 * 5)
    end
    total / 4
end
```

```
pop_ate = population_ate()
@printf "Realized-sample ATE (this draw): %.4f\n" True_Psi
@printf "Exact population ATE:           %.4f\n" pop_ate
```

```
Realized-sample ATE (this draw): 0.1840
Exact population ATE:           0.2028
```

Compare the estimators below against 0.2028, not against the realized draw.

```
# Load MLJ models to form our equivalent of SuperLearner
LogisticClassifier = @load LogisticClassifier pkg=MLJLinearModels verbosity=0
EvoTreeClassifier = @load EvoTreeClassifier pkg=EvoTrees verbosity=0
RandomForestClassifier = @load RandomForestClassifier pkg=DecisionTree verbosity=0

# Define a Stack (SuperLearner)
stack = Stack(
    metalearner = LogisticClassifier(),
    resampling = CV(nfolds=5),
    evo = EvoTreeClassifier(max_depth=4, nrounds=100),
    rf = RandomForestClassifier(n_trees=100, max_depth=5),
    glm = LogisticClassifier()
)
```

```
ProbabilisticStack(
  metalearner = LogisticClassifier(
        lambda = 2.220446049250313e-16,
        gamma = 0.0,
        penalty = :l2,
        fit_intercept = true,
        penalize_intercept = false,
        scale_penalty_with_samples = true,
        solver = nothing),
  resampling = CV(
        nfolds = 5,
        shuffle = false,
        rng = TaskLocalRNG()),
  measures = nothing,
  cache = true,
  acceleration = CPU1{Nothing}(nothing),
  evo = EvoTreeClassifier(
        loss = :mlogloss,
        metric = :mlogloss,
        nrounds = 100,
        bagging_size = 1,
        early_stopping_rounds = 9223372036854775807,
```

```
        L2 = 1.0,
        lambda = 0.0,
        gamma = 0.0,
        eta = 0.1,
        max_depth = 4,
        min_weight = 1.0,
        rowsample = 1.0,
        colsample = 1.0,
        nbins = 64,
        tree_type = :binary,
        seed = 123,
        device = :cpu),
  rf = RandomForestClassifier(
        max_depth = 5,
        min_samples_leaf = 1,
        min_samples_split = 2,
        min_purity_increase = 0.0,
        n_subfeatures = -1,
        n_trees = 100,
        sampling_fraction = 0.7,
        feature_importance = :impurity,
        rng = TaskLocalRNG()),
  glm = LogisticClassifier(
        lambda = 2.220446049250313e-16,
        gamma = 0.0,
        penalty = :l2,
        fit_intercept = true,
        penalize_intercept = false,
        scale_penalty_with_samples = true,
        solver = nothing))
```

8.2.3. Plug-in Estimator from Outcome Regression

```
Random.seed!(1)
X_mu = select(df, :w1, :w2, :w3, :w4, :A)
mach_mu = machine(stack, X_mu, categorical(df.Y))
MLJ.fit!(mach_mu, verbosity=0)

# predict the PO Y^1
X_mu1 = copy(X_mu); X_mu1.A .= 1
X_mu0 = copy(X_mu); X_mu0.A .= 0

# Predict probability of Y=1 (assuming 1 is the positive class)
mu1hat = pdf.(MLJ.predict(mach_mu, X_mu1), 1)
mu0hat = pdf.(MLJ.predict(mach_mu, X_mu0), 1)
```

```
plug_in_ate = mean(mu1hat .- mu0hat)
@printf "ATE (plug-in) = %.4g\n" plug_in_ate
```

```
ATE (plug-in) = 0.1759
```

8.2.4. AIPW ATE

```
Random.seed!(123)
X_pi = select(df, :w1, :w2, :w3, :w4)
mach_pi = machine(stack, X_pi, categorical(df.A))
MLJ.fit!(mach_pi, verbosity=0)

pi1hat = pdf.(MLJ.predict(mach_pi, X_pi), 1)

# Bound propensity scores to avoid extreme weights
pi1hat = max.(min.(pi1hat, 0.999), 0.001)
pi0hat = 1.0 .- pi1hat

# Current prediction on observed data
mu1hat_obs = mu1hat
mu0hat_obs = mu0hat

IF_1 = ((df.A ./ pi1hat) .* (df.Y .- mu1hat_obs) .+ mu1hat_obs)
IF_0 = (((1 .- df.A) ./ pi0hat) .* (df.Y .- mu0hat_obs) .+ mu0hat_obs)
IF = IF_1 .- IF_0

aipw_1 = mean(IF_1); aipw_0 = mean(IF_0)
aipw_manual = aipw_1 - aipw_0

ci_lb = mean(IF) - quantile(Normal(), 0.975) * std(IF) / sqrt(length(IF))
ci_ub = mean(IF) + quantile(Normal(), 0.975) * std(IF) / sqrt(length(IF))
res_manual_aipw = [aipw_manual, ci_lb, ci_ub]
@printf "ATE (AIPW)    = %.4g\n" aipw_manual
@printf "95%% CI        = [%.4g, %.4g]\n" ci_lb ci_ub
```

```
ATE (AIPW)    = 0.1918
95% CI        = [0.1311, 0.2525]
```

8.2.5. AIPW ATE Using CausalEstimate.jl

```
using CausalEstimate
```

```
r(x) = round(x, sigdigits=4)

psi_ate = ATE(outcome=:Y, treatment=:A, confounders=[:w1, :w2, :w3, :w4])
aipw_result = estimate(psi_ate, AIPW(crossfit=5), df)

(
    ATE    = r(estimate(aipw_result)),
    CI     = r.(confint(aipw_result)),
    pvalue = r(pvalue(aipw_result)),
)
```

```
(ATE = 0.2072, CI = (0.1206, 0.2939), pvalue = 2.776e-6)
```

8.2.6. Regression-imputation ATT (manual)

Note that the manual estimator below is evaluated on the **treated subset** `df1`, where `(1 .- df1.A)` is identically zero. The augmentation term `((1-A)/ )(Y- )` therefore vanishes and the estimator collapses to the regression-imputation ATT, `mean(Y_treated - ^ (X_treated))`. That is consistent if the control outcome model is correct, but it is **not** the doubly-robust / augmented ATT, and the influence-function SE understates uncertainty because it ignores estimation of and . The proper ATT efficient influence function (Hahn 1998) is `(1/p)[A(Y- ) - (1-A)· /(1- )·(Y- )]` evaluated over the **full** sample; the `CausalEstimate.jl` call in the next subsection computes the cross-fit, doubly-robust version.

```
Random.seed!(1)
df2 = copy(df)
df2.pi0hat = pi0hat
df2.pi1hat = pi1hat

df0 = filter(row -> row.A == 0, df2)
df1 = filter(row -> row.A == 1, df2)

# Fit outcome model only on controls
X_mu0 = select(df0, :w1, :w2, :w3, :w4)
mach_mu0 = machine(stack, X_mu0, categorical(df0.Y))
MLJ.fit!(mach_mu0, verbosity=0)

# Predict counterfactual for treated
X_mu1 = select(df1, :w1, :w2, :w3, :w4)
mu0hat_treated = pdf.(MLJ.predict(mach_mu0, X_mu1), 1)

IF_0_att = (((1 .- df1.A) ./ df1.pi0hat) .* (df1.Y .- mu0hat_treated) .+ mu0hat_treated)
IF_att = df1.Y .- IF_0_att

aipw_manual_att = mean(IF_att)
ci_lb_att = mean(IF_att) - quantile(Normal(), 0.975) * std(IF_att) / sqrt(length(IF_att))
```

```
ci_ub_att = mean(IF_att) + quantile(Normal(), 0.975) * std(IF_att) / sqrt(length(IF_att))
res_manual_aipw_att = [aipw_manual_att, ci_lb_att, ci_ub_att]
res_manual_aipw_att
```

```
3-element Vector{Float64}:
 0.20145553037952954
 0.1700985774823039
 0.2328124832767552
```

8.2.7. AIPW ATT Using CausalEstimate.jl

```
psi_att = ATT(outcome=:Y, treatment=:A, confounders=[:w1, :w2, :w3, :w4])
att_result = estimate(psi_att, AIPW(crossfit=5), df)

(
    ATT    = r(estimate(att_result)),
    CI     = r.(confint(att_result)),
    pvalue = r(pvalue(att_result)),
)
```

```
(ATT = 0.189, CI = (0.06404, 0.3139), pvalue = 0.00303)
```

8.3. TMLE

TMLE is a variant of IF based estimator. It is also a doubly robust estimator. See Mark van der Laan and his coauthors for the recent development of TMLE.

We should expect AIPW and TMLE to give similar answers.

```
Random.seed!(42)
tmle_result = estimate(psi_ate, TMLE(crossfit=5), df)

(
    TMLE   = r(estimate(tmle_result)),
    CI     = r.(confint(tmle_result)),
    pvalue = r(pvalue(tmle_result)),
)
```

```
(TMLE = 0.1775, CI = (0.08638, 0.2687), pvalue = 0.0001349)
```

```
@printf "Population ATE (exact):    %.4f\n" pop_ate
@printf "Realized-sample ATE:       %.4f\n" True_Psi
```

```
Population ATE (exact):    0.2028
Realized-sample ATE:       0.1840
```

8.4. DoubleML

DoubleML (Double/Debiased Machine Learning) was proposed by Chernozhukov et al. (2018). An R package `DoubleML` implements the framework; as of this writing there is no native Julia package, but the algorithm is straightforward to implement directly. The double machine learning framework consists of three key ingredients: Neyman orthogonality, high-quality machine learning estimation, and sample splitting.

They consider a partially linear model:

$$y_i = \theta d_i + g_0(x_i) + \eta_i \tag{8.6}$$

$$d_i = m_0(x_i) + v_i \tag{8.7}$$

This model is quite general, except it does not allow interaction of d and x; therefore no heterogeneous treatment effect across x. But "DoubleML" implements more than partially linear model, it actually allows for heterogeneous treatment effects, in models such as interactive regression model.

The basic idea of doubleML is to use any machine learning algorithm to estimate outcome equation ($l_0(X) = E(Y|X)$) and treatment equation ($m_0(X) = E(D|X)$). Then get the residuals, namely $\hat{W} = Y - \hat{l}_0(X)$ and $\hat{V} = D - \hat{m}_0(X)$.

Then regress $\hat{W}$ on $\hat{V}$. Based on FWL theorem, you get $\hat{\theta}$.

An important component here is to specify a Neyman-orthogonal score function. The *partialling-out* form, which is what the residual-on-residual regression below implements, is

$$\psi(W;\theta,\eta) = (Y - l(X) - \theta\,(D - m(X)))\,(D - m(X)), \tag{8.8}$$

with $l(X) = E[Y \mid X]$ and $m(X) = E[D \mid X]$. The estimator $\hat{\theta}$ solves the equation that the sample mean of this score function is 0.

The estimator's variance is the sandwich $J^{-1}\,E[\psi^2]\,J^{-1}$ with Jacobian $J = E[\partial\psi/\partial\theta] = -E[(D - m(X))^2]$ — this is what the code computes by dividing the score variance by $\left(\frac{1}{n}\sum \hat{V}_i^2\right)^2$.

```
using MLJ
using Random

# We implement DoubleML PLR (Partially Linear Regression) manually with cross-fitting
Random.seed!(123)
nsplits = 5
n_samples = nrow(df)
s = shuffle(1:n_samples)
folds = [s[1 + floor(Int, (k-1)*n_samples/nsplits) : floor(Int, k*n_samples/nsplits)] for k in

# Base learner (Random Forest for regression)
RandomForestRegressor = @load RandomForestRegressor pkg=DecisionTree verbosity=0
```

```
learner = RandomForestRegressor(n_trees=500, max_depth=5)

W_res = zeros(n_samples)
V_res = zeros(n_samples)

for fold in 1:nsplits
    test_idx = folds[fold]
    train_idx = setdiff(1:n_samples, test_idx)

    # Fit outcome model l_0(X) = E(Y | X)
    mach_l = machine(learner, W[train_idx, :], df.Y[train_idx])
    MLJ.fit!(mach_l, verbosity=0)
    y_pred = MLJ.predict(mach_l, W[test_idx, :])
    W_res[test_idx] = df.Y[test_idx] .- y_pred

    # Fit treatment model m_0(X) = E(D | X)
    mach_m = machine(learner, W[train_idx, :], df.A[train_idx])
    MLJ.fit!(mach_m, verbosity=0)
    a_pred = MLJ.predict(mach_m, W[test_idx, :])
    V_res[test_idx] = df.A[test_idx] .- a_pred
end
```

```
# FWL theorem: regress residual W_res on residual V_res
theta_hat = sum(W_res .* V_res) / sum(V_res .^ 2)

# Variance of score function (Neyman-orthogonal score)
psi = (W_res .- theta_hat .* V_res) .* V_res
var_theta = var(psi) / (mean(V_res .^ 2)^2)
se_theta = sqrt(var_theta / n_samples)

ci_lb = theta_hat - 1.96 * se_theta
ci_ub = theta_hat + 1.96 * se_theta
@printf "DML PLR Estimate = %.4g (SE = %.4g)\n" theta_hat se_theta
@printf "95%% CI           = [%.4g, %.4g]\n" ci_lb ci_ub
```

```
DML PLR Estimate = 0.192 (SE = 0.03206)
95% CI           = [0.1292, 0.2549]
```

9. Heterogeneous Treatment Effects with Machine Learning

```
using DataFrames
using Distributions
using Random
using Statistics
using LinearAlgebra
using Printf
using MLJ
using MLJDecisionTreeInterface
using GLM
using CairoMakie
CairoMakie.activate!(type = "png")
```

The estimands chapter defined the conditional average treatment effect

$$\mathrm{CATE}(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x] \tag{9.1}$$

as the effect of treatment for units with covariates $X = x$. In simulation we can compute CATE from known potential outcomes. In real data we observe only one potential outcome per unit, so CATE has to be estimated from (X, D, Y).

Here I focus on meta-learners: recipes that turn a regression method into a CATE estimator. I implement S-, T-, X-, R-, and DR-learners using `MLJ` and `DecisionTree.jl` random forests.

Julia does not currently have an equivalent of R's `grf::causal_forest` with honest sample splitting and CATE confidence intervals. For that workflow, use the R companion chapter.

9.1. The data-generating process

I use the same CATE function as the estimands chapter, $\tau(x) = 1 + 2x_1$ (the rest of the DGP differs: five covariates and confounding through the observed X_2).

```
Random.seed!(42)
n = 5000
p = 5
```

```
X    = rand(n, p)
# Treatment depends on the OBSERVED covariate X2 (a backdoor confounder we
# adjust for), so unconfoundedness given X holds and the estimators below are
# consistent for the true CATE/ATE.
ps   = @. 1 / (1 + exp(-(-0.3 + 1.5 * X[:, 2])))
D    = Float64.(rand(n) .< ps)
tau  = @. 1 + 2 * X[:, 1]
Y0   = @. 0.5 * X[:, 2] + randn()
Y1   = Y0 .+ tau
Y    = ifelse.(D .== 1, Y1, Y0)

@printf("n = %d, true ATE = %.3f\n", n, mean(tau))
@printf("True CATE at X1 = 0.2: %.2f\n", 1 + 2 * 0.2)
@printf("True CATE at X1 = 0.8: %.2f\n", 1 + 2 * 0.8)
```

```
n = 5000, true ATE = 2.009
True CATE at X1 = 0.2: 1.40
True CATE at X1 = 0.8: 2.60
```

Treatment is related to the observed covariate X_2, which also affects the outcome — a backdoor confounder. Because X_2 is observed and in the conditioning set, unconfoundedness holds and the estimators below are consistent for the true CATE/ATE.

9.2. A wrapper for fitting random forests

To keep the code short, define a helper that fits a random forest and returns predictions.

```
const RFR = @load RandomForestRegressor pkg=DecisionTree verbosity=0

"""
    rf_fit_predict(Xtrain, ytrain, Xpredict; n_trees=500, weights=nothing)
Fit a random forest on (Xtrain, ytrain) and return predictions at Xpredict.
DecisionTree's `RandomForestRegressor` does not accept per-sample weights, so
when `weights` are supplied we approximate a weighted fit by resampling the
training rows with probability proportional to the weights (weighted bootstrap).
"""
function rf_fit_predict(Xtrain, ytrain, Xpredict; n_trees::Int=500,
                        weights=nothing)
    if weights !== nothing
        w = Float64.(weights)
        m = size(Xtrain, 1)
        cdf = cumsum(w) ./ sum(w)            # weighted-bootstrap CDF
        idx = [searchsortedfirst(cdf, rand()) for _ in 1:m]
        Xtrain = Xtrain[idx, :]
        ytrain = ytrain[idx]
```

```
    end
    learner = RFR(n_trees=n_trees, max_depth=-1)
    Xtrain_t = MLJ.table(Xtrain)
    Xpred_t  = MLJ.table(Xpredict)
    mach = machine(learner, Xtrain_t, Float64.(ytrain))
    fit!(mach, verbosity=0)
    return MLJ.predict(mach, Xpred_t)
end

"""
    rf_oof(Xtrain, ytrain, Xpredict, folds; n_trees=500, weights=nothing, rows=...)

Out-of-fold version of `rf_fit_predict`. For each fold `k`, fit on the rows *not*
in `k` and predict only the fold-`k` rows of `Xpredict`. Restrict the training
rows further with `rows` (used below to fit an arm-specific model while still
predicting for everybody).

Every CATE estimate in the meta-learner comparison below goes through this
function rather than `rf_fit_predict`, and the reason matters. A random forest asked to predict
same rows it was trained on partly reproduces its own training targets, so an
in-sample CATE is contaminated by the noise in whatever pseudo-outcome it was fit
to. That inflates apparent accuracy for learners whose target is close to `Y`
and destroys it for learners whose target is a high-variance pseudo-outcome --
which is exactly the DR- and R-learners. It also makes any downstream *ranking*
on the estimate invalid; see the GATES section for what that does.
"""
function rf_oof(Xtrain, ytrain, Xpredict, folds; n_trees::Int=500,
                weights=nothing, rows=trues(length(folds)))
    out = zeros(size(Xpredict, 1))
    for k in unique(folds)
        train = (folds .!= k) .& rows
        test  = folds .== k
        w = weights === nothing ? nothing : Float64.(weights)[train]
        out[test] = rf_fit_predict(Xtrain[train, :], ytrain[train],
                                   Xpredict[test, :]; n_trees=n_trees, weights=w)
    end
    return out
end

# One fold assignment, shared by every learner below so that the comparison
# across learners is like-for-like.
Random.seed!(7)
folds = rand(1:5, n)
nothing
```

9.3. S-learner ("single")

Fit one regression of Y on (D, X), then predict the difference between $D = 1$ and $D = 0$ for each x:

```
X_with_D = hcat(D, X)
X1_test  = hcat(ones(n), X)
X0_test  = hcat(zeros(n), X)

mu1_S = rf_oof(X_with_D, Y, X1_test, folds)
mu0_S = rf_oof(X_with_D, Y, X0_test, folds)
tau_S = mu1_S .- mu0_S

@printf("S-learner CATE correlation with truth: %.3f\n", cor(tau_S, tau))
```

```
S-learner CATE correlation with truth: 0.884
```

The S-learner is simple. Its weakness is that the model may treat D as a minor predictor and shrink treatment effects toward zero.

9.4. T-learner ("two")

Fit two separate regressions, one on treated units and one on controls:

```
idx_T = D .== 1
idx_C = D .== 0

mu1_T = rf_oof(X, Y, X, folds; rows = idx_T)   # treated-arm model, predicted for all
mu0_T = rf_oof(X, Y, X, folds; rows = idx_C)   # control-arm model, predicted for all
tau_T = mu1_T .- mu0_T

@printf("T-learner CATE correlation with truth: %.3f\n", cor(tau_T, tau))
```

```
T-learner CATE correlation with truth: 0.882
```

The T-learner gives treatment and control separate outcome models. It can work well, but it can extrapolate badly when treated and control covariate distributions do not overlap.

9.5. X-learner (Künzel et al. 2019)

The X-learner starts from the T-learner. It imputes missing potential outcomes, creates pseudo-treatment effects, smooths them over X, and then combines the two arms with propensity-score weights.

```
# Step 1: pseudo-outcomes, stored full-length so the arm models can be fit
# out-of-fold with `rows` (entries outside each arm are never used for training)
D1_pseudo = zeros(n); D1_pseudo[idx_T] = Y[idx_T] .- mu0_T[idx_T]   # observed - imputed Y(0)
D0_pseudo = zeros(n); D0_pseudo[idx_C] = mu1_T[idx_C] .- Y[idx_C]   # imputed Y(1) - observed

# Step 2: regress pseudo-outcomes on X in each arm
tau_X1 = rf_oof(X, D1_pseudo, X, folds; rows = idx_T)
tau_X0 = rf_oof(X, D0_pseudo, X, folds; rows = idx_C)

# Step 3: weight by propensity score
e_hat = clamp.(rf_oof(X, D, X, folds), 0.02, 0.98)

tau_X = e_hat .* tau_X0 .+ (1 .- e_hat) .* tau_X1
@printf("X-learner CATE correlation with truth: %.3f\n", cor(tau_X, tau))
```

```
X-learner CATE correlation with truth: 0.928
```

The X-learner is useful when one treatment arm is much smaller than the other.

9.6. R-learner (Nie and Wager 2021)

The R-learner partials out the main effects of X from both Y and D:

$$\tilde{Y}_i = \frac{Y_i - \hat{m}(X_i)}{D_i - \hat{e}(X_i)}, \qquad \text{weight}_i = (D_i - \hat{e}(X_i))^2, \tag{9.2}$$

where $\hat{m}(x) = \mathbb{E}[Y \mid X = x]$ and $\hat{e}(x) = \mathbb{E}[D \mid X = x]$ are cross-fitted nuisance estimates.

```
# Nuisances cross-fitted on the shared folds defined in the helper chunk
m_hat    = rf_oof(X, Y, X, folds)
e_hat_cv = clamp.(rf_oof(X, D, X, folds), 0.02, 0.98)

pseudo_R   = (Y .- m_hat) ./ (D .- e_hat_cv)
weights_R  = (D .- e_hat_cv) .^ 2

# The R-learner is a *weighted* regression of pseudo_R on X with weights
# (D - e_hat)^2. DecisionTree's RF does not accept per-sample weights, so we
# pass them through the weighted-bootstrap path of `rf_fit_predict` (reached here
# via `rf_oof`, which forwards `weights`). This downweights
```

```
# observations with D close to e_hat, whose pseudo_R blows up (division by a
# near-zero denominator) and would otherwise dominate the split criterion.
tau_R = rf_oof(X, pseudo_R, X, folds; weights = weights_R)
@printf("R-learner CATE correlation with truth: %.3f\n", cor(tau_R, tau))
```

```
R-learner CATE correlation with truth: 0.714
```

The R-learner is often stable when overlap is reasonable because nuisance model errors have only second-order effects on the target.

9.7. DR-learner (Kennedy 2020)

The DR-learner uses an AIPW-style pseudo-outcome and then regresses it on X:

$$\tilde{Y}_i^{DR} = \hat{\mu}_1(X_i) - \hat{\mu}_0(X_i) + \frac{D_i(Y_i - \hat{\mu}_1(X_i))}{\hat{e}(X_i)} - \frac{(1 - D_i)(Y_i - \hat{\mu}_0(X_i))}{1 - \hat{e}(X_i)}. \tag{9.3}$$

```
mu1_cf = rf_oof(X, Y, X, folds; rows = D .== 1)
mu0_cf = rf_oof(X, Y, X, folds; rows = D .== 0)
e_cf   = clamp.(rf_oof(X, D, X, folds), 0.02, 0.98)

pseudo_DR = @. (mu1_cf - mu0_cf) +
               D * (Y - mu1_cf) / e_cf -
               (1 - D) * (Y - mu0_cf) / (1 - e_cf)

tau_DR = rf_oof(X, pseudo_DR, X, folds)
@printf("DR-learner CATE correlation with truth: %.3f\n", cor(tau_DR, tau))
```

```
DR-learner CATE correlation with truth: 0.769
```

9.8. Comparing all five estimators

```
fig = Figure(size = (1000, 700))
labs = ["S-learner", "T-learner", "X-learner", "R-learner", "DR-learner"]
preds = [tau_S, tau_T, tau_X, tau_R, tau_DR]

for (i, (lab, pred)) in enumerate(zip(labs, preds))
    row, col = divrem(i - 1, 3) .+ (1, 1)
    ax = Axis(fig[row, col],
              xlabel = "X1", ylabel = "Estimated CATE",
              title = lab)
```

```
    scatter!(ax, X[:, 1], pred, color = (:steelblue, 0.2), markersize = 4)
    lines!(ax, 0:0.01:1, x -> 1 + 2x, color = :firebrick, linestyle = :dash,
           linewidth = 2)
end
fig
```

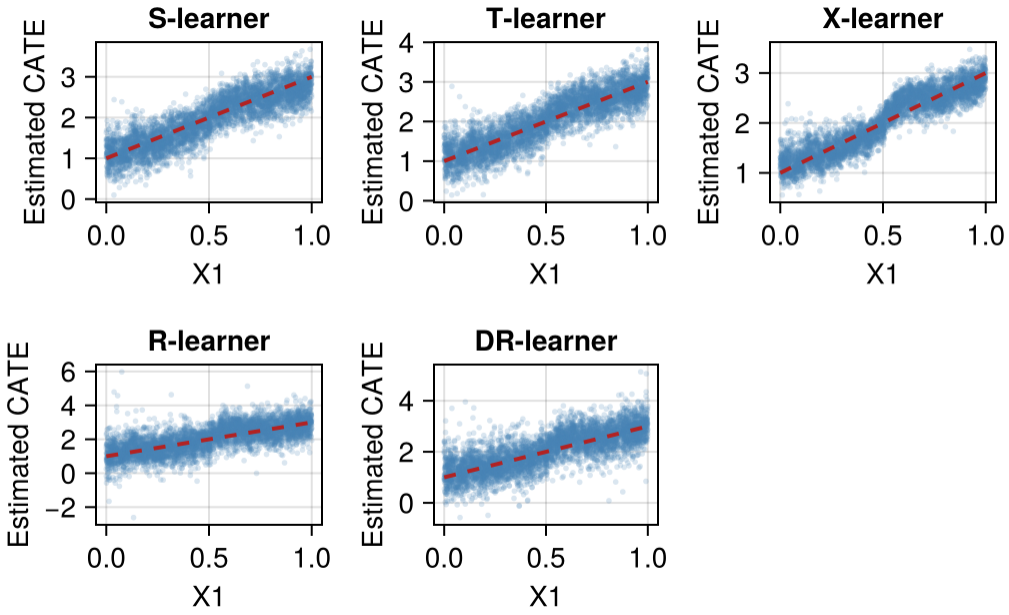


Figure 9.1.: Estimated CATE vs true $\tau(x) = 1 + 2x$ for each meta-learner. Red dashed line = ground truth.

9.9. Best Linear Projection (BLP)

Even if CATE is nonlinear, we often want a regression-style summary: which covariates are associated with larger effects? The best linear projection is the OLS regression of pseudo-outcomes on covariates:

$$(\beta_0^*, \beta^*) = \arg\min_{\beta_0,\beta} \mathbb{E}\left[(\tau(X) - \beta_0 - X'\beta)^2\right], \qquad \mathrm{BLP}(X) = \beta_0^* + X'\beta^*. \tag{9.4}$$

The arg min returns the coefficient vector (β_0^*, β^*); the best linear projection itself is the fitted function $\beta_0^* + X'\beta^*$.

A doubly-robust BLP uses the DR-learner pseudo-outcomes:

```
blp_df = DataFrame(hcat(pseudo_DR, X), [:tau_pseudo, :X1, :X2, :X3, :X4, :X5])
blp_fit = lm(@formula(tau_pseudo ~ X1 + X2 + X3 + X4 + X5), blp_df)
```

```
# `display`, not `println`. StatsBase defines only
# show(io, ::MIME"text/plain", ::CoefTable), so println() falls through to the
# generic struct show and prints the raw constructor -- `CoefTable(Any[[...]])`
# -- instead of the formatted table.
display(coeftable(blp_fit))
```

	Coef.	Std. Error	t	Pr(>	t	)
(Intercept)	1.03165	0.135462	7.62	<1e-13	0.76609	1.29722
X1	2.05583	0.116642	17.63	<1e-66	1.82716	2.28449
X2	-0.0285376	0.117208	-0.24	0.8076	-0.258318	0.201242
X3	-0.0205871	0.118377	-0.17	0.8619	-0.252657	0.211483
X4	-0.0204671	0.118625	-0.17	0.8630	-0.253025	0.212091
X5	0.0117126	0.118624	0.10	0.9214	-0.220842	0.244268

Here the coefficient on X_1 should be close to 2, and the coefficients on the other variables should be close to 0.

9.10. GATES: sorted group ATEs

GATES (group average treatment effects) is a simple way to report heterogeneity (Chernozhukov et al. 2018). Sort observations by predicted CATE, split them into bins, and estimate the ATE in each bin. (The same paper's CLAN — classification analysis — is the natural companion: compare average *covariates* between the most- and least-affected bins; here that would show high X_1 in the top quintile.)

```
nq = 5

# The ranking score MUST be out-of-fold, which `tau_DR` now is (it comes from
# `rf_oof`). This is not a technicality. When this chapter ranked on an in-sample
# forest prediction -- fit on (X, pseudo_DR) and predicted back at the same X --
# the bins sorted on pseudo_DR's own noise and then averaged that same noise, so
# the top bin collected positive noise and the bottom bin negative noise. The
# table ran from -1.22 to 5.28, entirely outside the true CATE range of [1, 3],
# with tight confidence intervals around impossible values.
tau_rank = tau_DR   # already out-of-fold, from rf_oof above

quintile_edges = quantile(tau_rank, range(0, 1, length = nq + 1))
quintiles      = searchsortedfirst.(Ref(quintile_edges), tau_rank) .- 1
quintiles      = clamp.(quintiles, 1, nq)

# AIPW-style ATE within each quintile using the DR pseudo-outcomes
function quintile_ate(tau_pseudo, idx)
    n_q = sum(idx)
    est = mean(tau_pseudo[idx])
```

```
    sd  = std(tau_pseudo[idx]) / sqrt(n_q)
    return (est = est, se = sd)
end

clan_df = DataFrame(quintile = 1:nq,
                    ATE = [quintile_ate(pseudo_DR, quintiles .== q).est for q in 1:nq],
                    SE  = [quintile_ate(pseudo_DR, quintiles .== q).se  for q in 1:nq],
                    # This is a simulation, so report the true within-bin average of
                    # tau alongside the estimate. It makes the table self-checking:
                    # every entry must lie in [1, 3] here, and Truth is the column to
                    # compare ATE against.
                    Truth = [mean(tau[quintiles .== q]) for q in 1:nq])
clan_df.lo = clan_df.ATE .- 1.96 .* clan_df.SE
clan_df.hi = clan_df.ATE .+ 1.96 .* clan_df.SE

@printf("%-9s %8s %7s %8s %8s %8s\n", "Quintile", "ATE", "SE", "95% LB", "95% UB", "Truth")
for row in eachrow(clan_df)
    @printf("%-9d %8.3f %7.3f %8.3f %8.3f %8.3f\n",
            row.quintile, row.ATE, row.SE, row.lo, row.hi, row.Truth)
end
```

```
Quintile       ATE      SE   95% LB   95% UB    Truth
1            1.350   0.075    1.204    1.497    1.393
2            1.602   0.081    1.443    1.760    1.620
3            2.220   0.081    2.061    2.379    2.041
4            2.417   0.074    2.273    2.561    2.388
5            2.612   0.075    2.464    2.759    2.604
```

Quintile 5 has a larger ATE than quintile 1, and because the ranking score is out-of-fold the magnitudes are trustworthy too: compare the `ATE` column against `Truth`, the actual within-bin average of $\tau(x) = 1 + 2X_1$. Both halves of the procedure — forming the groups and estimating the group means — now use predictions from models that did not see the observation being scored, since `pseudo_DR` is built from cross-fitted nuisances and `tau_DR` comes from `rf_oof`.

Do not expect the intervals to have exact nominal coverage even so, and the table shows why it is useful to print `Truth`: the reported standard errors treat the bin membership as fixed, when in fact the *boundaries* were estimated from the same data, and they take no account of the uncertainty in `tau_DR` itself. One quintile here has a truth just outside its interval, which is about what that omission would predict. The estimates are close enough to be read as a heterogeneity profile; the intervals are indicative rather than exact. For inference you would want the groups formed on a genuinely held-out split, or `grf`'s `rank_average_treatment_effect`.

This is worth insisting on because the failure is silent. Ranking on an in-sample score leaves the group means unbiased *on average* — they still average to the ATE — while making the spread meaningless, so nothing in the output looks wrong except the numbers themselves. A useful habit is to sanity-check group ATEs against the range the estimand can possibly take: here every quintile mean must lie in $[1, 3]$, and a table with a negative bottom bin is refuted before you read its standard errors.

9.11. Variable importance — which covariate drives heterogeneity?

A simple diagnostic is to regress the DR pseudo-outcome on each covariate separately and compare R^2. It answers a different question from the BLP, and the two are easy to run together and confuse.

The BLP is a *joint* projection with inference attached. Its target, Equation 9.4, is defined without reference to any fitted object: it is the best linear approximation to $\tau(x)$ using all covariates at once, it comes in the units of the outcome, it has a sign, and `coeftable` gives it a standard error we can test against zero. The importance measure below is *marginal* and descriptive: one univariate R^2 per covariate, with no units, no sign, and no standard error.

The difference that matters in practice is joint versus marginal. A covariate that is merely correlated with the true effect modifier picks up a high univariate R^2, because on its own it does predict the pseudo-outcome, while its BLP coefficient is near zero once the real modifier is in the regression. The projection splits shared variation between correlated covariates; the marginal measure gives each of them full credit for it. Here the five covariates are drawn independently, so the two agree.

Both share one blind spot worth naming, since it is not obvious: this importance measure is itself a *linear* fit, so a covariate that modifies the effect non-monotonically — large effects at both ends of its range, small in the middle — scores near zero on both the BLP and the R^2. A tree-based importance of the kind `grf` reports would flag it, because trees can split on it. Neither diagnostic here can.

```
function single_var_r2(target, x)
    df_one = DataFrame(t = target, x = x)
    fit = lm(@formula(t ~ x), df_one)
    1 - sum(abs2.(residuals(fit))) / sum(abs2.(target .- mean(target)))
end

vi = [single_var_r2(pseudo_DR, X[:, j]) for j in 1:p]
vi_df = DataFrame(variable = ["X$j" for j in 1:p], R2 = vi)
sort!(vi_df, :R2, rev = true)
println(vi_df)
```

```
5×2 DataFrame
 Row   variable  R2
       String    Float64

   1   X1        0.0585715
   2   X4        2.32397e-5
   3   X5        1.20668e-5
   4   X2        2.02436e-6
   5   X3        5.36056e-7
```

X_1 should be the most important variable in this simulation.

9.12. Policy learning: who should we treat?

A CATE estimate is not yet a policy. If treatment has a cost, the policy question is who should be treated. Define a treatment cost c in outcome units:

$$\pi^*(x) = \mathbb{1}\{\tau(x) > c\}. \tag{9.5}$$

For interpretability, we can restrict the policy to a single threshold rule.

```
# cost = 2 makes the problem non-degenerate: tau(x) = 1 + 2 x1 is in [1, 3],
# so the optimal rule is "treat iff x1 > 0.5" (about half the population).
# A cost below 1 would make treat-everyone optimal and there would be
# nothing for a policy to learn.
cost = 2.0

# Welfare of a rule = average gain over treating nobody, evaluated with the
# DR scores: mean over units of treat(x) * (pseudo_DR - cost)
welfare(treat) = mean(treat .* (pseudo_DR .- cost))

welfare_all  = welfare(trues(n))            # treat everyone
treat_est    = tau_DR .> cost               # rule from the ESTIMATED CATE
treat_true   = tau .> cost                  # oracle rule (simulation only)

# Simple threshold-rule family on X1
threshes = 0:0.05:1
welfares = [welfare(X[:, 1] .> t) for t in threshes]
best_t   = threshes[argmax(welfares)]

@printf("Treatment rates: estimated rule %.2f, oracle rule %.2f\n",
        mean(treat_est), mean(treat_true))
@printf("Welfare (treat everyone):                    %.3f\n", welfare_all)
@printf("Welfare (oracle rule  (x) > %.0f):            %.3f\n", cost, welfare(treat_true))
@printf("Welfare (best X1-threshold rule, X1 > %.2f): %.3f\n",
        best_t, maximum(welfares))
@printf("Welfare (^-rule, SAME scores, biased):       %.3f\n", welfare(treat_est))
@printf("Welfare (^-rule, evaluated on true  ):       %.3f\n",
        mean(treat_est .* (tau .- cost)))
```

```
Treatment rates: estimated rule 0.53, oracle rule 0.51
Welfare (treat everyone):                    0.040
Welfare (oracle rule  (x) > 2):            0.300
Welfare (best X1-threshold rule, X1 > 0.45): 0.301
Welfare (^-rule, SAME scores, biased):       0.261
Welfare (^-rule, evaluated on true  ):       0.215
```

The threshold rule gives up flexibility, but it is easy to explain — and it recovers the oracle threshold of 0.5 almost exactly.

One number above deserves a warning. The $\hat{\tau}$-rule "evaluated" with the same DR scores that selected it appears to beat even the oracle rule — which is impossible in expectation. Selecting units whose noisy score is high and then averaging those same scores is optimistic by construction. In a simulation we can evaluate the rule against the true $\tau(x)$ instead (the last line): the honest value is positive — better than treating everyone — but well below the oracle, because the noisy CATE estimates misclassify many units near the threshold. Both lessons matter: never evaluate a rule on the scores that chose it (in real data, estimate the rule and evaluate its welfare on separate folds), and do not expect an estimated rule to attain oracle welfare. Here the simple X_1-threshold rule, which searches a small one-dimensional family, gets essentially the oracle value — restricting the policy class is a form of regularisation.

For more on policy trees with IPW and AIPW losses (using R's `policytree` package), see the companion blog chapter on `policytree`. For a cross-software comparison of CATE estimators (including Stata 19's new `cate` command), see the Stata CATE blog chapter.

9.13. Causal forests with panel data

With panel data, unit effects can be correlated with treatment and covariates. A cross-sectional meta-learner can then be biased. The fixed effect adjustment is to demean by unit before applying the learner.

```
Random.seed!(2024)
n_firms = 200
n_t     = 5
N       = n_firms * n_t

firm_id  = repeat(1:n_firms, inner = n_t)
unit_fe  = randn(n_firms) .* 1.5
V1_firm  = randn(n_firms) .* 1.0
V1_panel = V1_firm[firm_id] .+ 0.3 .* randn(N)
# The unit effect drives BOTH treatment and the outcome (classic
# fixed-effect confounding); unit_fe is unobserved to the learner.
W_panel  = Float64.(rand(N) .<
           @. 1 / (1 + exp(-(0.3 * V1_firm[firm_id] + 0.5 * unit_fe[firm_id]))))
tau_panel = @. 0.5 + 1.0 * V1_panel
Y_panel  = unit_fe[firm_id] .+ V1_panel .+ tau_panel .* W_panel .+ randn(N)

df_panel = DataFrame(firm = firm_id, V1 = V1_panel, W = W_panel, Y = Y_panel)
# Y(1) - Y(0) = tau_panel exactly, so the true ATE is mean(tau_panel).
true_panel_ate = mean(tau_panel)
@printf("True panel ATE: %.3f\n", true_panel_ate)
```

```
True panel ATE: 0.655
```

A naive T-learner ignores the firm fixed effects. Because `unit_fe` raises both the treatment probability and the outcome, and is not in the learner's covariates, the naive estimate is badly biased upward:

```
X_panel_naive = reshape(df_panel.V1, N, 1)
idx_T_p = df_panel.W .== 1
idx_C_p = df_panel.W .== 0

# Note these panel fits are in-sample, unlike the meta-learners above. Splitting
# out-of-fold here would have to split by *firm* rather than by row, since two
# observations of the same firm are not independent. The comparison this section
# makes -- naive against within-transformed -- is driven by fixed-effect
# confounding, which is far larger than any in-sample optimism, so the point
# survives; do not read these two numbers as clean CATE accuracy figures.
mu1_naive = rf_fit_predict(X_panel_naive[idx_T_p, :], df_panel.Y[idx_T_p],
                           X_panel_naive)
mu0_naive = rf_fit_predict(X_panel_naive[idx_C_p, :], df_panel.Y[idx_C_p],
                           X_panel_naive)
tau_naive = mu1_naive .- mu0_naive

@printf("Naive panel T-learner ATE: %.3f  (true = %.3f)\n",
        mean(tau_naive), true_panel_ate)
```

```
Naive panel T-learner ATE: 2.014  (true = 0.655)
```

The within transformation removes firm-level variation and leaves the within-firm variation:

```
# Within transformation: subtract firm means
df_dm = combine(groupby(df_panel, :firm),
                :Y  => (y -> y .- mean(y)) => :Y_dm,
                :W  => (w -> w .- mean(w)) => :W_dm,
                :V1 => (v -> v .- mean(v)) => :V1_dm)

X_dm = reshape(df_dm.V1_dm, N, 1)
idx_T_dm = df_dm.W_dm .> 0
idx_C_dm = df_dm.W_dm .<= 0

mu1_dm = rf_fit_predict(X_dm[idx_T_dm, :], df_dm.Y_dm[idx_T_dm], X_dm)
mu0_dm = rf_fit_predict(X_dm[idx_C_dm, :], df_dm.Y_dm[idx_C_dm], X_dm)
tau_dm = mu1_dm .- mu0_dm

# The two demeaned groups differ in W_dm by less than a full 0 -> 1 switch
# (treated-above-average vs below-average within firm), so the raw contrast
# is attenuated; rescale by the W_dm gap, as in a Wald estimator.
w_gap = mean(df_dm.W_dm[idx_T_dm]) - mean(df_dm.W_dm[idx_C_dm])
@printf("Within-transform T-learner ATE: %.3f  (true = %.3f)\n",
        mean(tau_dm) / w_gap, true_panel_ate)
```

```
Within-transform T-learner ATE: 0.689  (true = 0.655)
```

The within transformation removes the fixed effects, and the rescaled contrast lands near the truth in this DGP. The cost is variance, because identification now comes from within-firm variation.

A caveat on what this estimator targets. Splitting on $W_{dm} > 0$ vs $W_{dm} \leq 0$ and dividing the T-learner contrast by the average W_{dm} gap is *not* a general fixed-effect CATE estimator. It is a heuristic within-firm contrast whose target depends on the distribution of demeaned treatment values and on the grouping rule, and the rescaling is a Wald-style approximation rather than an identified within estimand. It works here because the treatment effect is linear in V_1 and the DGP is benign. Properly identified heterogeneous panel effects require a proper orthogonal score or a dedicated panel causal-forest / DML construction; treat this section as intuition, not a turnkey method.

So how much does ignoring the panel cost? On this DGP the naive T-learner returns 2.014 against a true panel ATE of 0.655 — about three times the truth — while the within transform recovers 0.689. The magnitude is specific to this design, since it scales with how strongly the unit effect drives both treatment and outcome, but the direction is not: a unit effect that raises both leaves the naive estimate biased upward. Read those two numbers as the size of the confounding being removed, not as clean CATE accuracy figures, for the estimand reasons given just above.

For R's `grf::causal_forest`, pass `clusters = firm_id` so the honest sample split keeps each firm's observations together. Julia does not currently have an equivalent; see the companion blog chapter on causal forests in panel data and the R companion to this chapter for the GRF-based workflow.

9.14. Summary

- Meta-learners turn regression tools into CATE estimators.
- In Julia they can be built with `MLJ` random forests, but there is no full `grf` equivalent yet.
- BLP and GATES summarize heterogeneity in regression-style output.
- Policy learning turns CATE estimates into treatment rules.
- In applied work, compare at least two CATE estimators and report simple heterogeneity summaries, not only a CATE scatterplot.

10. Continuous and Multivalued Treatments

```
using DataFrames
using Distributions
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
```

So far most examples use a binary treatment. Many treatments are not binary: dose, dollars, hours, or a treatment with several levels. In those cases "the treatment effect" is not one number. We usually want a dose-response curve,

$$\mu(d) = \mathbb{E}[Y(d)], \quad d \in \mathcal{D}. \tag{10.1}$$

Here I implement two approaches directly in Julia: the generalized propensity score (Hirano-Imbens 2004) and a doubly-robust extension. For modified treatment policies and time-varying treatments, R's `lmtp` package is still the better practical tool; see the R companion chapter and the LMTP blog chapter.

10.1. Setup

```
Random.seed!(42)
n  = 2000
X1 = randn(n)
X2 = randn(n)
# Continuous treatment, depends on X1, X2
D  = @. 2.0 + 0.5 * X1 + 0.5 * X2 + randn()
# Dose-response: concave in D, with diminishing returns. Not "hump-shaped" over
# the observed range: the vertex is at d = 2/0.3 = 6.67, beyond the largest dose
# drawn below, so the slope 2 - 0.3d stays positive throughout the support and no
# downturn is identified from these data.
true_mu(d) = 1.0 + 2.0 * d - 0.15 * d^2
Y  = @. true_mu(D) + 0.3 * X1 - 0.3 * X2 + 0.5 * randn()
```

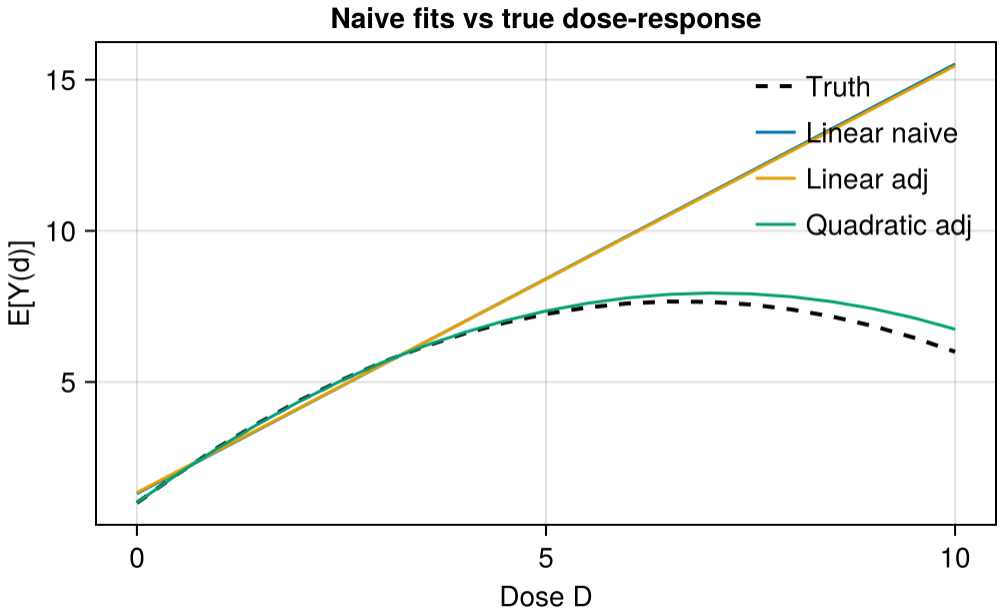
```
df = DataFrame(Y = Y, D = D, X1 = X1, X2 = X2)
@printf("Treatment range: [%.2f, %.2f]\n", minimum(D), maximum(D))
@printf("True  (d=2) = %.2f,  (d=4) = %.2f,  (d=6) = %.2f\n",
        true_mu(2), true_mu(4), true_mu(6))
```

```
Treatment range: [-2.01, 5.97]
True  (d=2) = 4.40,  (d=4) = 6.60,  (d=6) = 7.60
```

10.2. Naive regression vs adjusted regression

A linear fit on D alone has two problems: it ignores confounding by X, and it imposes a straight line on a nonlinear dose-response curve.

```
df.D2 = df.D .^ 2
linear_naive = lm(@formula(Y ~ D),                    df)
linear_adj   = lm(@formula(Y ~ D + X1 + X2),          df)
poly_adj     = lm(@formula(Y ~ D + D2 + X1 + X2),     df)

ds = 0.0:0.5:10.0

predict_mu(fit) = Float64.(predict(fit, DataFrame(D = collect(ds),
                                                  D2 = collect(ds) .^ 2,
                                                  X1 = 0, X2 = 0)))
fits = (
    linear_naive = predict_mu(linear_naive),
    linear_adj   = predict_mu(linear_adj),
    poly_adj     = predict_mu(poly_adj),
)
truth = true_mu.(ds)

fig = Figure(size = (700, 400))
ax = Axis(fig[1, 1], xlabel = "Dose D", ylabel = "E[Y(d)]",
          title = "Naive fits vs true dose-response")
lines!(ax, ds, truth, color = :black, linestyle = :dash,
       linewidth = 2, label = "Truth")
lines!(ax, ds, fits.linear_naive, label = "Linear naive")
lines!(ax, ds, fits.linear_adj,   label = "Linear adj")
lines!(ax, ds, fits.poly_adj,     label = "Quadratic adj")
axislegend(ax, position = :rt, framevisible = false)
fig
```



The quadratic adjusted model is better in this simulation because the true curve is quadratic. In real applications, the functional form is usually not known.

10.3. Generalized Propensity Score (Hirano-Imbens 2004)

The **generalized propensity score** is the conditional density of the treatment given covariates:

$$r(d, x) = f_{D|X}(d \mid x). \tag{10.2}$$

Two results from Hirano and Imbens (2004) are what make this object useful, and both are worth stating before we compute anything.

The first is the assumption they need, which is weaker than it looks. *Weak unconfoundedness* requires $Y(d) \perp D \mid X$ separately for each dose d. It does not require the whole path $\{Y(d) : d\}$ to be jointly independent of D given X, which would be a much stronger demand at a continuous treatment.

The second is the balancing property. Conditional on the scalar $r(d, X)$, the covariates carry no further information about whether a unit received dose d:

$$f_D(d \mid r(d, X)) = f_D(d \mid r(d, X),\, X). \tag{10.3}$$

So a one-dimensional score does the job of the full covariate vector, exactly as in the binary case. Put the two together and the dose-response function is

$$\mu(d) = E[Y(d)] = E\Big[\, E(Y \mid D = d,\; r(d, X))\Big], \tag{10.4}$$

where the outer expectation runs over the distribution of $r(d, X)$ in the whole sample. Read the argument of r carefully, because it is where this estimator is usually got wrong: at each counterfactual dose d we must recompute every unit's score *at that* d, not reuse the score at the dose the unit happened to receive. The `estimate_dose` function below does exactly that, which is why the GPS is recomputed inside it rather than taken from `df.gps`.

One more point of theory explains why the GPS enters the outcome model as a regressor rather than as a weight. The GPS is a conditional *density*, not a probability, so it carries the units of $1/D$; weights of the form $1/\hat{r}$ are therefore not scale-free and change if the dose is rescaled. Dividing by the marginal density of D would restore scale invariance, but Hirano and Imbens sidestep the issue altogether by putting the score in the outcome model.

In practice we start with a normal model for $D \mid X$, $D \mid X \sim \mathcal{N}(\beta_0 + \beta' X, \sigma^2)$. Fitting D on X_1 and X_2 by OLS gives $\hat{\beta}$ and $\hat{\sigma}$, and the score at any dose d for unit i is the normal density evaluated there,

$$\hat{r}(d, x_i) = \frac{1}{\hat{\sigma}\sqrt{2\pi}} \exp\left\{ -\frac{(d - x_i'\hat{\beta})^2}{2\hat{\sigma}^2} \right\}. \tag{10.5}$$

That is the whole calculation: one linear regression, then one density evaluation per unit per dose.

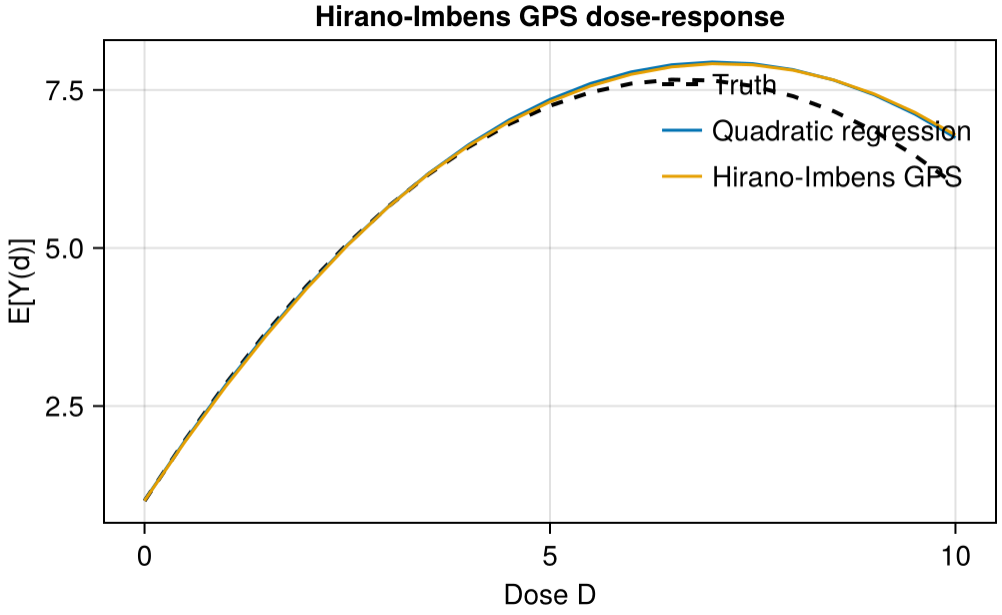
```
gps_model = lm(@formula(D ~ X1 + X2), df)
d_pred    = predict(gps_model)
sigma_e   = sqrt(sum(abs2.(residuals(gps_model))) / dof_residual(gps_model))

df.gps = @. pdf(Normal(d_pred, sigma_e), df.D)
@printf("GPS summary: min=%.4f, mean=%.4f, max=%.4f\n",
        minimum(df.gps), mean(df.gps), maximum(df.gps))
```

```
GPS summary: min=0.0009, mean=0.2806, max=0.3962
```

The Hirano-Imbens estimator has two steps. First regress Y on flexible functions of D and the estimated GPS. Then, for each dose d, average the predicted outcome over the sample.

```
# Stage 2: regress Y on flexible function of D and the GPS
df.gps2 = df.gps .^ 2
hi_fit = lm(@formula(Y ~ D + D2 + gps + gps2 + D & gps), df)

# Dose-response at each d: average over the empirical GPS distribution at that d
function estimate_dose(d)
    gps_at_d = @. pdf(Normal(d_pred, sigma_e), d)
    nd = DataFrame(D = fill(d, n), D2 = fill(d^2, n),
                   gps = gps_at_d, gps2 = gps_at_d .^ 2)
    mean(predict(hi_fit, nd))
end

hi_dose = [estimate_dose(d) for d in ds]
```

```
fig2 = Figure(size = (700, 400))
ax2 = Axis(fig2[1, 1], xlabel = "Dose D", ylabel = "E[Y(d)]",
           title = "Hirano-Imbens GPS dose-response")
lines!(ax2, ds, truth, color = :black, linestyle = :dash,
       linewidth = 2, label = "Truth")
lines!(ax2, ds, fits.poly_adj, label = "Quadratic regression")
lines!(ax2, ds, hi_dose, label = "Hirano-Imbens GPS")
axislegend(ax2, position = :rt, framevisible = false)
fig2
```



In this example the GPS estimator tracks the nonlinear curve much better than the naive linear regression.

10.4. Doubly-robust dose-response estimation

A kernel-localized AIPW estimator — in the spirit of Kennedy et al. (2017), who develop a more elaborate pseudo-outcome version with formal guarantees — is the continuous-treatment analogue of AIPW. It combines an outcome model $\hat{\mu}(d, x)$ with the GPS $\hat{r}(d \mid x)$:

$$\hat{\mu}_{DR}(d) = \frac{1}{n}\sum_i \hat{\mu}(d, X_i) + \sum_i \tilde{w}_i(d)\,(Y_i - \hat{\mu}(D_i, X_i)), \qquad \tilde{w}_i(d) = \frac{w_i(d)}{\sum_j w_j(d)}, \quad w_i(d) = \frac{K_h(D_i - d)}{\hat{r}(D_i \mid X_i)} \tag{10.6}$$

where K_h is a kernel around dose d. The code self-normalizes the kernel weights (a Hájek/stabilized form, $\sum_i \tilde{w}_i = 1$) rather than dividing by n, which stabilizes the correction term in finite samples.

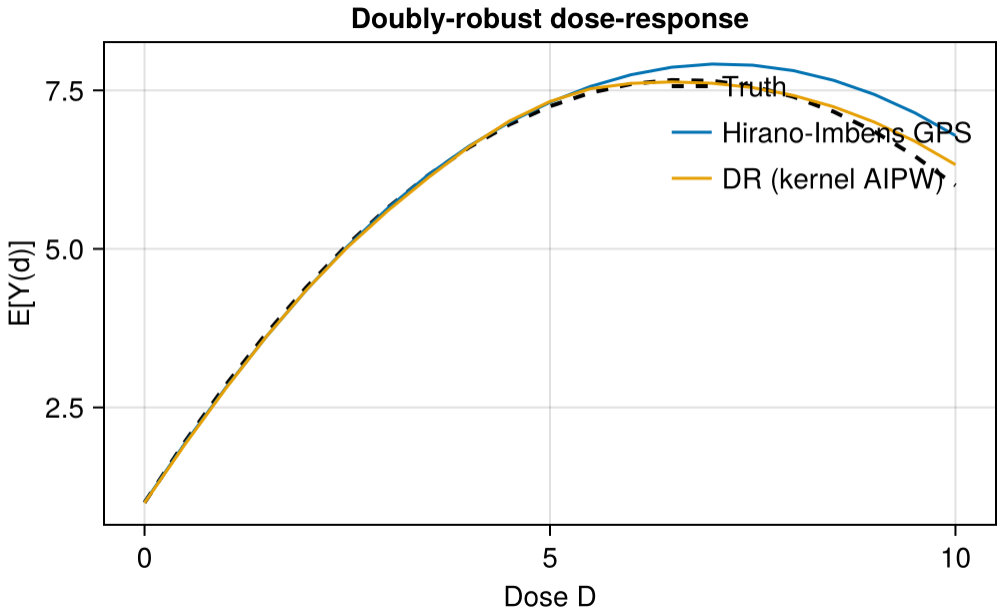
Two features of this estimator are worth understanding. The first is why it is doubly robust, and the algebra is the same as in the binary case worked through in Section 8.2.1: the first term is a plug-in built from the outcome model, the second subtracts that model's own residual reweighted by the score, so a first-order error in $\hat{\mu}$ enters twice with opposite signs and cancels, and what survives involves the *product* of the error in $\hat{\mu}$ and the error in $\hat{r}$. If either nuisance is right, the product is zero.

The second is why a bandwidth appears here when none appeared for the ATE. The ATE is a single number and is estimable at the $\sqrt{n}$ rate. But $\mu(d)$ at a *point* d is a regression function, and no $\sqrt{n}$ estimator of it exists without smoothness assumptions, because the data hold only finitely many observations near any given dose. The kernel is how we borrow from neighbouring doses, and h sets the exchange rate: too wide pulls in doses where the truth is different, too narrow leaves too few observations to average. Under the usual second-order smoothness the best trade is $h \propto n^{-1/5}$ and the rate is $n^{-2/5}$, slower than $\sqrt{n}$ by construction. Bandwidth sensitivity is therefore a cost of the method, not a tuning detail.

Kennedy et al. (2017) handle the localisation more carefully: instead of smoothing the correction term, they construct a pseudo-outcome whose conditional mean given $D = d$ is exactly $\mu(d)$, then run a single nonparametric regression of that pseudo-outcome on D. All the smoothing sits in one place, which is what lets them state formal guarantees. The version here keeps the doubly-robust structure and is easier to read, with weaker theory behind it.

```
# Outcome model: ^(D, X)
mu_fit = lm(@formula(Y ~ D + D2 + X1 + X2 + D & X1 + D & X2), df)

# Predict ^(D_i, X_i) for the observed treatments
mu_at_obs = predict(mu_fit)

# DR dose-response at a grid
function dr_estimate(d; h = 0.5)
    nd = DataFrame(D = fill(d, n), D2 = fill(d^2, n),
                   X1 = df.X1, X2 = df.X2)
    mu_at_d = predict(mu_fit, nd)
    kernel  = @. pdf(Normal(0, h), df.D - d)
    weight  = @. kernel / df.gps
    weight ./= mean(weight)
    correction = @. weight * (df.Y - mu_at_obs)
    mean(mu_at_d) + mean(correction)
end

dr_dose = [dr_estimate(d) for d in ds]

fig3 = Figure(size = (700, 400))
ax3 = Axis(fig3[1, 1], xlabel = "Dose D", ylabel = "E[Y(d)]",
           title = "Doubly-robust dose-response")
lines!(ax3, ds, truth, color = :black, linestyle = :dash,
       linewidth = 2, label = "Truth")
lines!(ax3, ds, hi_dose, label = "Hirano-Imbens GPS")
```

```
lines!(ax3, ds, dr_dose, label = "DR (kernel AIPW)")
axislegend(ax3, position = :rt, framevisible = false)
fig3
```



Read this self-normalized kernel correction as an *illustrative* kernel-AIPW construction, not a turnkey doubly robust theorem. The informal "consistent if either the outcome model or the GPS is correct" statement is the discrete-treatment intuition; for continuous treatments a formal doubly robust dose-response result additionally needs bandwidth, nuisance-convergence-rate, smoothness, and orthogonalization conditions (see Kennedy et al. on continuous-treatment DR estimators). The bandwidth h trades bias against variance.

10.5. Multivalued treatments

For a discrete multivalued treatment $D \in \{0, 1, \dots, K\}$, the GPS is a multinomial probability:

$$\hat{r}(d, x) = \hat{P}(D = d \mid X = x). \tag{10.7}$$

A multinomial regression supplies $\hat{r}$. The dose-response curve becomes the average outcome under each treatment level.

```
# Simulate a 4-level treatment
Random.seed!(99)
n_mv = 2000
X_mv = randn(n_mv, 2)
util  = hcat(zeros(n_mv),
             @.(0.2 + 0.5 * X_mv[:, 1]),
```

```
            @.(0.3 + 0.4 * X_mv[:, 2]),
            @.(0.5 + 0.3 * X_mv[:, 1] + 0.3 * X_mv[:, 2]))
# Softmax to get probabilities
exp_util = exp.(util)
prob_mv  = exp_util ./ sum(exp_util, dims = 2)
D_mv  = [rand(Distributions.Categorical(prob_mv[i, :])) - 1 for i in 1:n_mv]
Y_mv  = @. 0.5 * D_mv + 0.3 * X_mv[:, 1] - 0.2 * X_mv[:, 2] + randn()

# Unadjusted group means by treatment level (NOT causal: treatment assignment
# depends on X, and Y also depends on X, so these are confounded comparisons).
mv_df = DataFrame(Y = Y_mv, D = D_mv, X1 = X_mv[:, 1], X2 = X_mv[:, 2])
by_d = combine(groupby(mv_df, :D), :Y => mean => :y_mean, :Y => length => :n)
@printf("%-6s %12s %12s\n", "D", "Unadj Mean(Y)", "N")
for r in eachrow(by_d)
    @printf("%-6d %12.3f %12d\n", r.D, r.y_mean, r.n)
end
```

```
D      Unadj Mean(Y)            N
0            -0.022          406
1             0.622          486
2             0.879          521
3             1.569          587
```

The table above shows **unadjusted** group means, not causal dose-response estimates: because treatment assignment depends on X and Y also depends on X, the level-to-level differences are confounded. To recover causal contrasts one would weight by the multinomial GPS (or model the outcome). In practice, I would report adjusted contrasts between levels, such as $D = 1$ versus $D = 0$ or $D = 2$ versus $D = 1$.

10.6. When to reach for these methods

These methods are useful when:

1. Treatment is continuous or multivalued.
2. The research question is about the shape of the dose-response curve, not just one binary contrast.

For binary treatments, the standard Estimation and Heterogeneous Effects methods are sufficient.

For time-varying continuous treatments and modified-policy interventions, use LMTP in R for now. A Julia version would be useful, but this book does not need to reimplement it.

10.7. Summary

- Continuous and multivalued treatments usually require a dose-response curve, not a single coefficient.
- GPS extends propensity-score logic to continuous treatments by modeling the density of treatment given covariates.
- The kernel-AIPW doubly-robust estimator adds an outcome model to the GPS.
- For modified treatment policies and time-varying continuous treatments, use R's `lmtp` package.

11. Bayesian Causal Inference

```
using DataFrames
using Distributions
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
```

Bayesian methods return a posterior distribution, not only a point estimate and a standard error. For causal inference this is useful when we want uncertainty about individual treatment effects, policy values, or other derived quantities.

The two common models in this area are:

1. **BART for causal inference** (Hill 2011) — Bayesian Additive Regression Trees as a flexible outcome model.
2. **Bayesian Causal Forests** (BCF; Hahn-Murray-Carvalho 2020) — BART extended to separate prognostic and treatment-effect components.

Both have mature R packages (`BART`, `bcf`). Julia does not currently have a comparable BART/BCF workflow. So this chapter shows the basic Bayesian g-computation idea with a conjugate linear model, and points to R for the tree-based versions.

> **Related reading**: For a hands-on Bayesian model with `Numpyro`, see the Using Numpyro chapter in *Topics on Econometrics and Causal Inference*. The R companion gives the BART and BCF version.

11.1. Setup

```
Random.seed!(42)
n = 1500
p = 3

X = randn(n, p)
ps = @. 1 / (1 + exp(-(-0.3 + 0.5 * X[:, 1] + 0.3 * X[:, 2])))
```

```
D  = Float64.(rand(n) .< ps)
tau = @. 1 + 2 * X[:, 1]   # true CATE
Y0  = @. 0.5 * X[:, 2] + 0.3 * X[:, 3] + randn()
Y1  = Y0 .+ tau
Y   = ifelse.(D .== 1, Y1, Y0)

df = DataFrame(Y = Y, D = D, X1 = X[:, 1], X2 = X[:, 2], X3 = X[:, 3])
@printf("True ATE = %.3f\n", mean(tau))
```

```
True ATE = 0.901
```

11.2. Bayesian g-computation

The simplest Bayesian causal inference recipe is Bayesian g-computation:

1. Specify a Bayesian model for $\mathbb{E}[Y \mid D, X]$.
2. Sample from the posterior of the regression parameters.
3. For each posterior draw, compute counterfactual outcomes $\hat{Y}(1), \hat{Y}(0)$.
4. The posterior of the ATE is the distribution of the average difference across posterior draws.

A Bayesian linear regression with a flat prior is enough to show the idea:

```
"""
    bayes_linreg(X, y; n_samples=2000)
Bayesian linear regression with a Normal-Inverse-Gamma conjugate prior
(flat prior → the posterior is centered at the OLS estimates, with spread
close to the OLS standard errors). Returns `n_samples` posterior
draws of ( ,  ²).
"""
function bayes_linreg(X::Matrix, y::Vector; n_samples::Int = 2000)
    n, p = size(X)
    XtX_inv = inv(X' * X)
    beta_hat = XtX_inv * X' * y
    sse = sum(abs2.(y .- X * beta_hat))
    #  ² ~ InverseGamma((n-p)/2, sse/2)
     ²_post = [rand(InverseGamma((n - p) / 2, sse / 2)) for _ in 1:n_samples]
    #   |  ² ~ MultivariateNormal(beta_hat,  ² * XtX_inv)
    betas = zeros(n_samples, p)
    for i in 1:n_samples
        cov_mat =  ²_post[i] .* XtX_inv
        betas[i, :] = rand(MvNormal(beta_hat, Symmetric(cov_mat)))
    end
    return betas,  ²_post
end

# Fit on (D, X1, X2, X3, D*X1, D*X2, D*X3) - allow treatment effect heterogeneity
```

```
DesignMat = hcat(ones(n), df.D, df.X1, df.X2, df.X3,
                 df.D .* df.X1, df.D .* df.X2, df.D .* df.X3)
betas, σ²_draws = bayes_linreg(DesignMat, df.Y; n_samples = 2000)

# Counterfactual prediction matrices
X_treat   = hcat(ones(n), ones(n),  df.X1, df.X2, df.X3,
                 df.X1, df.X2, df.X3)           # D = 1, so D*X = X
X_control = hcat(ones(n), zeros(n), df.X1, df.X2, df.X3,
                 zeros(n), zeros(n), zeros(n))  # D = 0

# Posterior ITEs: n_samples x n
ite_post = (X_treat * betas')' .- (X_control * betas')'
# ATE posterior: average over n at each draw
ate_post = vec(mean(ite_post, dims = 2))

@printf("Bayesian g-computation:\n")
@printf("  Posterior mean ATE:  %.3f\n", mean(ate_post))
@printf("  Posterior SD:        %.3f\n", std(ate_post))
@printf("  95%% CrI:             [%.3f, %.3f]\n",
        quantile(ate_post, 0.025), quantile(ate_post, 0.975))
@printf("  True ATE:            %.3f\n", mean(tau))
```

```
Bayesian g-computation:
  Posterior mean ATE:  0.904
  Posterior SD:        0.056
  95% CrI:             [0.798, 1.016]
  True ATE:            0.901
```

With a flat prior, this credible interval is close to the usual frequentist interval. With informative priors, the posterior reflects both the prior and the data.

11.2.1. Individual-level credible intervals

```
ite_mean = vec(mean(ite_post, dims = 1))
ite_lo   = [quantile(ite_post[:, i], 0.025) for i in 1:n]
ite_hi   = [quantile(ite_post[:, i], 0.975) for i in 1:n]

# Sort by X1 for plotting
ord = sortperm(df.X1)

fig = Figure(size = (700, 400))
ax = Axis(fig[1, 1], xlabel = "X1", ylabel = "ITE",
          title = "Posterior ITEs vs truth")
band!(ax, df.X1[ord], ite_lo[ord], ite_hi[ord],
```

```
      color = (:steelblue, 0.2), label = "95% CrI")
scatter!(ax, df.X1, ite_mean, color = (:steelblue, 0.4), markersize = 3,
         label = "Posterior mean ITE")
lines!(ax, [-3, 3], 1 .+ 2 .* [-3, 3], color = :firebrick,
       linestyle = :dash, linewidth = 2, label = "True  (x)")
axislegend(ax, position = :rb, framevisible = false)
fig
```

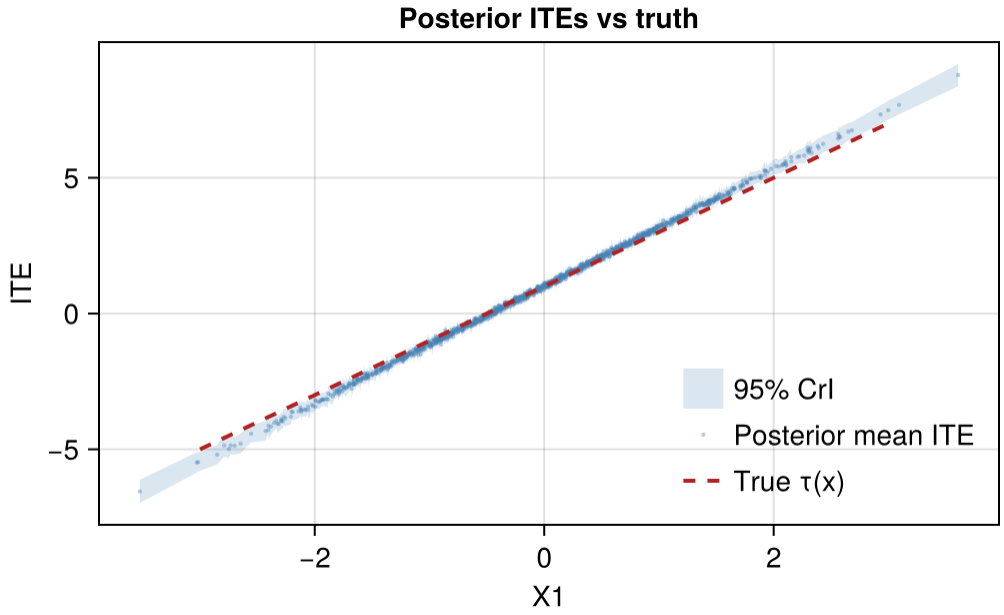


Figure 11.1.: Posterior mean ITE vs true (x) = 1 + 2 X . Blue band shows 95% pointwise credible interval across posterior draws.

The blue ribbon is the pointwise 95% credible interval. The red dashed line is the true treatment-effect function.

11.3. Why use BART/BCF in R instead?

The linear g-computation above is useful when:

- the outcome model can be approximated with a few interactions;
- the effect modifiers are known in advance.

For flexible Bayesian causal inference, R's `BART` and `bcf` are better tools. They use sums of regression trees and provide:

- flexible outcome models without manually specifying interactions;
- posterior draws of individual treatment effects;

- in BCF, separate priors for baseline prediction and treatment-effect heterogeneity.

Compare the rough sketch:

Capability	Julia (this chapter)	R (BART/BCF)
Bayesian point estimate of ATE		
Posterior credible intervals		
Flexible nonparametric outcome model	(limited to linear + interactions)	
Native ITE posterior with regularisation	(linear-model heterogeneity only)	
Separation of prognostic and treatment effects	–	(BCF)

For now, Julia users who need BART/BCF should call out to R. Reimplementing these models in Julia would be a separate package project.

11.4. When to use Bayesian methods

Reason to use Bayesian	Reason to use Frequentist
Prior information matters	Asymptotic theory is well-understood
Need full posterior for downstream decisions	Computational cost matters at scale
Small sample, weak identification	Effects are well-identified
Want individual-level credible intervals	Existing frequentist toolkit suffices

Bayesian methods are most useful when the posterior itself will be used: for treatment recommendations, policy values, or hierarchical models. If the target is just a well-identified ATE in a large sample, the frequentist tools are usually simpler.

11.5. Connections

- The Heterogeneous Treatment Effects chapter covers frequentist meta-learners. BCF is the Bayesian counterpart to causal forests.
- The Sensitivity Analysis chapter's Cinelli-Hazlett bounds are frequentist; the Bayesian analog puts a prior on the unmeasured-confounder strength (McCandless et al. 2007).
- For a hands-on Numpyro example of Bayesian modelling, see Using Numpyro.

11.6. Summary

- Bayesian g-computation computes counterfactual outcomes for each posterior draw.
- A conjugate linear model is easy to implement in Julia and is useful for exposition.
- For BART and BCF, use the R packages for now.
- For a Python/Numpyro Bayesian model, see the Using Numpyro chapter.

12. Distributional Treatment Effects

```
using DataFrames
using Distributions
using Random
using Statistics
using CairoMakie
CairoMakie.activate!(type = "png")
```

The previous chapters focus mostly on the average treatment effect. But many questions are not only about the mean. A policy can raise average earnings because high earners gain a lot, while low earners gain little. A clinical intervention can improve the median outcome but do nothing for the lower tail. To see this, we need distributional effects.

12.1. Beyond the average

The objects are the two marginal distributions $F_{Y(1)}$ and $F_{Y(0)}$: the outcome distribution if everyone were treated and if nobody were treated. From them we can compute quantile differences, density shifts, and inequality measures.

The most common summary is the **quantile treatment effect** (QTE):

$$\mathrm{QTE}(\tau) = F^{-1}_{Y(1)}(\tau) - F^{-1}_{Y(0)}(\tau), \quad \tau \in (0,1). \tag{12.1}$$

$\mathrm{QTE}(\tau)$ reports the difference between the τ-th quantile of the treated and control distributions. It is *not* the same as the average effect *at* a given pre-treatment quantile — QTE does not track individuals; it compares quantile to quantile across two marginal distributions.

Consider a treatment whose effect grows with the baseline outcome:

```
Random.seed!(42)
n = 5000

# Pre-treatment outcome (counterfactual under no treatment)
Y0 = rand(LogNormal(0, 0.6), n)

# Heterogeneous treatment effect: bigger at the top of the Y0 distribution
true_te = 0.2 .* Y0 .+ 0.1
Y1 = Y0 .+ true_te
```

```
ate     = mean(Y1 .- Y0)
qte_25  = quantile(Y1, 0.25) - quantile(Y0, 0.25)
qte_50  = quantile(Y1, 0.50) - quantile(Y0, 0.50)
qte_75  = quantile(Y1, 0.75) - quantile(Y0, 0.75)
qte_90  = quantile(Y1, 0.90) - quantile(Y0, 0.90)

@printf("ATE          = %.3f\n", ate)
@printf("QTE(0.25)    = %.3f\n", qte_25)
@printf("QTE(0.50)    = %.3f\n", qte_50)
@printf("QTE(0.75)    = %.3f\n", qte_75)
@printf("QTE(0.90)    = %.3f\n", qte_90)
```

```
ATE          = 0.339
QTE(0.25)    = 0.232
QTE(0.50)    = 0.296
QTE(0.75)    = 0.397
QTE(0.90)    = 0.535
```

The ATE is one number. The QTEs show that the effect is much larger near the top of the distribution.

```
 _grid = 0.05:0.025:0.95
qte_curve = [quantile(Y1,  ) - quantile(Y0,  ) for   in  _grid]

fig = Figure(size = (820, 360), fontsize = 13)

ax1 = Axis(fig[1, 1], xlabel = "Outcome Y", ylabel = "Density",
           title = "Outcome distributions")
density!(ax1, Y0, color = (:steelblue, 0.4), label = "Y(0)")
density!(ax1, Y1, color = (:firebrick, 0.4), label = "Y(1)")
axislegend(ax1, position = :rt, framevisible = false)

ax2 = Axis(fig[1, 2], xlabel = "Quantile  ", ylabel = "QTE( )",
           title = "Quantile treatment effects")
lines!(ax2, collect( _grid), qte_curve, color = :black, linewidth = 2)
hlines!(ax2, [ate], color = :gray40, linestyle = :dash)
text!(ax2, 0.05, ate + 0.02; text = "ATE", color = :gray40, fontsize = 11)

fig
```

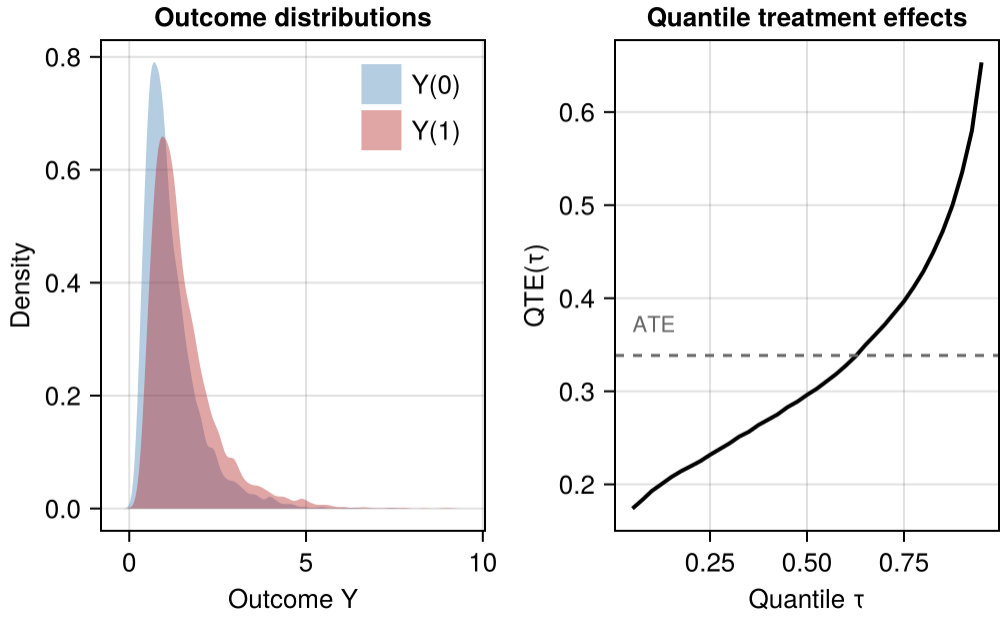


Figure 12.1.: Treated and control marginal distributions. The QTE curve traces the horizontal distance between the two CDFs at each quantile.

The treatment shifts and stretches the distribution. The QTE curve shows that low quantiles move little and high quantiles move more.

12.2. Identifying counterfactual distributions

QTE requires the two counterfactual marginal distributions. Observational data gives only $F_{Y|D=1}$ and $F_{Y|D=0}$. Identification needs the same kind of assumptions as ATE, but applied to the whole distribution.

Common approaches are:

Inverse propensity weighting on the CDF. Estimate the propensity score $\hat{\pi}(x)$ and weight observations to construct counterfactual empirical CDFs:

$$\hat{F}_{Y(d)}(y) = \frac{\sum_{i=1}^{n} w_i^{(d)} \mathbf{1}\{D_i = d\}\mathbf{1}\{Y_i \le y\}}{\sum_{i=1}^{n} w_i^{(d)} \mathbf{1}\{D_i = d\}}, \qquad w_i^{(d)} = \frac{1}{\hat{\pi}(x_i)^d (1-\hat{\pi}(x_i))^{1-d}}. \tag{12.2}$$

We self-normalize (divide by the sum of the weights rather than by n) so that $\hat{F}$ is a proper CDF that reaches 1; the unnormalized Horvitz–Thompson form tends to 1 only in expectation and need not reach 1 in any given sample, which destabilizes the upper quantiles. Then $\hat{\mathrm{QTE}}(\tau) = \hat{F}^{-1}_{Y(1)}(\tau) - \hat{F}^{-1}_{Y(0)}(\tau)$. This is the Firpo (2007) estimator for QTE under unconfoundedness.

Distribution regression. Model $P(Y \le y \mid X, D)$ as a flexible function of X for each threshold y (Chernozhukov, Fernández-Val, Melly 2013). Integrating out X gives the counterfactual marginal.

Distribution regression handles the entire distribution simultaneously rather than estimating each quantile separately.

Generative / engression approaches. Train a stochastic model that learns $Y \mid X, D$ as a distribution rather than a conditional mean. Sampling from the trained model gives counterfactual draws, from which any distributional summary follows. This is the route taken by `Engression.jl` and by `Endid.jl` for the DiD setting.

12.3. Distributional DiD with Engression

For panel data, distributional DiD extends the DiD idea from means to distributions. Instead of estimating only the counterfactual mean outcome, it estimates a counterfactual outcome distribution.

`Endid.jl` uses Engression for this. It learns a stochastic model for $Y \mid X$ rather than only a conditional mean. Counterfactual draws from that model give QTEs.

> **Note**
>
> The block below is **not executed** (`eval: false`). `Endid.jl` and its Engression backend are not part of this book's `Project.toml`, so the example will not run in the book's default environment. To run it, add `Endid` (and its dependencies) to a separate project environment first; see the `Endid.jl` repository for installation instructions.

```
using Endid

# Panel data: y outcome, id unit, time period, post 1 if post-treatment
# D = 1 marks treated units (optional; otherwise inferred from post)
res = endid(
    df, :y, :id, :time, :post;
    dvar       = :D,
    controls   = [:age, :income],
    nboot      = 100,
    num_epochs = 500,
    hidden_dim = 64,
)

println(res)   # ATT + bootstrap SE
res.qte        # DataFrame of QTE estimates at default quantiles 0.1:0.1:0.9
plot(res)      # QTE curve with confidence band
```

The `endid` function returns an `EndidResult` containing:

- `att` — the average treatment effect on the treated, with bootstrap SE and CI
- `qte` — a `DataFrame` of quantile treatment effects on the treated, one row per quantile in the grid (default `0.1:0.1:0.9`), each with bootstrap standard errors
- `model` — the trained Engression model (for further sampling / introspection)

- `design` — `"common_timing"` or `"staggered"` depending on which interface was used

For staggered adoption (different treatment times across cohorts), use `endid_staggered` with a `gvar` column carrying each unit's first-treatment period.

12.4. When QTE differs from ATT: a panel simulation

Now simulate panel data where the treatment effect depends on unit type:

```
Random.seed!(11)
n_units = 500
n_time  = 4
T0      = 2                # last pre-treatment period
treated_frac = 0.5
n_treated    = Int(round(n_units * treated_frac))

# Unit type (latent ability); drives both Y and the treatment effect size
ability = rand(Normal(0, 1), n_units)

# Build long panel
rows = NamedTuple[]
for i in 1:n_units, t in 1:n_time
    post = t > T0 ? 1 : 0
    treated = i <= n_treated ? 1 : 0
    # Heterogeneous TE: large at the top of the ability distribution
    te = (treated == 1 && post == 1) ? 0.4 * ability[i] + 0.5 : 0.0
    y = ability[i] + 0.3 * t + te + 0.4 * randn()
    push!(rows, (id = i, time = t, y = y, post = post, D = treated))
end
df = DataFrame(rows)
first(df, 5)
```

	id	time	y	post	D
	Int64	Int64	Float64	Int64	Int64
1	1	1	-0.946791	0	1
2	1	2	-0.253741	0	1
3	1	3	0.619691	1	1
4	1	4	1.32825	1	1
5	2	1	-0.603713	0	1

A standard ATT regression reports the mean effect. Here that hides the fact that low-type and high-type units have different effects.

```
# Compute empirical QTE on the simulated data by comparing treated post-period
# outcomes to a benchmark constructed from pre-period treated outcomes shifted
# by the control units' time trend (the standard DiD-style counterfactual,
# applied at each quantile rather than at the mean).
```

```
post_treated = df[(df.D .== 1) .& (df.post .== 1), :y]
pre_treated  = df[(df.D .== 1) .& (df.post .== 0), :y]
post_control = df[(df.D .== 0) .& (df.post .== 1), :y]
pre_control  = df[(df.D .== 0) .& (df.post .== 0), :y]

 _grid = 0.1:0.1:0.9
qte_did = [
    (quantile(post_treated,  ) - quantile(pre_treated,  )) -
    (quantile(post_control,  ) - quantile(pre_control,  ))
    for   in  _grid
]

result = DataFrame(quantile = collect( _grid),
                   QTE_DiD  = qte_did)
result
```

	quantile	QTE_DiD
	Float64	Float64
1	0.1	0.0542613
2	0.2	0.330757
3	0.3	0.282768
4	0.4	0.454995
5	0.5	0.441014
6	0.6	0.658351
7	0.7	0.697324
8	0.8	0.833802
9	0.9	1.02507

The curve below is a difference-in-quantiles: it subtracts quantile-by-quantile the treated change from the control change. This is a descriptive device, not an identified QTE. The difference of quantiles is not the quantile of the difference, so it equals the true QTE only under rank invariance.

Its *average*, though, is not a separate quantity: because $\int_0^1 F^{-1}(\tau)\,d\tau = E[Y]$, averaging this curve over the whole unit interval returns exactly the ordinary difference-in-means DiD. On the nine-point grid used here the average is 0.531 against a difference-in-means DiD of 0.538, and the gap closes as the grid is refined (0.534 on a 199-point grid, 0.536 on a 999-point grid) — it is discretization, not a difference in estimand. So the honest summary is that the quantile-DiD curve tells you nothing new about the *level* of the effect and everything about its *shape*: here, larger differences at the top of the outcome distribution.

```
fig = Figure(size = (640, 360), fontsize = 13)
ax  = Axis(fig[1, 1], xlabel = "Quantile  ", ylabel = "Difference-in-quantiles",
           title = "Difference-in-quantiles (descriptive quantile-DiD)")
lines!(ax, result.quantile, result.QTE_DiD, color = :firebrick, linewidth = 2)
scatter!(ax, result.quantile, result.QTE_DiD, color = :firebrick, markersize = 7)
hlines!(ax, [mean(result.QTE_DiD)], color = :gray40, linestyle = :dash)
text!(ax, 0.12, mean(result.QTE_DiD) + 0.02;
```

```
      text = "mean = difference-in-means DiD", color = :gray40, fontsize = 11)
fig
```

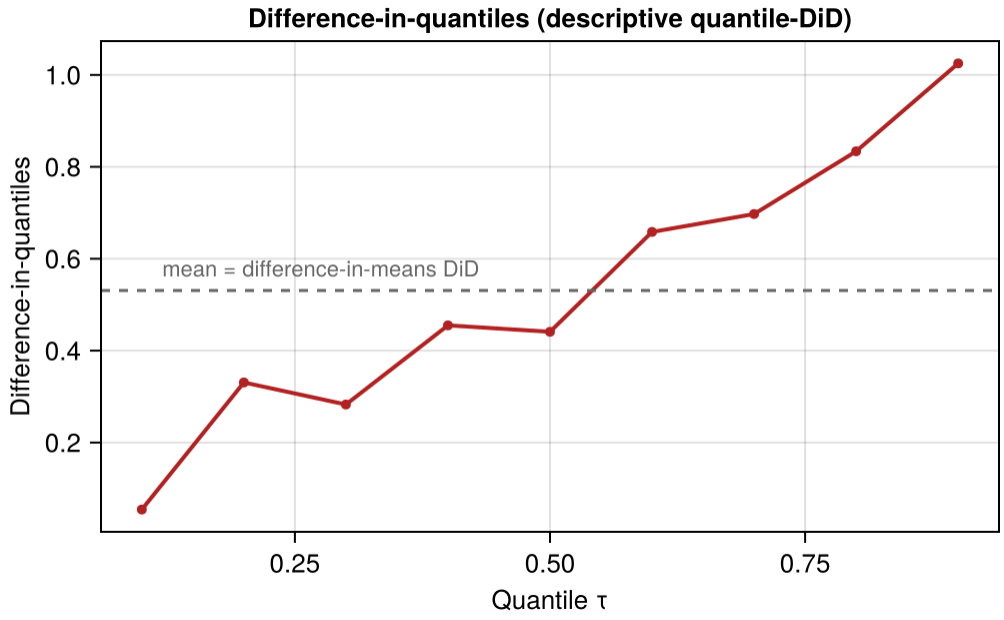


Figure 12.2.: Difference-in-quantiles (a descriptive quantile-DiD), computed at each quantile. This is not an identified QTE: the difference of quantiles equals the quantile of the difference, and so identifies the QTE, only under rank invariance. Its average, by contrast, *is* the ordinary difference-in-means DiD – averaging a quantile function returns a mean – so the information the curve adds is its shape, not its level.

The simple quantile-DiD above ignores covariate adjustment and bootstrap inference. `Endid.jl` adds both.

12.5. Beyond QTE: posterior bands as distributional output

TASC also produces more than a point estimate, but for a different object: the counterfactual path of one treated unit. QTE and TASC answer different questions:

- **QTE / Engression DiD**: how does the treatment effect *vary across the marginal distribution of outcomes* when many units are treated?
- **TASC posterior band**: how uncertain is the *single counterfactual path* when one unit is treated?

Use QTE when many treated units may have heterogeneous effects. Use TASC when the problem is a single treated unit with serial dependence.

12.6. Summary

- ATE summarizes the mean. QTE summarizes changes across the outcome distribution.
- QTE identification uses the same assumptions as ATE, but for CDFs.
- `Endid.jl` estimates distributional DiD through Engression.
- Report QTEs when the effect may differ across wages, test scores, health outcomes, sales, or other outcome distributions.

Part III.

Designs

13. Difference-in-Differences

```
using DataFrames
using DataFramesMeta
using Panelest
using Vcov
using StatsModels
import StatsAPI: coeftable
using ReadStatTables
using CSV
using SynthDiD
using CairoMakie
CairoMakie.activate!(type = "png")
using Statistics
using Printf
using RegressionTables
using Random
using Distributions
```

When unconfoundedness fails, we need weaker assumptions. The parallel trends assumption leads to the difference-in-differences estimator.

> **Related reading**: For a side-by-side comparison of DiD, synthetic control, synthetic DiD, and multivariate SC on a *single* panel — and how the choice of estimator changes the answer — see the Same data, different estimators chapter of *Topics on Econometrics and Causal Inference*.

13.1. $T = 2$ Panel Data

Suppose we have two periods, $t = (1, 2)$. Some units are treated just prior to period 2. For each individual i, there are four potential outcomes:

$$[Y_{i1}(0), Y_{i1}(1), Y_{i2}(0), Y_{i2}(1)] \tag{13.1}$$

Let D denote the treated group. The ATT in period 2 is

$$\tau_{2,att} = E(Y_2(1) - Y_2(0)|D = 1) \tag{13.2}$$

The first term is observed. The second is the counterfactual.

13.2. Parallel Trends Assumption

$$E[Y_2(0) - Y_1(0)|D = 1] = E[Y_2(0) - Y_1(0)|D = 0] \tag{13.3}$$

or

$$E[\Delta Y(0)|D = 1] = E[\Delta Y(0)|D = 0] \tag{13.4}$$

In words, the change in untreated potential outcomes between periods 1 and 2 is the same for treated and control groups.

13.3. No Anticipation Assumption

$$E[Y_1(1) - Y_1(0)|D = 1] = 0 \tag{13.5}$$

This is to say, in period 1, there is no treatment effect for the treated group.

13.4. DiD is Identified with PT and NA

With Parallel Trends, we notice

$$E[Y_2(0)|D = 1] = E[Y_1(0)|D = 1] + E[Y_2(0) - Y_1(0)|D = 0] \tag{13.6}$$

Then, with No Anticipation,

$$\begin{aligned} E[Y_2(0)|D = 1] &= E[Y_1(1)|D = 1] + E[Y_2(0) - Y_1(0)|D = 0] \\ &= E[Y_1|D = 1] + E[Y_2 - Y_1|D = 0] \end{aligned} \tag{13.7}$$

$$\tau_{2,att} = E[Y_2 - Y_1|D = 1] - E[Y_2 - Y_1|D = 0] \tag{13.8}$$

13.5. Advantages and Disadvantages of DiD

Advantages:

- No need to assume unconfoundedness. We "only" need PT and NA. Or say we don't need treatment itself to be independent of potential outcomes, but we do need it to be independent to the change in potential outcome at least for untreated state. This is importantly weaker in a lot of situations. There can be selection bias. If the selection bias does not change over time, then DiD can handle it.

Disadvantages:

- PT and NA might be violated.
- PT is not scale free, in the sense that even if outcome can have PT, but then non-linear transformation of outcome won't have PT (for example log of Y).

13.6. Traditional DiD

In practice, DiD in many period setting is usually done with

$$Y_{it} = \alpha_i + \phi_t + W_{it}\beta + \epsilon_{it} \tag{13.9}$$

here $W_{it} = (1[t = 2] \cdot D_i)$, which is the interaction of post-treatment indicator and treatment group indicator.

This is usually called Two Way Fixed Effect (TWFE). There are multiple ways to implement the same model in practice.

TWFE can be done by "Pooled OLS". That is, using OLS on time dummies and firm (individual) dummies. Wooldridge (2021) shows it's equivalent to use treatment dummies, instead of individual dummies. He also shows that this model can be equivalently implemented with fixed effect model and random effect model.

13.7. TWFE with Staggered Treatment Timing

The problem comes in when there is different timing of treatment. People used to still use

$$Y_{it} = \alpha_i + \phi_t + W_{it}\beta + \epsilon_{it} \tag{13.10}$$

where W_{it} now is a dummy when an individual i gets treated at time t.

However, what is β here?

13.8. Goodman-Bacon Decomposition

Goodman-Bacon (2021) showed that β in the TWFE is a weighted average of many different treatment effects, between treated cohorts, and control units, both can be different at different time points. The weights are a function of the size of the subsample, relative size of treatment and control units, and the timing of treatment in the sub sample. The decomposition weights themselves are non-negative and sum to one; the problem is that the units treated earlier are used as controls later. When those "forbidden" 2x2 comparisons are expressed in terms of the underlying cohort-time ATTs, some ATTs receive negative weight (de Chaisemartin and D'Haultfœuille 2020). Therefore there is no meaningful interpretation of β: it does not need to be a convex combination of treatment effects.

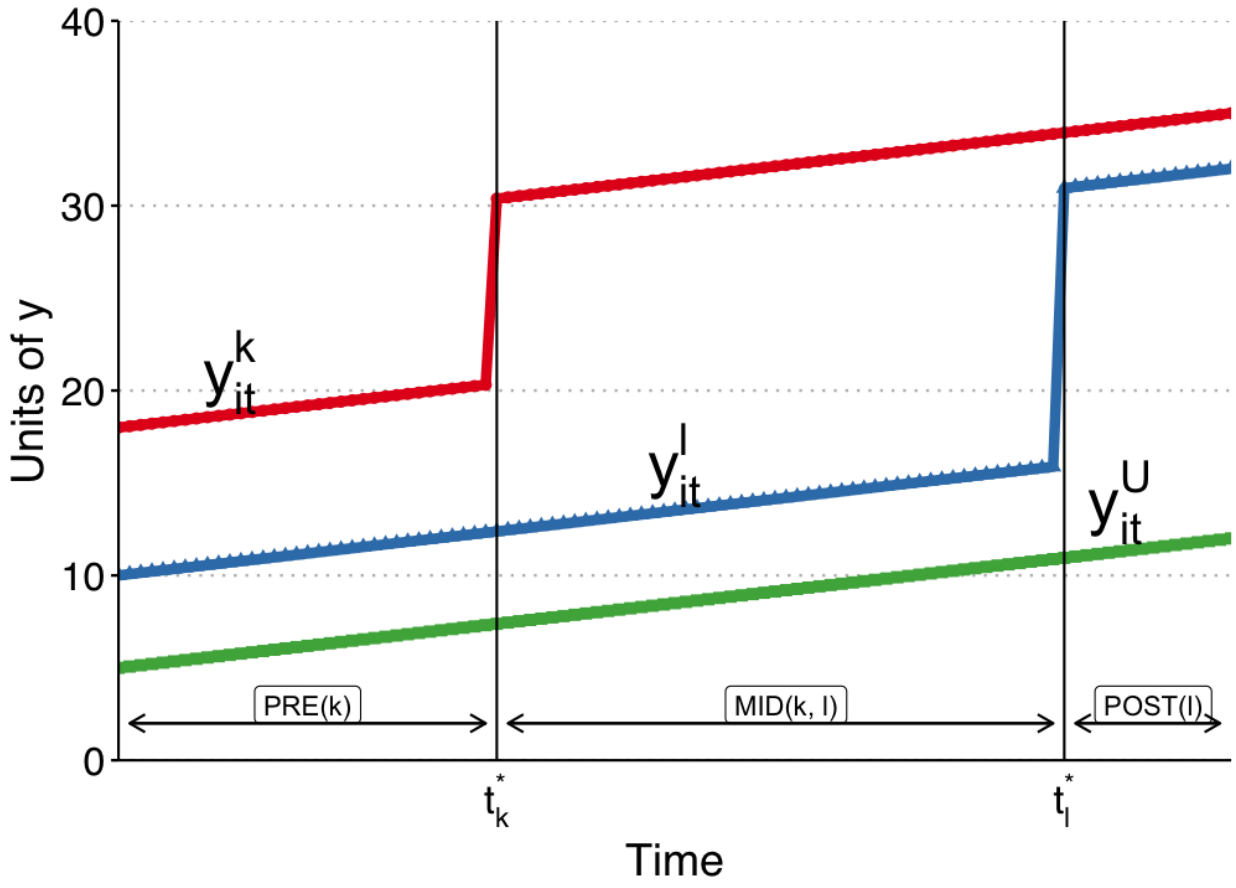


Figure 13.1.: Bacon decomposition

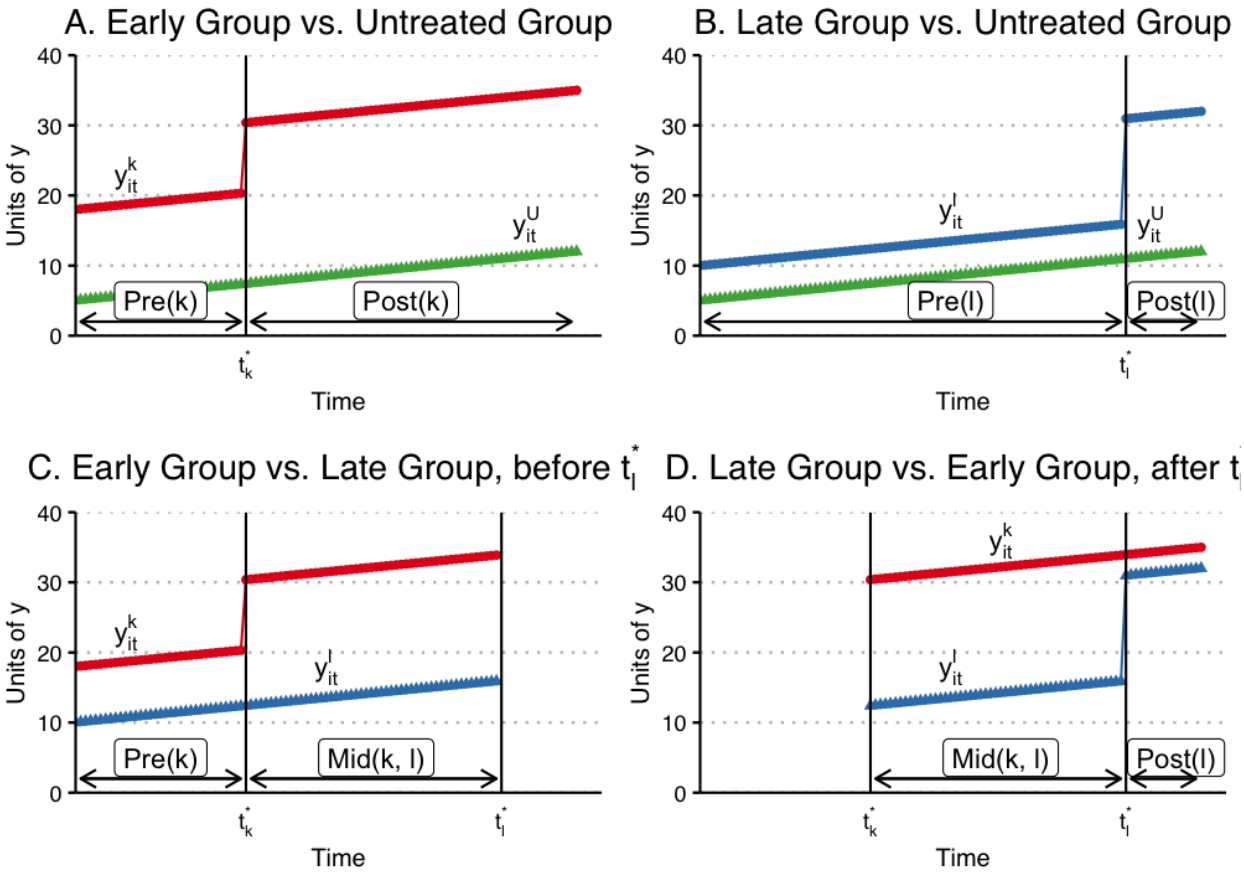


Figure 13.2.: Bacon decomposition

13.9. Wooldridge's ETWFE

There are a lot of ways to deal with staggered DiD situation. Wooldridge (2021) is basically saying: This is not a problem of TWFE, it's a mis-use of TWFE. The reason we get non-sensible result of β is that we know there is heterogeneous treatment effect, in the sense the treatment effect differs across cohort, but we force them to be the same. If we relax it, it can work. As he shows, this works when we specify cohort effects accordingly.

$$y_{it} = \alpha_i + \phi_t + \sum_{g=g_0}^{G} \sum_{t=g}^{T} \lambda_{g,t} \times 1(g,t) + \epsilon_{i,t} \tag{13.11}$$

Here g is a cohort indicator, a cohort is determined by the time of getting treatment. ETWFE is allowing each cohort to have different effect at each different time point after being treated. The baseline group is the never treated group. If there is no never treated group, it can easily changed to comparing to the last treated group.

13.10. Example 1: US Teen Employment

We'll use the mpdta dataset on US teen employment from the did package. "Treatment" in this dataset refers to an increase in the minimum wage rate. Our goal is to estimate the effect of this minimum wage treatment (treat) on the log of teen employment (lemp). Notice that the panel ID is at the county level (countyreal), but treatment was staggered across cohorts (first_treat) so that a group of counties were treated at the same time. In addition to these staggered treatment effects, we also observe log population (lpop) as a potential control variable.

```
# Load mpdta dataset from the did R package (Callaway & Sant'Anna 2021)
# US teen employment data; treatment = minimum wage increase
mpdta = CSV.read("data/mpdta.csv", DataFrame)
rename!(mpdta, "first.treat" => "first_treat")
first(mpdta, 6)
```

	year	countyreal	lpop	lemp	first_treat	treat
	Int64	Int64	Float64	Float64	Int64	Int64
1	2003	8001	5.89676	8.46147	2007	1
2	2004	8001	5.89676	8.33687	2007	1
3	2005	8001	5.89676	8.34022	2007	1
4	2006	8001	5.89676	8.37816	2007	1
5	2007	8001	5.89676	8.48735	2007	1
6	2003	8019	2.23238	4.99721	2007	1

```
combine(groupby(mpdta, :year), nrow)
combine(groupby(mpdta, :treat), nrow)
combine(groupby(mpdta, :first_treat), nrow)
```

	first_treat	nrow
	Int64	Int64
1	0	1545
2	2004	100
3	2006	200
4	2007	655

```
# String columns enable StatsModels to generate cohort×time dummies (ETWFE)
mpdta.first_treat_str = string.(mpdta.first_treat)
mpdta.year_str = string.(mpdta.year)

mod = feols(mpdta, @formula(lemp ~ lpop + first_treat_str & year_str + fe(countyreal) + fe(yea
```

```
show_regression_html(mod)
```

	Estimate	Std. Error	t value	Pr(>|t|)
lpop	-3.414e-33	2.694e-33	-1.267	0.205

	Estimate	Std. Error	t value	Pr(>|t|)
first_treat_str: 0 & year_str: 2003	-0.016	0.010	-1.594	0.111
first_treat_str: 2004 & year_str: 2003	0.048	0.016	2.992	0.003 **
first_treat_str: 2006 & year_str: 2003	-0.010	0.017	-0.611	0.541
first_treat_str: 2007 & year_str: 2003	-0.021	0.012	-1.802	0.072 .
first_treat_str: 0 & year_str: 2004	-0.023	0.008	-2.982	0.003 **
first_treat_str: 2004 & year_str: 2004	0.031	0.014	2.199	0.028 *
first_treat_str: 2006 & year_str: 2004	-0.011	0.010	-1.070	0.285
first_treat_str: 2007 & year_str: 2004	0.003	0.009	0.305	0.761
first_treat_str: 0 & year_str: 2005	-0.006	0.006	-0.979	0.328
first_treat_str: 2004 & year_str: 2005	-0.013	0.010	-1.244	0.214
first_treat_str: 2006 & year_str: 2005	0.003	0.009	0.327	0.744
first_treat_str: 2007 & year_str: 2005	0.016	0.008	2.050	0.040 *
first_treat_str: 0 & year_str: 2006	0.019	0.008	2.334	0.020 *
first_treat_str: 2004 & year_str: 2006	-0.054	0.013	-4.082	4.468e-05 ***
first_treat_str: 2006 & year_str: 2006	0.024	0.010	2.363	0.018 *
first_treat_str: 2007 & year_str: 2006	0.011	0.009	1.170	0.242
first_treat_str: 0 & year_str: 2007	0.026	0.009	2.824	0.005 **

	Estimate	Std. Error	t value	Pr(>|t|)
first_treat_str: 2004 & year_str: 2007	-0.011	0.016	-0.720	0.472
first_treat_str: 2006 & year_str: 2007	-0.006	0.013	-0.476	0.634
first_treat_str: 2007 & year_str: 2007	-0.009	0.011	-0.766	0.444

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

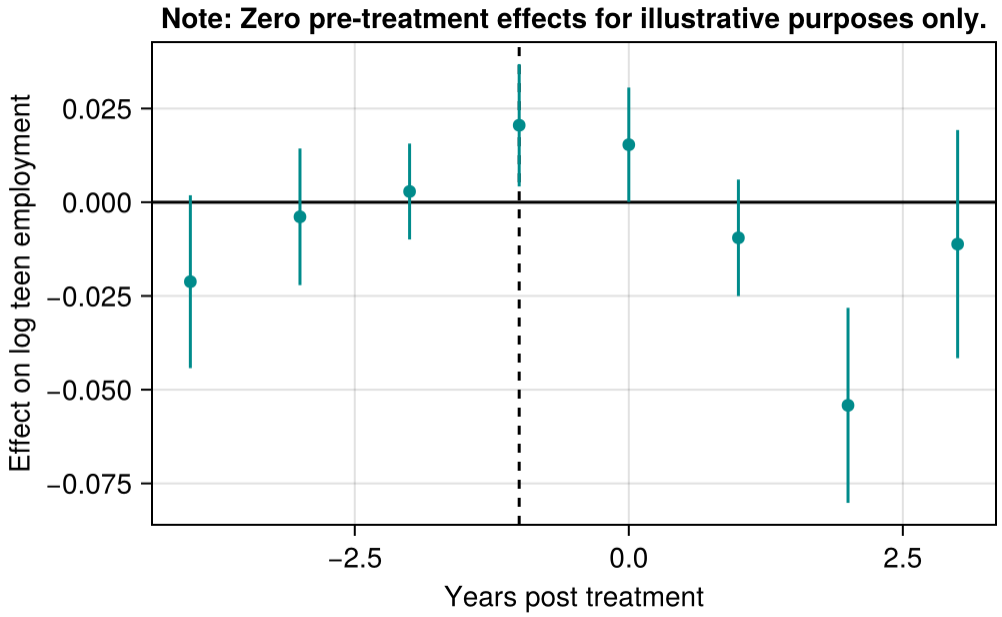
```
emfx(mod)
```

	type	estimate	std_error	conf_low	conf_high
	String	Float64	Float64	Float64	Float64
1	ATT	-0.00547767	0.00252612	-0.0104288	-0.000526576

```
mod_es = emfx(mod, type = "event")
mod_es
```

	event	estimate	std_error	conf_low	conf_high
	Int64	Float64	Float64	Float64	Float64
1	0	0.0153342	0.00778068	8.43689e-5	0.030584
2	1	-0.00949622	0.0079286	-0.025036	0.00604351
3	2	-0.0541802	0.0132735	-0.0801957	-0.0281647
4	3	-0.0111736	0.015526	-0.041604	0.0192568

```
mod_es2 = emfx(mod, type = "event", post_only = false)

fig = Figure()
ax = Axis(fig[1, 1],
    xlabel = "Years post treatment",
    ylabel = "Effect on log teen employment",
    title = "Note: Zero pre-treatment effects for illustrative purposes only.")

hlines!(ax, 0, color = :black)
vlines!(ax, -1, color = :black, linestyle = :dash)

rangebars!(ax, mod_es2.event, mod_es2.conf_low, mod_es2.conf_high, color = :darkcyan)
scatter!(ax, mod_es2.event, mod_es2.estimate, color = :darkcyan)

fig
```



13.11. Example 2: Staggered DiD

```
did_data = DataFrame(readstat("data/did_staggered_6.dta"))
@rtransform! did_data :first_treat = begin
    if :d4 == 1
        2004
    elseif :d5 == 1
        2005
    elseif :d6 == 1
        2006
    else
        0
    end
end

first(select(did_data, :id, :year, :y, :x, :d4, :d5, :d6, :te4, :te5, :te6, :first_treat), 6)
```

	id	year	y	x	d4	d5	d6	te4	te5	te6	first_treat
	Int16	Int16	Float32	Float32	Int8	Int8	Int8	Float32	Float32	Float32	Int64
1	1	2001	18.3596	0.899984	0	0	0	0.0	0.0	0.0	0
2	1	2002	18.1016	0.899984	0	0	0	0.0	0.0	0.0	0
3	1	2003	18.5956	0.899984	0	0	0	0.0	0.0	0.0	0
4	1	2004	16.5003	0.899984	0	0	0	1.43223	0.0	0.0	0
5	1	2005	19.9573	0.899984	0	0	0	2.57319	2.24723	0.0	0
6	1	2006	19.8659	0.899984	0	0	0	5.06752	4.23316	0.785892	0

```
combine(groupby(did_data, :first_treat), nrow)
```

	first_treat	nrow
	Int64	Int64
1	0	1572
2	2004	714
3	2005	498
4	2006	216

```
@rtransform! did_data :treated_cohort1 = begin
    if :d4 == 1 && :f04 == 1
        "d4f04"
    elseif :d4 == 1 && :f05 == 1
        "d4f05"
    elseif :d4 == 1 && :f06 == 1
        "d4f06"
    else
        missing
    end
end

@rtransform! did_data :treated_cohort2 = begin
    if :d5 == 1 && :f05 == 1
        "d5f05"
    elseif :d5 == 1 && :f06 == 1
        "d5f06"
    else
        missing
    end
end

@rtransform! did_data :treated_cohort3 = begin
    if :d6 == 1 && :f06 == 1
        "d6f06"
    else
        missing
    end
end

combine(groupby(dropmissing(did_data, :treated_cohort1), :treated_cohort1), :te4 => mean => :me
```

	treated_cohort1	mean4
	String	Float32
1	d4f04	3.75709
2	d4f05	4.01757
3	d4f06	4.57895

```
combine(groupby(dropmissing(did_data, :treated_cohort2), :treated_cohort2), :te5 => mean => :me
combine(groupby(dropmissing(did_data, :treated_cohort3), :treated_cohort3), :te6 => mean => :me
```

	treated_cohort3	mean6
	String	Float32
1	d6f06	1.84886

```
# String columns needed for cohort×time ETWFE dummies
did_data.first_treat_str = string.(did_data.first_treat)
did_data.year_str = string.(did_data.year)

# this replicates Jeff's results with pooled ols or xtreg, with covariate x.
mod = feols(did_data, @formula(y ~ x + first_treat_str & year_str + fe(id) + fe(year)), vcov =
show_regression_html(mod)
```

	Estimate	Std. Error	t value	Pr(>|t|)
x	-1.313e-32	5.787e-32	-0.227	0.820
first_treat_str: 0 & year_str: 2001	0.913	0.133	6.850	7.378e-12 ***
first_treat_str: 2004 & year_str: 2001	-1.124	0.159	-7.086	1.381e-12 ***
first_treat_str: 2005 & year_str: 2001	-0.425	0.172	-2.465	0.014 *
first_treat_str: 2006 & year_str: 2001	0.636	0.242	2.625	0.009 **
first_treat_str: 0 & year_str: 2002	0.802	0.129	6.194	5.854e-10 ***
first_treat_str: 2004 & year_str: 2002	-1.198	0.178	-6.724	1.767e-11 ***
first_treat_str: 2005 & year_str: 2002	-0.291	0.181	-1.610	0.107
first_treat_str: 2006 & year_str: 2002	0.687	0.233	2.950	0.003 **
first_treat_str: 0 & year_str: 2003	0.644	0.137	4.707	2.510e-06 ***
first_treat_str: 2004 & year_str: 2003	-1.187	0.168	-7.073	1.512e-12 ***

	Estimate	Std. Error	t value	Pr(>|t|)
first_treat_str: 2005 & year_str: 2003	0.198	0.211	0.939	0.348
first_treat_str: 2006 & year_str: 2003	0.345	0.260	1.324	0.186
first_treat_str: 0 & year_str: 2004	0.224	0.138	1.628	0.104
first_treat_str: 2004 & year_str: 2004	1.611	0.205	7.860	3.832e-15 ***
first_treat_str: 2005 & year_str: 2004	-1.361	0.181	-7.512	5.806e-14 ***
first_treat_str: 2006 & year_str: 2004	-0.474	0.258	-1.834	0.067 .
first_treat_str: 0 & year_str: 2005	-0.966	0.146	-6.629	3.389e-11 ***
first_treat_str: 2004 & year_str: 2005	1.299	0.213	6.109	1.000e-09 ***
first_treat_str: 2005 & year_str: 2005	0.916	0.223	4.102	4.092e-05 ***
first_treat_str: 2006 & year_str: 2005	-1.249	0.254	-4.910	9.123e-07 ***
first_treat_str: 0 & year_str: 2006	-1.617	0.169	-9.585	< 2e-16 ***
first_treat_str: 2004 & year_str: 2006	0.600	0.232	2.587	0.010 **
first_treat_str: 2005 & year_str: 2006	0.963	0.249	3.864	1.116e-04 ***
first_treat_str: 2006 & year_str: 2006	0.055	0.354	0.155	0.877

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
emfx(mod)
```

	type	estimate	std_error	conf_low	conf_high
	String	Float64	Float64	Float64	Float64
1	ATT	0.907151	0.0582209	0.793041	1.02126

```
mod_es = emfx(mod, type = "event")
mod_es
```

	event	estimate	std_error	conf_low	conf_high
	Int64	Float64	Float64	Float64	Float64
1	0	0.860449	0.177864	0.511843	1.20905
2	1	1.13086	0.165562	0.806369	1.45536
3	2	0.599837	0.231825	0.145468	1.05421

```
mod_es2 = emfx(mod, type = "calendar")
mod_es2
```

	calendar	estimate	std_error	conf_low	conf_high
	Int64	Float64	Float64	Float64	Float64
1	2004	1.6105	0.20489	1.20892	2.01208
2	2005	1.10755	0.122831	0.866807	1.3483
3	2006	0.539101	0.0562459	0.428862	0.649341

13.12. Nonlinear ETWFE: Count and Binary Outcomes

ETWFE is not limited to linear outcomes. When Y is a count or binary variable, the key identifying assumption shifts to **Conditional Parallel Trends on the linear index**: parallel trends on $\log E[Y_{it}(\infty)]$ for Poisson, or on $\text{logit}\, E[Y_{it}(\infty)]$ for logit. This is Wooldridge's (2023, 2026) CPT assumption stated on $G^{-1}(E[Y])$.

The correct Wooldridge ETWFE specification for a nonlinear model uses: - **Cohort FE** (one intercept per treatment cohort, including never-treated): captures selection into treatment - **Year FE** (common time trend): identified from never-treated units - **Post-treatment cohort×year dummies** (one per treated cohort per post-treatment year): capture the ATTs on the log scale

Critically, this is a **pooled model**, not a within-estimator. In the *linear* Gaussian examples above, unit FE and cohort FE give numerically identical ATTs (Wooldridge 2021). In nonlinear models, however, unit fixed effects raise the incidental-parameters problem and make average partial effects uncomputable — the unit intercepts cannot be averaged out of the nonlinear link. Wooldridge (2023) instead uses cohort intercepts (a Mundlak device), which is what `etwfe` implements; that is why the cohort-FE requirement matters only for Poisson/logit ETWFE.

13.12.1. Simulated example: staggered count data

We simulate panel data with three treatment cohorts (treated in years 3, 4, 5) and a never-treated group. The true treatment effect is multiplicative: $\exp(0.4) - 1 \approx 49\%$ increase in counts. Parallel trends holds on the log scale.

```
Random.seed!(42)
N_pois = 600; Tmax_pois = 6
unit_fe_vals = randn(N_pois) .* 0.3
rows_p = [(id=i, year=t) for i in 1:N_pois for t in 1:Tmax_pois]
df_count = DataFrame(rows_p)
sort!(df_count, [:id, :year])

cohort_grp = ((df_count.id .- 1) .÷ 150) .+ 1
df_count.first_treat = [3, 4, 5, 0][cohort_grp]   # 0 = never treated
df_count.treated = (df_count.first_treat .> 0) .& (df_count.year .>= df_count.first_treat)
df_count.unit_fe = unit_fe_vals[df_count.id]
df_count.log_mu = 1.0 .+ 0.2 .* df_count.year .+ df_count.unit_fe .+ 0.4 .* df_count.treated
df_count.count = rand.(Poisson.(exp.(df_count.log_mu)))

combine(groupby(df_count, [:first_treat, :treated]), :count => mean => :mean_count)
```

	first_treat	treated	mean_count
	Int64	Bool	Float64
1	0	0	5.96222
2	3	0	3.73
3	3	1	10.5767
4	4	0	4.12222
5	4	1	11.1311
6	5	0	4.54333
7	5	1	12.63

13.12.2. Poisson ETWFE

`etwfe()` handles formula construction, cohort FE + year FE, and post-treatment dummy creation automatically. Pass `family = "poisson"` for count outcomes.

```
mod_pois = etwfe(df_count, @formula(count ~ 1);
                 gvar = :first_treat, tvar = :year,
                 family = "poisson", vcov = Vcov.cluster(:id))

# Show the coefficient table explicitly. Letting the returned struct be the
# cell's display value prints a raw
# `ETWFEResult(Panelest Model: ... CoefTable(Any[[...]]))` dump, because neither
# ETWFEResult nor CoefTable defines a text/plain show method that Jupyter picks
# up in that position.
show_regression_html(mod_pois.model)
```

	Estimate	Std. Error	t value	Pr(>|t|)
_D_g3_t3	0.429	0.051	8.382	< 2e-16 ***
_D_g3_t4	0.417	0.051	8.226	< 2e-16 ***

	Estimate	Std. Error	t value	Pr(>|t|)
_D_g3_t5	0.430	0.056	7.657	1.902e-14 ***
_D_g3_t6	0.461	0.051	9.004	< 2e-16 ***
_D_g4_t4	0.464	0.044	10.596	< 2e-16 ***
_D_g4_t5	0.355	0.050	7.087	1.367e-12 ***
_D_g4_t6	0.415	0.048	8.630	< 2e-16 ***
_D_g5_t5	0.462	0.047	9.812	< 2e-16 ***
_D_g5_t6	0.427	0.044	9.729	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

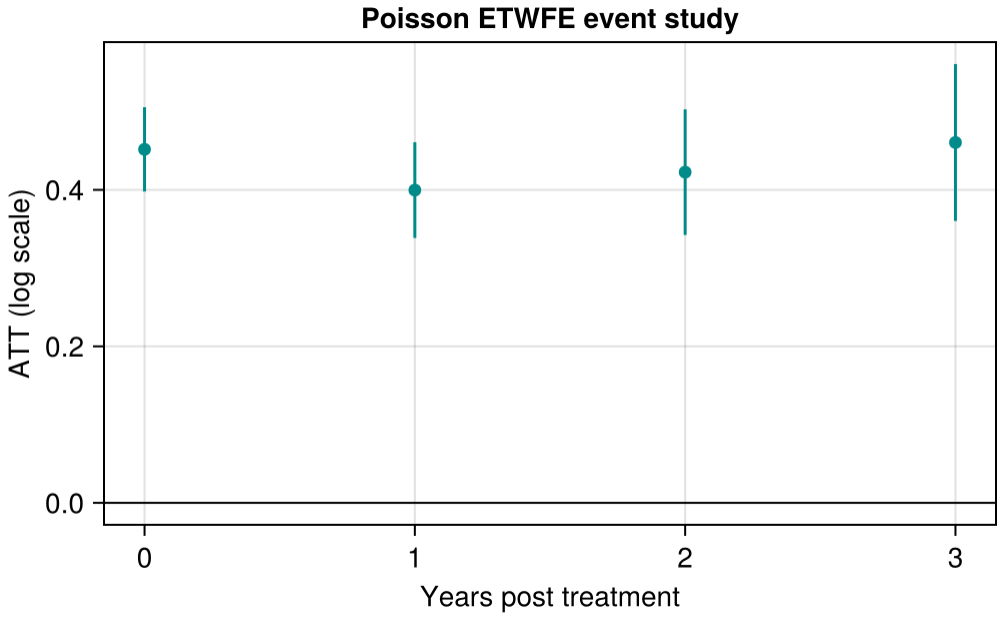
```
att = emfx(mod_pois)
@printf("ATT (log scale):  %.4f  (SE = %.4f)\n", att.estimate[1], att.std_error[1])
@printf("IRR - 1:          %.4f  (true   0.492)\n", exp(att.estimate[1]) - 1)
```

```
ATT (log scale):  0.4289  (SE = 0.0298)
IRR - 1:          0.5356  (true   0.492)
```

The ATT is on the **log scale** (log incidence rate ratio). Exponentiate and subtract 1 for the proportional increase in counts.

13.12.3. Event study

```
mod_pois_es = emfx(mod_pois, type = "event")

fig_p = Figure()
ax_p  = Axis(fig_p[1, 1],
    xlabel = "Years post treatment",
    ylabel = "ATT (log scale)",
    title  = "Poisson ETWFE event study")
hlines!(ax_p, 0, color = :black, linewidth = 1)
rangebars!(ax_p, mod_pois_es.event,
           mod_pois_es.conf_low, mod_pois_es.conf_high, color = :darkcyan)
scatter!(ax_p, mod_pois_es.event, mod_pois_es.estimate, color = :darkcyan)
fig_p
```



13.12.4. Comparison: linear ETWFE on log(count + 1)

```
df_count.log_count = log.(df_count.count .+ 1)

mod_lin = etwfe(df_count, @formula(log_count ~ 1);
                gvar = :first_treat, tvar = :year,
                vcov = Vcov.cluster(:id))
emfx(mod_lin)
```

	type	estimate	std_error	conf_low	conf_high
	String	Float64	Float64	Float64	Float64
1	ATT	0.384516	0.0283914	0.32887	0.440162

The linear model estimates the ATT on the $\log(Y + 1)$ scale — a different and less interpretable quantity than the Poisson log IRR.

13.13. Spatial Interference

Everything in this chapter so far assumes SUTVA: one unit's outcome does not depend on another unit's treatment. That is not always plausible. A treated municipality may affect deforestation in nearby untreated municipalities. A vaccinated household may reduce transmission to nearby households. A labor-market policy in one commuting zone may change flows to its neighbors. With this kind of spatial spillover, the usual DiD estimand no longer has the interpretation we want. Xu (2023) shows that ignoring interference can produce an estimand that is neither a direct effect nor a spillover effect.

Xu (2023, 2026) develops a doubly robust DiD that keeps the identifying logic of conditional parallel trends but adds an exposure mapping $G_{it} = G(i, W_{-it})$ for unit i's neighborhood treatment status. The estimand is the direct ATT at exposure level g,

$$\tau_g(t,c) = \frac{1}{|S_M|} \sum_{i \in S_M} \mathbb{E}[y_{it}(c,g) - y_{it}(\infty,g) \mid z_i, C_i = c, G_{it} = g], \tag{13.12}$$

where C_i is unit i's first treatment period (so $\{C_i > t\}$ is the not-yet-directly-treated comparison group), $S_M = \{C_i = c\} \cup \{C_i > t\}$, and g is a chosen exposure level. The DR plug-in averages three pieces over S_M: an IPW term from the treated cohort, an IPW term from the not-yet-treated, and a regression-imputation term, with three propensity models (η_{tc} for cohort, η_{tcg} and $\eta_{t\infty g}$ for exposure) and two outcome-change regressions. It is consistent if either all three propensities or both outcome models are correctly specified.

The companion package `DidInterference.jl` implements this. The 2×2 base case (Xu 2023):

```
using DidInterference, DataFrames

res = did_int_2x2(panel;
    yname     = :Y_post,
    yname_pre = :Y_pre,
    treat     = :W,
    exposure  = :G,
    g         = 1,
    covariates = [:z1, :z2],
    trim       = 0.01)   # Xu (2026) uses 0.01 in the Brazil application

println(res.estimate, "  ", res.ci)
```

For staggered adoption (Xu 2026), `did_int_staggered()` loops over `(cohort, time)` cells with $t \geq c$, restricts to S_M in each cell, fits the DR estimator, and aggregates across cells using joint-IF stacking (per-cell IFs share the never-treated comparison group, so independence-style SEs underestimate uncertainty noticeably). An R port with the same interface is available as `didint`; its Brazil Amazon `Lista de Municípios Prioritários` vignette replicates Section III of Xu (2026) using the public Assunção-McMillan-Murphy-Souza-Rodrigues replication archive on Zenodo.

13.14. Persistent Outcomes and Heterogeneous Dynamics

Standard event-study TWFE regressions like

$$Y_{it} = \alpha_i + \gamma_t + \sum_j D_{it}^j \delta_j + X_{it}'\beta + U_{it} \tag{13.13}$$

implicitly assume the residual has no serial dependence once unit and time fixed effects are absorbed. When outcomes are genuinely persistent — earnings, employment, consumption, anything with habits or adjustment costs — that assumption fails. The event-time dummies absorb persistence on

top of the true causal effect, producing spurious pre-trends and biased post-treatment estimates whose bias grows geometrically with the event-time horizon (Botosaru and Liu 2025, Figure 1).

Botosaru and Liu (2025) introduce a **dynamic panel with correlated random coefficients** that handles both persistence and unit-level heterogeneity in dynamic responses:

$$Y_{it} = \rho_Y Y_{i,t-1} + \alpha_i + \sum_j D_{it}^j \delta_{ij} + X_{it}'\beta + U_{it}, \tag{13.14}$$

with a parsimonious AR(1) on the event-time effects to keep dimensionality manageable:

$$\delta_{ij} = \rho_\delta \delta_{i,j-1} + \varepsilon_{ij}, \quad j \geq 1. \tag{13.15}$$

The latent $\lambda_i = (\alpha_i, \delta_{i0})$ is a unit-specific correlated random coefficient. A two-step semiparametric estimator: step 1 is **quasi-maximum likelihood** for the common parameters under a Gaussian working assumption on λ_i (consistent under misspecification); step 2 is **Tweedie / Gaussian-conjugate empirical Bayes** for the unit-level posterior trajectories $\{\delta_{i,j}\}_{j=0}^J$.

A companion note (Botosaru and Liu 2026) adds a **homogeneous-feedback extension**: covariates X_{it} are allowed to adjust endogenously to past Y and treatment. Under the assumption that the covariate adjustment rule does not depend on λ_i given the observable history, the likelihood factors into a structural piece and a feedback piece, yielding a clean decomposition of dynamic treatment effects into **direct** (through $\delta_{i,j}$) and **indirect** (through covariate feedback) components.

The package `TVHTE.jl` implements both papers:

```
using TVHTE

# Fit on a panel with staggered cohorts and a covariate
fit = tvhte(panel_Y, baseline_Y; t0 = cohort, J = max_event_time, X = panel_X)

# Feedback model + direct/indirect counterfactual decomposition
fb = fit_feedback(panel_Y, baseline_Y, panel_X, baseline_X)
cf = simulate_counterfactual(fit, fb, alt_cohort)
```

An R port with the same API is at `tvhte`; both packages have deployed docs at https://xiangao.github.io/TVHTE.jl/ and https://xiangao.github.io/tvhte/.

Synthetic control, synthetic DiD, and TASC are closely related enough that we treat them together in the next design chapter.

14. Synthetic Control, DiD, and TASC

```
using DataFrames
using SynthDiD
using CairoMakie
CairoMakie.activate!(type = "png")
using Statistics
using Printf

using TASC
```

Here I put three related panel-data designs together: classical synthetic control, synthetic DiD, and Time-Aware Synthetic Control (TASC). All compare a treated unit to a donor pool over time. They differ in what structure they put on the untreated counterfactual.

14.1. Synthetic Control

The biggest problem with DiD is the parallel trends assumption. There is not really a test for it, just like the unconfoundedness assumption. It is less demanding than unconfoundedness, but it is still hard to justify in many applications.

Synthetic control starts from a different idea. Instead of giving every control unit the same weight, it constructs a synthetic control unit from the donor pool:

$$\hat{Y}_{1t}(0) = \sum_{i=2}^{N} \hat{\omega}_i Y_{it}, \tag{14.1}$$

where the weights are chosen to make the synthetic unit close to the treated unit before treatment. The usual Abadie-style version restricts the donor weights to be nonnegative and to sum to one. The identifying hope is not that the average control follows the treated unit, but that some weighted average of controls does.

Classical synthetic control is time-agnostic in an important sense: the pre-treatment periods enter the weight-fitting problem as a set of matching moments. If we permute the order of the pre-treatment periods, the fitted donor weights are unchanged. This is often a strength because it avoids imposing a time-series model, but it can leave predictive information unused when there is a stable trend.

14.2. Synthetic DiD

Arkhangelsky et al. (2021) combine the idea of DiD and synthetic control. Standard DiD assigns equal weight to all control units and all pre-treatment time periods. Synthetic control assigns data-driven weights to donor units. Synthetic DiD uses both unit weights and time weights.

The three estimators below are written as a *unifying weighted-regression sketch*: the same two-way fixed-effect regression, with different weights switched on. This is a useful way to see how DiD, synthetic control, and synthetic DiD relate, but it is **not** the estimators' actual optimization problems. In particular, the synthetic-control weights $\widehat{\omega}_i^{sc}$ are not free regression weights — in Abadie's formulation they are chosen by a separate optimization that matches pre-treatment outcomes (and predictors) of the treated unit, subject to $\omega_i \geq 0,\ \sum_i \omega_i = 1$. Likewise the SDID unit and time weights $\widehat{\omega}_i^{sdid}, \widehat{\lambda}_t^{sdid}$ come from their own penalized pre-period optimization problems. Treat the displays below as schematic; see the package documentation (or Arkhangelsky et al. 2021) for the exact weight-selection problems.

The DiD objective can be written as

$$(\hat{\tau}^{did}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \underset{\tau,\mu,\alpha,\beta}{\arg\min} \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2. \tag{14.2}$$

Synthetic control can be viewed as a weighted version that puts the donor weights into the comparison:

$$(\hat{\tau}^{sc}, \hat{\mu}, \hat{\beta}) = \underset{\tau,\mu,\beta}{\arg\min} \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - \mu - \beta_t - W_{it}\tau)^2 \widehat{\omega}_i^{sc}. \tag{14.3}$$

Synthetic DiD adds time weights:

$$(\hat{\tau}^{sdid}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \underset{\tau,\mu,\alpha,\beta}{\arg\min} \sum_{i=1}^{N}\sum_{t=1}^{T}(Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2 \widehat{\omega}_i^{sdid}\widehat{\lambda}_t^{sdid}. \tag{14.4}$$

The unit weights try to make the donor pool look like the treated unit. The time weights put more emphasis on pre-treatment periods that are more informative about the post-treatment period.

14.3. Time-Aware Synthetic Control

TASC was proposed by Rho et al. (2026). It starts from the same panel-data structure, but it models the donor and treated outcomes through a latent time-series system:

$$x_t = Ax_{t-1} + q_t, \qquad q_t \sim N(0, Q), \tag{14.5}$$

$$y_t = Hx_t + r_t, \qquad r_t \sim N(0, R). \tag{14.6}$$

Here y_t is the vector of outcomes across units at time t, and x_t is a low-dimensional latent state. The matrix H maps the latent state into unit outcomes, so HX is a low-rank signal. The matrix A is the new piece relative to classical synthetic control: it says that the latent time factors evolve according to a stable trend.

The counterfactual step is simple. TASC learns the state-space model from the pre-treatment panel. After treatment, the treated unit's observed outcome is treated as missing. The filter and smoother continue to use the donor units to infer the latent state, and the treated row of H maps that state into the untreated counterfactual:

$$\hat{Y}_{1t}(0) = h_1^\top \hat{x}_t, \qquad t > T_0. \tag{14.7}$$

The estimated treatment effect is then

$$\hat{\tau}_t = Y_{1t} - \hat{Y}_{1t}(0). \tag{14.8}$$

Relative to classical synthetic control, TASC uses the order of time. Relative to synthetic DiD, it puts the time dependence in a generative latent-state model rather than in a set of pre-period weights. This can help when trends are stable and outcomes are noisy, but it is also a stronger modeling assumption.

14.4. Example: California Proposition 99

California's Proposition 99 raised cigarette taxes and funded anti-smoking programs. The usual synthetic-control exercise compares California's cigarette sales after 1988 to a donor pool of other states. The outcome is per-capita cigarette sales in packs.

```
california = california_prop99()
setup = panel_matrices(california, :State, :Year, :PacksPerCapita, :treated)

tau_sdid = synthdid_estimate(setup.Y, setup.N0, setup.T0)
tau_sc   = sc_estimate(setup.Y, setup.N0, setup.T0)
tau_did  = did_estimate(setup.Y, setup.N0, setup.T0)
nothing
```

`SynthDiD.jl` orders the donor states first and treated states last. `TASC.jl` expects the treated unit to be first, so we explicitly reorder the matrix before fitting the state-space model. The paper's Proposition 99 example uses a low latent dimension; here we set `d = 2`.

```
Y_tasc = vcat(setup.Y[(setup.N0 + 1):end, :], setup.Y[1:setup.N0, :])

tasc_model = fit_tasc(
    Y_tasc;
    d = 2,
    T0 = setup.T0,
```

```
    n_em = 200,
    tol = 1e-3,
)

tasc_pred = predict_counterfactual(tasc_model, Y_tasc)
tasc_att = mean(tasc_pred.effect[(setup.T0 + 1):end])
nothing
```

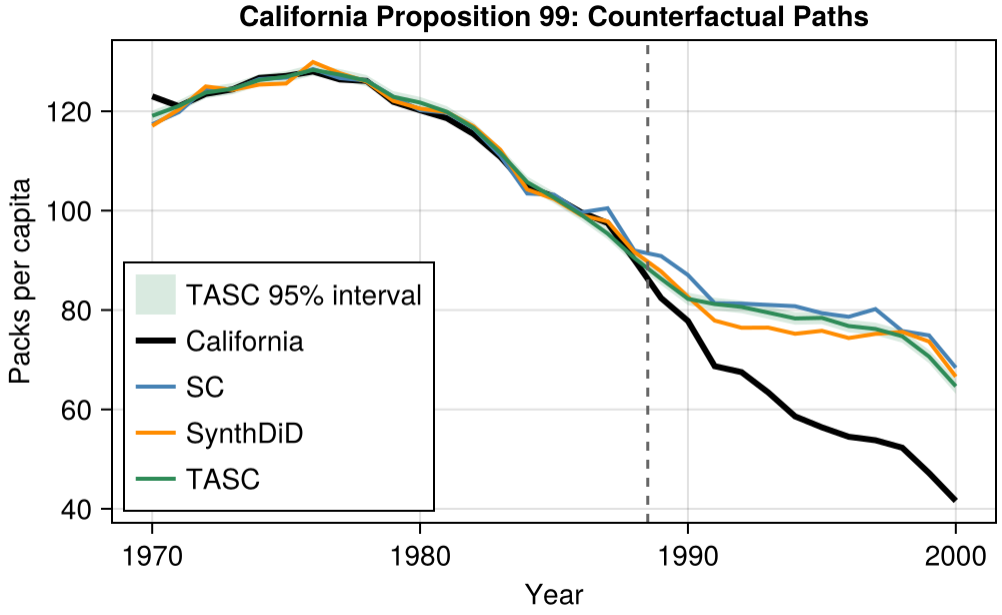
```
estimates = DataFrame(
    Method = ["Diff-in-Diff", "Synthetic Control", "Synthetic Diff-in-Diff", "TASC"],
    Estimate = [tau_did.estimate, tau_sc.estimate, tau_sdid.estimate, tasc_att],
)

DataFrames.transform(estimates, :Estimate => ByRow(x -> round(x, digits = 2)) => :EstimateRound
```

	Method	Estimate	EstimateRounded
	String	Float64	Float64
1	Diff-in-Diff	-27.3491	-27.35
2	Synthetic Control	-19.6197	-19.62
3	Synthetic Diff-in-Diff	-15.6038	-15.6
4	TASC	-17.1127	-17.11

The estimates are all negative: California's observed cigarette sales fell below the estimated no-Proposition-99 counterfactual. The methods differ because they impose different restrictions on the untreated path. DiD compares average changes. Synthetic control chooses donor weights. Synthetic DiD adds time weights. TASC uses a low-rank state-space model and updates the latent state with post-treatment donor observations.

```
years = Int.(setup.times)
treated = vec(setup.Y[(setup.N0 + 1), :])
sc_counterfactual = vec(tau_sc.weights.omega' * setup.Y[1:setup.N0, :])
# SynthDiD fits with omega_intercept=true, so its synthetic match is c +  'Y_donors
# for a fitted level shift c (the omega weights demean the donor columns). Add the
# pre-period level shift back so the plotted line matches the path SynthDiD compares
# the treated unit against; otherwise the line is vertically offset.
sdid_raw = vec(tau_sdid.weights.omega' * setup.Y[1:setup.N0, :])
sdid_intercept = mean(treated[1:setup.T0]) - mean(sdid_raw[1:setup.T0])
sdid_counterfactual = sdid_raw .+ sdid_intercept
tasc_counterfactual = tasc_pred.target
tasc_se = sqrt.(max.(tasc_pred.variance, 0.0))
tasc_lower = tasc_counterfactual .- 1.96 .* tasc_se
tasc_upper = tasc_counterfactual .+ 1.96 .* tasc_se

fig = Figure(size = (760, 430))
ax = Axis(
    fig[1, 1],
    xlabel = "Year",
```

```
    ylabel = "Packs per capita",
    title = "California Proposition 99: Counterfactual Paths",
)

band!(ax, years, tasc_lower, tasc_upper, color = (:seagreen, 0.18), label = "TASC 95% interval")
lines!(ax, years, treated, color = :black, linewidth = 3, label = "California")
lines!(ax, years, sc_counterfactual, color = :steelblue, linewidth = 2, label = "SC")
lines!(ax, years, sdid_counterfactual, color = :darkorange, linewidth = 2, label = "SynthDiD")
lines!(ax, years, tasc_counterfactual, color = :seagreen, linewidth = 2, label = "TASC")
vlines!(ax, years[setup.T0] + 0.5, color = :gray40, linestyle = :dash)

axislegend(ax, position = :lb)
fig
```



The figure emphasizes where the counterfactual enters. After 1988, the black line is observed California. The colored lines are estimates of California's untreated path. The treatment-effect path is the vertical gap between the black line and each counterfactual.

15. Randomization Inference for Synthetic Control

```
using DataFrames
using SynthDiD
using Statistics
using Random
using CairoMakie
CairoMakie.activate!(type = "png")

using TASC
```

Synthetic control is often used when there is one treated unit: California, West Germany, the Basque Country. Usual large-sample standard errors are not very helpful because there is no large sample of treated units. Two approaches are common:

- **Randomization inference**: use donor units as placebos.
- **Posterior inference**: put a model on the counterfactual path and compute posterior uncertainty.

Here I use both on the Proposition 99 panel.

15.1. The placebo test

The placebo idea is simple. Take each control unit in turn, pretend it was treated, and re-run synthetic control. This produces a placebo gap for each donor. The collection of placebo gaps is the empirical null distribution under no effect.

If California's gap is large relative to the placebo gaps, that is evidence against the no-effect null.

```
california = california_prop99()
setup = panel_matrices(california, :State, :Year, :PacksPerCapita, :treated)
Y    = setup.Y
N0   = setup.N0           # 38 donor states
T0   = setup.T0           # 1989 onward is post-treatment
N1   = size(Y, 1) - N0    # 1 treated state (California)
years = Int.(setup.times)

# California's observed effect (SC estimate)
```

```
tau_sc = sc_estimate(Y, N0, T0)
ca_obs = vec(Y[N0 + 1, :])
ca_cfact = vec(tau_sc.weights.omega' * Y[1:N0, :])
ca_effect = mean(ca_obs[(T0 + 1):end]) - mean(ca_cfact[(T0 + 1):end])

@printf("California ATT (SC): %.2f packs per capita\n", ca_effect)
```

```
California ATT (SC): -19.62 packs per capita
```

Now run the placebo: each control state takes a turn as the "treated" unit.

```
function run_placebo(Y, N0, T0, treated_idx)
    # Swap treated_idx into the last row (treated slot); remaining donors fill 1..N0-1
    donor_idx = setdiff(1:N0, treated_idx)
    Y_perm    = vcat(Y[donor_idx, :], Y[treated_idx:treated_idx, :])
    fit       = sc_estimate(Y_perm, length(donor_idx), T0)
    treated   = vec(Y_perm[end, :])
    cfact     = vec(fit.weights.omega' * Y_perm[1:length(donor_idx), :])
    return treated, cfact
end

placebo_effects = Float64[]
placebo_paths   = Matrix{Float64}(undef, N0, size(Y, 2))
placebo_cfacts  = Matrix{Float64}(undef, N0, size(Y, 2))

for i in 1:N0
    treated_i, cfact_i = run_placebo(Y, N0, T0, i)
    placebo_paths[i, :]  = treated_i .- cfact_i
    placebo_cfacts[i, :] = cfact_i
    push!(placebo_effects, mean(treated_i[(T0 + 1):end] .- cfact_i[(T0 + 1):end]))
end

# California's treatment effect path
ca_gap = ca_obs .- ca_cfact

@printf("California ATT:        %.2f\n", ca_effect)
@printf("Placebo distribution mean: %.2f\n", mean(placebo_effects))
@printf("Placebo distribution sd:   %.2f\n", std(placebo_effects))
```

```
California ATT:        -19.62
Placebo distribution mean: 0.32
Placebo distribution sd:   10.76
```

15.1.1. The two-sided p-value

A Fisher-style exact p-value is the fraction of placebo effects at least as extreme (in absolute value) as the observed effect:

```
# Fisher exact convention: count California itself in numerator and denominator,
# so the p-value floor is 1/(N0+1) = 1/39 rather than 0.
p_value = (1 + sum(abs.(placebo_effects) .>= abs(ca_effect))) / (length(placebo_effects) + 1)
@printf("Two-sided p-value: %.3f\n", p_value)
```

```
Two-sided p-value: 0.077
```

With 38 donor states, the smallest possible p-value is $1/39 \approx 0.026$ (California plus 38 placebos). A p-value at this floor means the observed effect is more extreme than every placebo — strong evidence against the sharp null.

15.1.2. The placebo plot

The usual plot overlays the placebo gaps in grey and California's gap in black. If the black line is outside the grey cloud, California looks unusual.

```
fig = Figure(size = (740, 380), fontsize = 13)
ax  = Axis(fig[1, 1], xlabel = "Year", ylabel = "Gap: observed − synthetic",
           title = "Placebo gap trajectories")
for i in 1:N0
    lines!(ax, years, placebo_paths[i, :],
           color = (:grey, 0.35), linewidth = 1)
end
lines!(ax, years, ca_gap, color = :black, linewidth = 2.5, label = "California")
hlines!(ax, [0.0]; color = :gray40, linestyle = :dot, linewidth = 1)
vlines!(ax, [years[T0] + 0.5]; color = :gray40, linestyle = :dash, linewidth = 1)
axislegend(ax, position = :lb, framevisible = false)
fig
```

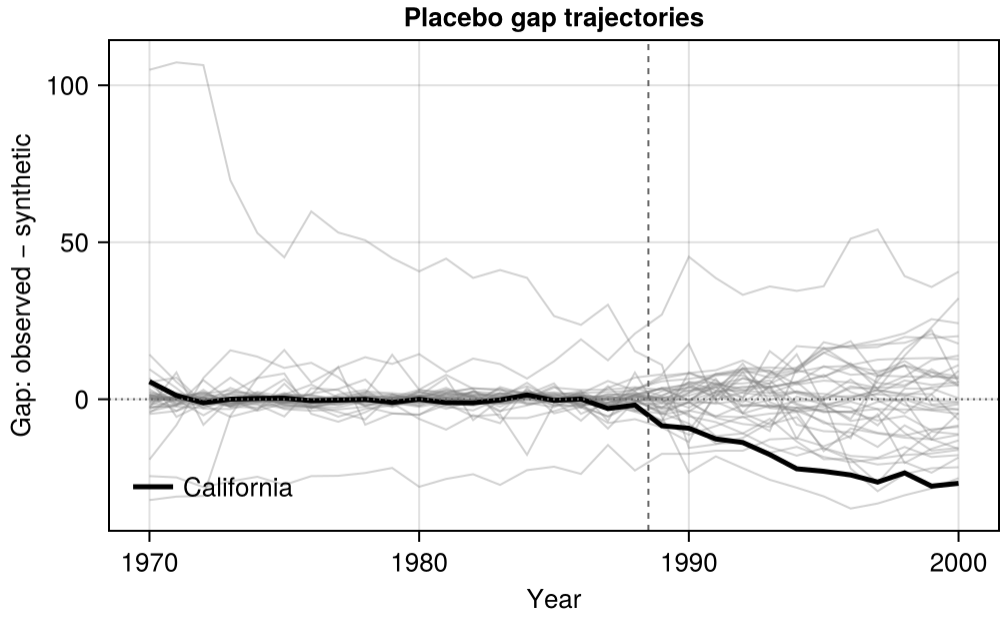


Figure 15.1.: California's treatment-effect gap (black) vs placebo gaps from each donor state (grey). The vertical dashed line marks 1989.

15.2. The MSPE ratio test

Some donor states fit California's pre-period trajectory better than others. A placebo state whose synthetic control fits its own pre-period poorly will produce a large *pre-period* gap by construction, inflating its placebo effect even under the sharp null. Abadie, Diamond, and Hainmueller (2015) propose normalising by pre-treatment fit:

$$\text{MSPE ratio}(\ell) = \frac{\frac{1}{T-T_0}\sum_{t>T_0}(Y_{\ell t} - \hat{Y}_{\ell t})^2}{\frac{1}{T_0}\sum_{t\le T_0}(Y_{\ell t} - \hat{Y}_{\ell t})^2}. \tag{15.1}$$

A large ratio means post-treatment gap is large *relative to* pre-treatment fit. The MSPE ratio test compares California's ratio to the distribution of placebo ratios.

```
function mspe_ratio(treated, cfact, T0)
    pre  = mean((treated[1:T0] .- cfact[1:T0]).^2)
    post = mean((treated[(T0 + 1):end] .- cfact[(T0 + 1):end]).^2)
    return post / max(pre, eps())
end

ca_ratio = mspe_ratio(ca_obs, ca_cfact, T0)

placebo_ratios = Float64[]
for i in 1:N0
```

```
    treated_i, cfact_i = run_placebo(Y, N0, T0, i)
    push!(placebo_ratios, mspe_ratio(treated_i, cfact_i, T0))
end

p_ratio = (1 + sum(placebo_ratios .>= ca_ratio)) / (length(placebo_ratios) + 1)
@printf("California MSPE ratio: %.2f\n", ca_ratio)
@printf("Placebo ratio 75th pct: %.2f\n", quantile(placebo_ratios, 0.75))
@printf("Placebo ratio max:     %.2f\n", maximum(placebo_ratios))
@printf("One-sided p-value (ratio): %.3f\n", p_ratio)
```

```
California MSPE ratio: 154.94
Placebo ratio 75th pct: 35.00
Placebo ratio max:     536.00
One-sided p-value (ratio): 0.077
```

The MSPE ratio discounts placebo states that were poorly fit before treatment.

```
fig = Figure(size = (640, 360), fontsize = 13)
ax  = Axis(fig[1, 1], xlabel = "Post/Pre MSPE ratio",
           ylabel = "Count", title = "MSPE ratio distribution")
hist!(ax, placebo_ratios, bins = 20, color = (:steelblue, 0.6))
vlines!(ax, [ca_ratio]; color = :firebrick, linewidth = 2, linestyle = :dash,
        label = "California")
axislegend(ax, position = :rt, framevisible = false)
fig
```

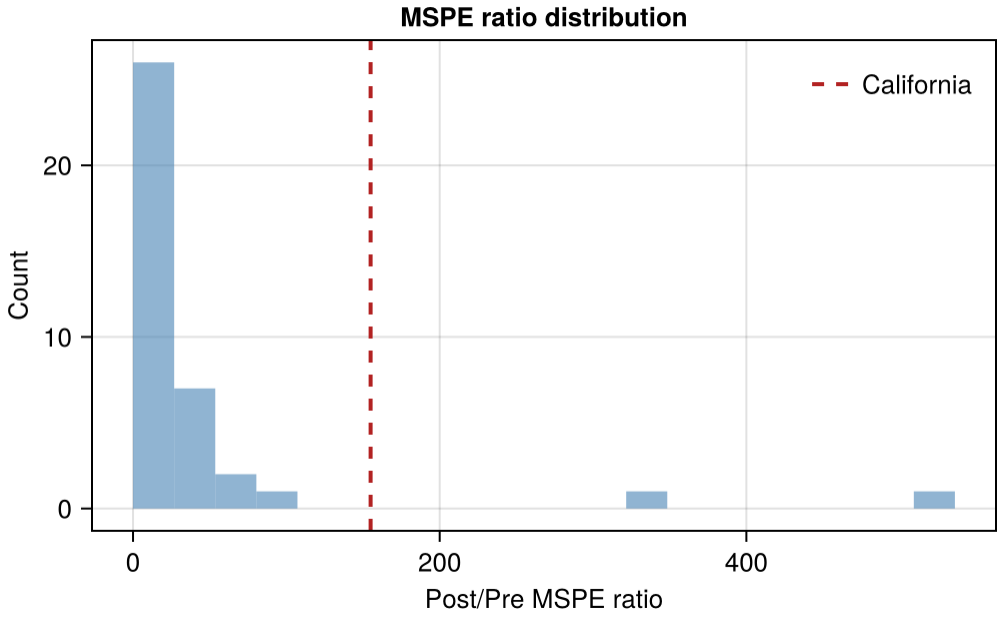


Figure 15.2.: Distribution of post/pre MSPE ratios across donor states (histogram). California's ratio is the red dashed line.

15.3. Time placebo

Another check moves the treatment date backward into the pre-period. If the method finds a large placebo effect before Proposition 99, the post-treatment effect is less credible.

```
# Pretend treatment started in 1980 instead of 1989
T0_placebo = findfirst(==(1980), years) - 1
Y_pre      = Y[:, 1:T0]   # cut off real treatment period
fit_pre    = sc_estimate(Y_pre, N0, T0_placebo)

ca_pre     = vec(Y_pre[N0 + 1, :])
ca_pre_cf  = vec(fit_pre.weights.omega' * Y_pre[1:N0, :])
placebo_effect_time = mean(ca_pre[(T0_placebo + 1):end] .- ca_pre_cf[(T0_placebo + 1):end])

@printf("Time-placebo effect (treatment moved to 1980): %.2f\n", placebo_effect_time)
@printf("Real effect (treatment in 1989):               %.2f\n", ca_effect)
```

```
Time-placebo effect (treatment moved to 1980): -3.31
Real effect (treatment in 1989):               -19.62
```

Here the time-placebo effect is much smaller than the real effect.

15.4. TASC posterior as Bayesian alternative

TASC models the panel with a linear Gaussian state-space process and returns a posterior distribution over the counterfactual path. This is model-based uncertainty, not a placebo distribution.

```
Y_tasc = vcat(Y[(N0 + 1):end, :], Y[1:N0, :])
tasc_model = fit_tasc(Y_tasc; d = 2, T0 = T0, n_em = 200, tol = 1e-3)
tasc_pred  = predict_counterfactual(tasc_model, Y_tasc)

tasc_cfact = vec(tasc_pred.target)
tasc_se    = sqrt.(max.(vec(tasc_pred.variance), 0.0))
tasc_lower = tasc_cfact .- 1.96 .* tasc_se
tasc_upper = tasc_cfact .+ 1.96 .* tasc_se

# At each post-treatment year, is California outside the band?
years_post = years[(T0 + 1):end]
outside    = (ca_obs[(T0 + 1):end] .< tasc_lower[(T0 + 1):end]) .|
             (ca_obs[(T0 + 1):end] .> tasc_upper[(T0 + 1):end])
@printf("Years California is outside the 95%% band: %d/%d\n",
        sum(outside), length(outside))
```

```
Years California is outside the 95% band: 12/12
```

```
# Convert placebo gaps to counterfactual paths for visualisation
fig = Figure(size = (820, 420), fontsize = 13)
ax  = Axis(fig[1, 1], xlabel = "Year", ylabel = "Packs per capita",
           title = "Frequentist placebo cloud vs Bayesian posterior band")

# Placebo counterfactuals (treated outcome minus gap, mapped to California's level)
for i in 1:N0
    cf = placebo_cfacts[i, :] .+ (ca_obs[T0] - placebo_cfacts[i, T0])
    lines!(ax, years, cf, color = (:grey, 0.20), linewidth = 1)
end

band!(ax, years, tasc_lower, tasc_upper;
      color = (:seagreen, 0.25), label = "TASC 95% interval")
lines!(ax, years, ca_obs;    color = :black,     linewidth = 3, label = "California")
lines!(ax, years, ca_cfact;  color = :steelblue, linewidth = 2, label = "SC counterfactual")
lines!(ax, years, tasc_cfact; color = :seagreen,  linewidth = 2, label = "TASC counterfactual")
vlines!(ax, [years[T0] + 0.5]; color = :gray40, linestyle = :dash, linewidth = 1)
axislegend(ax, position = :lb, framevisible = false)
fig
```

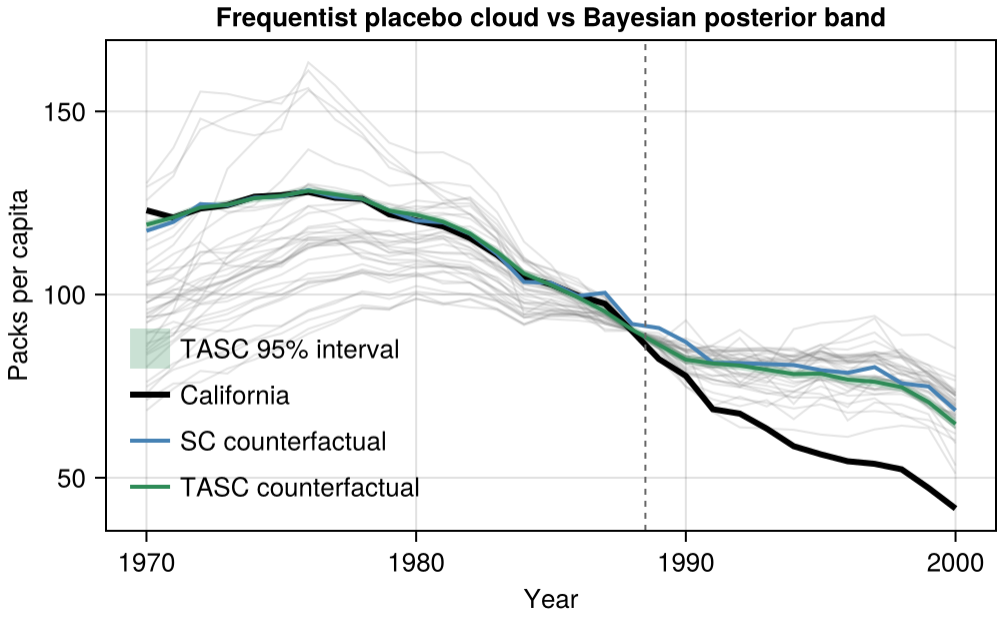


Figure 15.3.: California's observed path (black) with TASC posterior band (green), classical SC counterfactual (blue), and placebo gap envelope (grey).

15.5. How the two inference strategies compare

	Randomization inference	TASC posterior
What is uncertain?	Which unit was treated	Latent factor path
Null hypothesis	Sharp null: no effect anywhere	None — direct uncertainty
Validity requires	Donor pool comparable to treated	SSM correctly specified
Output	Permutation p-value	Posterior band
Smallest p possible	$1/(N_0+1)$ (here ~0.026)	N/A — continuous
Sensitivity to donor pool	High (small N0 → coarse p)	Low (band reflects modelling)

The two strategies answer different questions. Randomization inference asks whether California is unusual relative to donor-state placebos. TASC asks how uncertain the model is about California's counterfactual path. I would report both when using TASC.

15.6. Summary

- With one treated unit, randomization inference is more useful than asymptotic standard errors.
- Unit placebos give the empirical null distribution.
- MSPE ratios account for pre-treatment fit.
- Time placebos check whether the method finds effects before treatment.

- TASC posterior bands give model-based uncertainty for the counterfactual path.

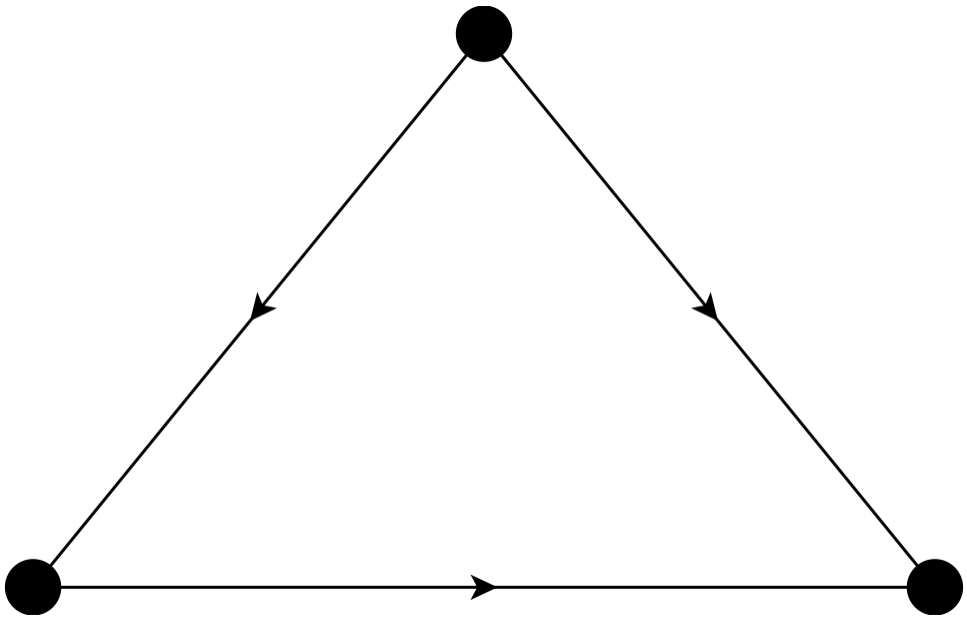
16. IV & Regression Discontinuity

```
using CairoMakie
CairoMakie.activate!(type = "png")
using GraphMakie, Graphs
using Distributions, Random, LinearAlgebra
using RDRobust
using Panelest
using Vcov
using DataFrames
using StatsModels, DataFramesMeta
import StatsAPI: coeftable
using Printf
using RegressionTables
```

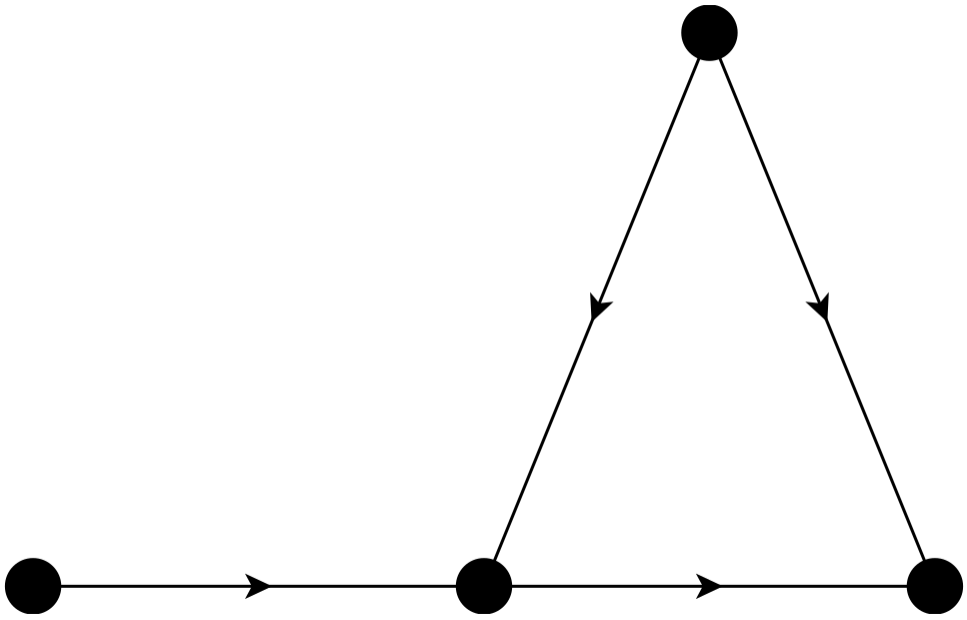
With an unobserved confounder, unconfoundedness fails. Even a randomized experiment can face this problem through noncompliance: subjects are randomly assigned but some do not comply, so the actually-treated group is self-selected and that selection may be correlated with the outcome.

16.1. Instrumental Variables

16.1.1. Uncontrolled Confounder



16.1.2. Why Does IV Work?



16.1.3. IV Under Potential Outcomes

- W has two potential outcomes W(1) and W(0), as a function of Z.
- Y has two potential outcomes Y(1) and Y(0), as a function of W.

Intention to Treat (ITT) is the average treatment effect of the treatment assignment Z. It is the difference between the average potential outcome under treatment and the average potential outcome under control. It is the average treatment effect of the treatment assignment Z, regardless of whether the subject complies with the assignment.

There are four possible groups of subjects in the case of binary treatment and binary instrument. The compliance type is defined by the latent pair of potential treatments $(W(0), W(1))$, not by an observed (Z, W) cell:

- complier: $W(0) = 0, W(1) = 1$
- always-taker: $W(0) = 1, W(1) = 1$
- never-taker: $W(0) = 0, W(1) = 0$
- defier: $W(0) = 1, W(1) = 0$

A given observed (Z, W) cell does not pin down the type (see the table below).

Table 16.1.: compliance groups

	W(1)=0	W(1)=1
W(0)=0	N	C
W(0)=1	D	A

These four groups (stratum) have different causal mechanisms. For example, the compliers are the only group that is affected by the treatment assignment Z in the same direction. The defiers are the only group that is affected by the treatment assignment Z in the opposite direction. The always-takers and never-takers are not affected by the treatment assignment Z.

Unfortunately we cannot identify the compliance group by looking at the data.

Z	W	G
0	0	C, N
0	1	A, D
1	0	N, D
1	1	C, A

16.1.4. Assumptions

- Exclusion restriction: writing the potential outcome as a function of both treatment w and instrument z, $Y(w, z)$, the instrument has no direct effect on the outcome: $Y(w, 1) = Y(w, 0)$
- Exogeneity of instrument: $Z \perp [W(0), W(1), Y(0), Y(1)]$
- Monotonicity (no defiers): $W(0) \leq W(1)$

16.1.5. LATE Identification

Since we excluded defiers, and the other two groups (always-takers and never-takers) are not affected by the treatment assignment Z. The only effect we can identify is the treatment effect on the compliers. This is called the Local Average Treatment Effect (LATE).

$$\begin{aligned}
&E[Y(Z=1) - Y(Z=0) | G = C] \\
&= \frac{E[Y(Z=1) - Y(Z=0)]}{P(W(1)=1, W(0)=0)} \\
&= \frac{E[Y|Z=1] - E[Y|Z=0]}{1 - P(W=0|Z=1) - P(W=1|Z=0)} \text{ (rule out groups A and N)} \\
&= \frac{E[Y|Z=1] - E[Y|Z=0]}{P(W=1|Z=1) - P(W=1|Z=0)} \\
&= \frac{E[Y|Z=1] - E[Y|Z=0]}{E[W|Z=1] - E[W|Z=0]}
\end{aligned} \tag{16.1}$$

16.1.6. Parametric Models

If we assume linearity, then this becomes the ratio of two OLS coefficients, sometimes called the Wald estimator.

$$\begin{aligned}
\tau_{LATE} &= \frac{E[Y|Z=1] - E[Y|Z=0]}{E[W|Z=1] - E[W|Z=0]} \\
&= \frac{Cov(Y, Z)}{Cov(W, Z)}
\end{aligned} \tag{16.2}$$

Why do covariates help? It is worth separating two distinct assumptions. Conditioning on covariates X can make the *as-if-random* (conditional independence) assumption more credible: the instrument may be unconfounded only within levels of X. The *exclusion restriction* – no direct effect of the instrument on the outcome – is a separate, substantive assumption that covariates do not generally repair. Adding X removes a direct $Z \to Y$ path only if that path runs through X, or if exclusion is conditional by design. So covariates help with conditional instrument independence, not with exclusion itself.

16.1.7. Example

```
Random.seed!(66)
nobs = 10000

# Here we have three variables w, m, z.
# m is the omitted variable, w and m are correlated.
# z is the instrument, which is correlated with w, but not m.
# u is independent of everything else.
covarMat = [
```

```
    1.0   0.36  0.64;
    0.36  1.0   0.0;
    0.64  0.0   1.0
]

mvn = MvNormal(zeros(3), covarMat)
mat = rand(mvn, nobs)'

data = DataFrame(w = mat[:, 1], m = mat[:, 2], z = mat[:, 3])
data.u = rand(Normal(0, 1), nobs)

# DGP
data.y = data.w .+ data.m .+ data.u

lm_biased = feols(data, @formula(y ~ w))
lm_full = feols(data, @formula(y ~ w + m))

# Manual 2SLS explicitly (educational)
# Stage 1: Predict endogenous variable w using instrument z
stage1 = feols(data, @formula(w ~ z))
data.w_hat = data.w .- stage1.residuals # Obtain fitted values

# Stage 2: Regress outcome y on predicted w_hat
tsls_manual = feols(data, @formula(y ~ w_hat));
```

```
# Biased OLS
println("  Biased OLS (w only)             ")
show_regression_html(lm_biased)

# Full OLS is good
println("\n  Full OLS (w + m)               ")
show_regression_html(lm_full)

# Manual 2SLS is good
# Note: the coefficients from manual 2SLS are correct, but standard
# errors are slightly incorrect because they use residuals from w_hat,
# not the true w.
println("\n  Manual 2SLS                    ")
show_regression_html(tsls_manual)
```

```
  Biased OLS (w only)
```

	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	-0.012	0.010	-1.164	0.244
w	1.357	0.010	135.823	< 2e-16 ***

	Estimate	Std. Error	t value	Pr(>|t|)

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1				

`Full OLS (w + m)`

	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	-0.009	0.010	-0.915	0.360
w	0.998	0.011	93.176	< 2e-16 ***
m	0.998	0.011	93.439	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1				

`Manual 2SLS`

	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	-0.006	0.010	-0.577	0.564
w_hat	1.000	0.016	64.426	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1				

16.1.8. Control Function Approach

The manual 2SLS above *implicitly* implements the **control function (CF)** approach. Understanding why makes the extension to nonlinear models (next chapter) transparent.

Algebraic motivation. Because `w` is endogenous, decompose the structural error:

$$\varepsilon = \rho v + \eta, \qquad v \equiv w - E[w \mid z], \qquad E(z\eta) = 0,\ E(zv) = 0. \tag{16.3}$$

If we *knew* v, controlling for it directly would eliminate the confounding channel. We don't know v, but the first-stage residual $\hat{v} = w - \hat{w}$ is a consistent estimate. Substituting into the structural equation:

$$y = \alpha + \beta w + \rho\,\hat{v} + \text{error}. \tag{16.4}$$

OLS on this **augmented** second stage gives a consistent $\hat{\beta}$ — identical to 2SLS by the Frisch–Waugh–Lovell (FWL) theorem. The coefficient $\hat{\rho}$ on $\hat{v}$ doubles as a **Durbin–Wu–Hausman (DWH) endogeneity test**: under H_0 (w is exogenous), $\rho = 0$.

```
import StatsBase: coef, stderror, coefnames

# Control function: add first-stage residual v̂ to the second stage OLS
data.v_hat = stage1.residuals

lm_cf = feols(data, @formula(y ~ w + v_hat))

# Compare point estimates from manual 2SLS and CF - should be identical by FWL
coef_tsls = coef(tsls_manual)[findfirst(==("w_hat"), coefnames(tsls_manual))]
coef_cf   = coef(lm_cf)[findfirst(==("w"),     coefnames(lm_cf))]

@printf("Manual 2SLS   ^_w = %.8f\n", coef_tsls)
@printf("Control funct ^_w = %.8f\n", coef_cf)

# DWH test: t-statistic on v̂
rho_hat = coef(lm_cf)[findfirst(==("v_hat"), coefnames(lm_cf))]
rho_se  = stderror(lm_cf)[findfirst(==("v_hat"), coefnames(lm_cf))]
@printf("\nDWH endogeneity test (^ on v̂): coef = %.4f  se = %.4f  t = %.3f\n",
        rho_hat, rho_se, rho_hat / rho_se)
```

```
Manual 2SLS   ^_w = 0.99987086
Control funct ^_w = 0.99987086

DWH endogeneity test (^ on v̂): coef = 0.6100  se = 0.0203  t = 30.077
```

The two $\hat{\beta}_w$ values agree to machine precision, confirming FWL. The large $|t|$ on $\hat{v}$ rejects $H_0 : \rho = 0$, confirming that `w` is endogenous.

> **Note**
>
> **Why FWL fails in nonlinear models.** FWL relies on the residual-maker matrix being idempotent and the second stage being linear — both fail once we replace OLS with Poisson or Probit. In a nonlinear second stage, $\hat{v}$ must enter *inside* the link function (e.g. $\exp(\mathbf{x}\beta + \rho\hat{v})$), which requires a structural assumption beyond instrument exogeneity. This is the subject of the next chapter.

16.2. When 2SLS with Covariates Is *Actually* LATE

The LATE identification above is clean because we have no covariates. The Wald estimator gives us LATE, full stop. But in practice we usually add covariates to defend conditional exogeneity:

$$Y = \beta W + \gamma^\top X + U, \qquad W = \pi Z + \delta^\top X + V. \tag{16.5}$$

Does 2SLS on this still give us a LATE? Angrist and Pischke (2009) say it gives a weighted average of covariate-specific LATEs. Blandhol et al. (2025) show that this is generally not true.

16.2.1. Why the Linear Specification Hides an Assumption

By Frisch-Waugh-Lovell, the IV estimand is

$$\beta_{iv} = \frac{E[Y\tilde{Z}]}{E[W\tilde{Z}]}, \qquad \tilde{Z} = Z - L[Z \mid X], \tag{16.6}$$

where $L[Z \mid X]$ is the **linear** projection of Z on X. So $\tilde{Z}$ is the part of Z that a linear regression on X cannot explain.

What if the true $E[Z \mid X]$ is not linear in X? Then $L[Z \mid X] \neq E[Z \mid X]$, and the residual instrument $\tilde{Z}$ still picks up some of the conditional mean. The IV estimand then mixes treatment effects across compliance groups. Blandhol et al. (2025) prove that β_{iv} is a non-negatively weighted average of conditional LATEs only if the rich covariates condition holds:

$$L[Z \mid X] = E[Z \mid X]. \tag{16.7}$$

This is a parametric assumption about $E[Z \mid X]$. It is implicit in the linear-in-X specification, and it is rarely defended. When it fails, β_{iv} picks up treatment effects for always-takers as well as compliers, and some always-taker terms enter with negative weight.

> **Warning**
>
> "2SLS with covariates estimates LATE" is true only if X enters saturated (one dummy per cell), or the true $E[Z \mid X]$ happens to be linear in X.

16.2.2. A Simulation

We build a DGP where $E[Z \mid X]$ is wildly nonlinear in X, so the linear-in-X specification has to fail. Then we compare four estimators against the true LATE.

```
using MLJ
using DecisionTree
using Random
using Statistics
using LinearAlgebra

Random.seed!(13)
n = 8000
plogis(z) = 1 / (1 + exp(-z))

# Single covariate on [-1, 1]
X = 2 .* rand(n) .- 1

# True conditional mean of Z is wildly nonlinear in X.
# A linear projection L[Z|X] cannot reproduce this.
```

```
pZ_true = plogis.(0.3 .+ 1.0 .* X .+ 2.0 .* sin.(2.5 .*   .* X))
Z       = Int.(rand(n) .< pZ_true)

# Potential treatment states with monotonicity (T1 >= T0).
# X enters the propensity, so X confounds W <-> Y.
U  = rand(n)
p0 = plogis.(-1.6 .+ 1.0 .* X)
p1 = min.(plogis.(-0.4 .+ 1.0 .* X) .+ 0.2, 0.95)
T0 = Int.(U .< p0)
T1 = Int.(U .< p1)
W  = ifelse.(Z .== 1, T1, T0)

G  = [t1 == 1 && t0 == 1 ? :AT :
      t1 == 0 && t0 == 0 ? :NT : :CP
      for (t1, t0) in zip(T1, T0)]

# Heterogeneous treatment effect - varies with X
   = 0.5 .+ 1.5 .* X
Y0 = 1.0 .+ 2.0 .* X .+ 0.8 .* X .^ 2 .+ randn(n)
Y1 = Y0 .+
Y  = ifelse.(W .== 1, Y1, Y0)

true_LATE = mean( [G .== :CP])
@printf("Compliance shares  AT=%.2f  CP=%.2f  NT=%.2f\n",
        mean(G .== :AT), mean(G .== :CP), mean(G .== :NT))
@printf("True unconditional LATE = %.3f\n", true_LATE)
```

```
Compliance shares  AT=0.19  CP=0.42  NT=0.39
True unconditional LATE = 0.617
```

Now we compare four estimators. Julia has no equivalent of R's `DoubleML` package, so we implement DDML PLIV manually with 5-fold cross-fitting using random-forest nuisance estimates. This is the same pattern the `nonparametric.qmd` chapter uses for DDML PLR.

```
# Helper: TSLS via the matrix expression (X is the exogenous matrix incl. constant).
function tsls_matrix(Y, W, Z, Xmat)
    Wmat = hcat(W, Xmat)
    Zmat = hcat(Z, Xmat)
    P    = Zmat * ((Zmat' * Zmat) \ Zmat')
    What = P * Wmat
         = (What' * Wmat) \ (What' * Y)
    return  [1]
end

# (a) Wald - biased: Z is not unconditionally exogenous because X confounds.
wald = (mean(Y[Z .== 1]) - mean(Y[Z .== 0])) /
```

```
       (mean(W[Z .== 1]) - mean(W[Z .== 0]))

# (b) TSLS with X entered linearly - the textbook spec.
b_lin  = tsls_matrix(Float64.(Y), Float64.(W), Float64.(Z),
                     hcat(ones(n), X))

# (c) TSLS with a rich polynomial basis - closer to saturation.
b_poly = tsls_matrix(Float64.(Y), Float64.(W), Float64.(Z),
                     hcat([X .^ k for k in 0:7]...))

# (d) DDML PLIV with random-forest nuisance estimates and 5-fold cross-fitting.
RandomForestRegressor = @load RandomForestRegressor pkg=DecisionTree verbosity=0
learner = RandomForestRegressor(n_trees=500, max_depth=5)

nsplits = 5
s       = shuffle(1:n)
folds   = [s[1 + floor(Int, (k-1)*n/nsplits) : floor(Int, k*n/nsplits)]
           for k in 1:nsplits]

mY = zeros(n); mW = zeros(n); mZ = zeros(n)
Xtab = DataFrame(X = X)

for fold_idx in 1:nsplits
    test  = folds[fold_idx]
    train = setdiff(1:n, test)
    for (out, dest) in ((Y, mY), (W, mW), (Z, mZ))
        mach = machine(learner, Xtab[train, :], Float64.(out[train]))
        MLJ.fit!(mach, verbosity = 0)
        dest[test] = MLJ.predict(mach, Xtab[test, :])
    end
end

rY = Y .- mY; rW = W .- mW; rZ = Z .- mZ
b_pliv = sum(rZ .* rY) / sum(rZ .* rW)

results = DataFrame(
    Estimator = ["True LATE",
                 "Wald (no covariates)",
                 "TSLS, X linear",
                 "TSLS, poly(X, 7)",
                 "DDML PLIV (manual, RF)"],
    Estimate  = [true_LATE, wald, b_lin, b_poly, b_pliv]
)
results
```

	Estimator	Estimate
	String	Float64
1	True LATE	0.616857
2	Wald (no covariates)	1.91029
3	TSLS, X linear	0.551637
4	TSLS, poly(X, 7)	0.585279
5	DDML PLIV (manual, RF)	0.581937

The linear-in-X 2SLS does not recover the true LATE. The polynomial 2SLS is closer, and DDML PLIV is close to the polynomial estimate. The Wald estimator is far off because Z is not unconditionally exogenous, and Wald has no way to use X.

What are the polynomial 2SLS and DDML PLIV estimating, if not LATE? Both target β_{rich}, a weighted average of conditional LATEs with weights proportional to $\text{Var}(Z \mid X) \cdot P(\text{complier} \mid X)$. It is non-negatively weighted, so it is weakly causal. But it is not the unconditional LATE we usually want.

16.2.3. Diagnostic: RESET on Z ~ X

Rich covariates says $E[Z \mid X]$ is linear in X. That is exactly the null of Ramsey's RESET test (Ramsey 1969). Julia has no direct equivalent of R's `lmtest::resettest()`, but the test is just an F-test of joint significance of higher powers of X.

```
using GLM

df_reset       = DataFrame(Z = Float64.(Z), X = X, X2 = X .^ 2, X3 = X .^ 3)
m_restricted   = lm(@formula(Z ~ X),                df_reset)
m_unrestricted = lm(@formula(Z ~ X + X2 + X3),      df_reset)
rss_r          = sum(residuals(m_restricted) .^ 2)
rss_u          = sum(residuals(m_unrestricted) .^ 2)
q              = 2          # restrictions
k_u            = 4          # parameters in unrestricted
F_stat         = ((rss_r - rss_u) / q) / (rss_u / (n - k_u))
pval           = 1 - cdf(FDist(q, n - k_u), F_stat)
@printf("RESET-style F on Z ~ X (joint test of X^2, X^3): F = %.2f, p = %.4g\n",
        F_stat, pval)
```

```
RESET-style F on Z ~ X (joint test of X^2, X^3): F = 131.57, p = 0
```

In our DGP it rejects strongly. The same test on real data is cheap, and it tells us whether the LATE interpretation is defensible before we report a 2SLS coefficient.

16.2.4. What to Do Instead

Blandhol et al. (2025) give four steps for empirical work.

1. **Reconsider the covariates.** If Z is unconditionally exogenous, drop them. A kitchen-sink set of controls makes rich covariates *less* likely.
2. **Run RESET on `Z ~ X`.** If it does not reject, the linear-IV-as-LATE interpretation is defensible.
3. **If RESET rejects, report DDML PLIV alongside 2SLS.** Julia has no first-class PLIV package, so the manual cross-fitting above is the recommended route. In R the package is `DoubleML`; in Stata it is `ddml`. DDML PLIV targets β_{rich}, a non-negatively weighted average of conditional LATEs.
4. **For a binary instrument, also estimate the unconditional LATE.** With instrument propensity score weighting (Słoczyński 2024),

$$\hat{\beta}_{late} = \frac{\sum_i Y_i[Z_i/\hat{p}(X_i) - (1-Z_i)/(1-\hat{p}(X_i))]}{\sum_i W_i[Z_i/\hat{p}(X_i) - (1-Z_i)/(1-\hat{p}(X_i))]}, \tag{16.8}$$

   where $\hat{p}(X) = P(Z = 1 \mid X)$ is estimated nonparametrically. Stata has `kappalate` (Słoczyński, Uysal, and Wooldridge). In Julia or R it is straightforward to build once $\hat{p}(X)$ is in hand.

16.2.5. A Note on the FRDD Example

The fuzzy-RDD example at the end of this chapter is 2SLS with race, state of birth, and quarter-of-birth fixed effects entered as covariates. The 2SLS estimate and the local `RDRobust` estimate differ. We called that "sensitivity to model selection" earlier. The rich-covariates story is at least as plausible an explanation, and the RESET test is the right first thing to run.

16.2.6. Bottom Line

"2SLS with covariates estimates LATE" needs the rich covariates condition. That condition is a parametric assumption on $E[Z \mid X]$ that researchers rarely defend, and a simple RESET test often rejects it. The honest alternatives are DDML PLIV for β_{rich} and IPSW (or DDML) for the unconditional LATE. Even when rich covariates holds, β_{rich} can be quite different from the unconditional LATE we usually care about.

16.3. Regression Discontinuity Design

RDD (regression discontinuity design) is a quasi-experimental design that is used to estimate causal effects of interventions when assignment to the intervention is determined by whether a subject's value on an observed covariate exceeds a threshold. The idea is that the assignment is as good as random, so we can estimate the causal effect of the intervention by comparing the outcomes of subjects who are just above and just below the threshold.

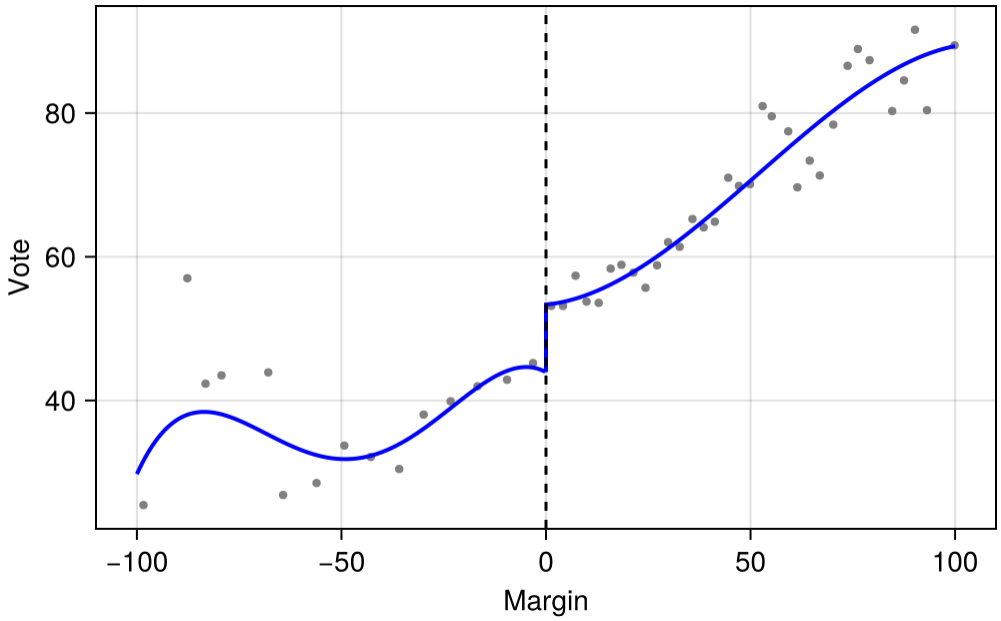
16.3.1. Sharp RDD

We have a continuous variable X, called the running variable, which determines the binary treatment W. The treatment is assigned according to a threshold c. The outcome variable Y is a function of X and W.

$$W = \mathbb{1}(X > c) \tag{16.9}$$

Lee (2008) studies the effect of incumbency advantage in elections. His identification strategy is based on the discontinuity generated by the rule that the party with a majority vote share wins. The forcing variable X_i is the difference in vote share between the Democratic and Republican parties in one election, with the threshold $c = 0$. The outcome variable Y_i is vote share at the second election.

```
using CSV
senate = CSV.read("data/rdrobust_senate.csv", DataFrame)

# Calculate RD plot data
plot_data = rdplot(senate.vote, senate.margin, c=0.0)

fig = Figure()
ax = Axis(fig[1, 1], xlabel="Margin", ylabel="Vote")
scatter!(ax, plot_data.vars_bins.rdplot_mean_x, plot_data.vars_bins.rdplot_mean_y,
         color=:gray, markersize=6)

# Plot polynomial fit for left side
poly_l = filter(r -> r.rdplot_x < 0.0, plot_data.vars_poly)
lines!(ax, poly_l.rdplot_x, poly_l.rdplot_y, color=:blue, linewidth=2)

# Plot polynomial fit for right side
poly_r = filter(r -> r.rdplot_x >= 0.0, plot_data.vars_poly)
lines!(ax, poly_r.rdplot_x, poly_r.rdplot_y, color=:blue, linewidth=2)

vlines!(ax, [0.0], color=:black, linestyle=:dash)
fig
```



16.3.2. Identification of SRDD

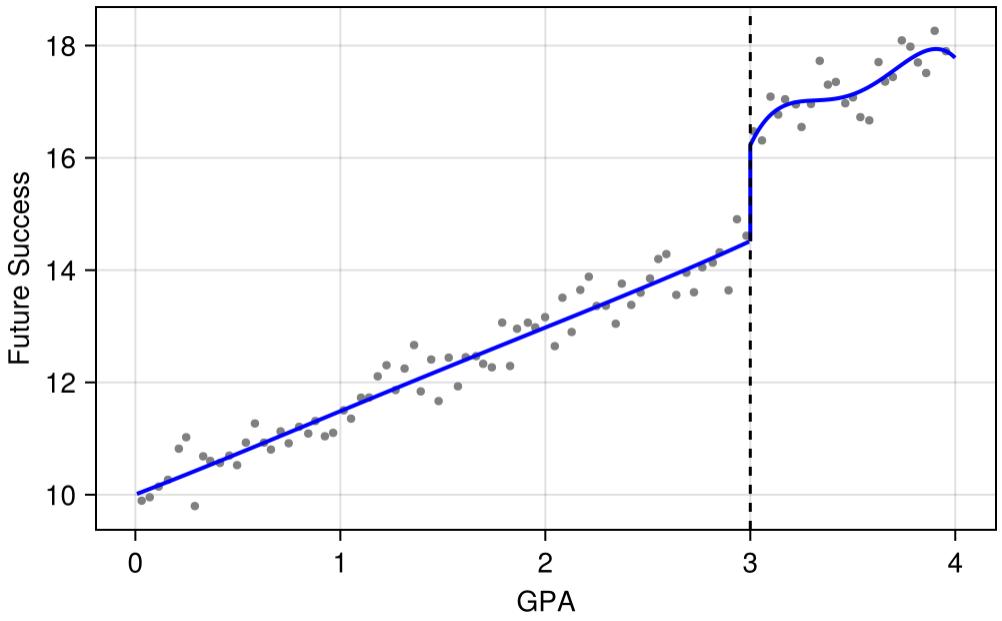
$$\tau_{RD} = \lim_{x \to c^+} E[Y|X = x] - \lim_{x \to c^-} E[Y|X = x] \tag{16.10}$$

In words, the treatment effect is the difference in the outcome from the right side and from the left side. This is the same as the difference in the outcome at the threshold, as if the treatment is assigned randomly.

16.3.3. Estimation of SRDD

```
GPA = rand(Uniform(0, 4), 1000)
# Treatment is W = 1(GPA > 3), so the +2 jump must sit on the right
# (above-cutoff) side to match the right-minus-left estimand above.
future_success = 10 .+ 1.5 .* GPA .+ 2 .* (GPA .> 3) .+ rand(Normal(0, 1), 1000)

plot_data_gpa = rdplot(future_success, GPA, c=3.0)

fig2 = Figure()
ax2 = Axis(fig2[1, 1], xlabel="GPA", ylabel="Future Success")
scatter!(ax2, plot_data_gpa.vars_bins.rdplot_mean_x, plot_data_gpa.vars_bins.rdplot_mean_y,
         color=:gray, markersize=6)

poly_gpa_l = filter(r -> r.rdplot_x < 3.0, plot_data_gpa.vars_poly)
lines!(ax2, poly_gpa_l.rdplot_x, poly_gpa_l.rdplot_y, color=:blue, linewidth=2)
```

```
poly_gpa_r = filter(r -> r.rdplot_x >= 3.0, plot_data_gpa.vars_poly)
lines!(ax2, poly_gpa_r.rdplot_x, poly_gpa_r.rdplot_y, color=:blue, linewidth=2)

vlines!(ax2, [3.0], color=:black, linestyle=:dash)
fig2
```




```
# Estimate the sharp RDD model
rdd_gpa = rdrobust(future_success, GPA, c=3.0)
@printf "Tau (conventional)   = %.4g\n" rdd_gpa.Estimate.tau_us[1]
@printf "P-value (robust)     = %.4f\n" rdd_gpa.pv.PValue[3]
```

```
Tau (conventional)   = 1.61
P-value (robust)     = 0.0003
```

16.3.4. Nonparametric Estimation

```
rdd_house = rdrobust(senate.vote, senate.margin, c=0.0)
@printf "Tau (conventional)   = %.4g\n" rdd_house.Estimate.tau_us[1]
@printf "P-value (robust)     = %.4f\n" rdd_house.pv.PValue[3]
```

```
Tau (conventional)   = 7.414
P-value (robust)     = 0.0000
```

16.3.5. Fuzzy RDD

In fuzzy RDD, the treatment is assigned according to a threshold c, but there exist non-compliance. This is similar to IV case.

$$\tau_{FRD} = \frac{\lim_{x \to c^+} E[Y|X = x] - \lim_{x \to c^-} E[Y|X = x]}{\lim_{x \to c^+} E[W|X = x] - \lim_{x \to c^-} E[W|X = x]} \tag{16.11}$$

For example, if food stamp eligibility is given to all households below a certain income, but not all households receive the food stamps. In other words, income does not solely determine the assignment. Cutoff point increases the probability of treatment but doesn't completely determine treatment.

FRDD is IV.

16.3.6. FRDD Example

Fetter (2013)'s main question of interest is how much of the increase in the home ownership rate in the midcentury US was due to mortgage subsidies given out by the government.

We're using the running variable quarter of birth (qob), which has been centered on the quarter of birth you'd need to be to be eligible for a mortgage subsidy for fighting in the Korean War (qob_minus_kw). This determines whether you were a veteran of either the Korean War or World War II (vet_wwko).

```
using CSV

# Load the real mortgages dataset (Fetter 2013) from causaldata R package
vet = CSV.read("data/mortgages.csv", DataFrame)

# Create an "above-cutoff" variable as the instrument
vet.above = vet.qob_minus_kw .> 0

# Impose a bandwidth of 12 quarters on either side
vet = vet[abs.(vet.qob_minus_kw) .< 12, :]

# Manual 2SLS explicitly (educational)
# We have Fixed Effects for state (bpl) and quarter of birth (qob).
# Endogenous variable: vet_wwko and qob_minus_kw * vet_wwko
# Instruments: above and qob_minus_kw * above
vet.vet_inter = vet.qob_minus_kw .* vet.vet_wwko
vet.above_inter = vet.qob_minus_kw .* vet.above

# Stage 1a: Predict vet_wwko
stage1a = feols(vet, @formula(vet_wwko ~ nonwhite + qob_minus_kw + above + above_inter + fe(bp
vet.vet_wwko_hat = vet.vet_wwko .- stage1a.residuals

# Stage 1b: Predict vet_inter
```

```
stage1b = feols(vet, @formula(vet_inter ~ nonwhite + qob_minus_kw + above + above_inter + fe(bp
vet.vet_inter_hat = vet.vet_inter .- stage1b.residuals

# Stage 2: Regress home_ownership on fitted endogenous variables
# Note: we use RobustVcov for heteroskedasticity-robust SEs
m_iv = feols(vet, @formula(home_ownership ~ nonwhite + qob_minus_kw + vet_wwko_hat + vet_inter_
show_regression_html(m_iv)
```

	Estimate	Std. Error	t value	Pr(>|t|)
nonwhite	-0.190	0.007	-27.951	< 2e-16 ***
qob_minus_kw	-0.007	0.002	-4.070	4.707e-05 ***
vet_wwko_hat	0.170	0.046	3.733	1.894e-04 ***
vet_inter_hat	-0.003	0.003	-1.092	0.275

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```
# Fuzzy RDD using rdrobust (local polynomial, MSE-optimal bandwidth)
# Note: rdrobust uses a data-driven bandwidth - no global FE needed
m_rdd = rdrobust(vet.home_ownership,
                 vet.qob_minus_kw,
                 fuzzy = vet.vet_wwko,
                 c = 0.0)
@printf "Manual 2SLS     tau = %.4g\n" m_iv.beta[3]
@printf "RDRobust (FRDD) tau = %.4g\n" m_rdd.Estimate.tau_us[1]
```

```
Manual 2SLS     tau = 0.1702
RDRobust (FRDD) tau = 1.879
```

These two results are very different, which is one of RDD's problems. It can be sensitive to model selection. For example, in the case of 2SLS, it's a linear model. In the case of `rdrobust`, it's a local regression, which can be sensitive to bandwidth selection.

17. Shift-Share Instrumental Variables

```
using DataFrames
using LinearAlgebra
using Statistics
using Random
using CairoMakie
CairoMakie.activate!(type = "png")
using Panelest
using StatsModels
using CSV

using ShiftShareIV
```

Shift-share, or Bartik, instruments answer one specific problem, so it is clearest to write the regression down first.

We want the effect of a local economic shock on a local outcome. For region ℓ,

$$Y_\ell \;=\; \beta X_\ell + Z_\ell'\gamma + u_\ell, \tag{17.1}$$

where X_ℓ is the shock the region actually experienced — local import competition per worker, in the China-shock literature — and Y_ℓ is the outcome, say the change in the local manufacturing employment share. The parameter we want is β.

The endogenous regressor is X_ℓ. Regions do not receive their shocks at random. A region whose import competition rises sharply may be a region whose industries were already losing ground, and whatever caused that decline sits in u_ℓ and moves Y_ℓ on its own. Then $\mathrm{Cov}(X_\ell, u_\ell) \neq 0$, and OLS mixes the effect of the shock with the local conditions that produced it. We cannot sign that bias in advance; it depends on whether those local conditions push the outcome up or down.

The shift-share instrument replaces the realised local shock with a predicted one, assembled from two pieces that are meant to have nothing to do with u_ℓ: each region's *baseline* industry shares, fixed before the outcome period, and *national* or foreign industry-level shocks, common across regions. A region with a large baseline share in an industry is mechanically more exposed to a shock hitting that industry, and that exposure is predicted rather than realised.

Building the instrument is the easy part. The difficult part is saying which of the two pieces carries the exogeneity assumption — the shares, the shocks, or both — because the two answers lead to different identification arguments, different diagnostics, and different standard errors. That is what the rest of the chapter is about.

17.1. The construction

For each location ℓ and industry k, let $s_{\ell k}$ be the share of local employment in industry k at baseline, and let g_k be a national growth rate (or trade shock) for that industry. The shift-share instrument is

$$B_\ell \;=\; \sum_{k=1}^{K} s_{\ell k}\, g_k. \tag{17.2}$$

This is "shift-share" because it combines a local share with an industry-level shift. Bartik (1991) used local industry mix to predict local employment growth. Autor, Dorn, and Hanson (2013) used local industry shares interacted with industry-level Chinese import growth in other high-income countries.

The first-stage regression is then

$$\Delta L_\ell \;=\; \pi_0 + \pi_1 B_\ell + X_\ell'\gamma + \epsilon_\ell, \tag{17.3}$$

with B_ℓ instrumenting for the observed local shock in a 2SLS for the outcome of interest.

17.2. Two identification views

There are two ways to justify a shift-share instrument. They put the exogeneity assumption on different objects.

17.2.1. Share view (GPSS 2020)

Treat the shares $s_{\ell k}$ as the source of identifying variation, with the shocks g_k acting as weights. The instrument is valid if shares are exogenous to the outcome (conditional on controls). GPSS prove that the shift-share IV is numerically equivalent to a GMM combination of K just-identified IVs, one per industry share:

$$\hat{\beta}^{SS} \;=\; \sum_{k=1}^{K} \hat{\alpha}_k\, \hat{\beta}_k, \tag{17.4}$$

where $\hat{\beta}_k$ is the just-identified IV using share $s_{\cdot k}$ alone and $\hat{\alpha}_k$ is the **Rotemberg weight**. This gives an important diagnostic: which industry shares are driving the estimate?

17.2.2. Shock view (BHJ 2022)

Treat the shocks g_k as the source of identifying variation. Shares are exposure weights. Under this view, shocks should be uncorrelated with the second-stage unobservables, conditional on industry-level controls. BHJ show how to rewrite the regression at the shock level.

The two views are not mutually exclusive. In an application, we should be clear which one is being defended and report diagnostics for both.

17.3. Inference

Standard OLS or cluster-robust standard errors on the regional regression **underestimate uncertainty** because the same industry shocks appear in many regions, inducing cross-region correlation that region-level clustering does not capture. Two corrections are now standard:

- **Cluster on shocks (BHJ)**: equivalent to running the regression at the industry-shock level. Implemented via the `bhj_collapse` function.
- **AKM SE (Adão, Kolesár, Morales 2019)**: explicit formula accounting for shock-level correlation; requires custom implementation or a dedicated package.

17.4. Simulation: build intuition

Use a small DGP with regions, industries, and a known causal effect.

```
# The CSVs below are 500 locations x 20 industries; beta_true is the DGP coefficient.
beta_true  = 0.5

df     = CSV.read("data/shift_share_sim.csv", DataFrame)
shares = Matrix(CSV.read("data/shift_share_shares.csv", DataFrame))
shocks = CSV.read("data/shift_share_shocks.csv", DataFrame).shock
X = df.X; Y = df.Y; u = df.u
first(df[:, [:region, :X, :Y, :u]], 6)
```

	region	X	Y	u
	Int64	Float64	Float64	Float64
1	1	-0.518194	0.11729	-0.308258
2	2	0.212703	-1.06765	-0.538806
3	3	0.652136	0.892118	1.37164
4	4	0.285676	0.861261	1.09974
5	5	-0.334493	-0.329605	-0.111544
6	6	0.37613	-1.72747	-1.75462

A naive OLS suffers from the confounder `u`:

```
ols  = feols(df, @formula(Y ~ X))
ivss = feiv(df, @formula(Y ~ 1), endo = :X, inst = :B)

@printf("%-25s %8s %8s\n", "Estimator", "Coef", "SE")
@printf("%-25s %8.3f %8.3f\n", "OLS (biased)",
        coef(ols)[end], stderror(ols)[end])
@printf("%-25s %8.3f %8.3f\n", "Shift-share IV",
        coef(ivss)[1], stderror(ivss)[1])
@printf("True beta: %.3f\n", beta_true)
```

```
Estimator                      Coef       SE
OLS (biased)                  1.451    0.079
Shift-share IV                0.531    0.120
True beta: 0.500
```

OLS is biased upward by the confounder; the shift-share IV recovers a value close to the true effect.

17.4.1. Rotemberg weights

The GPSS decomposition says the shift-share IV is a weighted average of K just-identified IVs (one per industry share). Compute the weights:

```
rw = rotemberg_weights(shares, shocks, X, Y)

@printf("%-10s %8s %8s %8s\n", "Industry", "Shock", " _k", " _k")
for row in eachrow(rw)
    @printf("%-10d %8.3f %8.3f %8.3f\n",
            row.industry, row.shock, row.alpha, row.beta_k)
end
@printf("\nGPSS identity: Σ  _k  _k = %.4f\n", sum(rw.alpha_beta))
@printf("Shift-share IV estimate: %.4f\n", coef(ivss)[1])
```

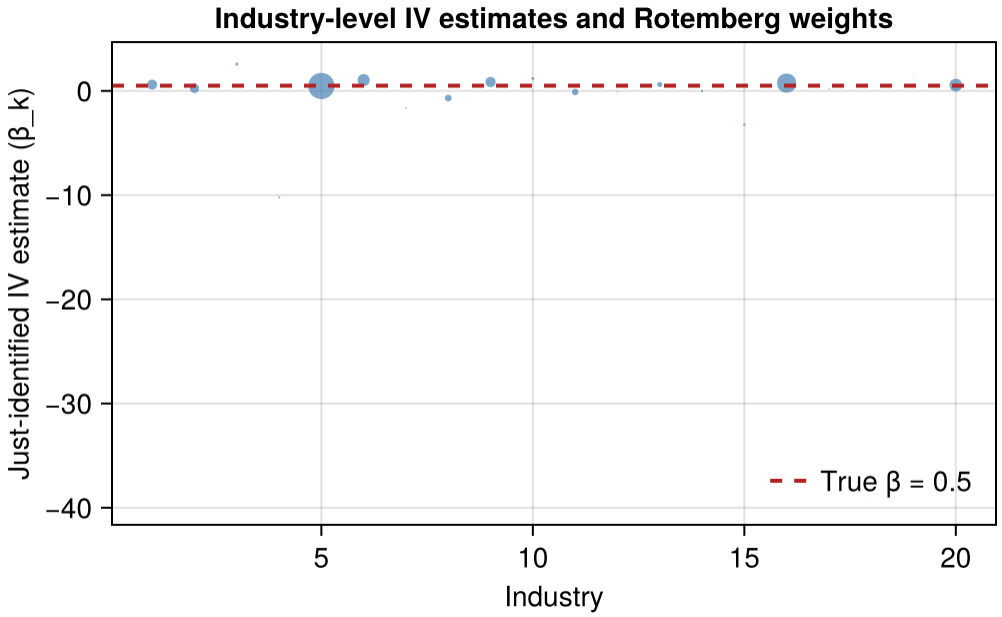
```
Industry       Shock       _k       _k
1              1.250    0.054    0.604
2             -0.959    0.048    0.244
3             -0.212    0.005    2.576
4              0.745    0.002  -10.225
5              2.663    0.387    0.469
6             -1.068    0.082    1.025
7              0.100   -0.001   -1.627
8             -0.718    0.025   -0.689
9              1.181    0.058    0.853
10            -0.197    0.004    1.188
11             0.741    0.023   -0.112
```

```
12             0.051   -0.001   -0.055
13            -0.442    0.014    0.611
14             0.270    0.003   -0.013
15             0.631    0.003   -3.240
16             2.271    0.204    0.746
17            -0.212    0.001    0.174
18            -0.115    0.000  -39.532
19             0.020   -0.000    1.234
20            -1.183    0.089    0.554

GPSS identity: Σ  _k  _k = 0.5309
Shift-share IV estimate: 0.5309
```

Two diagnostics matter:

1. **Weight concentration**: if one or two industries carry most of the weight, the identifying variation is essentially coming from those industries' shares. Their exogeneity is the assumption you are really leaning on.
2. **Heterogeneous** $\hat{\beta}_k$: in this benign simulation, the industry-specific IVs are all near the true effect (with sampling noise). In real data, if a few industries give wildly different estimates, the "average" shift-share IV is averaging over heterogeneous treatment effects you should report explicitly.

```
fig = Figure(size = (640, 380))
ax  = Axis(fig[1, 1],
           xlabel = "Industry",
           ylabel = "Just-identified IV estimate ( _k)",
           title  = "Industry-level IV estimates and Rotemberg weights")

# Bubble plot: position = beta_k, size = |alpha_k|
scatter!(ax, rw.industry, rw.beta_k,
         markersize = 30 .* sqrt.(abs.(rw.alpha)),
         color = (:steelblue, 0.7))
hlines!(ax, [beta_true], color = :firebrick, linestyle = :dash,
        linewidth = 2, label = "True   = $(beta_true)")
axislegend(ax, position = :rb, framevisible = false)
fig
```



17.4.2. What goes wrong when shares are endogenous

Now break the share-exogeneity assumption: make industry 1's share correlated with the second-stage error.

```
v = CSV.read("data/shift_share_bad_v.csv", DataFrame).v
shares_bad = copy(shares)
shares_bad[:, 1] .= max.(0.01, shares[:, 1] .+ 0.15 .* v)
shares_bad = shares_bad ./ sum(shares_bad, dims=2)   # renormalise

bad_noise = CSV.read("data/shift_share_bad_noise.csv", DataFrame)
B_bad = bartik_iv(shares_bad, shocks)
X_bad = B_bad .+ 0.3 .* u .+ bad_noise.noise_x
Y_bad = beta_true .* X_bad .+ u .+ 0.6 .* v .+ bad_noise.noise_y

df_bad  = DataFrame(X = X_bad, Y = Y_bad, B = B_bad)
iv_bad  = feiv(df_bad, @formula(Y ~ 1), endo = :X, inst = :B)

@printf("True beta: %.3f\n", beta_true)
@printf("Shift-share IV with bad share 1: %.3f\n", coef(iv_bad)[1])

# Would the Rotemberg weights have flagged industry 1?
rw_bad = rotemberg_weights(shares_bad, shocks, X_bad, Y_bad)
ord    = sortperm(abs.(rw_bad.alpha), rev = true)
rank1  = findfirst(==(1), rw_bad.industry[ord])
@printf("Rotemberg weight of industry 1: %.3f (rank %d of %d by |·|)\n",
        rw_bad.alpha[rw_bad.industry .== 1][1], rank1, nrow(rw_bad))
```

```
# Leave-one-industry-out: rebuild the instrument without industry 1
B_loo  = bartik_iv(shares_bad[:, 2:end], shocks[2:end])
df_loo = DataFrame(X = X_bad, Y = Y_bad, B = B_loo)
iv_loo = feiv(df_loo, @formula(Y ~ 1), endo = :X, inst = :B)
@printf("Leave-industry-1-out IV: %.3f\n", coef(iv_loo)[1])
```

```
True beta: 0.500
Shift-share IV with bad share 1: 0.798
Rotemberg weight of industry 1: 0.110 (rank 3 of 20 by |·|)
Leave-industry-1-out IV: 0.480
```

The estimate is biased, and the bias comes from one industry's share being related to the error. Note what the diagnostics do and do not catch here. The endogenous industry's Rotemberg weight is unremarkable — it ranks third of twenty — because the weights measure *influence* (whose exogeneity the estimate leans on most), not *endogeneity*. An industry with a modest weight can still move the estimate substantially when its share is correlated with the error. What isolates the problem is the **leave-one-industry-out** check: dropping industry 1 from the instrument restores an estimate close to the truth.

17.5. BHJ shock-level inference

Collapse the location-level data to the shock level and run a weighted IV at the shock (industry) level. This reframes the identifying assumption: shocks must be uncorrelated with the shock-level aggregated outcome residual.

```
collapsed = bhj_collapse(shares, shocks, Y, X)

# BHJ shock-level 2SLS: regress Y_agg ~ X_agg | shock, weighted by weight
df_collapsed = collapsed
iv_bhj = feiv(df_collapsed, @formula(Y_agg ~ 1),
              endo = :X_agg, inst = :shock, weights = :weight)

@printf("Location-level shift-share IV: %.3f\n", coef(ivss)[1])
@printf("BHJ shock-level IV:             %.3f\n", coef(iv_bhj)[1])
@printf("(Should be close; equivalent in the no-controls case)\n")
```

```
Location-level shift-share IV: 0.531
BHJ shock-level IV:             0.537
(Should be close; equivalent in the no-controls case)
```

The BHJ shock-level regression coincides with the location-level IV when there are no controls — up to implementation details of the weighting, so expect close rather than identical values in practice. Its value is interpretive: validity is now a statement about shocks (20 observations, one per industry) rather than locations (500 observations), and testing shock-level exogeneity is more transparent.

17.6. Empirical: the China shock (Autor, Dorn, Hanson 2013)

The standard shift-share application is ADH (2013). Commuting zones are exposed differently to Chinese import competition because their baseline industry mixes differ. The instrument is

$$\text{IV}_\ell = \sum_k \frac{L_{\ell k,\,1990}}{L_{\ell,\,1990}} \frac{\Delta M^{\text{other}}_k}{L_{k,\,1990}}, \tag{17.5}$$

where the shocks $\Delta M^{\text{other}}_k$ are growth in Chinese imports into other high-income countries. This leave-one-out construction removes US-specific demand from the shock.

The snippet below is schematic — the full ADH specification additionally needs period effects, the ADH control set, and population weights (a validated replication against the GPSS benchmark lives in the `ShiftShareIV.jl` repository).

```
# David Dorn distributes the replication data at ddorn.net
# Files needed:
#   workfile_china.csv  (commuting zone data, 1990-2007)
#   industry_shares.csv (czone × industry shares)

using CSV
cz     = DataFrame(CSV.File("data/workfile_china.csv"))
shares = Matrix(select(cz, r"share_"))   # L × K share matrix
shocks = cz.import_shock_other            # K-vector of "other-countries" shocks

B   = bartik_iv(shares, shocks)               # Bartik instrument
df_adh = hcat(cz, DataFrame(B = B))

# First-stage + second-stage
iv_adh = feiv(df_adh, @formula(d_sh_empl_mfg ~ 1),
              endo = :d_tradeusch_pw, inst = :B,
              vcov_type = Vcov.cluster(:statefip))

# Rotemberg decomposition
rw_adh = rotemberg_weights(shares, shocks, df_adh.d_tradeusch_pw,
                           df_adh.d_sh_empl_mfg)
# Top 5 industries by |alpha|:
sort(rw_adh, :alpha, rev=true)[1:5, :]

# BHJ shock-level regression
collapsed_adh = bhj_collapse(shares, shocks, df_adh.d_sh_empl_mfg,
                             df_adh.d_tradeusch_pw)
iv_bhj_adh = feiv(collapsed_adh, @formula(Y_agg ~ 1),
                  endo = :X_agg, inst = :shock, weights = :weight)
```

For a new shift-share paper, I would expect:

1. **Rotemberg weights** (via `rotemberg_weights`): identify which industries drive the estimate; flag weight concentration and heterogeneous $\widehat{\beta}_k$.
2. **BHJ shock-level regression** (via `bhj_collapse`): reframe validity as a statement about 20 industry shocks rather than 500 commuting zones.
3. **AKM standard errors** (Adão, Kolesár, Morales 2019): account for shock-level correlation in inference. Requires custom implementation; see the accompanying R chapter for an R-based version.

17.7. Summary

- Shift-share IV combines local shares and industry shocks.
- The GPSS view puts exogeneity on shares; the BHJ view puts it on shocks.
- Rotemberg weights show which industry shares drive the 2SLS estimate.
- Region-level clustering is usually too optimistic. Use shock-level inference and AKM standard errors when possible.
- `ShiftShareIV.jl` provides `bartik_iv`, `rotemberg_weights`, and `bhj_collapse`; `Panelest.jl` provides `feols` and `feiv`.

18. IV in Poisson with Fixed Effects

Poisson regression specifies the conditional mean as an exponential function. As a quasi-MLE it needs that mean to be correct, not the full count distribution. If a regressor is endogenous, the IV strategy has to respect the nonlinear mean — and this is where intuition carried over from 2SLS starts to mislead. This chapter works through what goes wrong, what to do instead, and how much the choice costs.

```
using Panelest
using Distributions
using DataFrames, DataFramesMeta
using StatsModels
import StatsAPI: coeftable
using Random, StatsBase, LinearAlgebra
using Printf
```

18.1. The problem, and three routes to it

Throughout, y_1 is a count, y_2 is the endogenous explanatory variable, and $\mathbf{z} = (\mathbf{z}_1, \mathbf{z}_2)$ collects the exogenous variables — $\mathbf{z}_1$ appearing in the structural equation and $\mathbf{z}_2$ excluded from it. The structural conditional mean is exponential,

$$E(y_1 \mid \mathbf{z}_1, y_2, c_1) = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + c_1), \tag{18.1}$$

with c_1 an unobservable correlated with y_2. Everything below is about how to remove that correlation.

18.1.1. A note on the notation: the subscript indexes *equations*, not variables

This chapter follows Wooldridge (2010) (sec. 18.5), because it quotes his equations (18.37)–(18.45) directly and the symbols need to match the source.

The subscript is the source of most of the confusion, so it is worth saying what it means: **it indexes the equation, not the variable.** y_1 is the variable explained by equation 1, the structural equation; y_2 is the variable explained by equation 2, the reduced form. The same convention governs c_1, v_2, δ_1 and π_2 — each subscript says which equation the object belongs to. This is the simultaneous-equations convention, and it scales: a second endogenous regressor is y_3, with no renaming.

One deliberate deviation: Wooldridge writes the coefficient on y_2 as γ_1; this chapter calls it α_1 throughout. Quoted equations are left exactly as he writes them.

18.1.2. The three estimators

Three estimators recur, and it is worth naming them by what they *do* rather than by letters, since the names then carry the argument:

Name	First stage	What enters the second stage	Family
Linear-residual CF	OLS projection of y_2 on $\mathbf{z}$	the residual $\hat{v}_2$, inside $\exp(\cdot)$	control function
Generalized-residual CF	probit or logit for y_2	the generalized residual $\hat{r}_2$, inside $\exp(\cdot)$	control function
Multiplicative-moment GMM	any model for $\Pr(y_2 = 1 \mid \mathbf{z})$	the fitted value $\hat{p}_2$, as an *instrument*	GMM / IV

Earlier drafts of this chapter called these Approach A, B and C. The letters are gone. Linear-residual CF was A, generalized-residual CF was B, and multiplicative-moment GMM was C. The third is not a control function at all: it does not modify the exponential mean, it changes the moment conditions. Grouping all three as "approaches" obscured that, and the distinction matters for when each is valid.

The plan: the theory first, in one place; then the three compared side by side; then simulations, continuous case and binary case.

18.2. Theory

18.2.1. The control function idea

18.2.1.1. A linear second stage: the CF *is* 2SLS

Set the exponential mean aside and take the linear analogue of Equation 18.1,

$$y_1 = \mathbf{z}_1\delta_1 + \alpha_1 y_2 + \varepsilon_1, \qquad E(\mathbf{z}'\varepsilon_1) = 0,$$

which is what defines ε_1. Two-stage least squares fits

$$\begin{aligned} \text{first stage:} \quad & y_2 = \mathbf{z}\pi_2 + v_2, \\ \text{second stage:} \quad & y_1 = \mathbf{z}_1\delta_1 + \alpha_1\hat{y}_2 + (\varepsilon_1 + \alpha_1\hat{v}_2), \end{aligned}$$

with $\hat{y}_2 = \mathbf{z}\hat{\pi}_2$ and $\hat{v}_2 = y_2 - \hat{y}_2$. The second-stage error is not ε_1: replacing y_2 by $\hat{y}_2$ moves $\alpha_1\hat{v}_2$ into it. Add $\hat{v}_2$ to an OLS second stage instead, and you recover the 2SLS coefficient exactly. The algebra:

1. **First stage.** $y_2 = \mathbf{z}\pi_2 + v_2$, by OLS. The residual satisfies $E(\mathbf{z}'\hat{v}_2) = 0$ by construction.

2. **Error decomposition.** Project ε_1 on v_2: $\varepsilon_1 = \rho v_2 + \eta_1$ with $\rho = \mathrm{Cov}(\varepsilon_1, v_2)/\mathrm{Var}(v_2)$. Then $E(v_2\eta_1) = 0$ by construction of the projection, and $E(\mathbf{z}'\eta_1) = E(\mathbf{z}'\varepsilon_1) - \rho E(\mathbf{z}'v_2) = 0$.
3. **Augmented second stage.** $y_1 = \mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho\hat{v}_2 + \eta_1$.

OLS on step 3 is the **control function estimator**, and Frisch–Waugh–Lovell makes it numerically identical to 2SLS. No assumption beyond $E(\mathbf{z}'\varepsilon_1) = 0$ is required: step 2 is a projection, so both of its orthogonality conditions are consequences of that moment condition and the first stage, not additions to them.

18.2.1.2. A nonlinear second stage: the CF is a genuine assumption

With an exponential mean, that safety net disappears. A direct IV/GMM approach solves moment conditions in $\mathbf{z}$ without touching the exponential mean, and the form of those moments matters:

- **Additive moments** $E[\mathbf{z}'(y_1 - \exp(\mathbf{x}\beta))] = 0$ assume an *additive* error orthogonal to $\mathbf{z}$.
- **Multiplicative moments** $E[\mathbf{z}'(y_1 e^{-\mathbf{x}\beta} - 1)] = 0$ (Mullahy 1997) assume a *multiplicative* error with mean independent of $\mathbf{z}$.

When the unobservable enters **inside** the exponential — y_1 has mean $\exp(\mathbf{x}\beta + \rho v)$, as in both DGPs of this chapter — the error is multiplicative, the additive moments are misspecified, and additive-moment IV is inconsistent. The multiplicative form is the robust default, and both are coded below so the difference is visible.

The control function does something different. It imposes the stronger *conditional mean assumption* (Wooldridge 2010, sec. 18.5; 2014), Wooldridge's eq. (18.40):

$$E[y_1 \mid \mathbf{z}_1,\, y_2,\, v_2] = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \rho v_2). \tag{18.2}$$

The unobservable enters the mean **linearly inside** $\exp(\cdot)$. Under this, substituting $\hat{v}_2$ for v_2 and running Poisson QMLE is consistent for α_1. Without it, nothing rescues the procedure.

Property	Linear (OLS/2SLS)	Nonlinear (Poisson CF)
Extra assumption beyond IV exogeneity	None — FWL applies	CF mean assumption required
CF vs. 2SLS/GMM numerically	**Identical**	**Different** — FWL fails
SE from second stage alone	Correct	**Understated** — bootstrap needed
Test of endogeneity	t-stat on $\hat{v}_2$ (DWH)	Same — t-stat on $\hat{v}_2$

Where the decomposition gets applied. The decomposition $\varepsilon_1 = \rho v_2 + \eta_1$ is the same in both cases. What changes is *where* it enters: additively outside any transformation in the linear model, inside $\exp(\cdot)$ in Poisson, so that the quasi-MLE score can absorb it. If $\rho \neq 0$ and $\hat{v}_2$ is omitted from the exponential mean, a multiplicative $e^{\rho v_2}$ term correlated with y_2 survives, biasing $\hat{\alpha}_1$ in the direction of ρ.

18.2.2. A continuous y_2: the linear reduced form is enough

For continuous y_2 the assumptions are the natural ones. Wooldridge's eqs. (18.38)–(18.39) are

$$y_2 = \mathbf{z}\pi_2 + v_2, \qquad c_1 = \rho_1 v_2 + e_1, \tag{18.3}$$

with v_2 **independent** of $\mathbf{z}$ and e_1 independent of $(\mathbf{z}, v_2)$. Those two independence conditions deliver Equation 18.2 directly, and the estimator follows: regress, take residuals, add them to the index.

Note which condition is doing the work. It is **independence**, not uncorrelatedness — and Wooldridge flags this at the point he states the assumption, before discreteness enters the discussion at all (Wooldridge 2010, 743):

> "(We could relax the independence assumptions to some degree, but **we cannot just assume that v_2 is uncorrelated with $\mathbf{z}$ and that e_1 is uncorrelated with v_2.**)"

The distinction is invisible for continuous y_2 — a linear model with a normal additive error satisfies both — which is exactly why it is easy to carry the procedure across to a binary y_2 without noticing that it has stopped holding.

18.2.2.1. Why independence, when the linear model needs only uncorrelatedness?

Because of the exponential. Substituting Equation 18.3 into Equation 18.1 gives

$$E(y_1 \mid \mathbf{z}, y_2, v_2) = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2) \cdot \exp(\rho_1 v_2) \cdot E[\exp(e_1) \mid \mathbf{z}, y_2, v_2], \tag{18.4}$$

and Equation 18.2 follows only if that final factor is a **constant** — which is why Wooldridge normalises $E[\exp(e_1)] = 1$ rather than $E(e_1) = 0$.

That is a condition on the *moment generating function* of e_1, that is, on every moment at once, which is essentially full independence. Compare the linear model, where the requirement is $E[\mathbf{z}'\varepsilon_1] = 0$ — a statement about one moment, delivered free by the OLS projection.

This is the same fact as "CF = 2SLS in a linear model, CF ≠ 2SLS in Poisson," seen from the assumption side rather than the algebra side. $\exp(\cdot)$ is what turns a moment condition into a distributional one, and everything difficult in this chapter follows from it.

18.2.2.2. Absorb or average in the first stage: the answer is the same

In a panel the linear first stage can handle the unit effects two ways, and it makes no difference which. Regress y_{it2} on $\mathbf{z}_{it}$ and unit dummies and you get the fixed-effects residual $\hat{v}_{it2}^{FE}$. Regress it on $\mathbf{z}_{it}$ and the unit means $\bar{\mathbf{z}}_i$ — the Mundlak device — and you get $\hat{v}_{it2}^{M}$. Put either into the second-stage FE-Poisson and $\hat{\alpha}_1$ and $\hat{\rho}_1$ come out **numerically identical** (Lin and Wooldridge 2019, app. A.1–A.2).

The reason is that Mundlak's regression reproduces the within slope, so the two residuals differ only by a unit-specific constant, $\hat{v}_{it2}^{FE} = \hat{v}_{it2}^{M} + g_i$. In the second-stage index that constant is absorbed:

$$\alpha_1 y_{it2} + \rho_1 \hat{v}_{it2}^{FE} + c_i = \alpha_1 y_{it2} + \rho_1 \hat{v}_{it2}^{M} + (c_i + \rho_1 g_i),$$

and c_i is a free parameter, so shifting it costs nothing. The model is reparameterised, not changed.

Two conditions on this. The averaging dimension in the first stage has to be the dimension the second stage absorbs. And it is a fact about *linear* first stages: once y_2 is binary the first stage is nonlinear, the residual no longer shifts by a constant, and choosing between dummies and Mundlak means becomes a real decision — which is the subject of the next section.

18.2.3. A binary y_2: why the linear reduced form fails

OLS always produces a residual uncorrelated with $\mathbf{z}$. That is a property of the population linear projection and it holds whatever y_2 looks like. But the control-function derivation needs v_2 *independent* of $\mathbf{z}$, which is a substantive assumption about the reduced form and not something OLS supplies.

For binary y_2 it cannot hold. Conditional on $\mathbf{z}$, the residual $v_2 = y_2 - \mathbf{z}\pi_2$ takes exactly two values,

$$v_2 \mid \mathbf{z} = \begin{cases} 1 - \mathbf{z}\pi_2 & \text{with probability } p(\mathbf{z}) \\ -\mathbf{z}\pi_2 & \text{with probability } 1 - p(\mathbf{z}), \end{cases} \tag{18.5}$$

so both the two support points *and* the probabilities of landing on them are functions of $\mathbf{z}$. Its variance, $\mathrm{Var}(v_2 \mid \mathbf{z}) = p(\mathbf{z})[1 - p(\mathbf{z})]$, is a function of $\mathbf{z}$ too. Independence fails by construction, not by approximation.

Wooldridge states this outright, at (Wooldridge 2010, 746):

> "The assumption that e_1 in (18.39) is independent of v_2 and **$\mathbf{z}$ rules out most cases where y_2 has some discreteness, such as with binary, count, or corner responses.** (In particular, the independence assumption likewise fails unless v_2 is independent of $\mathbf{z}$, and it would be very unusual for a discrete variable to be expressible as a linear equation with additive error independent of $\mathbf{z}$.)"

That sentence is compressed, so it is worth unpacking. It names one assumption, but the failure runs through three links:

1. **What the correction needs.** From Equation 18.4, e_1 must be independent of $(\mathbf{z}, y_2, v_2)$, so that $E[\exp(e_1) \mid \cdot]$ is constant.
2. **That silently requires $v_2 \perp \mathbf{z}$.** e_1 is defined as what is left of c_1 after projecting on v_2, so if v_2 depends on $\mathbf{z}$, e_1 inherits the dependence. This is what "the independence assumption likewise fails unless" refers to.
3. **And $v_2 \perp \mathbf{z}$ cannot hold for discrete y_2.** This is the step he leaves as "very unusual", and it is in fact airtight. Independence means the conditional distribution of v_2 given $\mathbf{z}$ is *the same for every* $\mathbf{z}$. But by Equation 18.5 the support is $\{-\mathbf{z}\pi_2,\ 1 - \mathbf{z}\pi_2\}$, and both points **slide as $\mathbf{z}$ changes**. A distribution whose support moves with $\mathbf{z}$ cannot be independent of it — unless $\pi_2 = \mathbf{0}$, in which case there is no first stage at all. The same argument covers count and corner responses: the mass point sits at $-\mathbf{z}\pi_2$ and travels.

So the sentence reads: *the correction needs* $e_1 \perp (v_2, \mathbf{z})$*; that needs* $v_2 \perp \mathbf{z}$*; and* $v_2 \perp \mathbf{z}$ *cannot hold when* y_2 *is discrete, because the residual's support is pinned to* $\mathbf{z}\pi_2$*.*

Note what does **not** break. $E[\mathbf{z}'v_2] = 0$ still holds exactly — the projection is fine, and OLS has done nothing wrong. It is independence that fails, and only the exponential mean cares about the difference.

18.2.3.1. The subtlety: discreteness of the residual is not the problem

It is tempting to summarise this as "the residual is binary, so it cannot be a control function." That argument proves too much, because **the generalized residual of the next section is two-valued given z as well.**

The difference is not the discreteness of the derived residual. It is *which object independence is asserted about.* The linear-residual CF asserts it of the **observed** v_2, where both the support points and their probabilities move with $\mathbf{z}$. Terza's setup asserts it of the **continuous latent** probit error, $v_2 \mid \mathbf{z} \sim \text{Normal}(0, 1)$ — which is the assumption every probit already makes, and costs nothing extra.

So the fix is not to find a continuous control function. It is to relocate the independence assumption to a latent variable where it is plausible.

18.2.3.2. What survives: the test, but not the correction

The linear-residual construction is not worthless for binary y_2. It remains **fully valid as a Hausman-type exogeneity test**, and Wooldridge is explicit about why (Wooldridge 2010, 744):

> "For the purposes of getting a limiting chi-square distribution, it does not matter where the linear combination $\hat{v}_2$ comes from. In other words, **under the null hypothesis none of the assumptions we made about** (c_1, v_2) **need to hold**: v_2 need not be independent of $\mathbf{z}$, and e_1 in equation (18.39) need not be independent of v_2. Therefore, as a test, this procedure is very robust, and it can be applied when y_2 contains binary, count, or other discrete variables. **Unfortunately, if** y_2 **is endogenous, the correction does not work without something like the assumptions made previously.**"

That is the cleanest statement of the position: under $H_0 : \rho = 0$ there is no endogeneity left to approximate, so the test needs no reduced-form assumption. Once $\rho \neq 0$, the correction needs one, and for binary y_2 it is unavailable.

18.2.4. Terza's exact correction

Terza (1998), exposited in Wooldridge (2010) (sec. 18.5) and Imbens and Wooldridge (2007) (sec. 3.3, eqs. 3.32–3.36), replaces the linear reduced form with a **threshold model** — Wooldridge's eq. (18.43):

$$y_2 = \mathbb{1}[\mathbf{z}\pi_2 + v_2 \geq 0], \qquad v_2 \mid \mathbf{z} \sim \text{Normal}(0, 1), \tag{18.6}$$

and *keeps* the independence assumption, now about the latent v_2. The derivation then runs in two steps. With (c_1, v_2) jointly normal, mean zero and independent of $\mathbf{z}$, write $c_1 = \rho_1 v_2 + e_1$ where $e_1 \mid \mathbf{z}, v_2 \sim \text{Normal}(0, \tau_1^2 - \rho_1^2)$. Conditioning on the latent error gives eq. (18.44),

$$E(y_1 \mid \mathbf{z}, v_2) = E[\exp(e_1)] \exp(\mathbf{x}_1\beta_1 + \rho_1 v_2) = \exp\big((\tau_1^2 - \rho_1^2)/2 + \mathbf{x}_1\beta_1\big) \exp(\rho_1 v_2), \tag{18.7}$$

using the lognormal mean. But v_2 is not observed — only y_2 is. Averaging over the truncated normal implied by y_2 gives eq. (18.45), and evaluating that expectation yields the exact result:

$$E(y_1 \mid \mathbf{z}, y_2) = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2) \cdot h(y_2, \mathbf{z}\pi_2, \rho) \tag{18.8}$$

$$h(y_2, \mathbf{z}\pi_2, \rho) = \exp(\rho^2/2) \left\{ y_2 \frac{\Phi(\rho + \mathbf{z}\pi_2)}{\Phi(\mathbf{z}\pi_2)} + (1 - y_2) \frac{1 - \Phi(\rho + \mathbf{z}\pi_2)}{1 - \Phi(\mathbf{z}\pi_2)} \right\} \tag{18.9}$$

This is estimated by plugging in the first-stage probit's $\hat{\pi}_2$ and jointly QMLE-fitting $(\delta_1, \alpha_1, \rho)$ against this mean — a genuinely nonlinear second stage, **not** a linear regressor addition.

ρ here is a coefficient, not a correlation. It is the coefficient in the linear projection of c_1 on v_2, i.e. $E[\exp(c_1) \mid v_2] = \exp(\rho v_2)$. With v_2 standard normal the two differ by a factor of σ_{c_1}, since $\rho = \text{Corr}(c_1, v_2)\, \sigma_{c_1}$. The distinction is not cosmetic: this chapter's binary DGP sets $c_1 = 0.2\, e_2$ with $v_2 = e_2$, so the *correlation* is exactly 1 while the ρ governing the approximation below is 0.2. Reading ρ as a correlation is also what $h(\cdot)$ forbids — it is ρ in the coefficient sense that makes $\exp(\rho^2/2)$ the lognormal correction and puts ρ on the same scale as the probit index.

18.2.4.1. Terza's FIML, and Stata's `etpoisson`

Terza (1998) offers three estimators for this model, and it is worth knowing which is which. The first is full-information maximum likelihood: write the Poisson density for y_1 and the probit for y_2, let the two errors be jointly normal, and integrate the unobservable out. This is what Stata's `etpoisson` fits, by Gauss–Hermite quadrature. The second is the two-step above — probit, then the exact mean by QMLE — which Terza calls a two-stage method of moments and describes as the *relatively robust partially parametric* option, since it needs the conditional mean and nothing else. The third sits between them, a nonlinear weighted least squares for the fully parametric case.

The distinction matters more in a panel than in a cross-section, for two reasons.

The first is what FIML assumes. Plain Poisson QMLE is consistent for the mean under *any* conditional distribution, because its first-order condition is the mean moment condition $\sum \mathbf{x}(y - \exp(\mathbf{x}\beta)) = 0$. That robustness is the reason `fepois` and `ppmlhdfe` are safe on count data that is nothing like Poisson. FIML's score instead carries the Poisson density inside the integral, so getting the mean right is not enough, and `vce(robust)` repairs inference rather than the estimator.

The second is the fixed effects. `etpoisson` has no `absorb()` option and no panel version, so unit effects must enter as dummies — and here the Poisson rescue does not apply. The closed-form concentration $\exp(\hat{\alpha}_i) = \sum_i y / \sum_i \exp(\mathbf{x}\beta)$ works because the unit effect factors out of the Poisson likelihood; integrating over a normal unobservable destroys that separability, so the unit effects become ordinary nonlinear parameters and the incidental-parameter problem returns in full. With a few observations per unit that is fatal, and non-convergence is the usual symptom.

Replacing the dummies with unit means makes it estimable again, and in simulation it behaves reasonably: close to unbiased when its own distributional assumption holds, and off by a few percent when the count is overdispersed or zero-inflated. But it then rests on the Mundlak projection *and* joint normality *and* the Poisson shape, where the control function needs only the first two, and it converges unreliably — in our own runs it failed on five to twelve percent of samples, and the failures were the samples with the most dispersion, so discarding them is a selection. It also identifies the treatment effect from within- *and* between-unit variation, where absorbing the dummies uses within-unit variation only. The efficiency it buys is not worth that stack.

18.2.5. The generalized-residual CF as a first-order approximation

Taylor-expand h in ρ around $\rho = 0$. Since $\frac{d}{d\rho}\exp(\rho^2/2)\big|_{\rho=0} = 0$, only the Φ-ratios contribute to first order:

$$\frac{\partial}{\partial\rho}\frac{\Phi(\rho + \mathbf{z}\pi_2)}{\Phi(\mathbf{z}\pi_2)}\bigg|_{\rho=0} = \frac{\phi(\mathbf{z}\pi_2)}{\Phi(\mathbf{z}\pi_2)} = \lambda(\mathbf{z}\pi_2), \qquad \frac{\partial}{\partial\rho}\frac{1-\Phi(\rho + \mathbf{z}\pi_2)}{1-\Phi(\mathbf{z}\pi_2)}\bigg|_{\rho=0} = -\lambda(-\mathbf{z}\pi_2) \tag{18.10}$$

where $\lambda(\cdot) = \phi(\cdot)/\Phi(\cdot)$ is the inverse Mills ratio — exactly the `gr2` computed via `pdf(nd, ) / cdf(nd, )` below. Since $h(y_2, \mathbf{z}\pi_2, 0) = 1$,

$$h(y_2, \mathbf{z}\pi_2, \rho) \approx 1 + \rho \cdot \underbrace{\left[y_2\lambda(\mathbf{z}\pi_2) - (1-y_2)\lambda(-\mathbf{z}\pi_2)\right]}_{=\ \hat{r}_2,\ \text{the generalized residual}} \tag{18.11}$$

and $\exp(\mathbf{x}_1\beta_1) \cdot h(\cdot) \approx \exp(\mathbf{x}_1\beta_1)(1 + \rho\hat{r}_2) \approx \exp(\mathbf{x}_1\beta_1 + \rho\hat{r}_2)$ for small $\rho\hat{r}_2$ — which is exactly "add $\hat{r}_2$ linearly inside $\exp(\cdot)$."

So the **generalized-residual CF is the first-order-in-ρ local approximation to Terza's exact correction**, with error $O(\rho^2)$. A direct large-n check against the R companion chapter's version of this DGP gives probability limits of 0.839 for the linear-residual CF and 0.822 for the generalized-residual CF against a true 0.8 — both biased, the latter roughly half the former, matching the theoretical ranking, and neither exact.

18.2.5.1. Why use the approximation rather than the exact correction?

Not because the exact correction is infeasible. It is worth being clear about this, because $h(\cdot)$ looks forbidding: the Φ-ratios and the $\exp(\rho^2/2)$ term make Equation 18.8 genuinely nonlinear in ρ, which suggests custom optimisation and no obvious way to combine it with the absorption that makes FE-Poisson tractable.

That reading is wrong, for a simple reason. Hold ρ fixed. Then $\log h(y_2, \mathbf{z}\hat{\pi}_2, \rho)$ is a *variable* — a known function of the data and the first-stage index — so the mean becomes

$$E(y_1 \mid \mathbf{z}, y_2, \alpha_i) = \exp(\mathbf{z}_1\delta_1 + \alpha_1 y_2 + \alpha_i + \log h),$$

which is an ordinary Poisson mean with an **offset**. Any Poisson routine that takes one will fit it with the fixed effects absorbed exactly as usual — `ppmlhdfe ..., absorb(id) offset(lnh)` in

Stata, `fepois(..., offset = ~lnh)` in R. Because h enters multiplicatively, the concentration that rescues FE-Poisson still goes through — $\exp(\hat{\alpha}_i) = \sum_i y_1 / \sum_i \exp(\mathbf{x}\beta)h$ — so there is no incidental-parameter problem. Only ρ itself is nonlinear, and it is one scalar: profile it on a grid and keep the value maximising the Poisson quasi-likelihood. This is Terza's two-step, and it is what Wooldridge (2010) (sec. 18.5) prescribes for the cross-section — "we can use NLS or a quasi-MLE, such as the Poisson or gamma QMLE" — with the fixed effects added.

So the honest reasons for the approximation are convenience and the test, not feasibility. It runs as one command with no profile and no bootstrap; the coefficient on $\hat{r}_2$ doubles as a Hausman-type t-test sitting next to the coefficient of interest, which is in fact how Wooldridge introduces it — as a variable-addition test of $\rho_1 = 0$, noting that its $\hat{\rho}_1$ "is not the estimate we obtain from Terza's two-step method." Joint nonlinear estimation is also not free of trouble: for the closely analogous binary-y_1 case Wooldridge notes the iterations can be hard to converge, with $\hat{\rho}_1$ drifting toward ± 1 (Wooldridge 2010, sec. 15.7.3), though a bounded profile grid makes that visible rather than fatal.

What matters is how large ρ is, and Equation 18.11 gives the rule: the error is $O(\rho^2)$. In a Monte Carlo at 2,000 units with four observations each, the approximation is indistinguishable from the exact fit at $|\rho| = 0.2$ — the value this chapter's binary DGP uses — and the exact version is unbiased throughout.

The size of the error is not symmetric in ρ, which is worth knowing before worrying about it. At $\rho = +0.5$ the approximation overstates the effect by about nine percent and at $\rho = +0.8$ by about thirty; at $\rho = -0.5$ the bias is invisible and at $\rho = -0.8$ it is about six percent *downward*. The asymmetry comes from h itself: a positive ρ scales the treated mean by a factor bounded by $1/\Phi(\mathbf{z}\pi_2)$, which grows without bound in the tail, while a negative ρ shrinks it smoothly toward zero. The $O(\rho^2)$ rate describes the behaviour near zero; it does not put the same coefficient on both sides.

So: report the approximation when $\hat{\rho}_1$ comes back small or negative, and fit the exact version alongside it when it is large and positive. Two cautions on the exact version. It is exact only under the joint normality of Equation 18.6 — when the unobservable depends on v_2 nonlinearly, fitting h can do worse than truncating it, because the profile absorbs curvature h cannot represent. And its standard errors need a cluster bootstrap, since the first stage is plugged in and ρ is profiled.

18.2.5.2. Probit or logit — and why logit is simpler than it looks

Nothing in the local-approximation argument is specific to the normal. The generalized residual is defined for *any* correctly specified binary-response GLM (Gourieroux et al. 1987), and for a **logit** it collapses. With $\Lambda(\cdot)$ the logistic CDF and $\lambda(\eta) = \Lambda(\eta)[1 - \Lambda(\eta)]$ its density,

$$\hat{r}_2 = y_2 \frac{\lambda(\hat{\eta})}{\Lambda(\hat{\eta})} - (1 - y_2) \frac{\lambda(\hat{\eta})}{1 - \Lambda(\hat{\eta})} = y_2[1 - \Lambda(\hat{\eta})] - (1 - y_2)\Lambda(\hat{\eta}) = y_2 - \Lambda(\hat{\eta}) = y_2 - \hat{p}_2, \quad (18.12)$$

i.e. **the logit generalized residual is just the plain response residual** $y_2 - \hat{p}_2$. No Mills-ratio machinery: a `logit` first stage, then `y2 .- predict(fit)`, *is* the generalized residual — not a shortcut to it.

Two limits. Terza's *exact* correction is probit-specific: $h(\cdot)$ comes from the truncated moments of the bivariate normal, and no analogue for a logistic v_2 appears in Terza (1998), Wooldridge (2010)

or Imbens and Wooldridge (2007). And the link-robustness that the GMM route enjoys — the fitted probability is a valid instrument even if the link is wrong, since any function of $\mathbf{z}$ is a valid instrument (Imbens and Wooldridge 2007) — belongs to using the fitted value as an *instrument*, not as a *residual*. It does not carry over to either control function, where the second-stage conditional mean still has to be right.

18.2.6. A different family: multiplicative-moment GMM

The third route abandons the control function. Any function of the exogenous $\mathbf{z}$ is a valid instrument, so the fitted probability $\hat{p}_2$ can instrument y_2 directly in a GMM problem that leaves the exponential mean untouched. Probit, logit and linear-probability fitted values are all equally valid for consistency; the choice affects only instrument strength. The moments are Mullahy's multiplicative ones,

$$E[\mathbf{z}'(y_1 e^{-\mathbf{x}\beta} - 1)] = 0, \tag{18.13}$$

which is the right choice when the unobserved heterogeneity sits inside $\exp(\cdot)$, as it does for counts.

Two consequences follow, and they are the whole tradeoff. **The validity of this route does not depend on the first-stage model being right** — $\hat{p}_2$ need only be a function of $\mathbf{z}$. But **it does not accommodate high-dimensional fixed effects**: absorbing them would require within-group demeaning of the regressor and instrument matrices before solving nonlinear moment conditions, which is not straightforward. Stata's `ivpoisson` similarly drops `absorb()`.

18.3. The three estimators compared

Two questions separate these estimators, and everything else they share. **Is it valid when y_2 is binary, and can it absorb high-dimensional fixed effects?**

	Linear-residual CF	Generalized-residual CF	Multiplicative-moment GMM
How it works	OLS residual $\hat{v}_2$ in the index	probit/logit generalized residual $\hat{r}_2$ in the index	fitted $\hat{p}_2$ as an *instrument*
Valid for binary y_2?	**No** — reduced-form assumption cannot hold	**Approximately** — error $O(\rho^2)$, derived	**Yes** — consistent
First stage must be correct?	moot	yes	**no**
Takes high-dimensional FE?	yes	yes	**no**

No estimator gets both. The generalized-residual CF comes closest, buying fixed effects at the price of an approximation whose error is at least *derived* and shrinks with ρ. The GMM route is the

only genuinely consistent one, and pays for it by giving up fixed effects — and, as the simulations show, by roughly tripling the variance. The linear-residual CF is the fallback: for binary y_2 it has no valid reduced form behind it, though it remains a fully valid *exogeneity test* and is accurate when ρ is small.

Common to all three: the second-stage exponential mean must be correctly specified, and the two control functions need bootstrapped standard errors because their residual is generated.

18.4. Simulation 1 — continuous endogenous variable

18.4.1. Data generating process

We simulate a panel where `time` (continuous, e.g. minutes on site) is endogenous: it is correlated with an unobserved factor `e` that also affects `visits`. The excluded instrument is `phone` (binary, e.g. phone-access indicator). Fixed effects are `ad` (20 groups) and `female`.

```
Random.seed!(42)
n_ad = 20; obs_per = 250; n = n_ad * obs_per

fe_ad  = randn(n_ad) .* 0.5
fe_grp = repeat(1:n_ad, inner = obs_per)
female = Int.(rand(n) .< 0.5)
phone  = Int.(rand(n) .< 0.4)
frfam  = rand(n)

# Common error drives endogeneity
e = randn(n)

# Continuous endogenous variable (linear first stage holds exactly)
time = 1.5 .* phone .+ 0.5 .* frfam .+ fe_ad[fe_grp] .+ e

# Poisson outcome - true coefficient on time = 0.8
true_  = 0.8
  = exp.(0.5 .+ true_  .* time .+ 0.4 .* frfam .+
         fe_ad[fe_grp] .+ 0.3 .* female .+ 0.5 .* e)
visits = [rand(Poisson( i)) for  i in  ]

df1 = DataFrame(visits=visits, time=time, phone=phone,
                frfam=frfam, female=female, ad=fe_grp)

println("n = $n   groups = $n_ad   visits mean = $(round(mean(visits),digits=2))")
```

```
n = 5000   groups = 20   visits mean = 15.93
```

18.4.2. Naive Poisson — the endogeneity bias

Ignoring the endogeneity of `time` gives a biased estimate of :

```
naive1 = fepois(df1, @formula(visits ~ time + frfam + fe(ad) + fe(female)))
println("Naive (biased):")
show_regression_html(naive1; title="Naive Poisson - time is endogenous")
```

```
Naive (biased):
```

	Estimate	Std. Error	t value	Pr(>|t|)
time	1.150	0.003	392.827	< 2e-16 ***
frfam	0.291	0.013	22.698	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

18.4.3. The linear-residual control function

Applying the framework above, the two-step procedure is:

1. **First stage** — linear projection of `time` on `phone`, `frfam`, and fixed effects via `feols`. The residual $\hat{v}_2 = \text{time} - \widehat{\text{time}}$ satisfies $E(\mathbf{z}'\hat{v}_2) = 0$ by construction. A linear first stage is appropriate here because `time` is continuous — the condition that fails for binary y_2 holds in this design.
2. **Second stage** — Poisson QMLE with $\hat{v}_2$ added as a regressor inside the exponential mean. Equation 18.2 holds exactly here because the DGP specifies $0.5\,e$ linearly inside $\exp(\cdot)$.

Note that unlike 2SLS in a linear model, $\hat{v}_2$ does *not* replace y_2 — both enter together. This is what makes the control function different from IV/GMM Poisson, confirmed numerically in Simulation 2.

```
# Step 1: linear projection
fs1 = feols(df1, @formula(time ~ phone + frfam + fe(ad) + fe(female)))
df1.v2hat = fs1.residuals

# Step 2: Poisson CF
m1_cf = fepois(df1, @formula(visits ~ time + v2hat + frfam + fe(ad) + fe(female)))
println("CF estimate (should recover   = $(true_ )):")
show_regression_html(m1_cf; title="CF: linear projection first stage")
```

```
CF estimate (should recover   = 0.8):
```

	Estimate	Std. Error	t value	Pr(>|t|)
time	0.810	0.005	158.862	< 2e-16 ***
v2hat	0.489	0.006	76.972	< 2e-16 ***
frfam	0.405	0.013	31.664	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

The coefficient on `time` recovers the true $\alpha_1 = 0.8$. The `v2hat` coefficient tests exogeneity — its t-statistic is the Hausman-type test.

18.4.4. Bootstrap standard errors

The second-stage standard errors are incorrect because the first-stage estimation error propagates. Julia's `Threads.@threads` parallelises the cluster bootstrap across all CPU cores:

```
function bootstrap_resample(df::DataFrame, B::Int; seed::Int=42)
    Random.seed!(seed)
    clusters = unique(df.ad)
    K = length(clusters)
    draws = [sample(clusters, K, replace=true) for _ in 1:B]
    return draws
end

function build_boot_df(df::DataFrame, drawn_clusters::Vector)
    frames = DataFrame[]
    for (id, c) in enumerate(drawn_clusters)
        sub = copy(df[df.ad .== c, :])
        sub.ad_boot = fill(id, nrow(sub))
        push!(frames, sub)
    end
    return vcat(frames...)
end

# Part 1 bootstrap: continuous time
function bootstrap_cf_continuous(df::DataFrame, B::Int=500; seed::Int=42)
    draws   = bootstrap_resample(df, B; seed=seed)
    results = zeros(B)
    Threads.@threads for b in 1:B
        df_b = build_boot_df(df, draws[b])
        fs   = feols(df_b,  @formula(time ~ phone + frfam + fe(ad_boot) + fe(female)))
        df_b.v2hat = fs.residuals
        ss   = fepois(df_b, @formula(visits ~ time + v2hat + frfam + fe(ad_boot) + fe(female)))
        idx  = findfirst(==("time"), coefnames(ss))
        results[b] = coef(ss)[idx]
    end
```

```
    return results
end

boot1 = bootstrap_cf_continuous(df1, 500)
naive_se1 = stderror(m1_cf)[findfirst(==("time"), coefnames(m1_cf))]
coef1     = coef(m1_cf)[findfirst(==("time"), coefnames(m1_cf))]

@printf("time coefficient:  %.4f  (true %.1f)\n", coef1, true_ )
@printf("Naive 2nd-stage SE: %.4f\n", naive_se1)
@printf("Bootstrap SE:       %.4f\n", std(boot1))
@printf("Bootstrap 95%% CI: [%.4f, %.4f]\n",
        quantile(boot1, 0.025), quantile(boot1, 0.975))
```

```
time coefficient:  0.8103  (true 0.8)
Naive 2nd-stage SE: 0.0051
Bootstrap SE:       0.0112
Bootstrap 95% CI: [0.7868, 0.8312]
```

The bootstrapped SE is larger than the naive second-stage SE, confirming that ignoring the first-stage uncertainty understates uncertainty.

18.5. Simulation 2 — binary endogenous variable

18.5.1. Data generating process

A clean probit DGP, with a fresh structural error `e2` independent of Simulation 1. The endogeneity coefficient = 0.2 is kept small so the first-order approximation stays tight and the estimators can be compared without that error dominating.

```
Random.seed!(123)

# Fresh structural error for Part 2 (independent of Part 1's e)
e2 = randn(n)

# Binary endogenous variable: clean probit latent index
# phone is the excluded instrument; no FE in latent index for clarity
time_hi = Int.(1.5 .* phone .+ 0.4 .* frfam .+ e2 .>= 0)

# Poisson outcome - true coefficient 0.8, endogeneity via  *e2 ( =0.2)
# Small   ensures the approximate CF assumption holds closely
 2 = 0.2
 2 = exp.(0.5 .+ true_  .* time_hi .+ 0.4 .* frfam .+
          fe_ad[fe_grp] .+ 0.3 .* female .+  2 .* e2)
visits2 = [rand(Poisson( 2i)) for  2i in  2]
```

```
df2 = DataFrame(visits=visits2, time_hi=time_hi, phone=phone,
                frfam=frfam, female=female, ad=fe_grp, e2=e2)

println("time_hi mean:      $(round(mean(time_hi), digits=3))")
println("Cor(time_hi, e2):  $(round(cor(time_hi, e2), digits=3))  (endogeneity)")
println("visits mean:       $(round(mean(visits2), digits=2))")
```

```
time_hi mean:      0.727
Cor(time_hi, e2):  0.613  (endogeneity)
visits mean:       4.78
```

18.5.2. Linear-residual CF

The estimator with no valid reduced-form assumption here, included to see how far wrong it goes.

```
# Step 1: linear projection of binary time_hi
fs_a = feols(df2, @formula(time_hi ~ phone + frfam + fe(ad) + fe(female)))
df2.v2hat = fs_a.residuals

# Step 2: Poisson CF
m_a = fepois(df2, @formula(visits ~ time_hi + v2hat + frfam + fe(ad) + fe(female)))
println("Linear-residual CF:")
show_regression_html(m_a; title="Linear-residual CF (OLS residual)")
```

```
Linear-residual CF:
```

	Estimate	Std. Error	t value	Pr(>|t|)
time_hi	0.813	0.036	22.719	< 2e-16 ***
v2hat	0.330	0.035	9.396	< 2e-16 ***
frfam	0.413	0.023	17.921	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

18.5.3. Generalized-residual CF

```
# Step 1: probit first stage
fs_b = feprobit(df2, @formula(time_hi ~ phone + frfam + fe(ad) + fe(female)))

# Generalized residual (Gourieroux et al.):  ( )/Φ( ) if y=1 (inverse Mills ratio),
# - ( )/(1-Φ( )) if y=0 (the reverse Mills ratio).
  = fs_b.eta
nd = Normal()
```

```
df2.gr2 = @. ifelse(df2.time_hi == 1,
    pdf(nd,  ) / cdf(nd,  ),
    -pdf(nd,  ) / (1.0 - cdf(nd,  )))

# Step 2: Poisson CF with generalized residual
m_b = fepois(df2, @formula(visits ~ time_hi + gr2 + frfam + fe(ad) + fe(female)))
println("Generalized-residual CF (probit):")
show_regression_html(m_b; title="Generalized-residual CF (probit)")
```

```
Generalized-residual CF (probit):
```

	Estimate	Std. Error	t value	Pr(>|t|)
time_hi	0.764	0.039	19.549	< 2e-16 ***
gr2	0.230	0.024	9.721	< 2e-16 ***
frfam	0.415	0.023	17.969	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

18.5.4. Probit or logit

Confirming Equation 18.12 numerically — the logit generalized residual is `time_hi .- phat`:

```
fs_b_logit = felogit(df2, @formula(time_hi ~ phone + frfam + fe(ad) + fe(female)))
phat_logit = fs_b_logit.mu
df2.gr2_logit = df2.time_hi .- phat_logit    # algebraically = generalized residual

m_b_logit = fepois(df2, @formula(visits ~ time_hi + gr2_logit + frfam + fe(ad) + fe(female)))
println("Generalized-residual CF with a logit first stage:")
show_regression_html(m_b_logit; title="Generalized-residual CF (logit)")

coef_b_probit = coef(m_b)[findfirst(==("time_hi"), coefnames(m_b))]
coef_b_logit  = coef(m_b_logit)[findfirst(==("time_hi"), coefnames(m_b_logit))]
@printf("Probit first stage: alpha1 = %.4f\n", coef_b_probit)
@printf("Logit  first stage: alpha1 = %.4f\n", coef_b_logit)
```

```
Generalized-residual CF with a logit first stage:
```

	Estimate	Std. Error	t value	Pr(>|t|)
time_hi	0.800	0.036	22.145	< 2e-16 ***
gr2_logit	0.346	0.036	9.698	< 2e-16 ***
frfam	0.413	0.023	17.878	< 2e-16 ***

	Estimate	Std. Error	t value	Pr(>|t|)

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1				

```
Probit first stage: alpha1 = 0.7636
Logit  first stage: alpha1 = 0.8000
```

18.5.5. The dummies in the first stage need a long panel

Both first stages above put the group dummies inside the probit or logit, which is safe here because each group has 250 observations. Applied panel work usually has the opposite shape: many units, a handful of periods each. A logit with unit dummies is badly biased in that case, and the size of the bias depends on observations per unit, not on how many units there are. Against a true slope of 1 it returns about 2.0 at two observations per unit, 1.37 at four, 1.15 at six, 1.04 at twenty-five, and essentially the truth by a hundred. A panel of a few thousand units with four or five observations each sits in the worst part of that range.

The problem is specific to the nonlinear first stage. Poisson with unit dummies is consistent with the number of periods fixed, and so is a linear first stage.

The fix is the Mundlak device: drop the unit dummies from the first stage and add unit means of the instrument and the time-varying controls instead. The instrument's own mean has to be among them, since the unit effect is being modelled as a function of those means and the instrument is one of the regressors it must be purged of. Conditional logit is also consistent with the number of periods fixed, but it eliminates the unit effects rather than estimating them, so it leaves no fitted index from which to build the generalized residual; the Mundlak version keeps the index. The construction is worked through in IV Poisson with many fixed effects.

Lin and Wooldridge (2019) treat this case directly for the exponential model with unobserved effects, and land in the same place. Their Procedure 3 adds a first-stage residual to FE Poisson and tests $\rho_1 = 0$; for a binary endogenous variable they write that one can "use as a control function the generalized residuals … where $\mathbf{w}_{it} = (1, \mathbf{z}_{it}, \bar{\mathbf{z}}_i)$" — the time-varying exogenous variables *and their unit means*, the instrument's mean included. They add the caveat in the same breath: the Mundlak device "might work reasonably well, but neither might be flexible enough," and they "leave investigations into the quality of CF approximations in discrete cases to future research." So the recommendation below is theirs as well as ours, and the open question they name is the one worth remembering: how good the approximation is when y_2 is discrete has not been settled, and Equation 18.11 says the error grows with ρ.

18.5.6. Multiplicative-moment GMM

We implement IV/GMM Poisson directly, keeping `time_hi` as the regressor and using probit fitted probabilities as the instrument. **Both moment forms** are coded: the additive moments are what "IV Poisson" often means in practice (e.g. Stata's `ivpoisson gmm` default), but because this chapter's DGP puts the unobservable inside $\exp(\cdot)$, only the multiplicative moments of Mullahy (1997) are consistent here.

```
"""
    ivpois(y, X, Z) -> ( , se)

IV/GMM Poisson (just-identified). X = [endog, exog], Z = [instrument, exog].
Damped IRLS: intercept initialised from log(mean(y)), half-steps for stability.
"""
function ivpois(y::Vector{Float64}, X::Matrix{Float64}, Z::Matrix{Float64};
                maxiter=500, tol=1e-8, step=0.5)
    p  = size(X, 2)
       = zeros(p)
     [p] = log(mean(y))    # intercept from marginal mean - prevents divergence

    for _ in 1:maxiter
          = exp.(clamp.(X *  , -30.0, 30.0))
        H = Z' * (  .* X)
        g = Z' * (y .-  )
        Δ = try H \ g catch; pinv(H) * g end
          .+= step .* Δ
        maximum(abs.(step .* Δ)) < tol && break
    end

      = exp.(clamp.(X *  , -30.0, 30.0))
    H = Z' * (  .* X)
    B = Z' * ((y .-  ).^2 .* Z)       # robust sandwich meat
    V = try inv(H) * B * inv(H)' catch; pinv(H) * B * pinv(H)' end
    return  , sqrt.(max.(0.0, diag(V)))
end

"""
    ivpois_mult(y, X, Z) ->

IV/GMM Poisson with MULTIPLICATIVE (Mullahy 1997) moments
Z'(y .* exp(-X ) .- 1) = 0 - consistent when the unobservable enters
inside the exponential mean (multiplicative error).
"""
function ivpois_mult(y::Vector{Float64}, X::Matrix{Float64}, Z::Matrix{Float64};
                     maxiter=1000, tol=1e-10, step=0.5)
    p  = size(X, 2)
       = zeros(p)
     [p] = log(mean(y))
    for _ in 1:maxiter
        w = y .* exp.(clamp.(-X *  , -30.0, 30.0))
        g = Z' * (w .- 1)
        H = Z' * (w .* X)
        Δ = try H \ g catch; pinv(H) * g end
          .+= step .* Δ
        maximum(abs.(step .* Δ)) < tol && break
```

```
    end
    return
end

# Probit (without FE, to avoid collinearity with instrument matrix)
fs_c  = feprobit(df2, @formula(time_hi ~ phone + frfam))
phat  = fs_c.mu

# Regressor matrix X: [time_hi, frfam, intercept]
# Instrument matrix Z: [phat,    frfam, intercept]  ← phat replaces time_hi
X_c = hcat(Float64.(df2.time_hi), Float64.(df2.frfam), ones(n))
Z_c = hcat(Float64.(phat),        Float64.(df2.frfam), ones(n))

_c, se_c = ivpois(Float64.(df2.visits), X_c, Z_c)
_cm      = ivpois_mult(Float64.(df2.visits), X_c, Z_c)
@printf("\nMultiplicative-moment GMM (no FE, for illustration)\n")
@printf("  additive moments:       time_hi = %7.4f   SE = %7.4f\n",
        _c[1], se_c[1])
@printf("  multiplicative moments: time_hi = %7.4f   (true %.1f)\n",
        _cm[1], true_ )
```

```
Multiplicative-moment GMM (no FE, for illustration)
  additive moments:       time_hi =  0.5273   SE =  0.0706
  multiplicative moments: time_hi =  0.7260   (true 0.8)
```

The additive-moment estimate is visibly biased — the DGP's unobservable ρe_2 sits inside $\exp(\cdot)$, so the additive moments are misspecified (a 30-replication check gives an additive-moment mean of about 0.61 against a multiplicative-moment mean of about 0.79, with true value 0.8). The multiplicative moments repair it.

18.5.7. Control function and GMM are not equivalent in Poisson

In a linear model, using $\hat{y}_2$ as an instrument (2SLS) is numerically identical to adding $\hat{v}_2 = y_2 - \hat{y}_2$ as a regressor, by Frisch–Waugh–Lovell. **In Poisson this equivalence breaks**, because the control function puts $\hat{v}_2$ inside $\exp(\cdot)$ while the GMM route uses $\hat{y}_2$ in the moment conditions without modifying the exponential mean.

```
# Use linear fitted values as instruments (GMM route, linear first stage)
ŷ2  = Float64.(df2.time_hi) .- Float64.(fs_a.residuals)
X_l = hcat(Float64.(df2.time_hi), Float64.(df2.frfam), ones(n))
Z_l = hcat(ŷ2,                    Float64.(df2.frfam), ones(n))

_l, _ = ivpois(Float64.(df2.visits), X_l, Z_l)
```

```
coef_a = coef(m_a)[findfirst(==("time_hi"), coefnames(m_a))]
@printf("Linear-residual CF (residual as regressor, with FE): ^  = %.4f\n", coef_a)
@printf("GMM, additive moments (linear fitted values, no FE): ^  = %.4f\n",  _l[1])
@printf("GMM, additive moments (probit fitted values, no FE): ^  = %.4f\n",  _c[1])
@printf("GMM, multiplicative moments (no FE):                 ^  = %.4f\n",  _cm[1])
@printf("True   = %.1f\n", true_ )
```

```
Linear-residual CF (residual as regressor, with FE): ^  = 0.8128
GMM, additive moments (linear fitted values, no FE): ^  = 0.6194
GMM, additive moments (probit fitted values, no FE): ^  = 0.5273
GMM, multiplicative moments (no FE):                 ^  = 0.7260
True   = 0.8
```

The estimates differ despite using the same first-stage linear projection, confirming that 2SLS $\neq$ CF for Poisson — and, within the GMM route, that the moment form matters as much as the instrument.

18.5.8. Bootstrap for the binary case

```
# Part 2 bootstrap: binary time_hi
function bootstrap_cf_binary(df::DataFrame, B::Int=500; seed::Int=42)
    draws   = bootstrap_resample(df, B; seed=seed)
    results = zeros(B)
    Threads.@threads for b in 1:B
        df_b = build_boot_df(df, draws[b])
        fs   = feols(df_b,  @formula(time_hi ~ phone + frfam + fe(ad_boot) + fe(female)))
        df_b.v2hat = fs.residuals
        ss   = fepois(df_b, @formula(visits ~ time_hi + v2hat + frfam + fe(ad_boot) + fe(female
        idx  = findfirst(==("time_hi"), coefnames(ss))
        results[b] = coef(ss)[idx]
    end
    return results
end

boot2 = bootstrap_cf_binary(df2, 500)
naive_se2 = stderror(m_a)[findfirst(==("time_hi"), coefnames(m_a))]

@printf("Linear-residual CF bootstrap (binary EEV, 500 reps, cluster by ad)\n")
@printf("  Point estimate:     %.4f  (true %.1f)\n", coef_a, true_ )
@printf("  Naive 2nd-stage SE: %.4f\n", naive_se2)
@printf("  Bootstrap SE:       %.4f\n", std(boot2))
@printf("  Bootstrap 95%% CI:  [%.4f, %.4f]\n",
        quantile(boot2, 0.025), quantile(boot2, 0.975))
```

```
Linear-residual CF bootstrap (binary EEV, 500 reps, cluster by ad)
  Point estimate:     0.8128  (true 0.8)
  Naive 2nd-stage SE: 0.0358
  Bootstrap SE:       0.0343
  Bootstrap 95% CI:  [0.7380, 0.8766]
```

18.5.9. All methods side by side

```
coef_naive2 = coef(
    fepois(df2, @formula(visits ~ time_hi + frfam + fe(ad) + fe(female)))
)[findfirst(==("time_hi"),
    coefnames(fepois(df2, @formula(visits ~ time_hi + frfam + fe(ad) + fe(female)))))]
coef_b = coef(m_b)[findfirst(==("time_hi"), coefnames(m_b))]

methods = ["True value", "Naive Poisson (binary y )",
           "Linear-residual CF (with FE)", "Generalized-residual CF (with FE)",
           "GMM, additive moments (no FE)",
           "GMM, multiplicative moments (no FE)"]
ests    = [true_ , coef_naive2, coef_a, coef_b,  _c[1],  _cm[1]]

println("\n  Binary EEV: coefficient on time_hi            ")
for (m, e) in zip(methods, ests)
    @printf("  %-42s  ^  = %6.4f\n", m, e)
end
```

```
  Binary EEV: coefficient on time_hi
  True value                                 ^  = 0.8000
  Naive Poisson (binary y )                  ^  = 1.0852
  Linear-residual CF (with FE)               ^  = 0.8128
  Generalized-residual CF (with FE)          ^  = 0.7636
  GMM, additive moments (no FE)              ^  = 0.5273
  GMM, multiplicative moments (no FE)        ^  = 0.7260
```

The two control functions recover the truth closely — with small $\rho = 0.2$ the approximate CF mean assumption is mild (the linear-residual CF has no valid first-stage assumption for binary y_2; the generalized-residual CF is correct only to first order in ρ, per the derivation above). Additive-moment IV Poisson does not: the unobservable enters inside $\exp(\cdot)$, so its moments are misspecified (Mullahy 1997).

The multiplicative-moment version repairs it, and in one specific sense does more than repair it: the `ad` and `female` effects it omits are independent multiplicative heterogeneity, absorbed by the intercept under the multiplicative moments ($\beta_0 \to \beta_0 + \log E[e^c]$), so dropping the fixed effects costs no slope *bias* at all. Mullahy's moments are *consistent* for this DGP, where the control functions are only approximately unbiased — 0.8015 at $n = 10^6$ (sd 0.0013), against plims of 0.839 and 0.822.

What consistency does not buy is precision, and that is why the table above can look as though it says the opposite. Without the fixed effects the multiplicative-moment estimator is roughly three times as noisy as either control function (sd 0.068 against 0.024 and 0.026, over 10 seeds at this chapter's $n = 5000$), so a single draw can leave the only consistent estimator of the three further from 0.8 than either CF estimator. At this sample size its RMSE is in fact the worst of the three (0.065, against 0.039 and 0.031); the ranking reverses as n grows. Read the comparison as one draw from that tradeoff, not as a verdict.

18.6. Practical guidance

- **With fixed effects and a binary y_2, use the generalized-residual CF.** Among the estimators that drop into FE-Poisson unmodified it is the theoretically motivated choice, and its error is *derived*, not assumed: $O(\rho^2)$, shrinking as the endogeneity shrinks. Put the unit dummies in its first stage only when each unit has many observations; with the short panels usual in applied work, use the Mundlak means instead (§ above).
- **Don't reach for `etpoisson` in a panel.** It fits Terza's FIML, which is the exact model, but it needs the Poisson distribution rather than just the mean, it has no `absorb()`, and unit dummies inside its quadrature likelihood bring back the incidental-parameter problem that FE-Poisson is normally free of (§ above).
- **Multiplicative-moment GMM trades bias for variance — decide which binds.** Mullahy's moments are *consistent* here, not merely accurate to first order, whenever the multiplicative heterogeneity is independent of $\mathbf{z}$. But consistency is not accuracy at a given sample size: over 10 seeds at $n = 5000$ it is nearly unbiased (mean 0.7985) and yet almost three times as noisy (sd 0.068 against 0.026), giving it the worst RMSE of the three. Prefer it once n is large enough that variance is not the binding constraint — and note that it does not accommodate high-dimensional fixed effects either way.
- **Use the multiplicative moments, not the additive ones.** When the unobservable sits inside $\exp(\cdot)$ — the natural case for counts — the additive moments are misspecified, and the simulation above shows the damage.
- **The linear-residual CF is a fallback, not a first choice.** It lands close to the truth at $\rho = 0.2$ here, but for binary y_2 it has no valid reduced-form assumption behind it. **Do not lean on it when ρ is not known to be small** — and note that ρ is estimated by the coefficient on the control function, so this is checkable rather than a matter of faith.
- **It remains fully valid as an exogeneity test.** Under $H_0 : \rho = 0$ none of the reduced-form assumptions are needed (Wooldridge 2010, 744). Use it to test; do not use it to correct.
- **Test, but do not let the test pick the model.** The obvious move is to run the t-test on the control function and, if it comes back insignificant, drop the residual and report the plain FE-Poisson. That is a pretest estimator and it is worse than either model it selects between. Guggenberger (2010) shows for the linear case that a Hausman pretest followed by OLS-or-2SLS, with the second-stage t-test read at its nominal level, has **asymptotic size one**. The mechanism is local endogeneity: let ρ drift to zero at rate $n^{-1/2}$ and the pretest usually fails to reject, while the bias of the estimator it falls back on is the same order as that estimator's standard error. No such theorem exists for the Poisson CF, but the mechanism

needs only a low-powered pretest and a fallback whose bias does not vanish faster than its standard error, and both are here. Report the naive and the control-function columns side by side.

- **Probit or logit both work for the generalized-residual CF.** Only Terza's exact correction needs normality.
- **Always bootstrap.** In every control-function specification above the second-stage standard errors are wrong, because the residual is generated.

18.7. Why Julia is faster

The parallel bootstrap above uses `Threads.@threads` which runs each of the 500 replications simultaneously across all CPU cores. A single replication calls `feols` + `fepois`, both implemented with the fast within-demeaning algorithm from `Panelest.jl`. On a machine with 8 cores this is roughly 6–8× faster than Stata's serial `bootstrap` command, and the speed advantage grows with replication count and sample size.

Part IV.

Longitudinal Causal Inference

19. G-Methods for Time-Varying Treatments

```
using DataFrames
using Distributions
using Random
using Statistics
using LinearAlgebra
using GLM
using Printf
```

Most chapters so far use one treatment decision. Many real problems are longitudinal. Treatment changes over time, covariates respond to past treatment, and those covariates then affect future treatment. Standard regression adjustment can fail in this setting.

> **Related reading**: The g-estimation with time-varying covariates chapter of *Topics on Econometrics and Causal Inference* derives the identification result and applies the parametric g-formula to a worked binary-treatment example. The LMTP framework for continuous and modified-policy interventions is covered in Longitudinal modified treatment policy (LMTP) and in the Continuous Treatments chapter.

The main tools are:

1. **G-formula**: simulate outcomes under a treatment regime.
2. **IPTW**: reweight observations by the probability of their treatment history.
3. **Marginal structural models**: fit a treatment-history regression in the weighted population.

For longitudinal TMLE, use R's `ltmle` package; there is no Julia equivalent yet. The g-formula and IPTW/MSM translate directly to Julia with `GLM.jl`.

19.1. The time-varying confounding problem

Consider a two-period example:

- L_0: baseline covariate (e.g. health status at study entry)
- A_0: treatment at time 0 (e.g. medication)
- L_1: covariate at time 1, **influenced by** A_0 (e.g. health status after first dose)
- A_1: treatment at time 1, depends on L_1
- Y: outcome at the end of follow-up

L_1 is the problem variable. It confounds the effect of A_1 on Y, so we would like to adjust for it. But it is also affected by A_0, so adjusting for it blocks part of the effect of A_0. Regression without L_1 is confounded; regression with L_1 over-controls.

19.1.1. Simulating the failure of standard regression

```
Random.seed!(1)
n = 5000

L0 = randn(n)
A0 = Float64.(rand(n) .< @. 1 / (1 + exp(-(-0.5 + 0.8 * L0))))
L1 = @. 0.5 * L0 + 1.0 * A0 + randn()
A1 = Float64.(rand(n) .< @. 1 / (1 + exp(-(-0.5 + 0.8 * L1))))
Y  = @. 0.5 * A0 + 0.5 * A1 + 0.5 * L1 + 0.5 * L0 + randn()

df = DataFrame(L0 = L0, A0 = A0, L1 = L1, A1 = A1, Y = Y)

# A0 has a direct effect (0.5) AND an indirect effect through L1
# (A0 -> L1 -> Y contributes 1.0 * 0.5 = 0.5), so A0's total effect is 1.0;
# A1's total effect is 0.5; always-vs-never total = 1.5.
@printf("True total effect of always-treated vs never-treated: 1.500\n\n")

naive_no_L1   = lm(@formula(Y ~ A0 + A1 + L0),       df)
naive_with_L1 = lm(@formula(Y ~ A0 + A1 + L0 + L1), df)

@printf("Naive (no L1):   A0=%.3f, A1=%.3f, total=%.3f\n",
        coef(naive_no_L1)[2], coef(naive_no_L1)[3],
        coef(naive_no_L1)[2] + coef(naive_no_L1)[3])
@printf("Naive (with L1): A0=%.3f, A1=%.3f, total=%.3f\n",
        coef(naive_with_L1)[2], coef(naive_with_L1)[3],
        coef(naive_with_L1)[2] + coef(naive_with_L1)[3])
```

```
True total effect of always-treated vs never-treated: 1.500

Naive (no L1):   A0=0.944, A1=0.850, total=1.794
Naive (with L1): A0=0.474, A1=0.478, total=0.951
```

Neither regression recovers the true total effect:

- Excluding L_1 leaves $L_1 \to A_1$ confounding that biases A_1.
- Including L_1 blocks the $A_0 \to L_1 \to Y$ pathway, so the coefficient on A_0 now measures only the *direct* effect, not the total effect.

This is exactly the setting g-methods were designed for.

19.2. Parametric g-formula

The g-formula writes the mean outcome under a treatment regime as:

$$\mathbb{E}[Y(\bar{a})] = \sum_{\bar{l}} \mathbb{E}[Y \mid \bar{a}, \bar{l}] \prod_{t=0}^{K} f(l_t \mid \bar{a}_{t-1}, \bar{l}_{t-1}), \tag{19.1}$$

where $\bar{a} = (a_0, a_1, \ldots, a_K)$ is a treatment regime and $\bar{l}$ is the covariate history. In practice we fit models for the time-varying covariates and the outcome, then simulate the data under the treatment regime.

```
# Step 1: Fit nuisance models
mod_L1 = lm(@formula(L1 ~ L0 + A0), df)
mod_Y  = lm(@formula(Y  ~ L0 + A0 + L1 + A1), df)

# Step 2: Simulate counterfactual outcomes under a regime (a0, a1).
# The L1 distribution under A0 = a0 differs from the observed L1.
function g_compute(a0::Real, a1::Real; n_mc::Int = 100)
    sigma_L1 = sqrt(sum(abs2.(residuals(mod_L1))) / dof_residual(mod_L1))
    Y_sim = zeros(n)
    for _ in 1:n_mc
        df_a0 = DataFrame(L0 = L0, A0 = fill(a0, n))
        L1_sim = predict(mod_L1, df_a0) .+ sigma_L1 .* randn(n)
        df_sim = DataFrame(L0 = L0, A0 = fill(a0, n),
                           L1 = L1_sim, A1 = fill(a1, n))
        Y_sim .+= predict(mod_Y, df_sim)
    end
    Y_sim ./= n_mc
    return mean(Y_sim)
end

Random.seed!(99)
E_Y11 = g_compute(1.0, 1.0)
E_Y00 = g_compute(0.0, 0.0)
@printf("G-formula estimate (always vs never): %.3f  (true = 1.500)\n",
        E_Y11 - E_Y00)
```

```
G-formula estimate (always vs never): 1.483  (true = 1.500)
```

The important step is simulating L_1 under the counterfactual treatment. We do not condition on the observed L_1, because observed L_1 is partly a consequence of observed treatment.

The g-formula can handle other regimes too. For example, treat only at $t = 0$:

```
Random.seed!(99)
E_Y10 = g_compute(1.0, 0.0)
@printf("Effect of treating only at t=0: %.3f  (true = 1.0)\n",
        E_Y10 - E_Y00)
```

```
Effect of treating only at t=0: 1.005  (true = 1.0)
```

19.3. IPTW for time-varying treatments

IPTW reweights observations by the inverse probability of their observed treatment history. The stabilized weight is:

$$SW_i = \prod_{t=0}^{K} \frac{f(A_t \mid \bar{A}_{t-1})}{f(A_t \mid \bar{A}_{t-1}, \bar{L}_t)}. \tag{19.2}$$

The numerator uses treatment history only. The denominator also conditions on the covariate history up to and including L_t — the covariates measured just before A_t. (Skipping L_t would leave the $L_t \to A_t$ confounding intact in the pseudo-population.)

```
# Numerator: P(A0) and P(A1 | A0)
n0 = glm(@formula(A0 ~ 1),  df, Binomial(), LogitLink())
n1 = glm(@formula(A1 ~ A0), df, Binomial(), LogitLink())

# Denominator: P(A0 | L0) and P(A1 | A0, L0, L1)
d0 = glm(@formula(A0 ~ L0),           df, Binomial(), LogitLink())
d1 = glm(@formula(A1 ~ A0 + L0 + L1), df, Binomial(), LogitLink())

p_n0 = predict(n0); p_n1 = predict(n1)
p_d0 = predict(d0); p_d1 = predict(d1)

# Probability of the OBSERVED treatment at each time
w0_num = ifelse.(df.A0 .== 1, p_n0, 1 .- p_n0)
w0_den = ifelse.(df.A0 .== 1, p_d0, 1 .- p_d0)
w1_num = ifelse.(df.A1 .== 1, p_n1, 1 .- p_n1)
w1_den = ifelse.(df.A1 .== 1, p_d1, 1 .- p_d1)

sw = (w0_num .* w1_num) ./ (w0_den .* w1_den)
@printf("Weight summary: min=%.3f, mean=%.3f, max=%.3f\n",
        minimum(sw), mean(sw), maximum(sw))

# Trim extreme weights at 99th percentile. Semicolon: without it this is the
# cell's last expression and Jupyter dumps all 5000 weights into the page.
sw_trim = min.(sw, quantile(sw, 0.99));
@printf("After trimming at the 99th percentile: max=%.3f\n", maximum(sw_trim))
```

```
Weight summary: min=0.284, mean=1.000, max=13.489
After trimming at the 99th percentile: max=3.370
```

19.4. Marginal structural models

With stabilized weights, the marginal structural model is a weighted regression of Y on treatment history:

```
df.sw = sw_trim
msm_fit = lm(@formula(Y ~ A0 + A1), df, wts = df.sw)

# display(), not println() -- see the note in the heterogeneous-effects chapter:
# println() on a CoefTable prints the raw constructor, not the table.
display(coeftable(msm_fit))
@printf("\nMSM total effect (always vs never): %.3f  (true = 1.500)\n",
        coef(msm_fit)[2] + coef(msm_fit)[3])

# For comparison: the untrimmed weights
df.sw_u = sw
msm_untrimmed = lm(@formula(Y ~ A0 + A1), df, wts = df.sw_u)
@printf("Untrimmed MSM total effect:         %.3f\n",
        coef(msm_untrimmed)[2] + coef(msm_untrimmed)[3])
```

	Coef.	Std. Error	t	Pr(>	t	)
(Intercept)	-0.0498963	0.0288233	-1.73	0.0835	-0.106403	0.00661038
A0	1.04559	0.0399352	26.18	<1e-99	0.967303	1.12388
A1	0.547731	0.0391661	13.98	<1e-42	0.470948	0.624515

```
MSM total effect (always vs never): 1.593  (true = 1.500)
Untrimmed MSM total effect:         1.486
```

The weighted population is constructed so treatment is no longer associated with past covariates. That is why the MSM targets the total effect. Note the trimmed estimate sits a little above the truth while the untrimmed one is closer: the weight models here are correctly specified, so the untrimmed weights are consistent, and capping the top 1% of weights reintroduces a bit of confounding bias in exchange for lower variance. Trimming is a bias-variance trade, not a free lunch.

19.5. Hernán-style g-estimation: when to use which

Estimator	Strengths	Weaknesses
G-formula	Handles arbitrary regimes; transparent	Sensitive to model misspecification on L_t and Y
IPTW + MSM	Simple, intuitive; valid with correct propensity model	Variance large with extreme weights

In practice, reporting both is useful. Agreement is reassuring; disagreement suggests model misspecification in at least one of them.

For longitudinal TMLE, use R's `ltmle` package. A Julia port would be useful but does not exist yet.

19.6. A robustness check: g-formula with covariates only

Even without IPTW + MSM, the g-formula is a useful check on standard regression. If standard regression and the g-formula disagree, that gap is a warning about time-varying confounding.

```
Random.seed!(123)
g_total = E_Y11 - E_Y00
msm_total = coef(msm_fit)[2] + coef(msm_fit)[3]

@printf("%-30s %.3f\n", "True total effect:", 1.5)
@printf("%-30s %.3f  (wrong)\n", "Standard regression (no L1):",
        coef(naive_no_L1)[2] + coef(naive_no_L1)[3])
@printf("%-30s %.3f  (wrong)\n", "Standard regression (with L1):",
        coef(naive_with_L1)[2] + coef(naive_with_L1)[3])
@printf("%-30s %.3f  (correct)\n", "G-formula:", g_total)
@printf("%-30s %.3f  (correct)\n", "MSM (IPTW):", msm_total)
```

```
True total effect:             1.500
Standard regression (no L1):   1.794  (wrong)
Standard regression (with L1): 0.951  (wrong)
G-formula:                     1.483  (correct)
MSM (IPTW):                    1.593  (correct)
```

The standard regressions are wrong in opposite directions; the g-methods both recover the truth. When standard and g-formula estimates diverge, the divergence is the bias.

19.7. When to reach for these methods

Use g-methods when:

1. There is a time-varying confounder L_t that is affected by past treatment A_{t-1} and itself affects subsequent treatment A_t.
2. The research question concerns a *sustained* or *dynamic* treatment regime, not a single decision at a single time.

This includes many clinical follow-up studies and longitudinal studies of education or labor-market policies.

For a one-shot treatment, the methods from the Estimation and Nonparametric Causal Methods chapters are sufficient. The added complexity of g-methods is only justified when the time-varying confounder problem is present.

19.8. Summary

- Standard regression fails when a time-varying confounder is itself affected by past treatment.
- G-formula simulates outcomes under treatment regimes.
- IPTW + MSM reweights the population and then fits a weighted treatment history model.
- Longitudinal TMLE is best handled in R's `ltmle` package for now.
- Disagreement across estimators is useful information, not just a nuisance.

Part V.

Survival & Time-to-Event

20. Survival Analysis with Causal Inference

```
using DataFrames
using Distributions
using GLM
using Statistics
using Random
using LinearAlgebra
using Printf
using CairoMakie
CairoMakie.activate!(type = "png")
using CSV
```

When the outcome is time-to-event, such as death, readmission, job exit, or machine failure, ordinary regression is not enough. Some observations are censored. We know the event had not happened by the end of follow-up, but we do not know the event time.

This Julia chapter implements Kaplan-Meier curves, IPW-adjusted survival curves, and RMST differences. For Cox models and doubly robust survival estimators, R's `survival` and `riskRegression` packages are still the more practical tools; see the R companion chapter.

20.1. The censoring problem

We observe $\min(T_i, C_i)$ and $\delta_i = \mathbb{1}\{T_i \leq C_i\}$, where T_i is the event time and C_i is the censoring time. The main quantity of interest is the survival function

$$S(t) = P(T > t) \tag{20.1}$$

the probability of surviving past time t. Causal contrasts are usually reported as:

- **Hazard ratio (HR)** — easy to estimate but hard to interpret causally (Hernán 2010).
- **Restricted mean survival difference (RMST)** — $\int_0^\tau [S_1(t) - S_0(t)]dt$ — interpretable as "years gained" up to time τ.
- **Survival probability difference at t** — direct probability statement, requires picking a follow-up time.

20.2. Simulated survival data

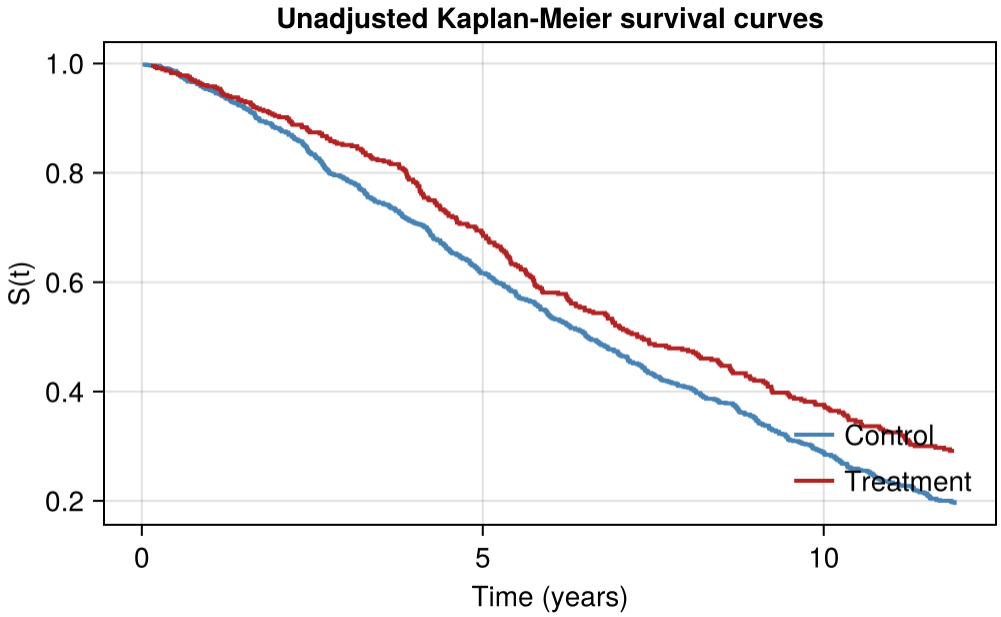
```
df = CSV.read("data/survival_sim.csv", DataFrame)
n  = nrow(df)
@printf("n = %d, events = %d (%.1f%%), censored = %d\n",
        n, Int(sum(df.event)), 100 * mean(df.event), Int(sum(1 .- df.event)))
```

```
n = 1500, events = 1010 (67.3%), censored = 490
```

20.3. Kaplan-Meier estimator

The Kaplan-Meier estimator computes $\hat{S}(t)$ nonparametrically. It is short enough to implement directly:

```
"""
    kaplan_meier(time, event)
Compute Kaplan-Meier survival estimates at the unique event times.
Returns (times, survival).
"""
function kaplan_meier(time, event)
    df_sort = sort(DataFrame(t = time, d = event), :t)
    event_times = unique(df_sort.t[df_sort.d .== 1])
    survival = Float64[]
    s = 1.0
    for t in event_times
        n_at_risk = sum(df_sort.t .>= t)
        n_events  = sum((df_sort.t .== t) .& (df_sort.d .== 1))
        s *= 1 - n_events / n_at_risk
        push!(survival, s)
    end
    return event_times, survival
end

t_trt, s_trt = kaplan_meier(df.time[df.trt .== 1], df.event[df.trt .== 1])
t_ctl, s_ctl = kaplan_meier(df.time[df.trt .== 0], df.event[df.trt .== 0])

fig = Figure(size = (700, 400))
ax = Axis(fig[1, 1], xlabel = "Time (years)", ylabel = "S(t)",
          title = "Unadjusted Kaplan-Meier survival curves")
stairs!(ax, t_ctl, s_ctl, color = :steelblue, linewidth = 2,
        label = "Control")
stairs!(ax, t_trt, s_trt, color = :firebrick, linewidth = 2,
        label = "Treatment")
axislegend(ax, position = :rb, framevisible = false)
fig
```



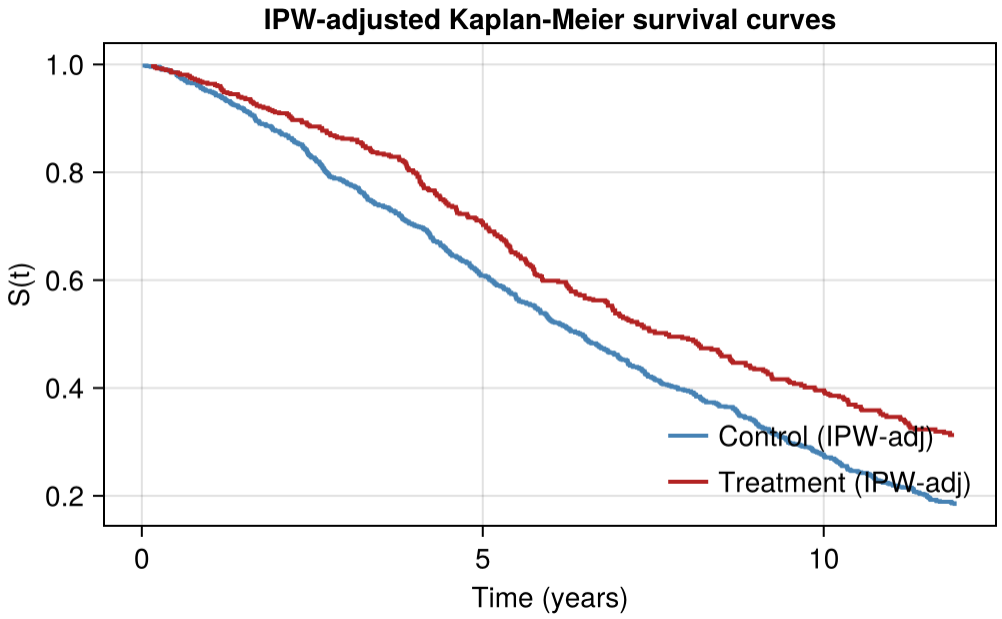
The unadjusted KM curves are descriptive. They include both the treatment effect and baseline imbalance.

20.4. IPW-adjusted survival

To adjust for confounding, weight observations by the inverse propensity score before computing KM:

```
ps_fit = glm(@formula(trt ~ age + sex), df, Binomial(), LogitLink())
df.ps  = predict(ps_fit)
df.ipw = ifelse.(df.trt .== 1, 1 ./ df.ps, 1 ./ (1 .- df.ps))
df.ipw .= min.(df.ipw, quantile(df.ipw, 0.99))

"""
    weighted_kaplan_meier(time, event, weights)
KM estimate with sample weights - used for IPW-adjusted survival.
"""
function weighted_kaplan_meier(time, event, weights)
    df_sort = sort(DataFrame(t = time, d = event, w = weights), :t)
    event_times = unique(df_sort.t[df_sort.d .== 1])
    survival = Float64[]
    s = 1.0
    for t in event_times
        w_at_risk = sum(df_sort.w[df_sort.t .>= t])
        w_events  = sum(df_sort.w[(df_sort.t .== t) .& (df_sort.d .== 1)])
        s *= 1 - w_events / w_at_risk
        push!(survival, s)
```

```
    end
    return event_times, survival
end

t_trt_w, s_trt_w = weighted_kaplan_meier(df.time[df.trt .== 1],
                                         df.event[df.trt .== 1],
                                         df.ipw[df.trt .== 1])
t_ctl_w, s_ctl_w = weighted_kaplan_meier(df.time[df.trt .== 0],
                                         df.event[df.trt .== 0],
                                         df.ipw[df.trt .== 0])

fig2 = Figure(size = (700, 400))
ax2 = Axis(fig2[1, 1], xlabel = "Time (years)", ylabel = "S(t)",
           title = "IPW-adjusted Kaplan-Meier survival curves")
stairs!(ax2, t_ctl_w, s_ctl_w, color = :steelblue, linewidth = 2,
        label = "Control (IPW-adj)")
stairs!(ax2, t_trt_w, s_trt_w, color = :firebrick, linewidth = 2,
        label = "Treatment (IPW-adj)")
axislegend(ax2, position = :rb, framevisible = false)
fig2
```



The IPW-adjusted curves remove the age and sex imbalance that is in the propensity score model.

20.5. Restricted Mean Survival Time

The **RMST** up to time τ is

$$\mathrm{RMST}(\tau) = E[\min(T, \tau)] = \int_0^\tau S(t)\, dt. \tag{20.2}$$

The contrast $\Delta_{\mathrm{RMST}}(\tau) = \mathrm{RMST}_1(\tau) - \mathrm{RMST}_0(\tau)$ is interpretable as years gained up to τ. It does not require proportional hazards.

Because the KM curve is a step function, its integral is computed exactly by a left rectangle sum (width times the survival value at the left endpoint of each segment):

```
function rmst(times, survival, tau)
    # Add time 0 and the truncation point, with appropriate S values
    t_vec = [0.0; times; tau]
    s_vec = [1.0; survival; survival[end]]
    # Restrict to times  tau
    keep = t_vec .<= tau
    t_use = t_vec[keep]
    s_use = s_vec[keep]
    # Add the tau endpoint, unless it's already the last element (t_vec was
    # constructed with tau appended, so `keep` already retains it -- pushing
    # again would duplicate it and create a zero-width trailing interval).
    if t_use[end] != tau
        push!(t_use, tau)
        push!(s_use, s_use[end])
    end
    # Exact integral of the KM step function: width × left-height
    sum(diff(t_use) .* s_use[1:end-1])
end

  = 5.0
rmst_trt_unadj = rmst(t_trt, s_trt,  )
rmst_ctl_unadj = rmst(t_ctl, s_ctl,  )
rmst_trt_adj   = rmst(t_trt_w, s_trt_w,  )
rmst_ctl_adj   = rmst(t_ctl_w, s_ctl_w,  )

@printf("Unadjusted RMST( =5):\n")
@printf("  Treatment: %.3f years    Control: %.3f years    Diff: %.3f\n",
        rmst_trt_unadj, rmst_ctl_unadj, rmst_trt_unadj - rmst_ctl_unadj)
@printf("IPW-adjusted RMST( =5):\n")
@printf("  Treatment: %.3f years    Control: %.3f years    Diff: %.3f\n",
        rmst_trt_adj, rmst_ctl_adj, rmst_trt_adj - rmst_ctl_adj)
```

```
Unadjusted RMST( =5):
  Treatment: 4.351 years    Control: 4.147 years    Diff: 0.203
IPW-adjusted RMST( =5):
  Treatment: 4.397 years    Control: 4.122 years    Diff: 0.275
```

The IPW-adjusted RMST difference is the adjusted years-gained estimate up to 5 years.

20.6. Cox proportional hazards (regression adjustment)

Julia's survival-regression tools are less mature than R's `survival::coxph`. In Julia, the choices are:

- Use `Survival.jl` for basic Cox PH (limited adjustment capabilities).
- Use a parametric Weibull/exponential model via `Distributions` and `Optim` directly.
- Call out to R for serious survival modelling.

For applied work where the Cox model with covariate adjustment is the target, R remains the practical choice. See the companion R chapter for the full Cox + IPCW + doubly-robust workflow.

20.7. Doubly-robust survival estimation

R's `riskRegression::ate` combines a Cox outcome model with a propensity score and censoring model. Julia does not currently have an equivalent.

For now, the practical Julia workflow is:

1. **Description**: unadjusted KM curves (this chapter).
2. **Adjusted survival**: IPW-weighted KM (this chapter).
3. **Causal contrasts**: RMST differences from the IPW-adjusted KM (this chapter).
4. **Cox HR with adjustment + doubly-robust estimators**: call out to R via `RCall.jl` or use the R companion chapter.

20.8. When to use these methods

Use these methods when:

1. The outcome is time-to-event with censoring.
2. The treatment effect on survival probabilities or years-of-life is the causal question.

For non-censored outcomes, the standard Estimation and Heterogeneous Effects methods are sufficient.

20.9. Summary

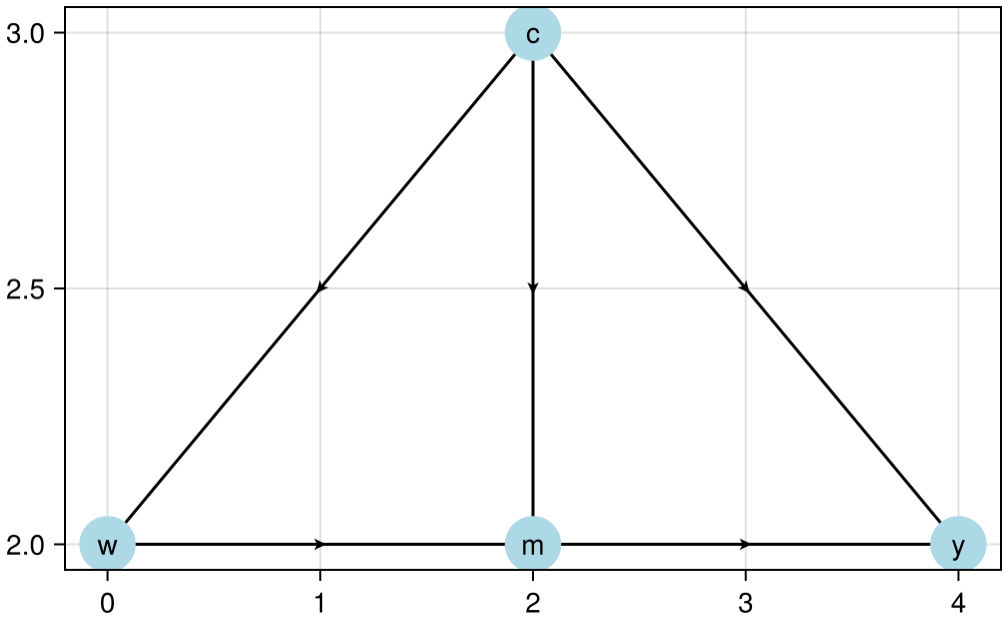
- Survival outcomes require methods that handle censoring.
- Kaplan-Meier estimates the survival curve nonparametrically.
- IPW-adjusted KM handles baseline confounding through weights.
- RMST differences are often the clearest causal summary.
- For Cox regression and doubly robust survival estimators, use R for now.

Part VI.

Mediation

21. Causal Mediation Analysis

```
using CairoMakie
CairoMakie.activate!(type = "png")
using GraphMakie
using Graphs
using GLM
using DataFrames
using StatsModels
using DataFramesMeta
using Random
using Distributions
using StatsFuns
using Statistics
using Lavaan
using Crumble
using Printf
```



21.1. Classical Mediation

Traditionally mediation model can be represented in the following equations:

$$Y = aW + bM + \epsilon_1 \tag{21.1}$$

$$M = cW + \epsilon_2 \tag{21.2}$$

That is, we'd like to study the effect of W on Y, and we see the effect can be a direct effect, and an indirect effect, through M.

Baron and Kenny's (http://davidakenny.net/cm/mediate.htm) method is done in four steps. Modern approach tends to use SEM (structural equation modeling) to model these two equations directly.

```
using Random
using DataFrames
using StatsModels
using Lavaan

Random.seed!(1234)
n = 10000
X = randn(n)
M = 0.5 .* X .+ randn(n)
Y = 0.7 .* M .+ 0.3 .* X .+ randn(n)
Data = DataFrame(X = X, M = M, Y = Y)

model = """
   Y ~ a*X + b*M
   M ~ c*X
   # direct effect (a)
   # indirect effect (b*c)
   bc := b*c
   # total effect
   total := a + (b*c)
"""
```

```
"   Y ~ a*X + b*M\n   M ~ c*X\n   # direct effect (a)\n   # indirect effect (b*c)\n   bc := b*
```

```
fit = sem(model, Data)
fit
```

```
lavaan 0.1.0-dev fit object
  Estimator:   ML
  Converged:   Yes
   ²(1) = 999999999999999983222784.000  [p = 0.000]
  Call summary() for full output.
```

21.1.1. Problems with Classical Mediation

- Lack of causal claim. We have to assume that there is no unmeasured confounder between M and Y. This is a strong assumption.
- Assumption of homogeneous effect.

21.2. Causal Mediation

21.2.1. CDE

Suppose we can set W and M at will to any (w, m), then we have the potential outcome $Y(w, m)$.

Controlled direct effect (CDE) is defined as

$$CDE(m) = E[Y(1, m) - Y(0, m)] \tag{21.3}$$

That is, setting M to m, what is the effect of W on Y?

21.2.2. Assumptions

- Conditional treatment randomization. Suppose we observe confounders X, which can be a joint set of confounders for the W to Y pathway, or the M to Y pathway.

$$Y(w, m) \perp W|X \tag{21.4}$$

This is the usual conditional independence (unconfoundedness, ignorability, etc.) assumption. This is saying the assignment of treatment, given covariates X, has nothing to do with potential outcomes.

- Conditional mediator randomization.

$$Y(w, m) \perp M|X, W = w \tag{21.5}$$

This is to say, within each strata of X, given treatment status, the assignment of mediator gives no information about potential outcome. This randomization is usually not implemented during many experiments (random trials). This is the assumption that makes a lot of mediation hard to make a causal claim.

- Positivity (overlap). There are two positivity assumptions:

$$P(W = w|X = x) > 0 \tag{21.6}$$

for all x. This is the usual positivity assumption.

$$P(M = m|X = x, W = w) > 0 \tag{21.7}$$

for all x, w, and m. This is the mediator positivity.

21.2.3. Estimation

CDE can be estimated using G-computation method, or IPW, or the doubly-robust methods, one of them is AIPW (augmented IPW).

21.2.4. G-computation

G-computation is to model the outcome equation:

$$\begin{aligned} CDE_G(m) = \sum_x [&E[Y \mid W=1, M=m, X=x] \\ &- E[Y \mid W=0, M=m, X=x]]\, P(X=x) \end{aligned} \tag{21.8}$$

21.2.5. IPW

IPW is to model the treatment assignment equation and the mediator assignment equation. The joint denominator factorizes *sequentially* – treatment given covariates, then mediator given treatment and covariates – which is the structure the estimator below uses:

$$E[Y(w,m)] = E\left[\frac{I(W=w)\, I(M=m)}{g_W(w|X)\, g_M(m|w,X)}\, Y\right] \tag{21.9}$$

Therefore,

$$CDE_{ipw}(m) = E\left[\frac{I(W=1, M=m)}{g_M(m|1,X)\, g_W(1|X)}\, Y - \frac{I(W=0, M=m)}{g_M(m|0,X)\, g_W(0|X)}\, Y\right] \tag{21.10}$$

where g_M is the probability of the mediator and g_W the probability of treatment. **This indicator-based form applies to a discrete (here binary) mediator.** For a continuous mediator the indicator $I(M=m)$ and probability g_M must be replaced by a conditional density (or a kernel / stochastic-intervention formulation); the estimator does not apply as written.

21.2.6. AIPW

$$CDE_{AIPW} = CDE_G(m) + B(\bar{Q}, g_M, g_W) \tag{21.11}$$

where $\bar{Q}$ is the mean outcome function.

$$\begin{aligned} B(\bar{Q}, g_M, g_W) = &\frac{1}{n}\sum_{i=1}^{n} \frac{I(M_i=m, W_i=1)}{g_M(m|1,X_i) g_W(1|X_i)} [Y_i - \bar{Q}(m,1,X_i)] \\ &- \frac{1}{n}\sum_{i=1}^{n} \frac{I(M_i=m, W_i=0)}{g_M(m|0,X_i) g_W(0|X_i)} [Y_i - \bar{Q}(m,0,X_i)] \end{aligned} \tag{21.12}$$

21.2.7. Example: G-computation

```
using GLM
using Distributions
using StatsFuns
using Statistics

Random.seed!(1234)
n = 5000
# confounder of A/Y
W1 = randn(n)
# confounder of M/Y
W2 = randn(n)
# treatment
A = rand.(Bernoulli.(logistic.(-1 .+ W1 ./ 2)))
# binary mediator
M = rand.(Bernoulli.(logistic.(-2 .+ A ./ 2 .+ W2 ./ 3)))
# binary outcome
Y = rand.(Bernoulli.(logistic.(-1 .+ A .- M ./ 2 .+ W1 ./ 3 .+ W2 ./ 3)))
full_data = DataFrame(W1 = W1, W2 = W2, A = A, M = M, Y = Y)

# fit outcome regression
or_fit = glm(@formula(Y ~ A + M + W1 + W2), full_data, Binomial(), LogitLink())

# new data setting A and M
data_A1_M0 = copy(full_data)
data_A0_M0 = copy(full_data)
data_A1_M0.A .= 1; data_A1_M0.M .= 0
data_A0_M0.A .= 0; data_A0_M0.M .= 0

# predict on new data
Qbar_A1_M0 = predict(or_fit, data_A1_M0)
Qbar_A0_M0 = predict(or_fit, data_A0_M0)

# gcomp estimate of CDE(0)
cde_gcomp = mean(Qbar_A1_M0 .- Qbar_A0_M0)
@printf "CDE(0) G-computation = %.4g\n" cde_gcomp
```

```
CDE(0) G-computation = 0.2113
```

21.2.8. Example: IPW

```
# model for P(A = 1 | W)
ps_fit1 = glm(@formula(A ~ W1 + W2), full_data, Binomial(), LogitLink())
```

```
P_A1_W = predict(ps_fit1)
P_A0_W = 1 .- P_A1_W

# model for P(M = 0 | A, W)
ps_fit2 = glm(@formula(M ~ A + W1 + W2), full_data, Binomial(), LogitLink())

# P(M = 0 | A = 1, W)
data_A1 = copy(full_data); data_A1.A .= 1
P_M0_A1_W = 1 .- predict(ps_fit2, data_A1)

# P(M = 0 | A = 0, W)
data_A0 = copy(full_data); data_A0.A .= 0
P_M0_A0_W = 1 .- predict(ps_fit2, data_A0)

# ipw estimate of CDE(0)
cde_ipw = mean( (A .== 1) ./ P_A1_W .* (M .== 0) ./ P_M0_A1_W .* Y ) -
          mean( (A .== 0) ./ P_A0_W .* (M .== 0) ./ P_M0_A0_W .* Y )
@printf "CDE(0) IPW = %.4g\n" cde_ipw
```

```
CDE(0) IPW = 0.226
```

21.2.9. Example: AIPW

```
# aipw estimate of E[Y(1,0)]
aiptw_EY_A1_M0 = mean(Qbar_A1_M0) +
mean( (A .== 1) ./ P_A1_W .* (M .== 0) ./ P_M0_A1_W .* (Y .- Qbar_A1_M0) )

# aipw estimate of E[Y(0,0)]
aiptw_EY_A0_M0 = mean(Qbar_A0_M0) +
mean( (A .== 0) ./ P_A0_W .* (M .== 0) ./ P_M0_A0_W .* (Y .- Qbar_A0_M0) )

# aipw estimate of CDE(0)
cde_aipw = aiptw_EY_A1_M0 - aiptw_EY_A0_M0
@printf "CDE(0) AIPW = %.4g\n" cde_aipw
```

```
CDE(0) AIPW = 0.2245
```

21.3. NIE and NDE: Natural Direct and Indirect Effects

CDE is to study the effect of treatment, given the level of mediator. Instead, Natural Effect is to set mediator to its natural value with the value of treatment, that is, $M = M(w)$.

$$\begin{aligned} ATE &= NIE + NDE \\ &= (E[Y(1, M(1))] - E[Y(1, M(0))]) \\ &\quad + (E[Y(1, M(0))] - E[Y(0, M(0))]) \end{aligned} \tag{21.13}$$

The advantage of NDE and NIE comparing to CDE is that it's more "natural"; that is, you don't set the level of mediator deterministically. And it can decompose the ATE into direct and indirect effects.

However, there is an additional assumption needed to identify NDE and NIE.

21.3.1. Additional Assumption

$$Y(w, m) \perp M(w^*)|X \tag{21.14}$$

This is the "cross-world" condition: the outcome under (w, m) is independent of M under w^*. These two situations cannot happen in the same world; you cannot set W to both w and w^*. No experiment can implement it.

This cross-world independence is *additional to*, not a substitute for, the usual identifying assumptions. The standard mediation formula for natural effects also requires:

- treatment ignorability (no unmeasured treatment-outcome confounding) given X;
- mediator ignorability (no unmeasured mediator-outcome confounding) given W and X;
- positivity for both treatment and mediator; and
- **no treatment-induced confounding** of the mediator-outcome relationship – no variable affected by W may confound M and Y. (When such a confounder exists, natural effects are not identified by this formula; interventional direct/indirect effects are used instead.)

Cross-world independence alone is not sufficient.

21.3.2. Estimation

```
# fit outcome regression (include interaction because we can)
or_fit = glm(@formula(Y ~ A + M + W1 + W2 + A&M + M&W1), full_data, Binomial(), LogitLink())

# need E(Y | A = 0/1, M = 0/1, W1 = W1i, W2 = W2i)
function get_EY_a_m_Wi(full_data, or_fit, a, m)
    data_Aa_Mm_Wi = copy(full_data)
    data_Aa_Mm_Wi.A .= a
    data_Aa_Mm_Wi.M .= m
    predict(or_fit, data_Aa_Mm_Wi)
end

EY_A0_M0_Wi = get_EY_a_m_Wi(full_data, or_fit, 0, 0)
EY_A0_M1_Wi = get_EY_a_m_Wi(full_data, or_fit, 0, 1)
EY_A1_M0_Wi = get_EY_a_m_Wi(full_data, or_fit, 1, 0)
```

```
EY_A1_M1_Wi = get_EY_a_m_Wi(full_data, or_fit, 1, 1)

# include interactions -- why not? (NOTE: A*W1 expands to A + W1 + A&W1, so
# the MAIN effects are included. Writing only A&W1 would omit the main effect
# of A and force the estimated NIE to be essentially zero by construction.)
med_fit = glm(@formula(M ~ A*W1 + W1*W2), full_data, Binomial(), LogitLink())

# estimates of P(M = m | A = a, W = W_i)
function get_Pm_a_Wi(full_data, med_fit, a, m)
    data_Aa_Wi = copy(full_data)
    data_Aa_Wi.A .= a
    p = predict(med_fit, data_Aa_Wi)
    if m == 1
        return p
    else
        return 1 .- p
    end
end

PM0_A0_Wi = get_Pm_a_Wi(full_data, med_fit, 0, 0)
PM1_A0_Wi = get_Pm_a_Wi(full_data, med_fit, 0, 1)
PM0_A1_Wi = get_Pm_a_Wi(full_data, med_fit, 1, 0)
PM1_A1_Wi = get_Pm_a_Wi(full_data, med_fit, 1, 1)

# E(E(Y | A = 1, M, W) | A = 1, W)
EY1M1_Wi = EY_A1_M1_Wi .* PM1_A1_Wi .+ EY_A1_M0_Wi .* PM0_A1_Wi
# E(E(Y | A = 0, M, W) | A = 1, W)
EY0M1_Wi = EY_A0_M1_Wi .* PM1_A1_Wi .+ EY_A0_M0_Wi .* PM0_A1_Wi
# E(E(Y | A = 1, M, W) | A = 0, W)
EY1M0_Wi = EY_A1_M1_Wi .* PM1_A0_Wi .+ EY_A1_M0_Wi .* PM0_A0_Wi
# E(E(Y | A = 0, M, W) | A = 0, W)
EY0M0_Wi = EY_A0_M1_Wi .* PM1_A0_Wi .+ EY_A0_M0_Wi .* PM0_A0_Wi

# estimate of E[Y(1, M(1))]
E_Y1M1 = mean(EY1M1_Wi)
# estimate of E[Y(0, M(1))]
E_Y0M1 = mean(EY0M1_Wi)
# estimate of E[Y(1, M(0))]
E_Y1M0 = mean(EY1M0_Wi)
# estimate of E[Y(0, M(0))]
E_Y0M0 = mean(EY0M0_Wi)

# NDE = E[Y(1,M(0))] - E[Y(0,M(0))], NIE = E[Y(1,M(1))] - E[Y(1,M(0))]
@printf "NDE = %.4g\n" (E_Y1M0 - E_Y0M0)
@printf "NIE = %.4g\n" (E_Y1M1 - E_Y1M0)
@printf "ATE = %.4g\n" (E_Y1M1 - E_Y0M0)
```

```
NDE = 0.2093
NIE = -0.01266
ATE = 0.1966
```

21.4. IIE and IDE: Interventional Direct and Indirect Effects

People are not happy with the cross-world assumption in general. Interventional effects avoid it. The device is a random draw: instead of the unit's own $M(w^*)$, use $M^*_{w^*}$, a draw from the *distribution* of $M(w^*)$ given $X = x$. Effects defined through such draws are identified without the cross-world assumption, even when a treatment-induced confounder Z sits between treatment and mediator — the situation in the example below.

One subtlety matters. The plain randomized-interventional decomposition (available in `Crumble.jl` as `effect = "RI"`) uses the draw in *every* term; its direct and indirect effects sum to an "overall interventional effect" that need **not** equal the ATE when Z is present, because the random draw breaks the within-unit dependence between M and Y. The **recanting twins** construction (`effect = "RT"`, used below) is built precisely to repair this: it decomposes the *actual* ATE,

$$ATE \;=\; E[Y(1, Z(1), M(1))] - E[Y(0, Z(0), M(0))] \;=\; IIE + IDE, \tag{21.15}$$

into direct and indirect components by walking a path of intermediate "twin" regimes for Z and M, each identified without cross-world independence. With `effect = "RT"`, the printed $IDE + IIE$ adds up to the ATE by construction.

> ⚠ Warning
>
> **Two caveats on the numbers below, both about `Crumble.jl` rather than about the estimand.** First, the direct and indirect standard errors are combined as $\sqrt{se_a^2 + se_b^2}$. That treats two estimators computed on the *same* sample as independent, so it ignores their covariance; the correct standard error comes from the variance of the difference of the influence curves, not from adding their variances. Second, the `"RT"` path currently computes both the natural and the randomized influence curves and then combines only the natural ones, so what is reported is not yet the recanting-twins decomposition described above — the identity $IDE + IIE = ATE$ holds arithmetically, because all three are built from the same three regime means, but it holds for the natural decomposition.
> Read the point estimates as an illustration of the workflow. For interventional effects to lean on, the `medoutcon` output in the R companion chapter is the better reference.

21.4.1. Example

```
using DataFrames
using StatsModels
using Random
using StatsFuns
```

```
using Distributions
using Crumble

Random.seed!(1584)

# produces a simple data set based on a causal model with mediation
function make_example_data(n_obs = 1000)
    # baseline covariates
    w_1 = rand.(Bernoulli(0.6), n_obs)
    w_2 = rand.(Bernoulli(0.3), n_obs)
    w_3_prob = min.(0.2 .+ (w_1 .+ w_2) ./ 3, 1.0)
    w_3 = rand.(Bernoulli.(w_3_prob))

    # exposure
    a_prob = logistic.(w_1 .+ w_2 .+ w_3 .- 2)
    a = rand.(Bernoulli.(a_prob))

    # mediator-outcome confounder affected by treatment
    z_prob = logistic.(-log(2) .- a .+ (w_1 .+ w_2 .+ w_3) ./ 3 .+ 0.2)
    z = rand.(Bernoulli.(z_prob))

    # mediator -- could be multivariate
    m_prob = logistic.(log(3) .* (w_1 .+ w_2) .+ 2 .* a .- 2 .* z)
    m = rand.(Bernoulli.(m_prob))

    # outcome
    y_prob = logistic.(1 ./ (w_1 .+ w_2 .+ w_3 .- z .+ a .+ m))
    y = rand.(Bernoulli.(y_prob))

    # construct output
    dat = DataFrame(W_1 = w_1, W_2 = w_2, W_3 = w_3, A = a, Z = z, M = m, Y = y)
    return dat
end
```

```
make_example_data (generic function with 2 methods)
```

```
# set seed and simulate example data
example_data = make_example_data()
w_names = ["W_1", "W_2", "W_3"]
m_names = ["M"]

# quick look at the data
first(example_data, 6)
```

	W_1	W_2	W_3	A	Z	M	Y
	Bool	Bool	Bool	Bool	Bool	Bool	Bool
1	0	1	0	0	0	1	1
2	0	0	0	0	0	1	1
3	0	0	0	0	0	1	1
4	0	0	0	0	0	1	1
5	1	0	0	0	0	0	1
6	1	1	1	1	1	1	0

```
# Estimate interventional direct and indirect effects using Crumble.jl
# effect = "RT" is the recanting twin estimand, which gives IDE and IIE
# while handling Z (mediator-outcome confounder affected by treatment)
result = crumble(example_data, ["A"];
                 outcome   = "Y",
                 mediators = m_names,
                 moc       = ["Z"],       # post-treatment confounder Z
                 covar     = w_names,
                 effect    = "RT",        # recanting twin
                 learners  = ["glm"])
result
```

```
CrumbleResult
  Effect type: RT

Estimates:
  Direct Effect                              -0.0005 (SE:   0.0241) [95% CI:  -
0.0477,   0.0467]
  Average Treatment Effect                   -0.0005 (SE:   0.0241) [95% CI:  -
0.0477,   0.0467]
  Indirect Effect                            -0.0000 (SE:   0.0241) [95% CI:  -
0.0473,   0.0472]
```

```
# Extract direct and indirect effect estimates
ide = result.estimates["direct"]
iie = result.estimates["indirect"]
@printf "IDE (Direct)   = %.4g (SE = %.4g)\n" ide["estimate"] ide["std.error"]
@printf "IIE (Indirect) = %.4g (SE = %.4g)\n" iie["estimate"] iie["std.error"]
@printf "ATE            = %.4g\n" result.estimates["ate"]["estimate"]
```

```
IDE (Direct)   = -0.0005093 (SE = 0.02408)
IIE (Indirect) = -2.789e-05 (SE = 0.0241)
ATE            = -0.0005372
```

Part VII.

Causal Discovery

22. Causal Discovery: Observed Variables

```
using CausalInference
using RecursiveCausalDiscovery
using CausalGraphs
using Graphs
using CairoMakie
CairoMakie.activate!(type = "png")
using Statistics
using LinearAlgebra
using Random
using Printf
using DataFrames
using CSV
using Distributions: Normal, quantile
```

Earlier chapters assumed the graph was known, or at least that we could draw a defensible DAG from subject knowledge. Causal discovery asks what can be learned about the graph from observational data.

22.1. The Problem

Given n i.i.d. observations of p variables $(X_1, \ldots, X_p)$, we want to recover the directed acyclic graph (DAG) G that generated the data.

There is an important limit: observational data alone cannot distinguish between DAGs with the same conditional independence structure. The set of observationally equivalent DAGs is a **Markov equivalence class**. The data can identify this class, represented by a CPDAG, not necessarily the one true DAG.

A CPDAG has the same skeleton (undirected edges) as all DAGs in its MEC. Some edges can be oriented (they have the same direction in every member of the MEC); others remain undirected because both directions are consistent with the CI structure.

22.1.1. Why some edges cannot be oriented

Three DAGs are observationally equivalent — they produce the same set of conditional independences — and therefore indistinguishable from data:

$$X \to Y \to Z \qquad X \leftarrow Y \leftarrow Z \qquad X \leftarrow Y \to Z \tag{22.1}$$

All three imply $X \perp\!\!\!\perp Z \mid Y$ and no other CI. Their CPDAG shows $X - Y - Z$ with no arrowheads.

The structure $X \to Y \leftarrow Z$ (a **collider** at Y) is *not* equivalent: here $X \perp\!\!\!\perp Z$ but $X \not\perp\!\!\!\perp Z \mid Y$. Because it has a unique CI signature, the $\to Y \leftarrow$ orientation is identifiable from data. Algorithms orient these colliders; non-collider edges often remain undirected.

22.1.2. What a CPDAG tells you

A directed edge $X \to Y$ in the CPDAG means: in *every* DAG consistent with the data, X causes Y. An undirected edge $X - Y$ means some consistent DAGs have $X \to Y$ and others have $X \leftarrow Y$ — data alone cannot decide.

For causal estimation: once you have a CPDAG, you can use background knowledge (temporal ordering, experimental context) to orient the remaining undirected edges, and then apply identification strategies from earlier chapters.

22.2. Simulation

We generate a random Gaussian linear DAG using an Erdős–Rényi random graph with 8 nodes and edge probability 0.35. Each edge $j \to i$ generates a structural equation

$$X_i = \sum_{j \in \mathrm{pa}(i)} \beta_{ji} X_j + \varepsilon_i, \quad \varepsilon_i \sim N(0,1) \tag{22.2}$$

where coefficients β_{ji} are drawn uniformly from $[-1, -0.25] \cup [0.25, 1]$.

```
Random.seed!(2025)

n_vars    = 8
n_samples = 2000
edge_prob = 0.35
labels    = ["X$i" for i in 1:n_vars]

adj_mat   = gen_er_dag_adj_mat(n_vars, edge_prob)
data_mat  = gen_gaussian_data(adj_mat, n_samples)

true_dag  = SimpleDiGraph(adj_mat)
true_skel = SimpleGraph(true_dag)

@printf "Variables: %d   Samples: %d   True edges: %d\n" n_vars n_samples ne(true_skel)
```

```
Variables: 8   Samples: 2000   True edges: 9
```

```
draw_graph(adjmat_to_admg(adj_mat, labels); direction="TB")
```

```
[SVG graph - display in an HTML environment]
```

Figure 22.1.: True DAG used for simulation.

22.3. CI Test Infrastructure

Both algorithms use a **Fisher-Z conditional independence test** appropriate for Gaussian data. Given the partial correlation $\hat{\rho}_{XY|Z}$, the test statistic is

$$T = \sqrt{n - |Z| - 3} \cdot \operatorname{atanh}(\hat{\rho}_{XY|Z}) \;\sim\; N(0,1) \quad \text{under } H_0 : X \perp\!\!\!\perp Y \mid Z \tag{22.3}$$

Choosing the significance level α involves a bias-variance tradeoff on the skeleton:

- **Low α (e.g., 0.001)**: conservative test — it fails to reject independence more easily, so it removes more edges. Fewer spurious edges (false positives) but may discard real ones (false negatives). Produces a sparser skeleton.
- **High α (e.g., 0.05)**: liberal test — it rejects independence (declares dependence) more often, so it keeps more edges. Fewer false removals but may retain spurious edges. Produces a denser skeleton.

With $n = 2000$ and $p = 8$ we use $\alpha = 0.01$, a common default. With smaller n or larger p the CI tests lose power and true edges get dropped, so consider raising α to avoid over-pruning.

We wrap `fisher_z` in a counter to track exactly how many CI tests each algorithm calls:

```
function make_counted_ci(data, sig)
    count = Ref(0)
    function ci(x, y, S, d)
        count[] += 1
        fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig)
    end
    ci, count
end
```

```
make_counted_ci (generic function with 1 method)
```

22.4. PC Algorithm

The **PC algorithm** (Spirtes & Glymour, 1991) starts from a complete undirected graph and removes edges whenever a conditional independence is found. It searches for separating sets Z of increasing size, conditioning on subsets of the current neighbors. After skeleton discovery, Meek's rules orient as many edges as possible.

How it works, step by step:

1. Start with a complete undirected graph (every pair connected).
2. For each pair (X, Y), test $X \perp\!\!\!\perp Y \mid Z$ for all $Z \subseteq \text{neighbors}(X) \setminus \{Y\}$, starting with $|Z| = 0$. Remove the edge if any such test passes.
3. Repeat with conditioning sets of increasing size until no new edges are removed.
4. Identify colliders: if $X - Z - Y$ and X, Y non-adjacent, orient as $X \to Z \leftarrow Y$ only if Z was not in the separating set of X and Y.
5. Apply Meek's orientation rules to propagate orientations without creating new colliders or cycles.

```
sig_level = 0.01

ci_pc, ctr_pc = make_counted_ci(data_mat, sig_level)
cpdag_pc = pcalg(n_vars, ci_pc, data_mat)
skel_pc  = SimpleGraph(cpdag_pc)

f1_skel_pc  = f1_score(true_skel, skel_pc)
f1_cpdag_pc = f1_score(true_dag,  cpdag_pc)

@printf "PC:    skeleton F1 = %.3f   CPDAG F1 = %.3f   CI tests = %d\n" f1_skel_pc f1_cpdag_pc
```

```
PC:    skeleton F1 = 0.750   CPDAG F1 = 0.500   CI tests = 200
```

> **Note**
>
> **Reading the output: F1 scores**
> The **F1 score** is the harmonic mean of precision and recall, ranging from 0 (worst) to 1 (perfect).
>
> - **Skeleton F1** measures whether the algorithm found the right edges regardless of direction. An edge is a true positive if it exists in both the estimated skeleton and the true one.
> - **CPDAG F1** is stricter: it compares the two graphs' directed-edge sets, with an undirected CPDAG edge represented as *both* directions. A correctly-adjacent but unoriented edge therefore contributes one true positive (the direction matching the true DAG) and one false positive (the reverse) — a partial penalty — while a wrongly oriented edge contributes a false positive and a false negative.
>
> A high skeleton F1 with lower CPDAG F1 means the algorithm found the right adjacencies but left some edges unoriented — the typical case when the MEC contains many equivalent DAGs.

Computational complexity: In the worst case, PC tests all subsets of neighbors, giving exponential growth with the maximum degree. In sparse graphs this is manageable; in dense graphs it becomes prohibitive.

22.5. RSL-D Algorithm

RSL-D (Recursive Structure Learning — Diamond-free; Mokhtarian et al., JMLR 2025) works by *recursively removing* variables that satisfy a **removability** criterion.

Two sets are central to the criterion:

- **Markov boundary** $\mathrm{Mb}(X)$: the minimal set S such that $X \perp\!\!\!\perp$ (all other variables) $\mid S$. Conditioning on $\mathrm{Mb}(X)$ screens X off from everything else in the graph. In a Gaussian DAG with no hidden variables, $\mathrm{Mb}(X)$ consists of X's parents, children, and co-parents (other parents of X's children) — equivalently, all variables directly adjacent to X in the skeleton.
- **Neighbourhood** $\mathrm{Ne}(X)$: the set of variables directly adjacent to X in the *current* skeleton (connected by an undirected edge at this stage of the algorithm).

Under causal sufficiency and faithfulness, $\mathrm{Mb}(X) = \mathrm{Ne}(X)$; the distinction arises during the algorithm's recursive steps as variables are removed one by one.

Variable X is removable if, for every pair $Y \in \mathrm{Mb}(X)$, $Z \in \mathrm{Ne}(X)$:

$$\exists\, W \subseteq \mathrm{Mb}(X) \setminus \{Y, Z\} \text{ such that } Y \perp\!\!\!\perp Z \mid W \tag{22.4}$$

(Lemma 3 of the paper). Removing variables one by one and recording the local neighbourhood structure recovers the full CPDAG.

A variable is removable if its local neighborhood can be learned before the rest of the graph is fully known. The algorithm removes such variables one at a time and records the local structure.

How it works, step by step:

1. Estimate the Markov boundary $\mathrm{Mb}(X)$ of each variable (all variables that, conditioned on, make X independent of everything else).
2. Find a removable variable X — one whose neighbourhood satisfies the criterion above.
3. Record the local skeleton structure around X, then remove X from the graph.
4. Repeat on the reduced variable set until all variables have been processed.
5. Reconstruct the full CPDAG from the recorded local structures.

The key efficiency advantage: RSL-D uses at most $O(p \cdot d \cdot |\mathrm{MB}|^2)$ CI tests (where d is the max neighbourhood size). PC can require exponentially many in the worst case as the separator-set search grows.

```
ci_rsl, ctr_rsl = make_counted_ci(data_mat, sig_level)
cpdag_rsl = rsld(data_mat, ci_rsl)
skel_rsl  = SimpleGraph(cpdag_rsl)

f1_skel_rsl  = f1_score(true_skel, skel_rsl)
f1_cpdag_rsl = f1_score(true_dag,  cpdag_rsl)

@printf "RSL-D: skeleton F1 = %.3f   CPDAG F1 = %.3f   CI tests = %d\n" f1_skel_rsl f1_cpdag_rs
```

```
RSL-D: skeleton F1 = 1.000   CPDAG F1 = 0.947   CI tests = 43
```

22.6. Comparison

> **Note**
>
> **Reading the CPDAG figure**
> In the side-by-side graph display:
>
> - A **directed edge** $X_i \to X_j$ means this orientation is invariant across all equivalent DAGs — the algorithm is confident about the direction.
> - A **bidirected (red) edge** $X_i \leftrightarrow X_j$ in the CPDAG display represents an *unoriented* edge: both $X_i \to X_j$ and $X_i \leftarrow X_j$ are statistically equivalent. This is **not** a hidden common cause — it means the data cannot determine direction. (Hidden common causes are a latent-variable concept, covered in the next chapter.)
> - A **missing edge** means the algorithm found a conditional independence separating those two variables.
>
> Compare the estimated CPDAG to the true DAG: extra edges are false positives (the algorithm wrongly retained a spurious association); missing edges are false negatives (a true relationship was incorrectly removed by a CI test).

```
DataFrame(
    Algorithm   = ["PC",              "RSL-D"],
    Skeleton_F1 = round.([f1_skel_pc,  f1_skel_rsl],  digits=3),
    CPDAG_F1    = round.([f1_cpdag_pc, f1_cpdag_rsl], digits=3),
    CI_Tests    = [ctr_pc[],           ctr_rsl[]],
)
```

```
side_by_side([
    (adjmat_to_admg(adj_mat, labels),  "True DAG"),
    (cpdag_to_admg(cpdag_pc,  labels), "PC - estimated CPDAG"),
    (cpdag_to_admg(cpdag_rsl, labels), "RSL-D - estimated CPDAG"),
]; note="Red  edges in the CPDAGs are unoriented (not confounded) - both directions are statis
```

MultiSVG("<div style=\"display:flex;gap:20px;flex-wrap:wrap;\njustify-
content:center;align-items:flex-start\"><div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">True DAG</p><svg width=\"22
4,4 -4,-256 217,-256 217,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"18\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"41\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X2->X1 -->\n<g id=\"edge1\" class=\"edge\">\n<title
215.67C23.63,-205.69 17.04,-192.65 14,-180 2.98,-134.08 8.24,-78.67 13.1,-
46.28\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"16.58,-46.65 14.71,-
36.22 9.67,-45.55 16.58,-46.65\"/>\n</g>\n<!-- X3 -->\n<g id=\"node3\" class=\"node\">\n<title>
anchor=\"middle\" x=\"95\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X3</text>\n</g>\n<!-- X2->X3 -->\n<g id=\"edge3\" class=\"edge\">\n<title
215.7C60.76,-207.39 68.56,-197.28 75.61,-188.14\"/>\n<polygon fill=\"blue\" stroke=\"blue\" po
190.16 81.81,-180.1 72.93,-185.88 78.47,-190.16\"/>\n</g>\n<!-- X5 -->\n<g id=\"node5\" class=
anchor=\"middle\" x=\"41\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X5</text>\n</g>\n<!-- X2->X5 -->\n<g id=\"edge5\" class=\"edge\">\n<title
215.7C41,-207.98 41,-198.71 41,-190.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"44
190.1 41,-180.1 37.5,-190.1 44.5,-190.1\"/>\n</g>\n<!-- X6 -->\n<g id=\"node6\" class=\"node\">
anchor=\"middle\" x=\"137\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X6</text>\n</g>\n<!-- X3->X6 -->\n<g id=\"edge6\" class=\"edge\">\n<title
143.7C110.26,-135.56 116.19,-125.69 121.58,-116.7\"/>\n<polygon fill=\"blue\" stroke=\"blue\" p
118.48 126.74,-108.1 118.59,-114.88 124.59,-118.48\"/>\n</g>\n<!-- X4 -->\n<g id=\"node4\" clas
anchor=\"middle\" x=\"195\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X4</text>\n</g>\n<!-- X7 -->\n<g id=\"node7\" class=\"node\">\n<title>X7</title>
anchor=\"middle\" x=\"41\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X5->X7 -->\n<g id=\"edge8\" class=\"edge\">\n<title
143.7C41,-135.98 41,-126.71 41,-118.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"44
118.1 41,-108.1 37.5,-118.1 44.5,-118.1\"/>\n</g>\n<!-- X7->X1 -->\n<g id=\"edge2\" clas
71.7C32.75,-63.9 29.67,-54.51 26.82,-45.83\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=
44.51 23.62,-36.1 23.42,-46.7 30.07,-44.51\"/>\n</g>\n<!-- X8 -->\n<g id=\"node8\" class=\"node
anchor=\"middle\" x=\"141\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X8</text>\n</g>\n<!-- X8->X3 -->\n<g id=\"edge4\" class=\"edge\">\n<title
215.7C124.28,-207.56 117.8,-197.69 111.89,-188.7\"/>\n<polygon fill=\"blue\" stroke=\"blue\" po
186.54 106.24,-180.1 108.81,-190.38 114.66,-186.54\"/>\n</g>\n<!-- X8->X6 -->\n<g id=\"e
215.56C149.93,-205.33 153.43,-192.08 155,-180 157.06,-164.13 157.61,-159.79 155,-
144 153.56,-135.28 150.87,-126.06 148,-117.8\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points
116.46 144.48,-108.3 144.67,-118.89 151.23,-116.46\"/>\n</g>\n<!-- X8->X7 -->\n<g id=\"e
215.76C140.41,-196.63 136.83,-165.63 122,-144 108.93,-124.94 86.53,-
111.06 68.59,-102.3\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"69.72,-
98.96 59.17,-97.96 66.79,-105.32 69.72,-98.96\"/>\n</g>\n</g>\n</svg> <div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">PC - estimated CPDAG</p><sv
4,4 -4,-184 240,-184 240,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"18\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X7 -->\n<g id=\"node7\" class=\"node\">\n<title>X7</title>
anchor=\"middle\" x=\"45\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X1->X7 -->\n<g id=\"edge4\" class=\"edge\">\n<title
62.23C30.44,-56.75 32.66,-50.98 34.77,-45.51\"/>\n<polygon fill=\"red\" stroke=\"red\" points=
61.11 24.67,-71.7 31.54,-63.63 25.01,-61.11\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\
46.69 38.4,-36.1 31.54,-44.17 38.07,-46.69\"/>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node
anchor=\"middle\" x=\"118\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X3 -->\n<g id=\"node3\" class=\"node\">\n<title>X3</title>
anchor=\"middle\" x=\"164\" y=\"-14.3\" font-family=\"Times,serif\" font-

Table 22.1.: Algorithm comparison on the simulated 8-node DAG (n = 2000, significance 0.01)

	Algorithm	Skeleton_F1	CPDAG_F1	CI_Tests
	String	Float64	Float64	Int64
1	PC	0.75	0.5	200
2	RSL-D	1.0	0.947	43

> **Note**
>
> **Interpreting the comparison table**
>
> - **Skeleton F1** is the primary metric for comparing algorithms: it measures edge recovery independent of orientation, since orientation accuracy is bounded by the MEC.
> - **CPDAG F1** rewards correctly oriented edges and penalizes leaving orientable edges undirected.
> - **CI tests** is the computational cost metric. Fewer tests means faster runtime, especially important when n is large and each test involves inverting covariance matrices.
>
> On this dataset RSL-D typically matches or exceeds PC's accuracy while using substantially fewer CI tests — the advantage grows with p and graph density.

22.7. Monte Carlo Evaluation

A single dataset can be lucky. We average over 100 replications to get stable estimates.

What to look for in the MC results:

- The **mean skeleton F1** is the headline: how often does each algorithm recover the true adjacencies?
- The **distribution of CI test counts** (shown in Figure 22.3) shows whether the efficiency advantage is consistent or only appears on average.
- High variance in F1 across replications suggests the algorithms are sensitive to the particular random graph — denser or sparser graphs drawn from the same Erdős–Rényi model can be very different problems.

```
function evaluate_once(seed; n_vars=8, n_samples=2000, edge_prob=0.35, sig=0.01)
    Random.seed!(seed)
    adj  = gen_er_dag_adj_mat(n_vars, edge_prob)
    data = gen_gaussian_data(adj, n_samples)
    skel = SimpleGraph(SimpleDiGraph(adj))

    ci_pc,  ctr_pc  = make_counted_ci(data, sig)
    ci_rsl, ctr_rsl = make_counted_ci(data, sig)

    cp_pc  = pcalg(n_vars, ci_pc,  data)
    cp_rsl = rsld(data, ci_rsl)
```

```
    (
        f1_pc  = f1_score(skel, SimpleGraph(cp_pc)),
        f1_rsl = f1_score(skel, SimpleGraph(cp_rsl)),
        ci_pc  = ctr_pc[],
        ci_rsl = ctr_rsl[],
    )
end

mc = [evaluate_once(s) for s in 1:100]

@printf "\nMonte Carlo averages (100 replications, n=%d, p=%d)\n" n_samples n_vars
@printf "%-8s  Skeleton F1  CI tests\n" "Method"
@printf "%-8s  %10.3f  %8.1f\n" "PC"    mean(r.f1_pc  for r in mc) mean(r.ci_pc  for r in mc)
@printf "%-8s  %10.3f  %8.1f\n" "RSL-D" mean(r.f1_rsl for r in mc) mean(r.ci_rsl for r in mc)
```

```
Monte Carlo averages (100 replications, n=2000, p=8)
Method    Skeleton F1  CI tests
PC              0.894     233.2
RSL-D           0.942      56.0
```

```
let
    fig = Figure(size=(650, 380))
    ax  = Axis(fig[1,1];
        xlabel = "CI tests",
        ylabel = "Frequency",
        title  = "CI Test Counts: PC vs RSL-D (100 replications)")
    hist!(ax, [r.ci_pc  for r in mc]; bins=20, color=(:steelblue, 0.65), label="PC")
    hist!(ax, [r.ci_rsl for r in mc]; bins=20, color=(:tomato, 0.65),    label="RSL-D")
    axislegend(ax)
    fig
end
```

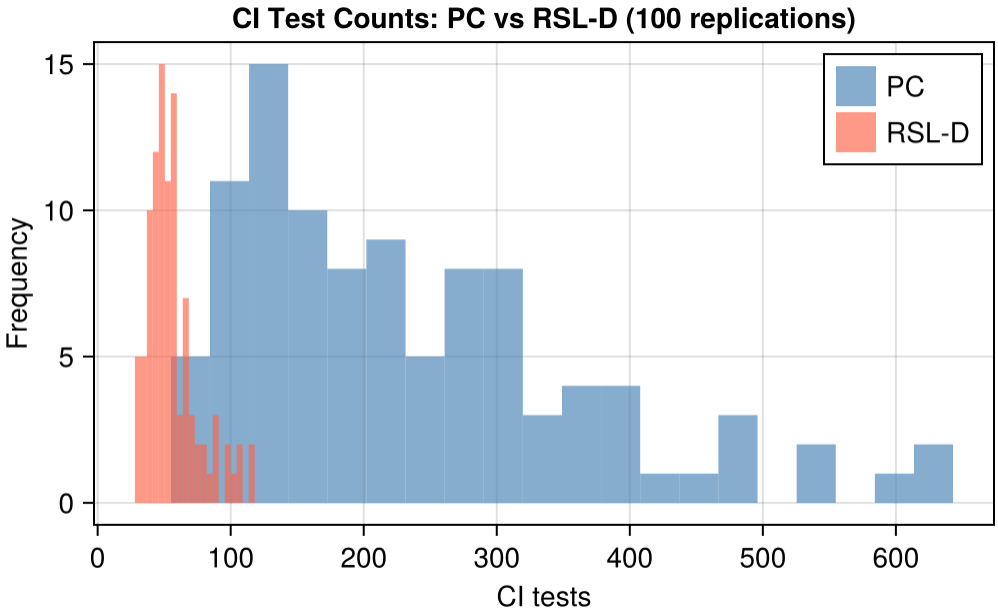


Figure 22.3.: Distribution of CI test counts over 100 replications. RSL-D consistently uses fewer CI tests.

22.8. Real Data: Student Performance in PISA

The simulation had a luxury real research never does: a known true DAG, so we could score recovery with F1. On real data there is **no ground truth**, so the questions change. Instead of "how close is the estimate to the truth?" we ask: do PC and RSL-D *agree*; what can *background knowledge* orient; and is the structure *stable* under resampling?

We illustrate on the OECD's **PISA 2022** assessment, restricted to **United States** students and six variables with a clear substantive ordering.

Node	Meaning	Role
`HISEI`	Highest parental occupational status (ISEI)	Family background
`HOMEPOS`	Home possessions index	Family background
`IMMIG`	Immigration status (1 native … 3 first-gen)	Demographic
`GRADE`	Grade relative to modal grade for age	Schooling
`GENDER`	1 = female, 0 = male	Demographic
`MATH`	Mean of the 10 mathematics plausible values	Outcome

```
pisa   = CSV.read("data/pisa_usa2022.csv", DataFrame)
plabs  = ["HISEI","HOMEPOS","IMMIG","GRADE","GENDER","MATH"]
Xp     = Matrix{Float64}(pisa[:, plabs])
wp     = Float64.(pisa.W)
np_, pp_ = size(Xp)
@printf "Students (complete cases): %d\n" np_
```

```
Students (complete cases): 3890
```

> **Note**
>
> **Three honesty caveats**, kept explicit:
>
> - **Plausible values.** Each student's ability is reported as 10 plausible values (posterior draws), not one score. We use their mean, understating the measurement variance; a full treatment runs discovery on each PV and pools.
> - **Categorical-as-Gaussian.** `IMMIG`, `GRADE`, `GENDER` are discrete; Fisher-Z assumes joint Gaussianity — a standard but imperfect approximation.
> - **One country, one wave.** Pooling would inject country/time structure that appears as spurious edges unless modelled as extra nodes.

22.8.1. Respecting the survey design

PISA is not a simple random sample — students carry final weights `W_FSTUWT`. The CI tests are functions of the **covariance matrix** and a **sample size**, so we supply design-consistent inputs: a **weighted covariance/correlation** matrix, and **Kish's effective sample size** $n_{\text{eff}} = (\sum_i w_i)^2 / \sum_i w_i^2$.

```
mu_p  = (Xp' * wp) ./ sum(wp)
Xc_p  = Xp .- mu_p'
Sig_p = (Xc_p' * (wp .* Xc_p)) ./ sum(wp)      # design-weighted covariance
sd_p  = sqrt.(diag(Sig_p)); Cw_p = Sig_p ./ (sd_p * sd_p')
neff  = sum(wp)^2 / sum(wp .^ 2)
@printf "Raw n = %d   Kish effective n = %.0f\n" np_ neff
```

```
Raw n = 3890   Kish effective n = 3481
```

```
let dfc = DataFrame(Variable = plabs)
    for (k, l) in enumerate(plabs); dfc[!, l] = round.(Cw_p[:, k], digits=2); end
    dfc
end
```

Table 22.3.: Design-weighted correlation matrix (PISA 2022, USA).

	Variable	HISEI	HOMEPOS	IMMIG	GRADE	GENDER	MATH
	String	Float64	Float64	Float64	Float64	Float64	Float64
1	HISEI	1.0	0.42	-0.17	0.05	-0.02	0.33
2	HOMEPOS	0.42	1.0	-0.18	0.05	0.03	0.4
3	IMMIG	-0.17	-0.18	1.0	0.07	0.01	-0.05
4	GRADE	0.05	0.05	0.07	1.0	0.08	0.17
5	GENDER	-0.02	0.03	0.01	0.08	1.0	-0.09
6	MATH	0.33	0.4	-0.05	0.17	-0.09	1.0

22.8.2. Two algorithms on the weighted data

The CI test recomputes correlations from a data matrix, so to honour the design we build a synthetic sample of n_{eff} rows whose empirical covariance equals the design-weighted Σ_w exactly (a Cholesky recolour). Both PC and RSL-D then run on these design-consistent inputs unchanged.

```
m_eff = round(Int, neff)
Random.seed!(1)
Z  = randn(m_eff, pp_); Z .-= mean(Z, dims=1)
Z  = Z * inv(cholesky(Symmetric(cov(Z))).U)     # whiten
Xw = Z * cholesky(Symmetric(Sig_p)).U .+ mu_p'  # recolour to weighted covariance
@printf "Synthetic covariance reproduces Σ_w (max abs diff = %.1e)\n" maximum(abs.(cov(Xw) .- S

sig_p = 0.01
ci_p(x, y, S, d) = fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig_p)

cpdag_pcp  = pcalg(pp_, ci_p, Xw)
cpdag_rslp = rsld(Xw, ci_p)
skel_pcp   = SimpleGraph(cpdag_pcp)
skel_rslp  = SimpleGraph(cpdag_rslp)

common = length(intersect(Set(Tuple.(sort.([ [e.src,e.dst] for e in edges(skel_pcp)]))),
                          Set(Tuple.(sort.([ [e.src,e.dst] for e in edges(skel_rslp)])))))
@printf "Skeleton edges - PC: %d   RSL-D: %d   in both: %d\n" ne(skel_pcp) ne(skel_rslp) commo
```

```
Synthetic covariance reproduces Σ_w (max abs diff = 9.1e-13)
Skeleton edges - PC: 9   RSL-D: 9   in both: 9
```

```
side_by_side([
    (cpdag_to_admg(cpdag_pcp,  plabs), "PC - estimated CPDAG"),
    (cpdag_to_admg(cpdag_rslp, plabs), "RSL-D - estimated CPDAG"),
]; note="Red  edges are unoriented (not confounded) - both directions are statistically equiva
```

MultiSVG("<div style=\"display:flex;gap:20px;flex-wrap:wrap;\njustify-
content:center;align-items:flex-start\"><div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">PC - estimated CPDAG</p><sv
4,4 -4,-328 189.5,-328 189.5,4 -4,4\"/>\n<!-- HISEI -->\n<g id=\"node1\" class=\"node\">\n<titl
anchor=\"middle\" x=\"29.5\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">HISEI</text>\n</g>\n<!-- HOMEPOS -->\n<g id=\"node2\" class=\"node\">\n<title>HO
anchor=\"middle\" x=\"70.5\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">HOMEPOS</text>\n</g>\n<!-- HISEI->HOMEPOS -->\n<g id=\"edge1\" class=\"ed
215.73C35.94,-197.64 41.94,-168.52 49.5,-144 52.18,-135.3 55.65,-126.01 58.99,-
117.67\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"62.23,-119 62.8,-
108.42 55.76,-116.33 62.23,-119\"/>\n</g>\n<!-- IMMIG -->\n<g id=\"node3\" class=\"node\">\n<ti
anchor=\"middle\" x=\"70.5\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">IMMIG</text>\n</g>\n<!-- HISEI->IMMIG -->\n<g id=\"edge2\" class=\"edge\"
215.69C11.12,-185.23 -7.7,-120.67 13.5,-72 18.43,-60.68 27.11,-50.68 36.23,-
42.51\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"38.49,-45.19 43.92,-
36.09 34,-39.81 38.49,-45.19\"/>\n</g>\n<!-- MATH -->\n<g id=\"node6\" class=\"node\">\n<title>
anchor=\"middle\" x=\"89.5\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">MATH</text>\n</g>\n<!-- HISEI->MATH -->\n<g id=\"edge3\" class=\"edge\">\
215.7C51.52,-207.3 60.3,-197.07 68.19,-187.86\"/>\n<polygon fill=\"blue\" stroke=\"blue\" point
189.97 74.84,-180.1 65.67,-185.42 70.99,-189.97\"/>\n</g>\n<!-- HOMEPOS->IMMIG -->\n<g i
71.7C70.5,-63.98 70.5,-54.71 70.5,-46.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"T
46.1 70.5,-36.1 67,-46.1 74,-46.1\"/>\n</g>\n<!-- GRADE -->\n<g id=\"node4\" class=\"node\">\n<
anchor=\"middle\" x=\"150.5\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">GRADE</text>\n</g>\n<!-- GRADE->IMMIG -->\n<g id=\"edge5\" class=\"edge\"
215.89C152.99,-185.2 152.36,-119.54 127.5,-72 121.68,-60.87 112.57,-
50.83 103.32,-42.56\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"105.5,-
39.82 95.6,-36.05 100.99,-45.17 105.5,-39.82\"/>\n</g>\n<!-- GRADE->MATH -->\n<g id=\"ed
215.7C128.11,-207.3 119.19,-197.07 111.16,-187.86\"/>\n<polygon fill=\"blue\" stroke=\"blue\" p
185.34 104.41,-180.1 108.34,-189.94 113.61,-185.34\"/>\n</g>\n<!-- GENDER -->\n<g id=\"node5\"
anchor=\"middle\" x=\"118.5\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">GENDER</text>\n</g>\n<!-- GENDER->GRADE -->\n<g id=\"edge7\" class=\"edge
287.7C130.05,-279.73 134.45,-270.1 138.49,-261.26\"/>\n<polygon fill=\"blue\" stroke=\"blue\" p
262.65 142.68,-252.1 135.34,-259.74 141.71,-262.65\"/>\n</g>\n<!-- GENDER->MATH -->\n<g
287.85C112.06,-277.49 109.01,-264.01 106.5,-252 102.19,-231.34 97.73,-
207.89 94.46,-190.25\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"97.84,-
189.3 92.59,-180.1 90.96,-190.56 97.84,-189.3\"/>\n</g>\n<!-- MATH->HOMEPOS -->\n<g id=
143.7C82.69,-135.9 80.14,-126.51 77.78,-117.83\"/>\n<polygon fill=\"blue\" stroke=\"blue\" poi
116.84 75.14,-108.1 74.38,-118.67 81.14,-116.84\"/>\n</g>\n</g>\n</svg> <div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">RSL-
D - estimated CPDAG</p><svg width=\"220pt\" height=\"332pt\"\n viewBox=\"0.00 0.00 220.25 332.0
4,4 -4,-328 216.25,-328 216.25,4 -4,4\"/>\n<!-- HISEI -->\n<g id=\"node1\" class=\"node\">\n<ti
anchor=\"middle\" x=\"52.25\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">HISEI</text>\n</g>\n<!-- HOMEPOS -->\n<g id=\"node2\" class=\"node\">\n<title>HO
anchor=\"middle\" x=\"52.25\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">HOMEPOS</text>\n</g>\n<!-- HOMEPOS->HISEI -->\n<g id=\"edge1\" class=\"ed
287.86C23.68,-248.63 -18.31,-149.73 9.25,-72 12.92,-61.68 19.54,-51.9 26.51,-
43.61\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"29.13,-45.94 33.21,-
36.16 23.92,-41.26 29.13,-45.94\"/>\n</g>\n<!-- IMMIG -->\n<g id=\"node3\" class=\"node\">\n<ti
anchor=\"middle\" x=\"52.25\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">IMMIG</text>\n</g>\n<!-- HOMEPOS->IMMIG -->\n<g id=\"edge2\" class=\"edge
287.85C52.25,-250.83 52.25,-163.18 52.25,-118.39\"/>\n<polygon fill=\"blue\" stroke=\"blue\" po
118.23 52.25,-108.23 48.75,-118.23 55.75,-118.23\"/>\n</g>\n<!-- MATH -->\n<g id=\"node6\" clas
anchor=\"middle\" x=\"115.25\" y=\"-230.3\" font-family=\"Times,serif\" font-

> **!** Important
>
> **Read the disagreement, not just the agreement.** A purely data-driven method may orient an edge as `MATH → HISEI` — a child's test score causing their parent's occupational status. That is causally backwards, and the data cannot protect us: which direction this edge takes is not identifiable from observational data alone, so any orientation the algorithm prints reflects its (possibly mistaken) v-structure decisions under misspecification, not evidence about the true direction. This is exactly why background knowledge is not optional.

22.8.3. Orienting with background knowledge

Sex and immigration background are fixed at birth; family socioeconomic status is established long before the test; grade precedes the assessment; the score is the outcome. That gives a **tier ordering**

$$\{\text{GENDER}, \text{IMMIG}\} \prec \{\text{HISEI}, \text{HOMEPOS}\} \prec \text{GRADE} \prec \text{MATH}. \tag{22.5}$$

A cross-tier edge must point earlier→later; a within-tier edge the ordering cannot decide. We impose this on the discovered **skeleton**, because the ordering is knowledge we hold independently of how the data oriented things.

```
tier_p = Dict("GENDER"=>0,"IMMIG"=>0,"HISEI"=>1,"HOMEPOS"=>1,"GRADE"=>2,"MATH"=>3)
di_e = Tuple{Symbol,Symbol}[]; bi_e = Tuple{Symbol,Symbol}[]
for e in edges(skel_pcp)
    a, b = plabs[e.src], plabs[e.dst]
    if tier_p[a] == tier_p[b]
        push!(bi_e, (Symbol(a), Symbol(b)))          # within-tier: undirected
    else
        lo, hi = tier_p[a] < tier_p[b] ? (a, b) : (b, a)
        push!(di_e, (Symbol(lo), Symbol(hi)))        # cross-tier: earlier → later
    end
end
oriented_admg = make_graph(vertices=Symbol.(plabs), di_edges=di_e, bi_edges=bi_e)
```

```
ADMG([:HISEI, :HOMEPOS, :IMMIG, :GRADE, :GENDER, :MATH], [(:IMMIG, :HISEI), (:HISEI, :MATH), (
```

> **⚠** Warning
>
> **Where data-driven orientation disagreed with us.** Left to itself, an algorithm may orient `IMMIG-GRADE` as `GRADE → IMMIG` — schooling causing immigration status, which is impossible. Such mistakes come from v-structure and propagation decisions made under misspecification; the data is not informative about these directions. Imposing the tier order overrides them. The lesson: **never ship a discovered orientation you have prior reason to disbelieve**.

```
draw_graph(oriented_admg; direction="LR")
```

```
[SVG graph - display in an HTML environment]
```

Figure 22.5.: PC skeleton oriented by the temporal/logical tiers. Cross-tier edges point earlier-to-later; HISEI–HOMEPOS stays undirected (same tier, shown red).

Every edge into `MATH` is now a candidate causal parent of achievement — the bridge from discovery to estimation (back-door adjustment, e.g. on the parents of the chosen treatment variable). The only edge the ordering leaves undirected, `HISEI - HOMEPOS`, is two same-tier measures of family background that temporal logic cannot separate.

22.8.4. Stability: does the skeleton survive resampling?

A single fit can be lucky. We resample students with replacement, re-estimate the **weighted** PC skeleton, and record how often each edge appears.

```
function weighted_skel(Xb, wb, labs, sig)
    mu = (Xb' * wb) ./ sum(wb); Xc = Xb .- mu'
    Σ  = (Xc' * (wb .* Xc)) ./ sum(wb)
    m  = round(Int, sum(wb)^2 / sum(wb .^ 2))
    Z  = randn(m, length(labs)); Z .-= mean(Z, dims=1)
    Z  = Z * inv(cholesky(Symmetric(cov(Z))).U)
    Xs = Z * cholesky(Symmetric(Σ)).U .+ mu'
    ci(x, y, S, d) = fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig)
    SimpleGraph(pcalg(length(labs), ci, Xs))
end

B_pisa = 200
adj_b  = Vector{Matrix{Float64}}(undef, B_pisa)   # one slot per replicate (race-free)
Threads.@threads for b in 1:B_pisa
    rng = MersenneTwister(1000 + b)
    idx = rand(rng, 1:np_, np_)
    g   = weighted_skel(Xp[idx, :], wp[idx], plabs, sig_p)
    A   = zeros(pp_, pp_)
    for e in edges(g); A[e.src, e.dst] += 1; A[e.dst, e.src] += 1; end
    adj_b[b] = A
end
edge_freq = sum(adj_b) ./ B_pisa
```

```
6×6 Matrix{Float64}:
 0.0    1.0   1.0    0.015  0.0   1.0
 1.0    0.0   1.0    0.0    0.17  1.0
 1.0    1.0   0.0    0.895  0.02  0.0
 0.015  0.0   0.895  0.0    0.97  1.0
```

```
 0.0    0.17  0.02   0.97   0.0   0.98
 1.0    1.0   0.0    1.0    0.98  0.0
```

```
edge_names = String[]; edge_vals = Float64[]
for i in 1:pp_-1, j in i+1:pp_
    if edge_freq[i, j] > 0
        push!(edge_names, "$(plabs[i]) - $(plabs[j])")
        push!(edge_vals,  edge_freq[i, j])
    end
end
ord = sortperm(edge_vals)
let
    fig = Figure(size=(680, 380))
    ax  = Axis(fig[1,1]; xlabel="Selection frequency across bootstrap resamples",
               yticks=(1:length(ord), edge_names[ord]), title="Bootstrap edge stability (weight
    barplot!(ax, 1:length(ord), edge_vals[ord]; direction=:x, color=(:steelblue, 0.8))
    vlines!(ax, [0.5]; linestyle=:dash, color=:gray)
    xlims!(ax, 0, 1)
    fig
end
```

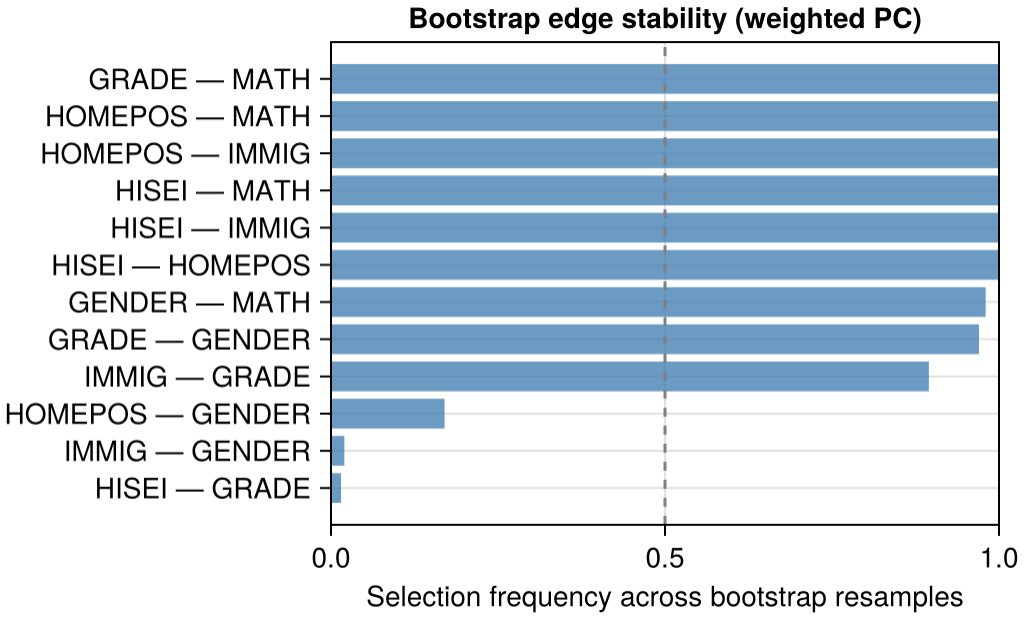


Figure 22.6.: Bootstrap edge-selection frequency (weighted PC, 200 resamples). Edges near 1.0 are robust; near 0.5 are sample-dependent.

The family-background→math edges (`HISEI`, `HOMEPOS`) are the most stable features: the SES gradient in achievement is not an artefact. On real data, causal discovery is best read as **hypothesis generation** — it proposes a skeleton and the orientations the data support, which you then reconcile with subject-matter knowledge before estimating effects.

22.9. Summary

Both PC and RSL-D target the Markov equivalence class. The practical difference is computational: RSL-D usually uses fewer CI tests, especially as the graph gets denser.

From CPDAG to causal estimation: Once a CPDAG is in hand, the workflow continues as follows:

1. **Use background knowledge to orient undirected edges.** Temporal ordering is the most common source: if X is measured before Y, the edge must be $X \to Y$. Domain expertise, experimental context, or exclusion restrictions can orient others.
2. **Check identifiability.** With a fully oriented DAG, apply the backdoor, front-door, or ID algorithm (Chapter 3) to determine whether the effect of interest is identifiable.
3. **Estimate.** Use the identified functional with the estimators from Chapter 4 (RA, IPW, AIPW).

If some edges remain undirected after exhausting background knowledge, the bound over the equivalence class is not automatic. Something has to enumerate what the class permits. For total effects under a linear Gaussian model that procedure is IDA (intervention calculus when the DAG is absent): walk the DAGs consistent with the CPDAG, compute the effect in each, and report the range of the resulting multiset. `CausalInference.jl` does not implement it – it gives you adjustment-set search (`find_backdoor_adjustment`, `list_covariate_adjustment` and relatives) rather than effect enumeration – so in Julia you either enumerate the consistent DAGs yourself with `Graphs.jl` and apply the identified functional to each, or call R's `pcalg::ida` through `RCall.jl`. The alternative is to collect interventional data and break the remaining equivalences.

> **Note**
>
> **What cannot be recovered from observational data alone**
> Both algorithms identify the CPDAG, not necessarily the true DAG. Some edge directions require extra assumptions, temporal ordering, or interventional data.
> The next chapter addresses a harder problem: recovering the causal skeleton when some variables are *unobserved*.

23. Causal Discovery: Latent Variables

```
using CausalInference
using RecursiveCausalDiscovery
using CausalGraphs
using Graphs
using CairoMakie
CairoMakie.activate!(type = "png")
using Statistics
using LinearAlgebra
using Random
using Printf
using DataFrames
using Combinatorics: powerset
using Distributions: Normal, quantile
```

The previous chapter assumed all causally relevant variables were observed. That is a strong assumption. In economics, ability, demand shocks, and peer effects are often unobserved.

When latent confounders exist, PC and RSL-D can give the wrong graph. Here I use algorithms designed for latent variables.

23.0.1. Why PC fails with latent variables

Suppose the true graph is $X \leftarrow U \rightarrow Y$ where U is unobserved. In the observed data, X and Y are marginally correlated (through U) and no observed conditioning set separates them. PC will incorrectly draw an edge $X - Y$ and may orient it as $X \rightarrow Y$ or $X \leftarrow Y$, neither of which exists in the truth.

With even one hidden common cause, the conditional independences among observed variables may not correspond to any DAG on the observed variables alone. We need a MAG/PAG representation.

23.1. Maximal Ancestral Graphs and PAGs

With latent variables, the appropriate representation is a **Maximal Ancestral Graph (MAG)**. Over a set of *observed* variables $\mathbf{O}$, a MAG encodes:

- $X \rightarrow Y$: X is an ancestor of Y in the full DAG
- $X \leftrightarrow Y$: X and Y have a common hidden ancestor (hidden confounder)
- No edge: $X \perp\!\!\!\perp Y \mid Z$ for some $Z \subseteq \mathbf{O} \setminus \{X, Y\}$

The MAG over observed variables summarizes the causal and confounding relationships visible in the observed data without naming the latent variables.

Just as DAGs have Markov equivalence classes represented by CPDAGs, MAGs have equivalence classes represented by **Partial Ancestral Graphs (PAGs)**. In a PAG, the mark $\circ$ on an edge endpoint means "could be arrowhead or tail in some member of the equivalence class."

PAG mark	Meaning	Economic interpretation
$X \to Y$	X causes Y in every equivalent MAG	Robust causal direction
$X \circ\!\!\to Y$	Some MAGs have $X \to Y$, others $X \leftrightarrow Y$ (the arrowhead at Y is invariant)	Direction uncertain
$X \leftrightarrow Y$	Hidden common cause in every equivalent MAG	Definite unmeasured confounder
$X \circ\!\!\circ Y$	Could be $\to$, $\leftarrow$, or $\leftrightarrow$	Maximally uncertain

Reading a PAG for policy purposes:

- A definite $X \to Y$ edge is the most useful finding: it means any intervention on X propagates to Y regardless of which specific MAG the data came from.
- A definite $X \leftrightarrow Y$ edge is a warning: before using regression of Y on X to estimate a causal effect, you must address the hidden confounder — through an instrument, a proxy, or a natural experiment.
- $\circ$ marks indicate what you *don't* know. Additional data, temporal ordering, or experimental variation can resolve them.

23.2. Simulation with Hidden Variables

We reuse the same 8-node Gaussian linear DAG from the previous chapter and hide 2 nodes, treating them as unobserved confounders.

```
Random.seed!(2025)

n_vars    = 8
n_samples = 2000
edge_prob = 0.35
latent    = [3, 6]
obs_idx   = setdiff(1:n_vars, latent)
all_labels = ["X$i" for i in 1:n_vars]
obs_labels = ["X$(obs_idx[i])" for i in 1:length(obs_idx)]

adj_mat   = gen_er_dag_adj_mat(n_vars, edge_prob)
data_full = gen_gaussian_data(adj_mat, n_samples)
data_obs  = data_full[:, obs_idx]
n_obs     = length(obs_idx)
```

```
@printf "Total variables: %d   Hidden: %s   Observed: %d\n" n_vars latent n_obs
```

```
Total variables: 8   Hidden: [3, 6]   Observed: 6
```

23.2.1. True structure over observed variables

Two observed variables are adjacent in the MAG iff no subset of the *observed* variables d-separates them in the full DAG.

```
true_dag = SimpleDiGraph(adj_mat)

function true_mag_skeleton(dag, observed)
    p = length(observed)
    skel = SimpleGraph(p)
    for i in 1:(p-1), j in (i+1):p
        u, v = observed[i], observed[j]
        other_obs = setdiff(observed, [u, v])
        separated = any(dsep(dag, u, v, S) for S in powerset(other_obs))
        !separated && add_edge!(skel, i, j)
    end
    return skel
end

true_obs_skel = true_mag_skeleton(true_dag, obs_idx)
@printf "True MAG skeleton edges (over %d observed nodes): %d\n" n_obs ne(true_obs_skel)
```

```
True MAG skeleton edges (over 6 observed nodes): 5
```

```
side_by_side([
    (admg_with_latent(adj_mat, all_labels, latent), "Full DAG (square = hidden)"),
    (skeleton_to_admg(true_obs_skel, obs_labels),   "True MAG skeleton (observed)"),
]; note="Square nodes are unobserved (latent). Plaintext nodes are observed.")
```

23.3. FCI Algorithm

FCI is the extension of PC for latent variables. It starts from a complete graph, removes edges with CI tests, and then uses orientation rules that can produce bidirected edges for hidden common causes.

How FCI extends PC:

1. **Phase 1 (skeleton):** Same as PC — remove edges via CI tests with growing conditioning sets.

MultiSVG("<div style=\"display:flex;gap:20px;flex-wrap:wrap;\njustify-
content:center;align-items:flex-start\"><div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">Full DAG (square = hidden)<
4,4 -4,-256 217,-256 217,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"18\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"41\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X2->X1 -->\n<g id=\"edge1\" class=\"edge\">\n<title
215.67C23.63,-205.69 17.04,-192.65 14,-180 2.98,-134.08 8.24,-78.67 13.1,-
46.28\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"16.58,-46.65 14.71,-
36.22 9.67,-45.55 16.58,-46.65\"/>\n</g>\n<!-- X3 -->\n<g id=\"node3\" class=\"node\">\n<title>
180 77,-180 77,-144 113,-144 113,-180\"/>\n<text text-anchor=\"middle\" x=\"95\" y=\"-
158.3\" font-family=\"Times,serif\" font-size=\"14.00\">X3</text>\n</g>\n<!-- X2->X3 -->
215.7C60.76,-207.39 68.56,-197.28 75.61,-188.14\"/>\n<polygon fill=\"blue\" stroke=\"blue\" poi
190.16 81.81,-180.1 72.93,-185.88 78.47,-190.16\"/>\n</g>\n<!-- X5 -->\n<g id=\"node5\" class=\
anchor=\"middle\" x=\"41\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X5</text>\n</g>\n<!-- X2->X5 -->\n<g id=\"edge5\" class=\"edge\">\n<title
215.7C41,-207.98 41,-198.71 41,-190.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"44
190.1 41,-180.1 37.5,-190.1 44.5,-190.1\"/>\n</g>\n<!-- X6 -->\n<g id=\"node6\" class=\"node\">
108 119,-108 119,-72 155,-72 155,-108\"/>\n<text text-anchor=\"middle\" x=\"137\" y=\"-
86.3\" font-family=\"Times,serif\" font-size=\"14.00\">X6</text>\n</g>\n<!-- X3->X6 -->
143.7C110.26,-135.56 116.19,-125.69 121.58,-116.7\"/>\n<polygon fill=\"blue\" stroke=\"blue\" p
118.48 126.74,-108.1 118.59,-114.88 124.59,-118.48\"/>\n</g>\n<!-- X4 -->\n<g id=\"node4\" clas
anchor=\"middle\" x=\"195\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X4</text>\n</g>\n<!-- X7 -->\n<g id=\"node7\" class=\"node\">\n<title>X7</title>
anchor=\"middle\" x=\"41\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X5->X7 -->\n<g id=\"edge8\" class=\"edge\">\n<title
143.7C41,-135.98 41,-126.71 41,-118.11\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"44
118.1 41,-108.1 37.5,-118.1 44.5,-118.1\"/>\n</g>\n<!-- X7->X1 -->\n<g id=\"edge2\" clas
71.7C32.75,-63.9 29.67,-54.51 26.82,-45.83\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\
44.51 23.62,-36.1 23.42,-46.7 30.07,-44.51\"/>\n</g>\n<!-- X8 -->\n<g id=\"node8\" class=\"node
anchor=\"middle\" x=\"141\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X8</text>\n</g>\n<!-- X8->X3 -->\n<g id=\"edge4\" class=\"edge\">\n<title
215.7C124.28,-207.56 117.8,-197.69 111.89,-188.7\"/>\n<polygon fill=\"blue\" stroke=\"blue\" po
186.54 106.24,-180.1 108.81,-190.38 114.66,-186.54\"/>\n</g>\n<!-- X8->X6 -->\n<g id=\"e
215.56C149.93,-205.33 153.43,-192.08 155,-180 157.06,-164.13 157.61,-159.79 155,-
144 153.56,-135.28 150.87,-126.06 148,-117.8\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points
116.46 144.48,-108.3 144.67,-118.89 151.23,-116.46\"/>\n</g>\n<!-- X8->X7 -->\n<g id=\"e
215.76C140.41,-196.63 136.83,-165.63 122,-144 108.93,-124.94 86.53,-
111.06 68.59,-102.3\"/>\n<polygon fill=\"blue\" stroke=\"blue\" points=\"69.72,-
98.96 59.17,-97.96 66.79,-105.32 69.72,-98.96\"/>\n</g>\n</g>\n</svg> <div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">True MAG skeleton (observed
4,4 -4,-328 117,-328 117,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"41\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"18\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X1->X2 -->\n<g id=\"edge1\" class=\"edge\">\n<title
277.95C30.37,-272.65 28.55,-267.1 26.81,-261.81\"/>\n<polygon fill=\"red\" stroke=\"red\" point
279.29 35.31,-287.7 35.52,-277.1 28.87,-279.29\"/>\n<polygon fill=\"red\" stroke=\"red\" points
260.51 23.62,-252.1 23.42,-262.7 30.07,-260.51\"/>\n</g>\n<!-- X7 -->\n<g id=\"node5\" class=\
anchor=\"middle\" x=\"41\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X1->X7 -->\n<g id=\"edge2\" class=\"edge\">\n<title
277.62C44.09,-269.39 44.69,-260.33 45,-252 46.81,-204.03 46.81,-191.97 45,-

2. **Phase 2 (initial orientation):** Mark all edge endpoints as ∘ (uncertain); orient unshielded colliders as in PC.
3. **Phase 3 (Possible-D-SEP pruning):** For each remaining edge $X - Y$, compute the **Possible-D-SEP** sets and run *additional* CI tests conditioning on their subsets, removing further edges. With latent variables, two non-adjacent observed nodes need not be separable by a subset of their neighbors — this stage is what makes FCI's skeleton differ from PC's, and it is the computationally expensive part. Colliders are then re-oriented on the pruned skeleton.
4. **Phase 4 (rule propagation):** Apply the 10 orientation rules (Zhang 2008) that propagate known marks without creating contradictions, including rules that can produce definite → and ↔ edges. These rules perform no CI tests and are cheap.

The extra marks let FCI say what is known and what remains uncertain, but the output is harder to read than a DAG.

```
sig_level = 0.01

count_fci = Ref(0)
ci_fci = (x, y, S, d) -> begin
    count_fci[] += 1
    fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig_level)
end

pag_fci  = fcialg(n_obs, ci_fci, data_obs)
skel_fci = SimpleGraph(pag_fci)

f1_fci = f1_score(true_obs_skel, skel_fci)
@printf "FCI:     skeleton F1 = %.3f   CI tests = %d\n" f1_fci count_fci[]
```

```
FCI:     skeleton F1 = 0.889   CI tests = 76
```

```
# No `println` here: plot_fci_graph_text prints the edge list as a side effect and
# returns nothing, so wrapping it in println() appends a literal "nothing" to the
# output -- which is what the rendered page used to show.
plot_fci_graph_text(pag_fci, ["X$(obs_idx[i])" for i in 1:n_obs])
```

```
X1<-X2   X1<-X7   X2o-oX5  X5o-oX7
```

Figure 23.2.

> **Note**
>
> **Reading the FCI text output**
> The text representation uses ASCII edge marks:
>
> - `X1 --> X2`: definite cause — X_1 is an ancestor of X_2 in every equivalent MAG (not

necessarily a *direct* effect: the path may run through latent mediators)
- `X1 <-> X2`: definite hidden common cause ($X_1 \leftrightarrow X_2$)
- `X1 o-> X2`: the X_1 endpoint is uncertain; X_2 endpoint is definitely an arrowhead
- `X1 o-o X2`: both endpoints uncertain; could be any of $\rightarrow$, $\leftarrow$, $\leftrightarrow$

In practice for economic research: focus first on `<->` edges — these flag definite confounding and tell you where IV or proxy strategies are needed. Then examine `-->` edges — these are causal claims that survive across all statistically equivalent structures.

When FCI gives wrong answers: FCI assumes the CI tests are perfectly accurate (no finite-sample error). In practice, with small n or many variables, some false CI decisions propagate through the orientation rules. The skeleton quality (F1) is generally more reliable than the orientation quality.

23.4. L-MARVEL Algorithm

L-MARVEL (Latent Markov Boundary Variable Elimination; Mokhtarian et al., JMLR 2025) extends the recursive MARVEL approach to handle latent confounders. Instead of exhaustive CI-test-based Markov boundary discovery, L-MARVEL uses a **Gaussian precision matrix** estimator:

$$\widehat{\mathrm{Mb}}(X) = \left\{ Y : \frac{|\hat{P}_{XY}|}{\sqrt{\hat{P}_{XX}\hat{P}_{YY}}} > \tau \right\} \tag{23.1}$$

where $\hat{P} = \hat{\Sigma}^{-1}$ is the precision matrix and τ is a threshold from the Fisher-Z distribution.

In a Gaussian linear model, a zero in the precision matrix means conditional independence given all other variables. L-MARVEL uses this fact to estimate Markov boundaries with fewer CI tests.

The removability criterion changes for the latent case (Theorem 32). Recall: $\mathrm{Mb}(X)$ is the Markov boundary of X — the minimal conditioning set that screens X off from all other variables (estimated here via the precision matrix above rather than by exhaustive CI tests) — and $\mathrm{Ne}(X)$ is the set of variables directly adjacent to X in the current skeleton. Variable X is removable iff for every $Y \in \mathrm{Mb}(X)$ and $Z \in \mathrm{Ne}(X)$:

$$\underbrace{\exists\, W \subseteq \mathrm{Mb}(X) \setminus \{Y, Z\} : Y \perp\!\!\!\perp Z \mid W}_{\text{Condition 1}} \quad \text{or} \quad \underbrace{\forall\, W \subseteq \mathrm{Mb}(X) \setminus \{Y, Z\} : Y \not\perp\!\!\!\perp Z \mid W \cup \{X\}}_{\text{Condition 2}} \tag{23.2}$$

Intuition for the two conditions:

- **Condition 1** is the same as the observed-variable (RSL-D) removability criterion: Y and Z can be separated by some subset of the Markov boundary.

- **Condition 2** is new and handles latent variables: it captures latent-induced dependencies through the theorem's removable-variable criterion. Loosely, when Y and Z are connected through a hidden common cause, no subset of $\mathrm{Mb}(X)$ separates them, yet conditioning on X changes the dependence. **This is not a generic d-separation rule** — it should not be read as "conditioning on X blocks hidden confounding," which would be false in general MAG/PAG reasoning. It is a theorem-specific condition (Theorem 32) on when X can be removed without losing latent-structure information; the formal statement above, not the intuition, is what the algorithm uses.

Together, the two conditions ensure that removing X does not destroy information about the latent structure connecting its neighbours.

```
count_lm = Ref(0)
ci_lm = (x, y, S, d) -> begin
    count_lm[] += 1
    fisher_z(x, y, collect(Int, S), Matrix{Float64}(d), sig_level)
end

lm_skel = l_marvel(data_obs, ci_lm)

f1_lm = f1_score(true_obs_skel, lm_skel)
@printf "L-MARVEL: skeleton F1 = %.3f   CI tests = %d\n" f1_lm count_lm[]
```

```
L-MARVEL: skeleton F1 = 1.000   CI tests = 14
```

> **Note**
>
> **L-MARVEL output: skeleton only**
> Unlike FCI, L-MARVEL returns only the **skeleton** — undirected edges with no arrowhead or bidirected marks. This is a deliberate design choice: the recursive removal procedure identifies *adjacencies* efficiently but does not run the full PAG orientation phase.
> This is still valuable. Knowing which pairs of variables are adjacent tells you which potential causal relationships to investigate further (with instruments, experiments, or domain knowledge). Variables with no edge are conditionally independent given some observed set — no causal relationship need be posited between them.
> If you need orientations as well as skeleton, run FCI. If you only need to screen out non-adjacent pairs (the most common use case in large-p problems), L-MARVEL's precision matrix approach is substantially faster.

23.5. Comparison

```
DataFrame(
    Algorithm   = ["FCI", "L-MARVEL"],
    Output      = ["PAG (orientations + skeleton)", "Skeleton only"],
```

Table 23.2.: Algorithm comparison on the latent-variable scenario (2 hidden nodes, n = 2000)

	Algorithm	Output	Skeleton_F1	CI_Tests
	String	String	Float64	Int64
1	FCI	PAG (orientations + skeleton)	0.889	76
2	L-MARVEL	Skeleton only	1.0	14

```
    Skeleton_F1 = round.([f1_fci, f1_lm], digits=3),
    CI_Tests    = [count_fci[], count_lm[]],
)
```

```
side_by_side([
    (skeleton_to_admg(true_obs_skel, obs_labels), "True MAG skeleton"),
    (skeleton_to_admg(skel_fci,      obs_labels), "FCI - estimated skeleton"),
    (skeleton_to_admg(lm_skel,       obs_labels), "L-MARVEL - estimated skeleton"),
]; note="All edges shown as undirected (skeleton comparison only).")
```

23.6. Monte Carlo Evaluation

```
function evaluate_latent(seed; n_vars=8, n_samples=2000, edge_prob=0.35, n_latent=2, sig=0.01)
    Random.seed!(seed)
    adj       = gen_er_dag_adj_mat(n_vars, edge_prob)
    data_full = gen_gaussian_data(adj, n_samples)
    dag       = SimpleDiGraph(adj)

    latent_vars = sort(randperm(n_vars)[1:n_latent])
    obs_vars    = setdiff(1:n_vars, latent_vars)
    data_obs    = data_full[:, obs_vars]
    n_obs       = length(obs_vars)

    true_skel = true_mag_skeleton(dag, obs_vars)

    cnt_fci = Ref(0)
    ci_fci  = (x, y, S, d) -> begin cnt_fci[] += 1; fisher_z(x, y, collect(Int, S), Matrix{Floa
    pag     = fcialg(n_obs, ci_fci, data_obs)
    f1f     = f1_score(true_skel, SimpleGraph(pag))

    cnt_lm  = Ref(0)
    ci_lm   = (x, y, S, d) -> begin cnt_lm[] += 1; fisher_z(x, y, collect(Int, S), Matrix{Float
    skel_lm = l_marvel(data_obs, ci_lm)
    f1l     = f1_score(true_skel, skel_lm)

    (f1_fci=f1f, f1_lm=f1l, ci_fci=cnt_fci[], ci_lm=cnt_lm[])
```

MultiSVG("<div style=\"display:flex;gap:20px;flex-wrap:wrap;\njustify-
content:center;align-items:flex-start\"><div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">True MAG skeleton</p><svg w
4,4 -4,-328 117,-328 117,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"41\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"18\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X1->X2 -->\n<g id=\"edge1\" class=\"edge\">\n<title
277.95C30.37,-272.65 28.55,-267.1 26.81,-261.81\"/>\n<polygon fill=\"red\" stroke=\"red\" point
279.29 35.31,-287.7 35.52,-277.1 28.87,-279.29\"/>\n<polygon fill=\"red\" stroke=\"red\" points
260.51 23.62,-252.1 23.42,-262.7 30.07,-260.51\"/>\n</g>\n<!-- X7 -->\n<g id=\"node5\" class=\"
anchor=\"middle\" x=\"41\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X1->X7 -->\n<g id=\"edge2\" class=\"edge\">\n<title
277.62C44.09,-269.39 44.69,-260.33 45,-252 46.81,-204.03 46.81,-191.97 45,-
144 44.69,-135.67 44.09,-126.61 43.46,-118.38\"/>\n<polygon fill=\"red\" stroke=\"red\" points=
277.58 42.62,-287.83 46.93,-278.15 39.95,-277.58\"/>\n<polygon fill=\"red\" stroke=\"red\" poin
117.85 42.62,-108.17 39.95,-118.42 46.93,-117.85\"/>\n</g>\n<!-- X5 -->\n<g id=\"node4\" class=
anchor=\"middle\" x=\"18\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X5</text>\n</g>\n<!-- X2->X5 -->\n<g id=\"edge3\" class=\"edge\">\n<title
205.67C18,-200.69 18,-195.49 18,-190.51\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"14.5
205.7 18,-215.7 21.5,-205.7 14.5,-205.7\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"21.5
190.1 18,-180.1 14.5,-190.1 21.5,-190.1\"/>\n</g>\n<!-- X4 -->\n<g id=\"node3\" class=\"node\">
anchor=\"middle\" x=\"95\" y=\"-302.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X4</text>\n</g>\n<!-- X5->X7 -->\n<g id=\"edge4\" class=\"edge\">\n<title
133.95C28.63,-128.65 30.45,-123.1 32.19,-117.81\"/>\n<polygon fill=\"red\" stroke=\"red\" point
133.1 23.69,-143.7 30.13,-135.29 23.48,-133.1\"/>\n<polygon fill=\"red\" stroke=\"red\" points=
118.7 35.38,-108.1 28.93,-116.51 35.58,-118.7\"/>\n</g>\n<!-- X8 -->\n<g id=\"node6\" class=\"n
anchor=\"middle\" x=\"41\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X8</text>\n</g>\n<!-- X7->X8 -->\n<g id=\"edge5\" class=\"edge\">\n<title
61.67C41,-56.69 41,-51.49 41,-46.51\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"37.5,-
61.7 41,-71.7 44.5,-61.7 37.5,-61.7\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"44.5,-
46.1 41,-36.1 37.5,-46.1 44.5,-46.1\"/>\n</g>\n</g>\n</svg> <div style=\"text-
align:center;flex:1\">\n<p style=\"margin:0 0 4px;font-weight:600\">FCI - estimated skeleton</p
4,4 -4,-256 171,-256 171,4 -4,4\"/>\n<!-- X1 -->\n<g id=\"node1\" class=\"node\">\n<title>X1</t
anchor=\"middle\" x=\"41\" y=\"-230.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X1</text>\n</g>\n<!-- X2 -->\n<g id=\"node2\" class=\"node\">\n<title>X2</title>
anchor=\"middle\" x=\"18\" y=\"-158.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X2</text>\n</g>\n<!-- X1->X2 -->\n<g id=\"edge1\" class=\"edge\">\n<title
205.95C30.37,-200.65 28.55,-195.1 26.81,-189.81\"/>\n<polygon fill=\"red\" stroke=\"red\" point
207.29 35.31,-215.7 35.52,-205.1 28.87,-207.29\"/>\n<polygon fill=\"red\" stroke=\"red\" points
188.51 23.62,-180.1 23.42,-190.7 30.07,-188.51\"/>\n</g>\n<!-- X7 -->\n<g id=\"node5\" class=\"
anchor=\"middle\" x=\"41\" y=\"-14.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X7</text>\n</g>\n<!-- X1->X7 -->\n<g id=\"edge2\" class=\"edge\">\n<title
205.62C44.09,-197.39 44.69,-188.33 45,-180 46.81,-132.03 46.81,-119.97 45,-
72 44.69,-63.67 44.09,-54.61 43.46,-46.38\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"39
205.58 42.62,-215.83 46.93,-206.15 39.95,-205.58\"/>\n<polygon fill=\"red\" stroke=\"red\" poin
45.85 42.62,-36.17 39.95,-46.42 46.93,-45.85\"/>\n</g>\n<!-- X5 -->\n<g id=\"node4\" class=\"no
anchor=\"middle\" x=\"18\" y=\"-86.3\" font-family=\"Times,serif\" font-
size=\"14.00\">X5</text>\n</g>\n<!-- X2->X5 -->\n<g id=\"edge3\" class=\"edge\">\n<title
133.67C18,-128.69 18,-123.49 18,-118.51\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"14.5
133.7 18,-143.7 21.5,-133.7 14.5,-133.7\"/>\n<polygon fill=\"red\" stroke=\"red\" points=\"21.5
118.1 18,-108.1 14.5,-118.1 21.5,-118.1\"/>\n</g>\n<!-- X4 -->\n<g id=\"node3\" class=\"node\">
anchor=\"middle\" x=\"95\" y=\"-230.3\" font-family=\"Times,serif\" font-

```
end

mc = [evaluate_latent(s) for s in 1:100]

@printf "\nMonte Carlo averages (100 replications, n=%d, p=%d, %d latent)\n" n_samples n_vars 2
@printf "%-10s  Skeleton F1  CI tests\n" "Method"
@printf "%-10s  %10.3f  %8.1f\n" "FCI"      mean(r.f1_fci for r in mc) mean(r.ci_fci for r in m
@printf "%-10s  %10.3f  %8.1f\n" "L-MARVEL" mean(r.f1_lm  for r in mc) mean(r.ci_lm  for r in m
```

```
Monte Carlo averages (100 replications, n=2000, p=8, 2 latent)
Method      Skeleton F1  CI tests
FCI               0.912     170.5
L-MARVEL          0.934      80.9
```

```
let
    fig = Figure(size=(650, 380))
    ax  = Axis(fig[1,1];
        xlabel = "CI tests",
        ylabel = "Frequency",
        title  = "CI Test Counts: FCI vs L-MARVEL (100 replications)")
    hist!(ax, [r.ci_fci for r in mc]; bins=20, color=(:steelblue, 0.65), label="FCI")
    hist!(ax, [r.ci_lm  for r in mc]; bins=20, color=(:tomato, 0.65),    label="L-MARVEL")
    axislegend(ax)
    fig
end
```

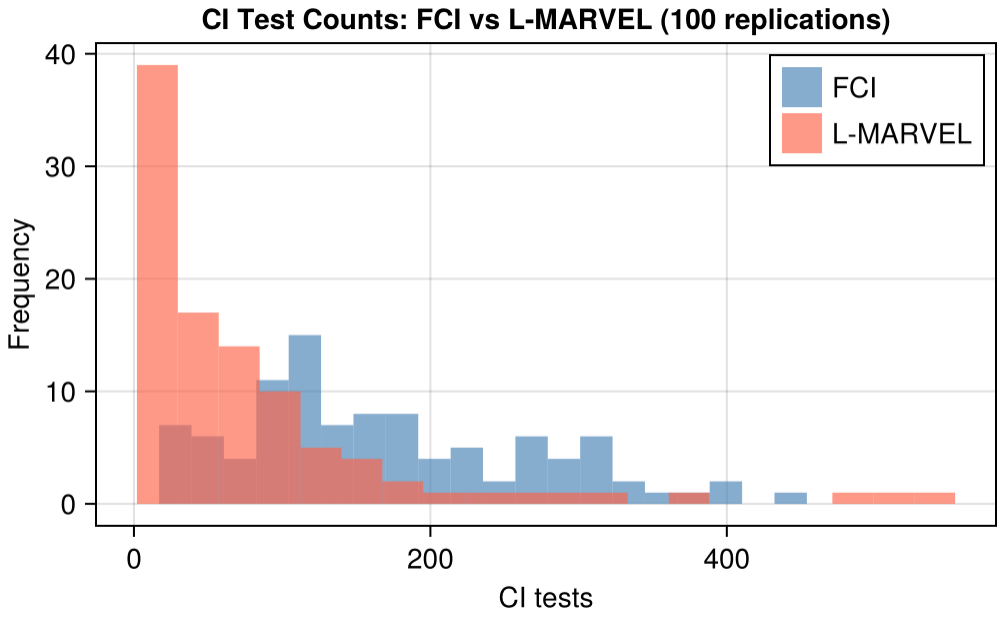


Figure 23.4.: CI test count distributions across 100 replications.

23.7. Summary

Property	PC / RSL-D	FCI	L-MARVEL
Latent confounders	assumes none	handles	handles
Output	CPDAG	PAG	Skeleton
Orientations			
MB estimation	CI tests	CI tests	Precision matrix
CI complexity	PC: $O(p^2 2^p)$ worst case; RSL-D: $O(p \cdot d \cdot \lvert\text{MB}\rvert^2)$	Same as PC	Fewer (Gaussian MB pre-filters)

FCI and L-MARVEL serve different purposes. Use FCI when orientations matter. Use L-MARVEL when the goal is a faster skeleton screen.

23.7.1. Practical decision guide

Use PC or RSL-D when: - You are confident in causal sufficiency (all relevant variables are measured) - You want a CPDAG that you can orient further with background knowledge - RSL-D is preferred over PC when $p > 10$ or the graph may be moderately dense

Use FCI when: - You suspect hidden confounders but don't know where - You need edge orientations and $\leftrightarrow$ marks to guide IV or proxy strategies - Sample size is large enough that CI-test errors don't cascade badly through orientation rules

Use L-MARVEL when: - You suspect latent confounders and have many variables ($p > 20$) - The goal is to prune the adjacency graph before applying domain knowledge or estimation - Gaussian data is appropriate (precision matrix estimator requires this)

23.7.2. From discovery to estimation

Causal discovery is a first step, not a final answer. A reasonable workflow is:

1. **Discover** — run FCI or L-MARVEL to get candidate adjacencies and, if using FCI, some edge marks
2. **Refine** — apply temporal ordering, institutional knowledge, and exclusion restrictions to orient remaining edges
3. **Identify** — check whether the effect of interest is identified given the refined graph (backdoor, front-door, ID algorithm)
4. **Estimate** — use TMLE, AIPW, or the estimators in earlier chapters

Causal discovery narrows the space of possible structures. Domain knowledge still does the hard work.

> ⚠ Warning
>
> **Sample size requirements**
> L-MARVEL's Gaussian MB estimator requires $n > p + 1$ samples. With many variables and limited data, fall back to CI-test-based MB estimation by passing a custom `mb_estimator` to `l_marvel`.

> ℹ Note
>
> **Going further: orientation in the latent case**
> FCI's PAG output distinguishes definite causes ($\rightarrow$), definite hidden common causes ($\leftrightarrow$), and uncertain cases ($\circ$). The PAG can be combined with background knowledge such as temporal ordering to further orient edges before estimation.

23.8. How Much Does Causal Discovery Help in Economics?

Recovering the MAG skeleton is still far from recovering the full DAG. Much is still missing:

What is recovered	What is still missing
L-MARVEL skeleton	Edge directions; direct cause vs. hidden common cause; latent nodes
FCI PAG	Unique MAG; latent nodes; most edges still carry $\circ$ marks
True MAG	Latent nodes and their connections
Full DAG	Nothing — this is the goal

The core econometric question is almost always "what is the causal effect of X on Y?" Causal discovery with latent variables does not answer this directly. A definite $X \leftrightarrow Y$ in a PAG tells you confounding exists — but not how to remove it. Without oriented edges you cannot even check whether the effect is identified, let alone estimate it.

Where these methods do add value in economics:

1. **Falsifying structural models.** If your model implies $X \perp\!\!\!\perp Z \mid W$, test it. A rejection means the model's independence assumptions are inconsistent with the data — discovery tells you *which* assumptions fail before you commit to a full structural estimation.
2. **Flagging where instruments are needed.** A definite $X \leftrightarrow Y$ in the PAG is a data-driven diagnostic: OLS of Y on X is biased, and you need an instrument, proxy, or natural experiment. Discovery does not supply the instrument but tells you where to look for one.
3. **High-dimensional variable selection.** With many candidate controls, knowing which pairs are conditionally independent prunes the problem before applying identification strategies. This is most useful in settings with $p > 20$ variables where theory does not specify the full graph.

4. **Hypothesis generation when theory is silent.** For new economic phenomena — platform markets, fintech, peer effects in novel settings — where theory does not give a strong causal ordering, discovery algorithms generate hypotheses worth investigating with better-powered research designs.

Causal discovery is a complement to IV, RD, DiD, and synthetic control, not a substitute. It is most useful early in a project, as a diagnostic and hypothesis-generating tool.

24. From Graph to Estimate

```
using CausalGraphs
using DataFrames
using Random
using Statistics
using CairoMakie
CairoMakie.activate!(type = "png")
```

The previous chapters covered discovery: algorithms that learn a graph from data. But a graph is not an estimate. After we have a graph, we still need to ask two questions:

1. **Identification**: is the target effect a function of the observed-data distribution?
2. **Estimation**: once the identifying formula is known, estimate it from the data.

`CausalGraphs.jl` implements both steps for **acyclic directed mixed graphs (ADMGs)** — directed graphs that may also have bidirected edges representing unmeasured common causes. ADMGs are the natural output of discovery algorithms like FCI when there are latent confounders. They are also the natural way to encode researcher hypotheses about which arrows are causal and which open paths represent hidden confounding.

Here are three cases: backdoor identification, front-door identification, and non-identification.

24.1. Backdoor: a-fixable effects

Start with the simple case. X is a common cause of treatment A and outcome Y, and X is observed. Backdoor adjustment applies. In the package terminology this is `a_fixable`.

```
# Build the graph: X is a common cause of A and Y
g_backdoor = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
)

id_backdoor = identify(g_backdoor, :A, :Y)
@printf("Identification strategy: %s\n", id_backdoor.strategy)
```

```
Identification strategy: a_fixable
```

The package recognises the backdoor structure: `X` blocks the only open backdoor path from A to Y, so adjusting on X identifies the ACE — the average causal effect, which is the ATE of the earlier chapters under a name `CausalEstimate.jl` uses for the field it returns.

```
Random.seed!(1)
n = 2000
X = randn(n)
A = Float64.(rand(n) .< 1 ./ (1 .+ exp.(-X)))      # treatment depends on X
Y = 2 .* A .+ X .+ 0.5 .* randn(n)                  # true ACE = 2
data_bd = DataFrame(X = X, A = A, Y = Y)

result_bd = estimate_causal(
    a = [1, 0],
    data      = data_bd,
    graph     = g_backdoor,
    treatment = :A,
    outcome   = :Y,
)

@printf("\nBackdoor estimation (true ACE = 2.0):\n")
@printf("  TMLE  ACE: %.3f  [95%% CI: %.3f, %.3f]\n",
        result_bd[:TMLE].ACE, result_bd[:TMLE].lower_ci, result_bd[:TMLE].upper_ci)
@printf("  AIPW  ACE: %.3f  [95%% CI: %.3f, %.3f]\n",
        result_bd[:Onestep].ACE, result_bd[:Onestep].lower_ci, result_bd[:Onestep].upper_ci)
@printf("  IPW   ACE: %.3f\n", result_bd[:IPW].ACE)
@printf("  GCOMP ACE: %.3f\n", result_bd[:Gcomp].ACE)
```

```
Backdoor estimation (true ACE = 2.0):
  TMLE  ACE: 2.018  [95% CI: 1.972, 2.065]
  AIPW  ACE: 2.018  [95% CI: 1.972, 2.065]
  IPW   ACE: 2.041
  GCOMP ACE: 2.018
```

All four estimators recover the true ACE in this simple example. TMLE and AIPW also return confidence intervals.

24.2. Front-door: p-fixable effects

Now suppose A and Y have an unmeasured common cause U, but there is a measured mediator M on the path $A \to M \to Y$. Backdoor adjustment fails because U is unobserved. The front-door formula can still identify the effect.

In ADMG notation, the unmeasured confounder appears as a bidirected edge between A and Y.

```
# A → M → Y, with unmeasured confounding between A and Y (bidirected edge)
g_frontdoor = make_graph(
    vertices = [:A, :M, :Y],
    di_edges = [(:A, :M), (:M, :Y)],
    bi_edges = [(:A, :Y)],
)

id_frontdoor = identify(g_frontdoor, :A, :Y)
@printf("Identification strategy: %s\n", id_frontdoor.strategy)
```

```
Identification strategy: p_fixable
```

The package returns `p_fixable`, recognizing the front-door structure. This uses a different functional than backdoor adjustment and must model M too.

```
Random.seed!(2)
n = 3000
U = randn(n)                                          # unmeasured confounder
A = Float64.(rand(n) .< 1 ./ (1 .+ exp.(-U)))         # A depends on U
M = Float64.(rand(n) .< 1 ./ (1 .+ exp.(-(0.5 .+ 2.0 .* A))))   # M depends on A only
Y = 1.5 .* M .+ 2.0 .* U .+ 0.5 .* randn(n)           # Y depends on M and U
data_fd = DataFrame(A = A, M = M, Y = Y)

# True ACE: effect of setting A=1 vs A=0
# E[Y|do(A=1)] - E[Y|do(A=0)] = 1.5 * (E[M|A=1] - E[M|A=0])
true_M_a1 = 1 / (1 + exp(-(0.5 + 2.0)))   # scalar P(M=1 | A=1)
true_M_a0 = 1 / (1 + exp(-0.5))           # scalar P(M=1 | A=0)
true_ace_fd = 1.5 * (true_M_a1 - true_M_a0)
@printf("True front-door ACE = %.3f\n\n", true_ace_fd)

result_fd = estimate_causal(
    a = [1, 0],
    data      = data_fd,
    graph     = g_frontdoor,
    treatment = :A,
    outcome   = :Y,
)

@printf("Front-door estimation:\n")
@printf("  TMLE  ACE: %.3f  [95%% CI: %.3f, %.3f]\n",
        result_fd[:TMLE].ACE, result_fd[:TMLE].lower_ci, result_fd[:TMLE].upper_ci)
```

```
True front-door ACE = 0.453

Front-door estimation:
  TMLE  ACE: 0.484  [95% CI: 0.401, 0.567]
```

The front-door functional pins down the effect even though U is unobserved. A naive mean difference is badly biased:

```
naive_ace = mean(Y[A .== 1]) - mean(Y[A .== 0])
@printf("Naïve mean difference (Y|A=1) - (Y|A=0): %.3f  (biased!)\n", naive_ace)
@printf("Front-door TMLE estimate:                %.3f\n", result_fd[:TMLE].ACE)
@printf("True ACE:                                %.3f\n", true_ace_fd)
```

```
Naïve mean difference (Y|A=1) - (Y|A=0): 2.132  (biased!)
Front-door TMLE estimate:                0.484
True ACE:                                0.453
```

The naive difference is biased by the path through U.

24.3. When identification fails

Not every graph identifies the effect. If A and Y have a hidden common cause and there is no front-door variable, the effect is not identified.

```
# A → Y with unmeasured confounding and no mediator
g_unident = make_graph(
    vertices = [:A, :Y],
    di_edges = [(:A, :Y)],
    bi_edges = [(:A, :Y)],
)

try
    id_unident = identify(g_unident, :A, :Y)
    @printf("Strategy: %s\n", id_unident.strategy)
catch e
    @printf("Identification failed: %s\n", sprint(showerror, e))
end
```

```
Strategy: not_identified
```

The package returns `not_identified` (or raises an informative error, depending on the version). The honest response in applied work is to report the non-identifiability, then either:

1. Find additional variables that close the unmeasured-confounding paths
2. Apply sensitivity analysis to bound the effect under maintained assumptions
3. Find an instrumental variable for a LATE-style identification (covered in the IV chapter)

Saying that the hidden confounding is probably small is sensitivity analysis, not identification.

24.4. End-to-end: discover, identify, estimate

The previous chapters' discovery algorithms (PC, FCI, RSL-D, L-MARVEL) return graphs that can be passed directly to `identify()` and `estimate_causal()`. The full pipeline is:

```
using CausalInference   # for PC algorithm

# Step 1: discover the graph (chapter 11)
graph_pc = pcalg(data, 0.05, gausscitest)

# Step 2: convert to ADMG (if needed; PC returns a CPDAG)
# Use the helpers in chapter 11 to convert CPDAG → ADMG

# Step 3: identify and estimate
id  = identify(graph_admg, :A, :Y)
result = estimate_causal(
    a = [1, 0],
    data = data,
    graph = graph_admg,
    treatment = :A,
    outcome = :Y,
)
```

In applied work, I would not trust the discovered graph blindly. A more reasonable workflow is:

1. **Hypothesise** a graph based on subject-matter knowledge
2. **Discover** a data-driven graph using PC/FCI/RSL-D
3. **Compare** the two — disagreement points to substantive questions
4. **Identify** under both graphs separately
5. **Estimate** under both, and report the range as part of the result

If the estimate changes a lot across plausible graphs, the data alone is not settling the question.

24.5. Why this matters

The graph-identification-estimation sequence matters because:

- **Backdoor adjustment is not always available.** Many real DAGs have hidden confounders. Front-door, nested, and general ID algorithms are the recourse, but they are too complex to derive by hand for non-trivial graphs.
- **The right estimator depends on the identification strategy.** TMLE-for-backdoor is different from TMLE-for-front-door. Automating the routing avoids the common mistake of plugging a graph into the wrong estimator.
- **Reporting becomes more disciplined.** Once the workflow forces you to specify a graph, it becomes harder to wave hands about "unconfoundedness" — you have to say exactly which variables are doing the unconfounding work.

24.6. Summary

- A graph is not an estimate. Identification is the bridge.
- `CausalGraphs.jl` routes ADMGs to backdoor, front-door, nested, or non-identified cases.
- `estimate_causal()` then runs the estimator that matches the identification strategy.
- In applied work, hypothesize a graph, use discovery as a check, identify, estimate, and report sensitivity to plausible graph choices.

Part VIII.

Appendix

25. Julia Packages Used in This Book

This appendix lists the Julia packages used in the book. Several were written alongside the examples and live under `~/projects/software`. They fill the gaps needed for the workflows here: fixed effects, semiparametric causal estimators, staggered DiD, synthetic control, TASC, regression discontinuity, SEM, and causal mediation.

The packages are regular Julia projects, and the manifest reaches them by path rather than through the registry. Three entries are relative – `Panelest`, `RDRobust` and `SynthDiD` resolve through `../../software/`. The other nine are absolute paths under `/home/xao/projects`, seven in `software/` and two, `CausalInference` and `RecursiveCausalDiscovery`, in `repo_cloned/` where they are local clones of registered packages. The manifest is therefore machine-specific: `Pkg.instantiate()` on another machine will fail until those paths exist or are re-pointed.

`LinearAlgebra`, `Random`, `Statistics`, `Printf`, `DelimitedFiles`, `Downloads` and `Logging` are Julia standard libraries. They sit on the default load path, so they need no `Project.toml` entry and no installation.

25.1. Working With The Environment

The reproducible workflow is:

```
cd ~/projects/books/causal_econometrics_julia
julia --project -e 'using Pkg; Pkg.instantiate()'
julia --project -e 'using CausalGraphs, CausalEstimate, Panelest, DiD, SynthDiD, RDRobust, Lava
quarto render
```

The standard `julia-1.12` Jupyter kernel uses `--project=@.`, so Quarto picks up the book environment when rendering from this directory. A custom sysimage can still be useful for speed, but it is optional and should be rebuilt whenever the local packages change.

25.2. Package Map

Package	Used For	Main Book Chapters
`Panelest.jl`	Fixed-effects OLS, GLM-style panel models, clustered covariance, IV/2SLS (`feiv`), and ETWFE post-processing (`emfx()`, `dataset()`)	Estimation, DiD, IV/RDD
`CausalGraphs.jl`	DAG/ADMG construction, graph-based identification, Pearl-Shpitser ID, and estimator routing	Identification, DAGs/ADMGs, Smoking cessation
`CausalEstimate.jl`	Unified TMLE and AIPW estimation: `estimate(ATE/ATT(...), TMLE/AIPW(...), df)` and graph-implied backdoor via `GraphID`	Estimation, Nonparametric, Smoking cessation
`CausalInference.jl` (local clone)	PC and FCI algorithms, Fisher-Z CI tests	Causal Discovery (both chapters)
`RecursiveCausalDiscovery.jl` (local clone)	RSL-D and L-MARVEL recursive discovery, F1 scoring	Causal Discovery (both chapters)
`SynthDiD.jl`	Synthetic control, standard DiD, and synthetic DiD estimators	DiD
`TASC.jl`	Time-Aware Synthetic Control with Kalman filtering, RTS smoothing, EM fitting, SC/RSC utilities, and TASC plot recipes	Synthetic control, DiD, and TASC
`RDRobust.jl`	Local-polynomial RD estimation, bandwidth selection, RD plot data, and fuzzy RD	IV/RDD
`Lavaan.jl`	SEM, CFA, defined parameters, fit measures, and lavaan-style mediation syntax	Mediation
`Crumble.jl`	Flexible causal mediation using cross-fitting, Riesz representers, and neural-network nuisance estimation	Mediation
`ShiftShareIV.jl`	Bartik/shift-share IV, Rotemberg weights, BHJ shock-level collapse	Shift-share IV
`DidInterference.jl`	DiD with spatial interference between units (`eval: false`)	DiD
`Endid.jl`	Distributional difference-in-differences (`eval: false`)	Distributional effects

Package	Used For	Main Book Chapters
`TVHTE.jl`	Time-varying heterogeneous event-study effects (`eval: false`)	DiD
`CairoMakie.jl` (registry)	Every figure in the book, via Makie's Cairo backend	Almost every chapter
`GraphMakie.jl` (registry)	Graph layouts for DAGs and mediation diagrams	Identification, IV/RDD, Mediation
`GeometryBasics.jl` (registry)	Geometric primitives for the identification figure	Identification
`CSV.jl` (registry)	Reading the example data files	Estimation, DiD, IV/RDD, Shift-share IV, Survival, Causal Discovery
`ReadStatTables.jl` (registry)	Reading Stata `.dta` example data	Estimation, DiD
`DataFramesMeta.jl` (registry)	`@chain`/`@transform` data wrangling	DiD, IV/RDD, Mediation, Poisson-IV
`RegressionTables.jl` (registry)	Formatted regression output	Estimation, DiD, IV/RDD
`Distributions.jl` (registry)	Draws and distributional calculations in the DGPs	Most chapters with a simulation
`StatsBase.jl` (registry)	Descriptive statistics and sampling helpers	Poisson-IV
`StatsFuns.jl` (registry)	Log/exp and distribution helper functions	Mediation
`MLJ.jl` (registry)	Machine-learning interface for nuisance estimation	Estimation, Nonparametric, Heterogeneous effects, IV/RDD
`MLJLinearModels.jl` (registry)	Linear and logistic learners for MLJ	Nonparametric
`MLJDecisionTreeInterface.jl` (registry)	Decision-tree learners for MLJ	Heterogeneous effects
`DecisionTree.jl` (registry)	Decision trees and random forests	Estimation, Nonparametric, IV/RDD
`EvoTrees.jl` (registry)	Gradient-boosted trees as a nuisance learner	Nonparametric
`GLMNet.jl` (registry)	Lasso and elastic net for high-dimensional nuisance models	Estimation

25.3. Panelest.jl

`Panelest.jl` handles the regression models with high-dimensional fixed effects. Its API stays close to the formula style used throughout the book:

```julia
using Panelest, Vcov

feols(df, @formula(y ~ x + fe(id) + fe(year)))
fepois(df, @formula(y ~ x + fe(id) + fe(year)))
feols(df, @formula(y ~ x + fe(id) + fe(year)), vcov = Vcov.cluster(:id))
```

The package exports `feols`, `fepois`, `felogit`, `feprobit`, `feglm`, `clogit`, `cre`, `feiv`, and `fe`. The fixed-effect term `fe(...)` is the central convenience feature. In the DiD chapter, it lets ETWFE specifications stay readable:

```julia
feols(
    mpdta,
    @formula(lemp ~ lpop + first_treat_str & year_str + fe(countyreal) + fe(year)),
    vcov = Vcov.cluster(:countyreal),
)
```

The current package also includes `feiv` for 2SLS-style workflows:

```julia
feiv(df, @formula(y ~ x_exo + fe(id)), endo = :x_endo, inst = :z)
```

25.4. CausalEstimate.jl

`CausalEstimate.jl` is the unified estimation front-end used in the estimation, nonparametric, and applied graph chapters. It combines TMLE and AIPW under one `estimate(...)` call, with cross-fitting and doubly robust inference built in.

```julia
using CausalEstimate, DataFrames

n = 1000
df = DataFrame(
    Y = randn(n),
    A = rand([0, 1], n),
    W = randn(n),
)

# AIPW for ATE
result = estimate(ATE(outcome=:Y, treatment=:A, confounders=[:W]), AIPW(crossfit=5), df)
estimate(result)       # point estimate
confint(result)        # (lower, upper) 95% CI
pvalue(result)         # two-sided p-value

# TMLE for ATE
result_tmle = estimate(ATE(outcome=:Y, treatment=:A, confounders=[:W]), TMLE(crossfit=5), df)

# ATT
result_att = estimate(ATT(outcome=:Y, treatment=:A, confounders=[:W]), AIPW(crossfit=5), df)
```

For graph-identified backdoor (a-fixable) effects, pass a `GraphID` layer:

```
using CausalEstimate, CausalGraphs

g = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
)

result = estimate(
    ATE(outcome = :Y, treatment = :A),
    GraphID(graph = g),
    AIPW(crossfit = 5),
    df,
)
```

`GraphID` reads the Markov pillow of the treatment from the graph and uses it as the adjustment set. For p-fixable (front-door), nested-fixable, and general ID-algorithm effects, use `CausalGraphs.estimate_causal(...)` directly.

25.5. CausalGraphs.jl

`CausalGraphs.jl` is the graph layer for the DAG/ADMG chapters. It constructs graphs, checks graph properties, runs `identify()`, and exposes the general Pearl-Shpitser `ID_algorithm()`:

```
using CausalGraphs

g = make_graph(
    vertices = [:X, :A, :Y],
    di_edges = [(:X, :A), (:X, :Y), (:A, :Y)],
)

identify(g, :A, :Y).strategy
```

For non-backdoor effects (p-fixable, nested-fixable, ID plug-in), `CausalGraphs.estimate_causal(...)` provides the full estimation pipeline:

```
res = CausalGraphs.estimate_causal(
    a = [1.0, 0.0],
    data = df,
    graph = g,
    treatment = :A,
    outcome = :Y,
)
```

```
res[:TMLE].ACE          # front-door NPS TMLE
res[:IDPlugin].ACE      # finite-support ID plug-in
```

25.6. ETWFE helpers in Panelest

`Panelest` also exports `emfx()` and `dataset()` for ETWFE post-processing:

```
using Panelest

att = emfx(model)
event_study = emfx(model, type = "event")
calendar    = emfx(model, type = "calendar")
```

`emfx()` reads the cohort-by-time interaction coefficients from a `feols` model and aggregates them into an overall ATT, event-time effects, or calendar-time effects. `dataset("mpdta")` returns a synthetic balanced panel with string cohort and year columns ready for StatsModels ETWFE interactions.

25.7. SynthDiD.jl

`SynthDiD.jl` implements the comparison among standard DiD, synthetic control, and synthetic DiD. The package exposes a matrix-oriented workflow:

```
using SynthDiD

df = california_prop99()
setup = panel_matrices(df, :State, :Year, :PacksPerCapita, :treated)

_sdid = synthdid_estimate(setup.Y, setup.N0, setup.T0)
_sc   = sc_estimate(setup.Y, setup.N0, setup.T0)
_did  = did_estimate(setup.Y, setup.N0, setup.T0)
```

The most important functions are `panel_matrices`, `synthdid_estimate`, `sc_estimate`, `did_estimate`, `effect_curve`, `placebo`, `vcov`, `se`, `california_prop99`, and `synthdid_controls`.

25.8. TASC.jl

`TASC.jl` implements Time-Aware Synthetic Control. It keeps the same panel-data spirit as synthetic control, but models the untreated outcome panel with a low-rank linear Gaussian state-space model:

```julia
using TASC

model = fit_tasc(Y; d = 2, T0 = 19)
pred = predict_counterfactual(model, Y)
```

The package exports `fit_tasc`, `predict_counterfactual`, `predict_post_intervention`, `tasc_plot`, classical synthetic-control baselines, HSVT/RSC-style denoising helpers, and synthetic data generators. It is registered in the environment's `Project.toml` as a developed dependency.

25.9. RDRobust.jl

`RDRobust.jl` is the RD layer used in the IV/RDD chapter. It mirrors the familiar `rdrobust` workflow:

```julia
using RDRobust

fit = rdrobust(y, x; c = 0.0)
bw = rdbwselect(y, x; c = 0.0)
plot_data = rdplot(y, x; c = 0.0)
```

It supports local-polynomial RD estimation, robust bias-corrected intervals, bandwidth selectors, covariate adjustment, clustered inference, multiple kernels, and fuzzy RD designs. In the book, `rdrobust` is used both for a sharp RD example and for a fuzzy RD comparison with a manual 2SLS specification.

25.10. Lavaan.jl

`Lavaan.jl` ports the R `lavaan` modeling style to Julia. It is used for classical SEM-based mediation:

```julia
using Lavaan

model = """
  m ~ a * x
  y ~ b * m + c * x
  indirect := a * b
  total := c + indirect
"""

fit = sem(model, df)
parameterEstimates(fit)
```

The public API includes `lavaan`, `cfa`, `sem`, `growth`, `sam`, `parameterEstimates`, `fitMeasures`, `standardizedSolution`, `lavInspect`, `lavTestLRT`, `modindices`, `rsquare`, `lavPredict`, and `simulateData`. The package supports the familiar lavaan operators `=~`, `~`, `~~`, `~1`, and `:=`.

25.11. Crumble.jl

`Crumble.jl` is the causal mediation package used for interventional and recanting-twin estimands. Its entry point is `crumble()`:

```
using Crumble

control = crumble_control(crossfit_folds = 5, epochs = 50)

result = crumble(
    df,
    ["A"];
    outcome = "Y",
    mediators = ["M"],
    moc = ["Z"],
    covar = ["W1", "W2"],
    effect = "RT",
    control = control,
)

result.estimates
```

The package exports `crumble`, `crumble_control`, `sequential_module`, and `CrumbleResult`. The supported effect families are natural (`"N"`), organic (`"O"`), randomized interventional (`"RI"`), and recanting twin (`"RT"`). In the mediation chapter, `"RT"` is used because it handles a mediator-outcome confounder affected by treatment.

25.12. Practical Advice

These packages are best treated as a coherent local research stack. When changing package code, rerun a direct load check from the book directory:

```
julia --project -e 'using CausalEstimate, CausalGraphs, Panelest, SynthDiD, RDRobust, Lavaan, C
```

For content changes, render the affected chapter first. For package API changes, render the full book after deleting the relevant `_freeze` chapter directory so Quarto does not reuse stale cached results.

References

Each chapter lists the works it cites at the bottom of its own page; the complete bibliography lives in `references.bib` in the book repository. Selected entries appear below. The main background references are Imbens and Rubin (2015) and Hernán and Robins (2020).

Angrist, Joshua D., and Jörn-Steffen Pischke. 2009. *Mostly Harmless Econometrics: An Empiricist's Companion*. Princeton University Press.

Blandhol, Christine, John Bonney, Magne Mogstad, and Alexander Torgovitsky. 2025. "When Is TSLS Actually LATE?"

Botosaru, Irene, and Laura Liu. 2025. "Time-Varying Heterogeneous Treatment Effects in Event Studies." *arXiv Preprint arXiv:2509.13698*. https://arxiv.org/abs/2509.13698.

———. 2026. "Event Studies with Feedback." *AEA Papers and Proceedings* 116: 70–74. https://doi.org/10.1257/pandp.20261110.

Chernozhukov, Victor, Mert Demirer, Esther Duflo, and Iván Fernández-Val. 2018. "Generic Machine Learning Inference on Heterogeneous Treatment Effects in Randomized Experiments." *NBER Working Paper 24678*.

Gourieroux, Christian, Alain Monfort, Eric Renault, and Alain Trognon. 1987. "Generalized Residuals." *Journal of Econometrics* 34 (1–2): 5–32.

Guggenberger, Patrik. 2010. "The Impact of a Hausman Pretest on the Asymptotic Size of a Hypothesis Test." *Econometric Theory* 26 (2): 369–82. https://doi.org/10.1017/S0266466609100014.

Hernán, Miguel A. 2010. "The Hazards of Hazard Ratios." *Epidemiology* 21 (1): 13–15.

Hernán, Miguel A., and James M. Robins. 2020. *Causal Inference: What If*. Chapman & Hall/CRC.

Hirano, Keisuke, and Guido W. Imbens. 2004. "The Propensity Score with Continuous Treatments." In *Applied Bayesian Modeling and Causal Inference from Incomplete-Data Perspectives*, 73–84. Wiley.

Imbens, Guido W., and Donald B. Rubin. 2015. *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press.

Imbens, Guido W., and Jeffrey M. Wooldridge. 2007. "What's New in Econometrics? Lecture 6: Control Functions and Related Methods." National Bureau of Economic Research Summer Institute. https://users.nber.org/~confer/2007/si2007/WNE/lect_6_controlfuncs.pdf.

Kennedy, Edward H. 2020. "Towards Optimal Doubly Robust Estimation of Heterogeneous Causal Effects." *arXiv Preprint arXiv:2004.14497*.

Kennedy, Edward H., Zongming Ma, Matthew D. McHugh, and Dylan S. Small. 2017. "Non-Parametric Methods for Doubly Robust Estimation of Continuous Treatment Effects." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 79 (4): 1229–45.

Künzel, Sören R., Jasjeet S. Sekhon, Peter J. Bickel, and Bin Yu. 2019. "Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning." *Proceedings of the National Academy of Sciences* 116 (10): 4156–65.

Lin, Wei, and Jeffrey M. Wooldridge. 2019. "Testing and Correcting for Endogeneity in Nonlinear Unobserved Effects Models." In *Panel Data Econometrics: Theory*, edited by Mike Tsionas,

21–43. Academic Press. https://doi.org/10.1016/B978-0-12-814367-4.00002-2.

Mullahy, John. 1997. "Instrumental-Variable Estimation of Count Data Models: Applications to Models of Cigarette Smoking Behavior." *Review of Economics and Statistics* 79 (4): 586–93.

Nie, Xinkun, and Stefan Wager. 2021. "Quasi-Oracle Estimation of Heterogeneous Treatment Effects." *Biometrika* 108 (2): 299–319.

Ramsey, J. B. 1969. "Tests for Specification Errors in Classical Linear Least-Squares Regression Analysis." *Journal of the Royal Statistical Society, Series B* 31 (2): 350–71.

Rho, Saeyoung, Cyrus Illick, Samhitha Narasipura, Alberto Abadie, Daniel Hsu, and Vishal Misra. 2026. "Time-Aware Synthetic Control." *arXiv Preprint arXiv:2601.03099*. https://arxiv.org/abs/2601.03099.

Rubin, Donald B. 1980. "Randomization Analysis of Experimental Data: The Fisher Randomization Test Comment." *Journal of the American Statistical Association* 75 (371): 591–93.

Słoczyński, Tymon. 2022. "Interpreting OLS Estimands When Treatment Effects Are Heterogeneous: Smaller Groups Get Larger Weights." *The Review of Economics and Statistics* 104 (3): 501–9.

———. 2024. "When Should We (Not) Interpret Linear IV Estimands as LATE?" *Quantitative Economics*.

Terza, Joseph V. 1998. "Estimating Count Models with Endogenous Switching: Sample Selection and Endogenous Treatment Effects." *Journal of Econometrics* 84: 129–54.

Wooldridge, Jeffrey M. 2010. *Econometric Analysis of Cross Section and Panel Data*. 2nd ed. MIT Press.

———. 2014. "Quasi-Maximum Likelihood Estimation and Testing for Nonlinear Models with Endogenous Explanatory Variables." *Journal of Econometrics* 182 (1): 226–34.

Xu, Ruonan. 2023. "Difference-in-Differences with Interference." *arXiv Preprint arXiv:2306.12003*. https://arxiv.org/abs/2306.12003.

———. 2026. "Dynamic Difference-in-Differences with Interference." *AEA Papers and Proceedings* 116: 58–63. https://doi.org/10.1257/pandp.20261108.